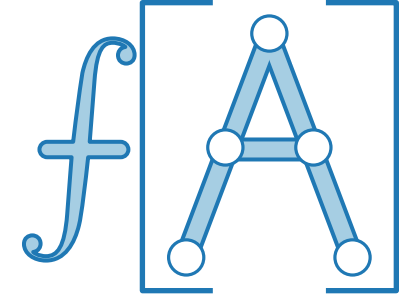
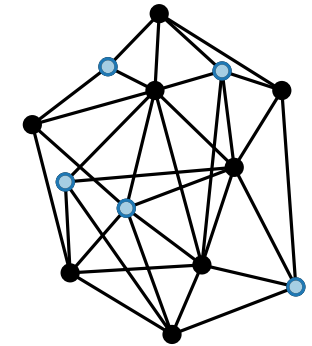
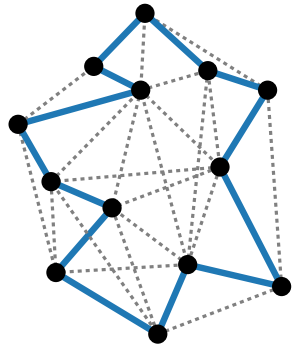


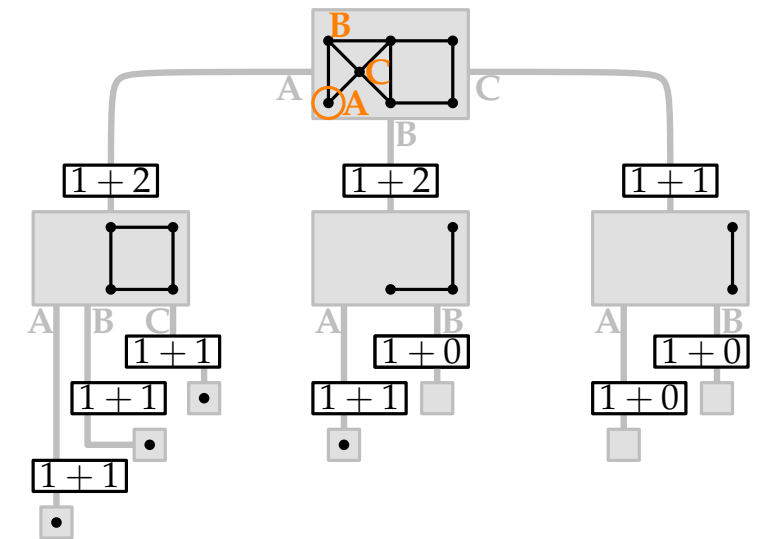
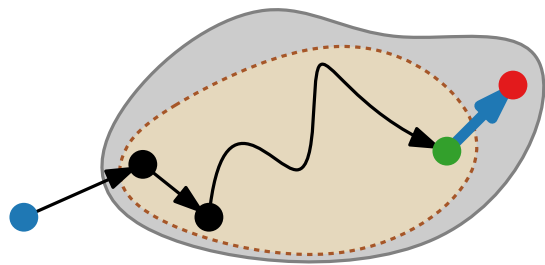


Fortgeschrittene Algorithmen

19. Vorlesung Exakte Algorithmen (für NP-schwere Probleme)

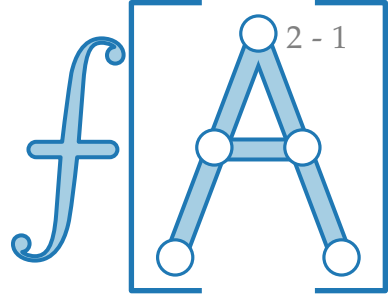


Teil I: Motivation



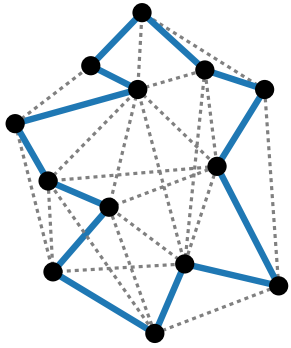
Beispiele von NP-schweren Problemen

Viele wichtige (praktische) Probleme sind NP-schwer, zum Beispiel...

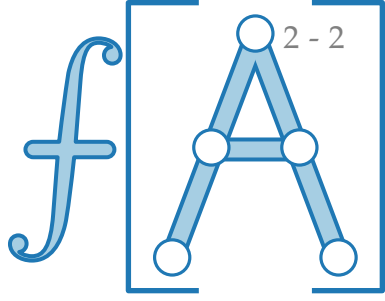


Beispiele von NP-schweren Problemen

Viele wichtige (praktische) Probleme sind NP-schwer, zum Beispiel...

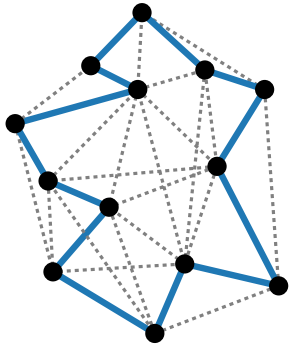


TSP

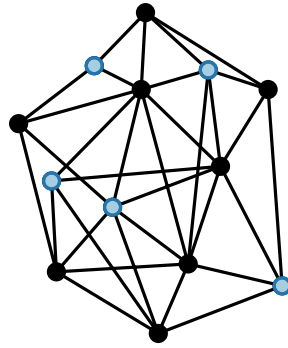


Beispiele von NP-schweren Problemen

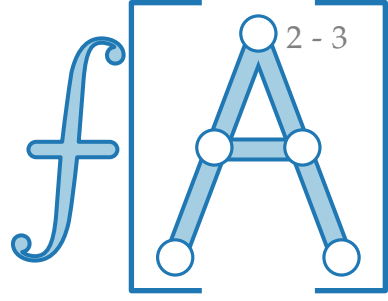
Viele wichtige (praktische) Probleme sind NP-schwer, zum Beispiel...



TSP

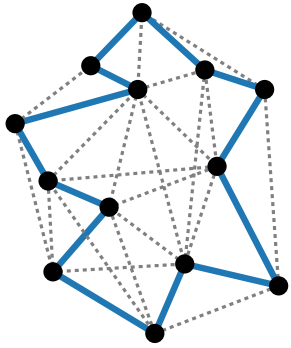


MIS

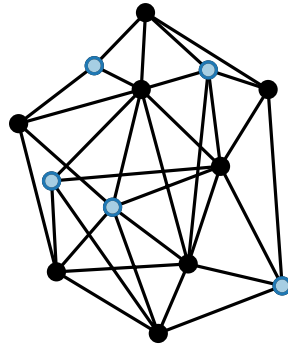


Beispiele von NP-schweren Problemen

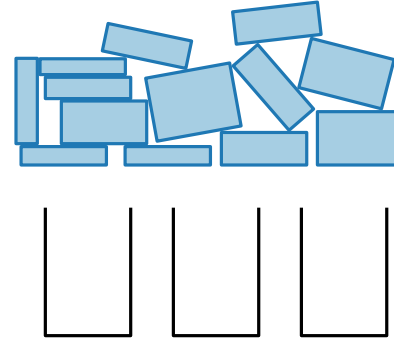
Viele wichtige (praktische) Probleme sind NP-schwer, zum Beispiel...



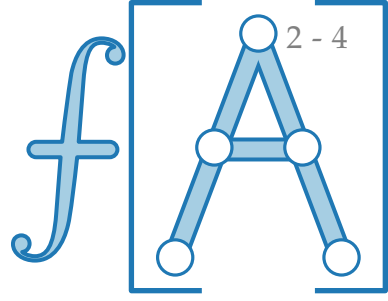
TSP



MIS

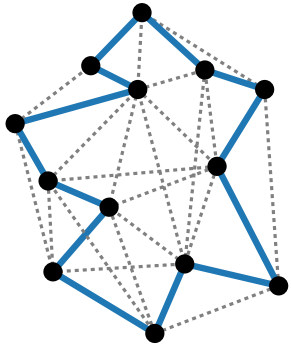
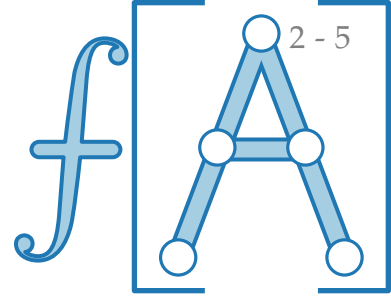


Bin Packing

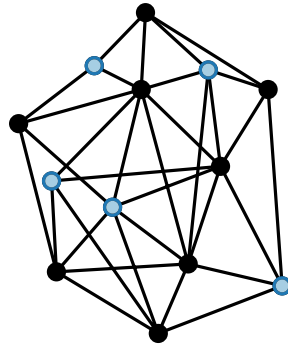


Beispiele von NP-schweren Problemen

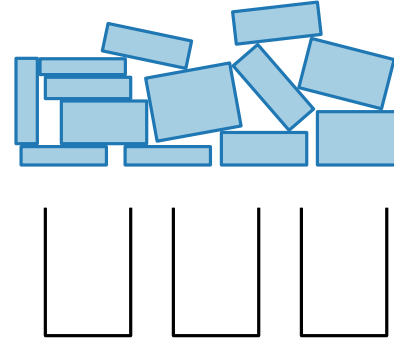
Viele wichtige (praktische) Probleme sind NP-schwer, zum Beispiel...



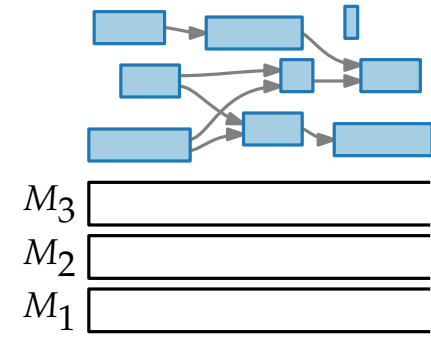
TSP



MIS

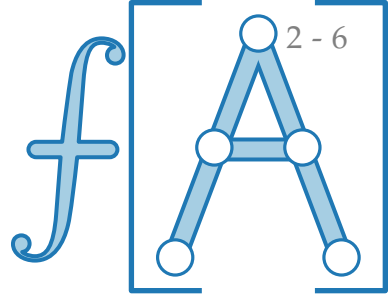


Bin Packing

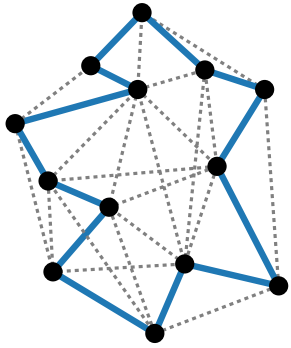


Ablaufplanung

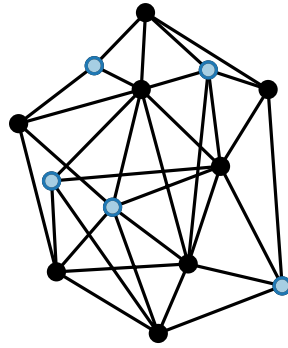
Beispiele von NP-schweren Problemen



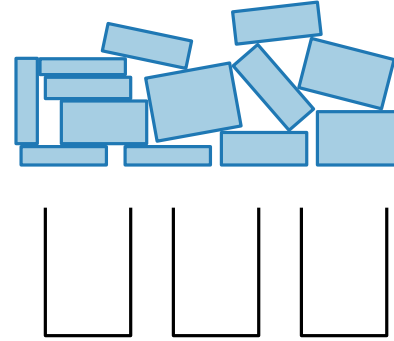
Viele wichtige (praktische) Probleme sind NP-schwer, zum Beispiel...



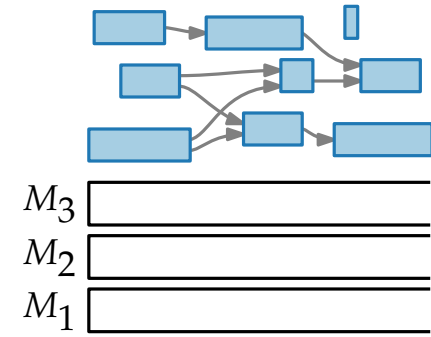
TSP



MIS



Bin Packing



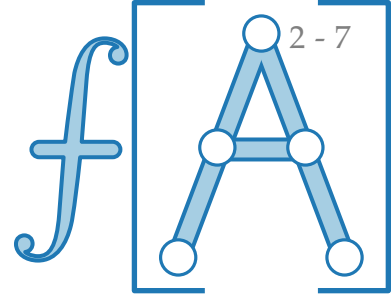
Ablaufplanung

$(x_1 \vee x_2 \vee \neg x_4) \wedge$
 $(\neg x_2 \vee x_3 \vee \neg x_4) \wedge$
 $(x_3 \vee x_7 \vee \neg x_8) \wedge$

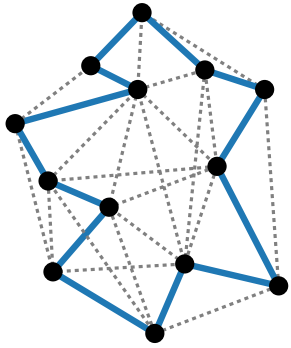
...

SAT

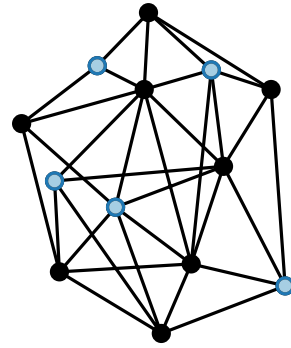
Beispiele von NP-schweren Problemen



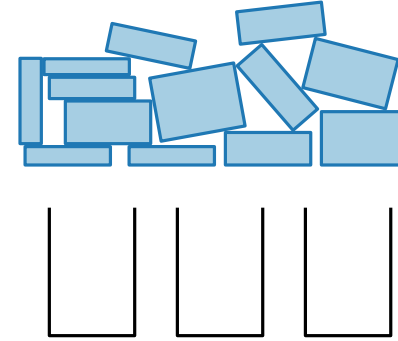
Viele wichtige (praktische) Probleme sind NP-schwer, zum Beispiel...



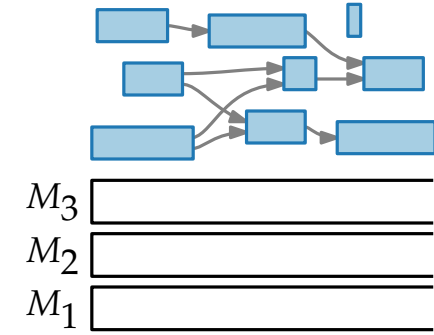
TSP



MIS



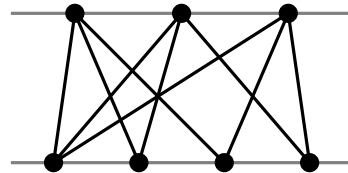
Bin Packing



Ablaufplanung

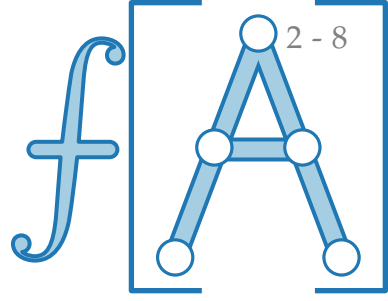
$(x_1 \vee x_2 \vee \neg x_4) \wedge$
 $(\neg x_2 \vee x_3 \vee \neg x_4) \wedge$
 $(x_3 \vee x_7 \vee \neg x_8) \wedge$
...

SAT

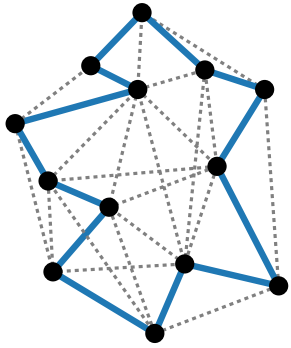


Graphenzeichnen

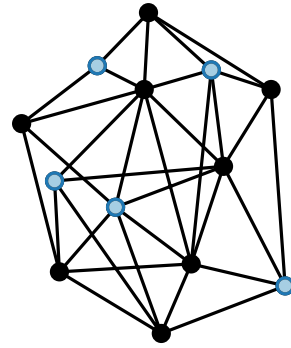
Beispiele von NP-schweren Problemen



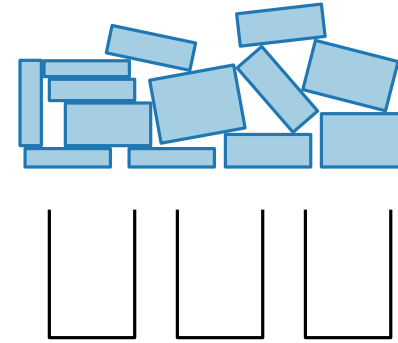
Viele wichtige (praktische) Probleme sind NP-schwer, zum Beispiel...



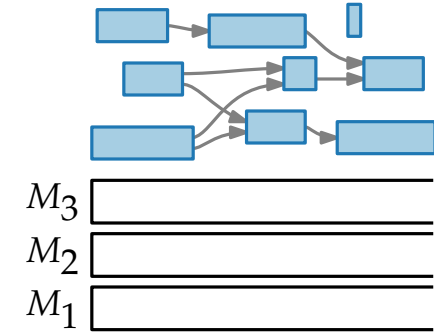
TSP



MIS



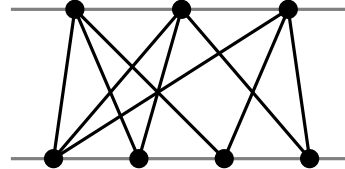
Bin Packing



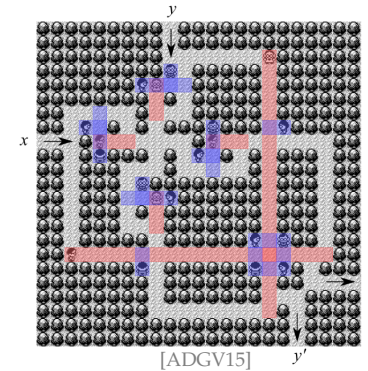
Ablaufplanung

$(x_1 \vee x_2 \vee \neg x_4) \wedge$
 $(\neg x_2 \vee x_3 \vee \neg x_4) \wedge$
 $(x_3 \vee x_7 \vee \neg x_8) \wedge$
...

SAT

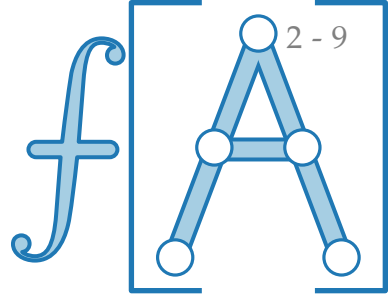


Graphenzeichnen

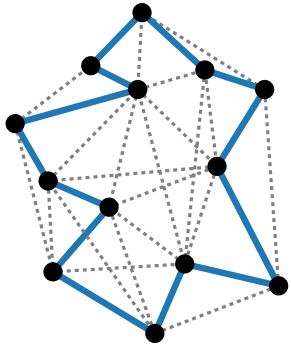


Spiele

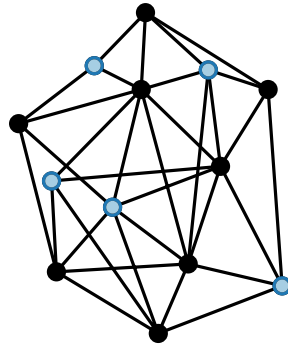
Beispiele von NP-schweren Problemen



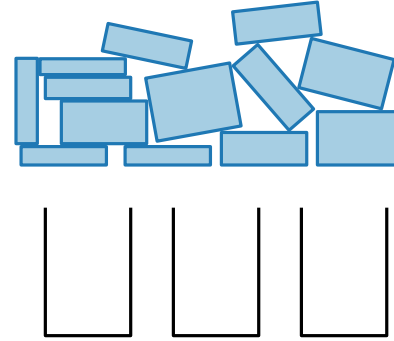
Viele wichtige (praktische) Probleme sind NP-schwer, zum Beispiel...



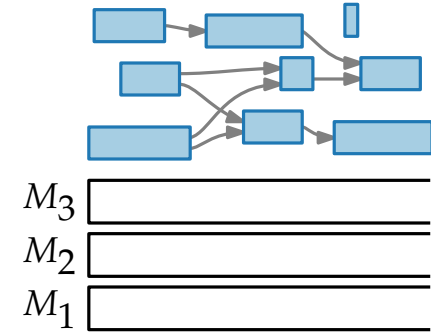
TSP



MIS



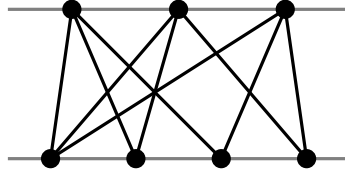
Bin Packing



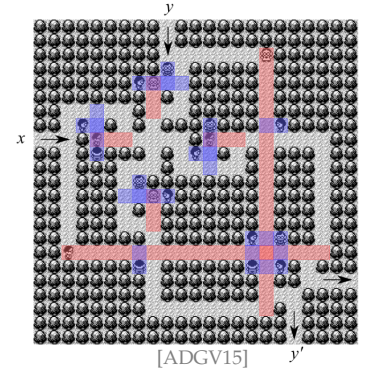
Ablaufplanung

$(x_1 \vee x_2 \vee \neg x_4) \wedge$
 $(\neg x_2 \vee x_3 \vee \neg x_4) \wedge$
 $(x_3 \vee x_7 \vee \neg x_8) \wedge$
...

SAT



Graphenzeichnen

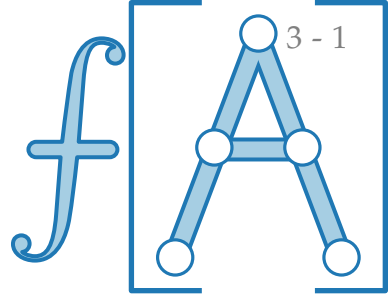


Spiele

...

Formale Sicht auf NP-Schwere

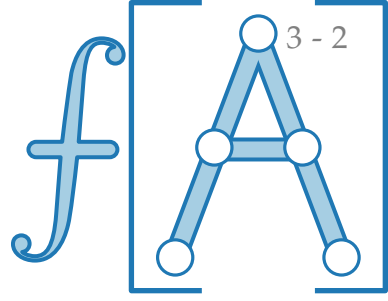
Aber was bedeutet NP-schwer/-vollständig eigentlich?



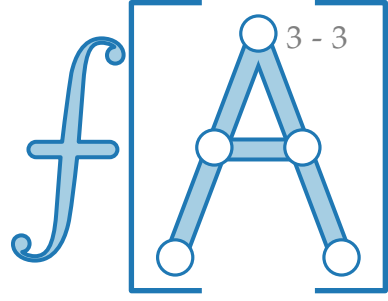
Formale Sicht auf NP-Schwere

Aber was bedeutet NP-schwer/-vollständig eigentlich?

- NP-schwer = nichtdeterministisch Polynomialzeit schwer



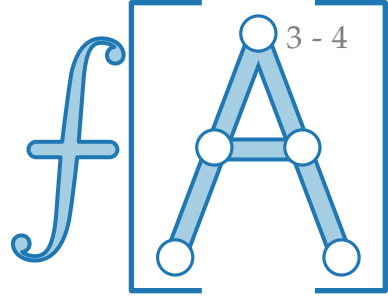
Formale Sicht auf NP-Schwere



Aber was bedeutet NP-schwer/-vollständig eigentlich?

- NP-schwer = nichtdeterministisch Polynomialzeit schwer
- Ein Entscheidungsproblem H ist NP-schwer, wenn es „mindestens so schwierig ist wie das schwierigste Problem in NP“.

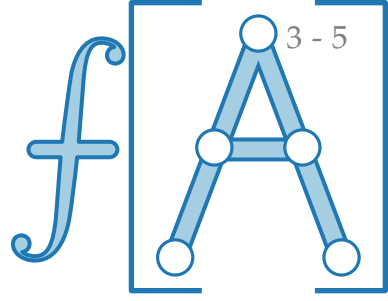
Formale Sicht auf NP-Schwere



Aber was bedeutet NP-schwer/-vollständig eigentlich?

- NP-schwer = nichtdeterministisch Polynomialzeit schwer
- Ein Entscheidungsproblem H ist NP-schwer, wenn es „mindestens so schwierig ist wie das schwierigste Problem in NP“.
- oder: Es gibt eine Polynomialzeitreduktion von einem NP-schweren Problem L auf H .

Formale Sicht auf NP-Schwere

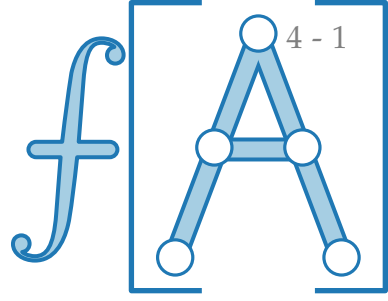


Aber was bedeutet NP-schwer/-vollständig eigentlich?

- NP-schwer = nichtdeterministisch Polynomialzeit schwer
- Ein Entscheidungsproblem H ist NP-schwer, wenn es „mindestens so schwierig ist wie das schwierigste Problem in NP“.
- oder: Es gibt eine Polynomialzeitreduktion von einem NP-schweren Problem L auf H .
- Wenn $P \neq NP$, dann können NP-schwere Probleme nicht in (deterministischer) Polynomialzeit gelöst werden.

Missverständnisse über NP-Schwere

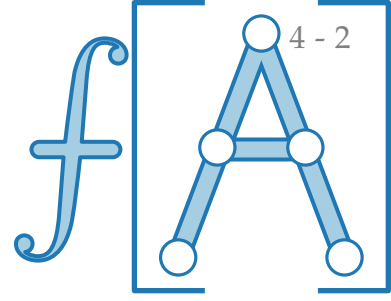
Häufige Missverständnisse [Mann '17]



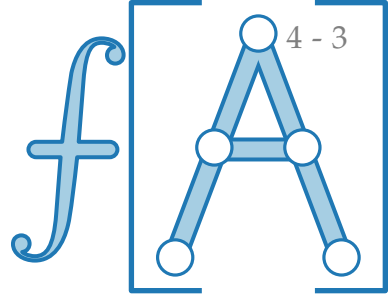
Missverständnisse über NP-Schwere

Häufige Missverständnisse [Mann '17]

- Wenn ein ähnliches Probleme NP-schwer ist, dann ist das betrachtete Problem auch NP-schwer.



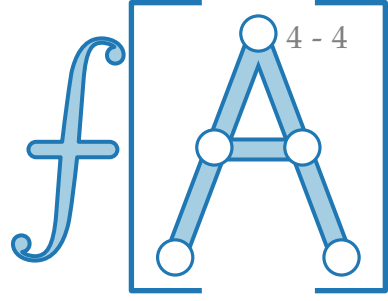
Missverständnisse über NP-Schwere



Häufige Missverständnisse [Mann '17]

- Wenn ein ähnliches Probleme NP-schwer ist, dann ist das betrachtete Problem auch NP-schwer.
- Probleme, die in der Praxis schwer zu lösen sind, sind NP-schwer.

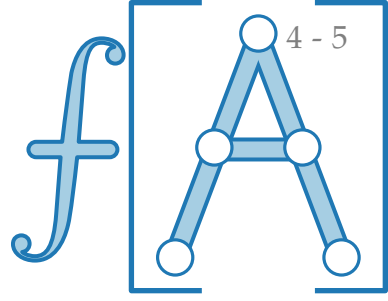
Missverständnisse über NP-Schwere



Häufige Missverständnisse [Mann '17]

- Wenn ein ähnliches Probleme NP-schwer ist, dann ist das betrachtete Problem auch NP-schwer.
- Probleme, die in der Praxis schwer zu lösen sind, sind NP-schwer.
- NP-schwere Probleme können nicht optimal gelöst werden.

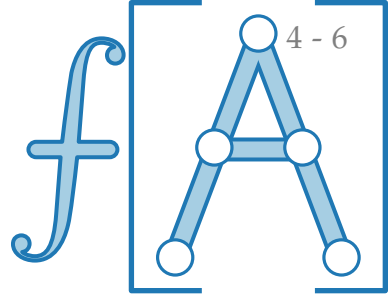
Missverständnisse über NP-Schwere



Häufige Missverständnisse [Mann '17]

- Wenn ein ähnliches Probleme NP-schwer ist, dann ist das betrachtete Problem auch NP-schwer.
- Probleme, die in der Praxis schwer zu lösen sind, sind NP-schwer.
- NP-schwere Probleme können nicht optimal gelöst werden.
- NP-schwere Probleme können nicht effizienter als durch erschöpfende Suche gelöst werden.

Missverständnisse über NP-Schwere

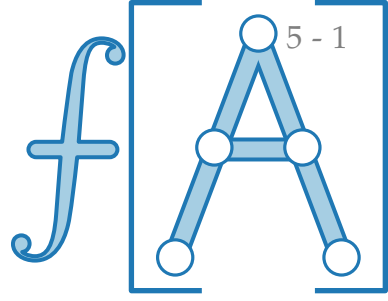


Häufige Missverständnisse [Mann '17]

- Wenn ein ähnliches Probleme NP-schwer ist, dann ist das betrachtete Problem auch NP-schwer.
- Probleme, die in der Praxis schwer zu lösen sind, sind NP-schwer.
- NP-schwere Probleme können nicht optimal gelöst werden.
- NP-schwere Probleme können nicht effizienter als durch erschöpfende Suche gelöst werden.
- Die einzigen praktisch sinnvollen Lösungen von NP-schweren Problemen sind Heuristiken.

NP-schwere Probleme lösen

Was können wir tun?

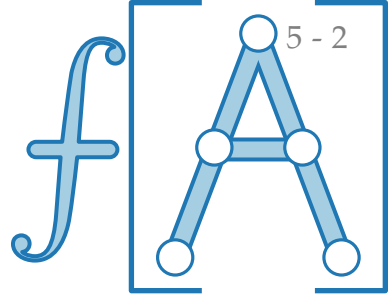


NP-schwer

NP-schwere Probleme lösen

Was können wir tun?

- Optimalität für Geschwindigkeit opfern

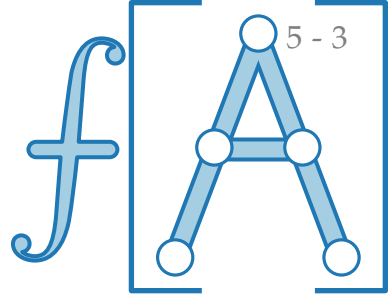


NP-schwer

NP-schwere Probleme lösen

Was können wir tun?

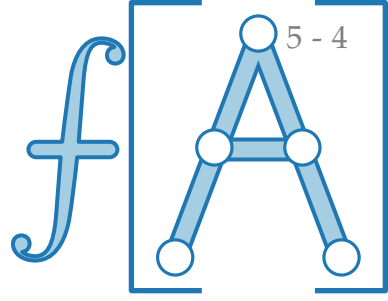
- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)



Heuristik

NP-schwer

NP-schwere Probleme lösen

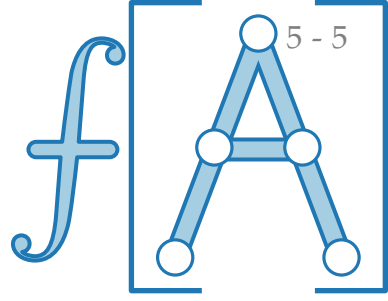


Was können wir tun?

- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)
 - Approximationsalgorithmen



NP-schwere Probleme lösen

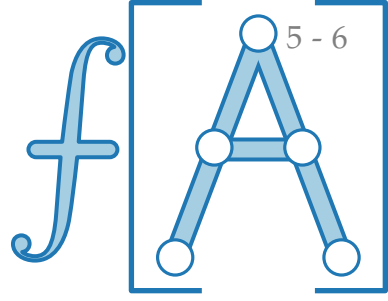


Was können wir tun?

- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)
 - Approximationsalgorithmen
- Optimale Lösungen

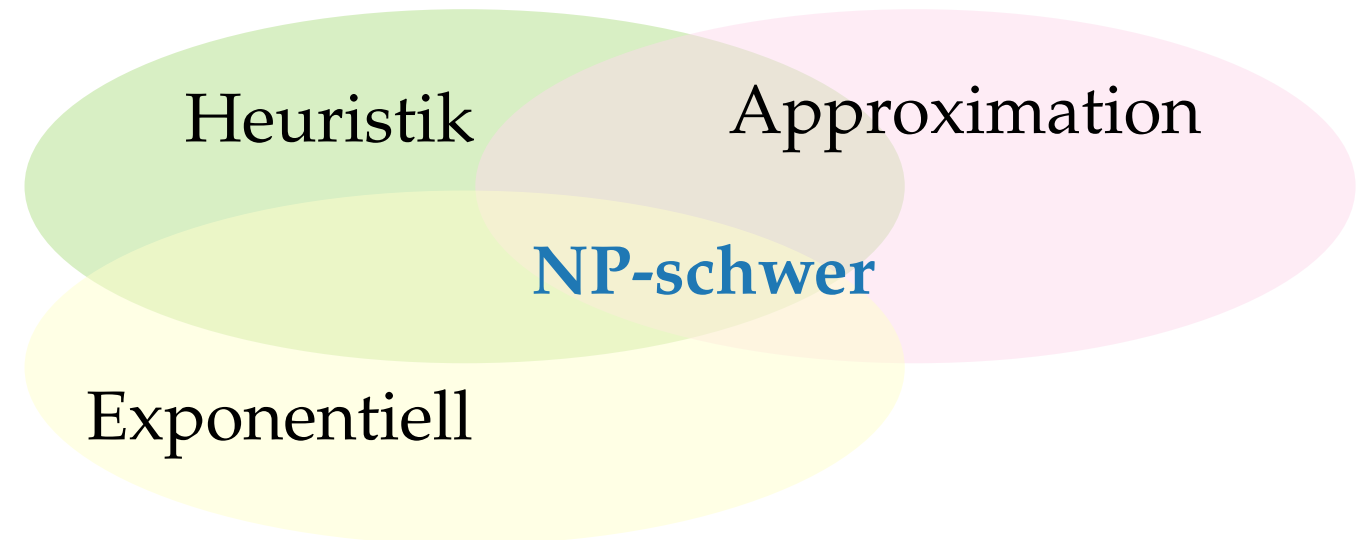


NP-schwere Probleme lösen

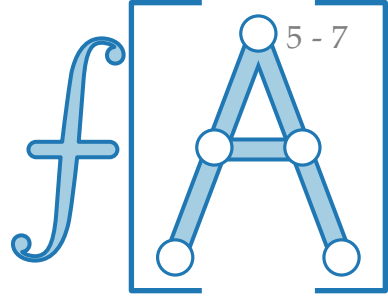


Was können wir tun?

- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)
 - Approximationsalgorithmen
- Optimale Lösungen
 - Exakte Exponentialzeitalgorithmen

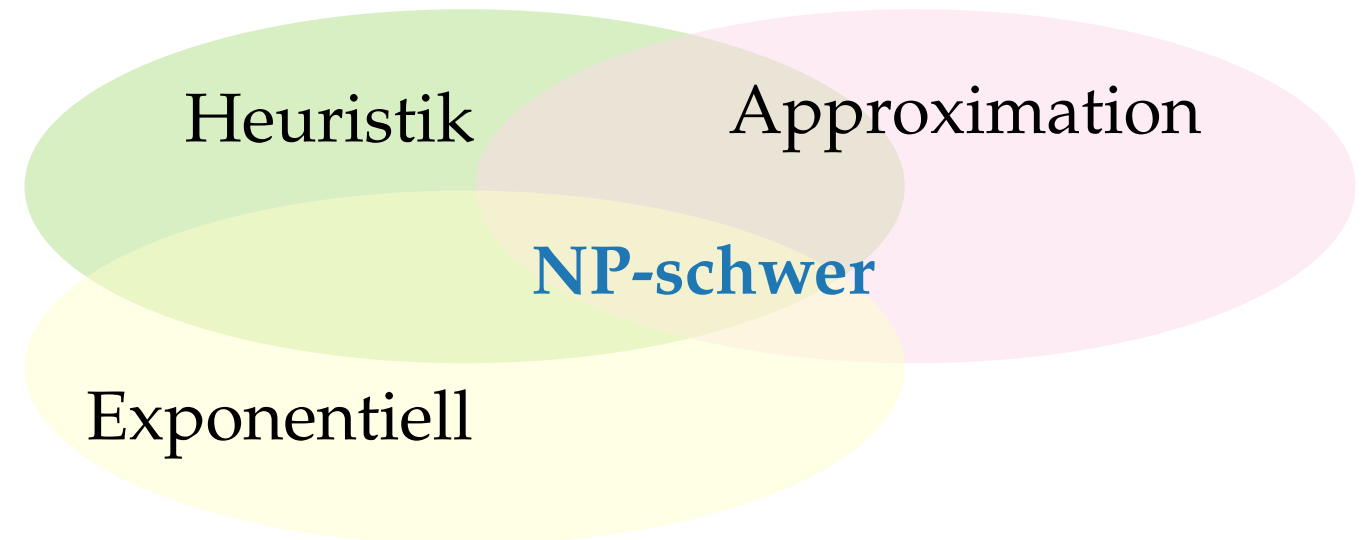


NP-schwere Probleme lösen

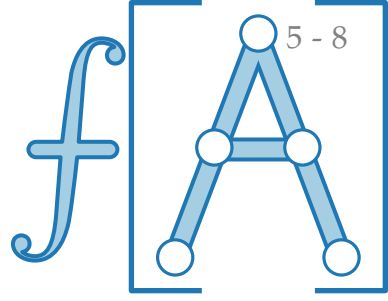


Was können wir tun?

- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)
 - Approximationsalgorithmen
- Optimale Lösungen
 - Exakte Exponentialzeitalgorithmen
 - feinkörnige Analyse

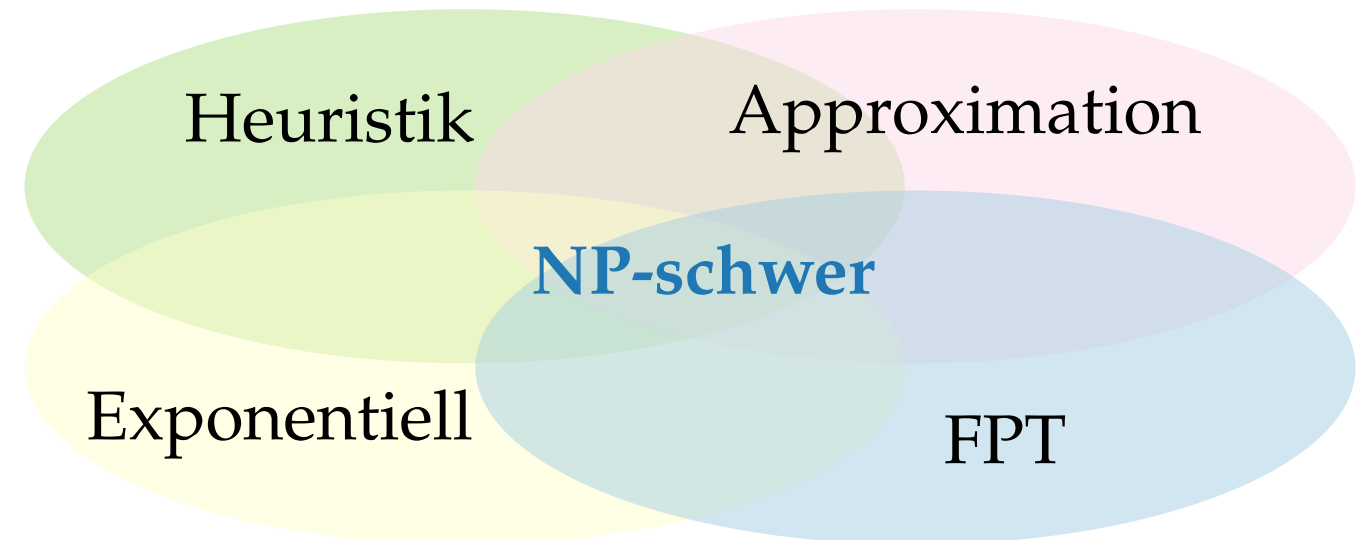


NP-schwere Probleme lösen

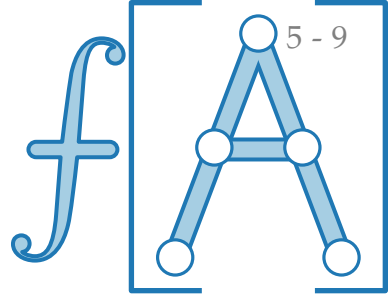


Was können wir tun?

- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)
 - Approximationsalgorithmen
- Optimale Lösungen
 - Exakte Exponentialzeitalgorithmen
 - feinkörnige Analyse
 - parametrisierte Algorithmen



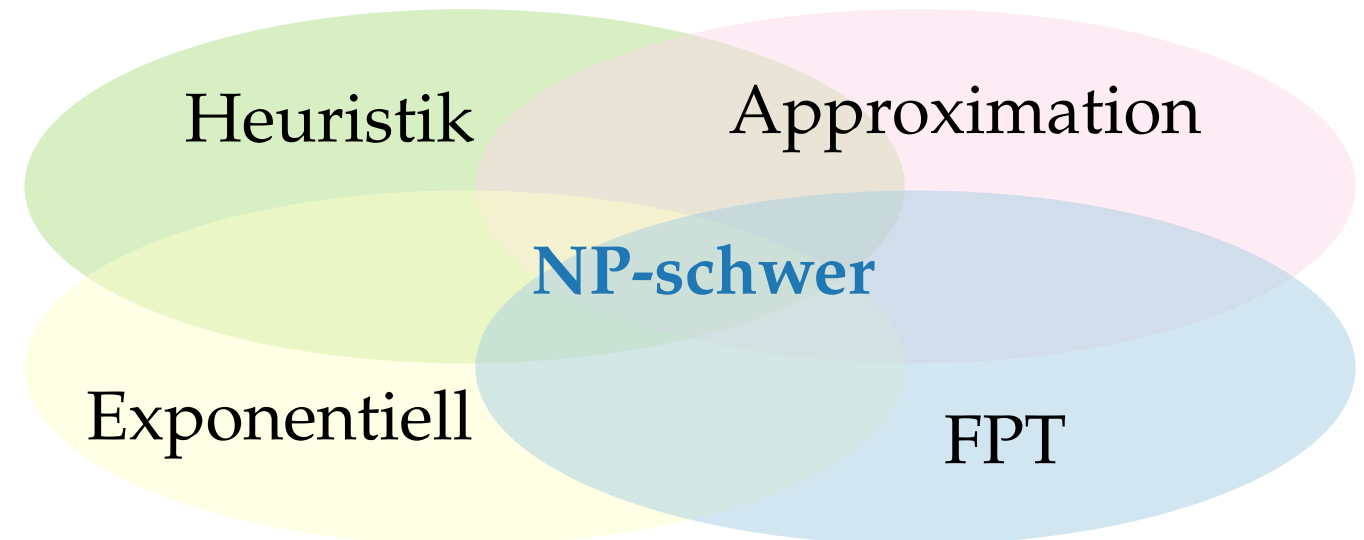
NP-schwere Probleme lösen



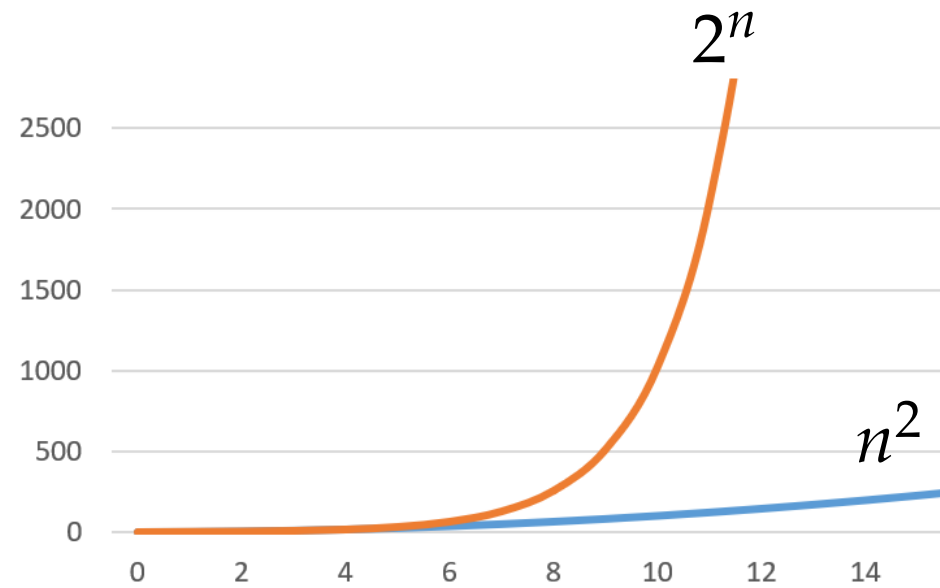
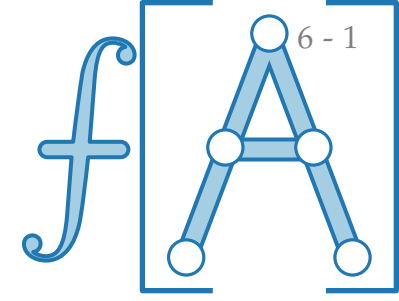
Was können wir tun?

- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)
 - Approximationsalgorithmen
- Optimale Lösungen
 - Exakte Exponentialzeitalgorithmen
 - feinkörnige Analyse
 - parametrisierte Algorithmen

heute

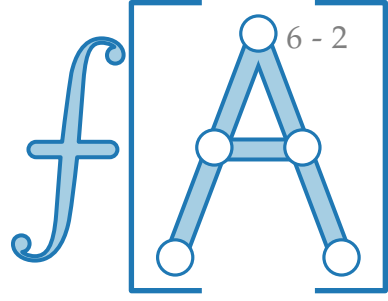


Motivation

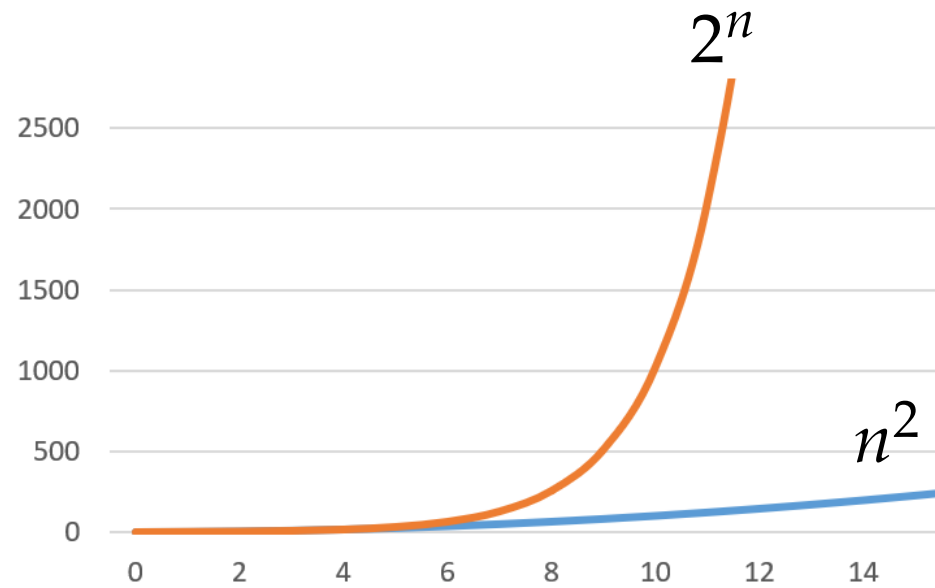


- effiziente vs. ineffiziente Algorithmen
- Polynomialzeit vs. Superpolynomialzeit

Motivation

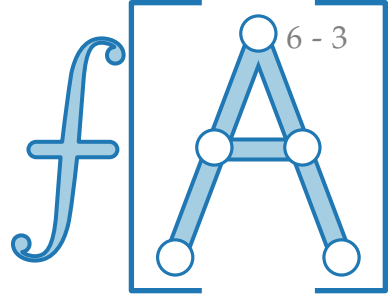


Exponentielle Laufzeit ... sollten wir einfach **aufgeben**?



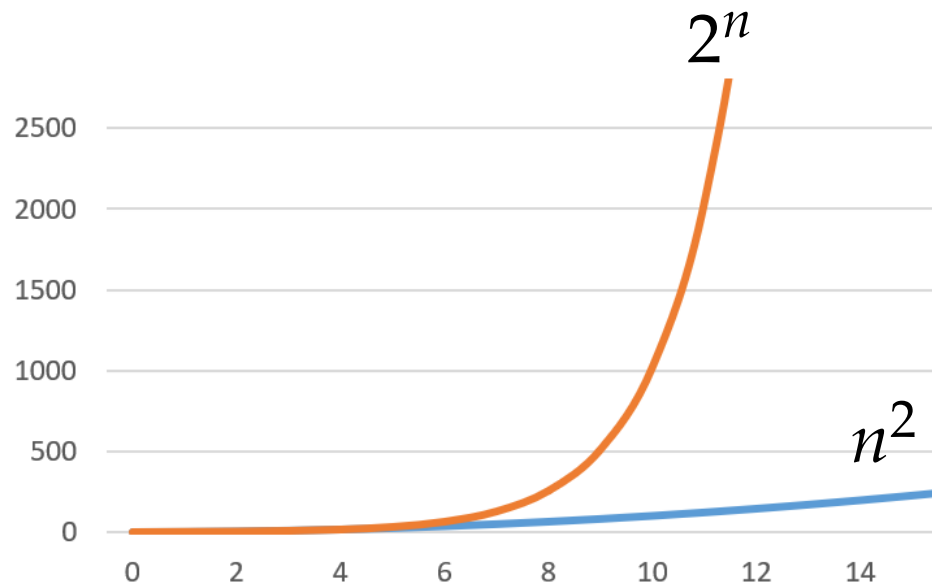
- effiziente vs. ineffiziente Algorithmen
- Polynomialzeit vs. Superpolynomialzeit

Motivation



Exponentielle Laufzeit ... sollten wir einfach **aufgeben**?

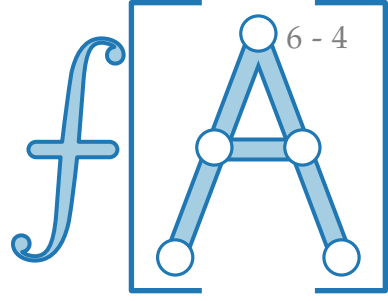
■ ... können „*schnell*“ für mittelgroße Instanzen sein:



■ effiziente vs.
ineffiziente Algorithmen

■ Polynomialzeit vs.
Superpolynomialzeit

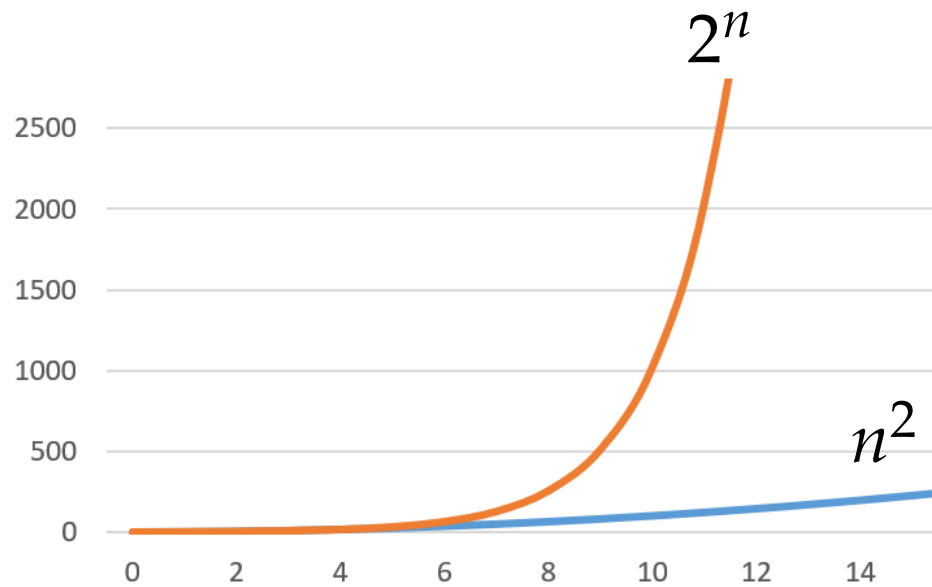
Motivation



Exponentielle Laufzeit ... sollten wir einfach **aufgeben**?

■ ... können „*schnell*“ für mittelgroße Instanzen sein:

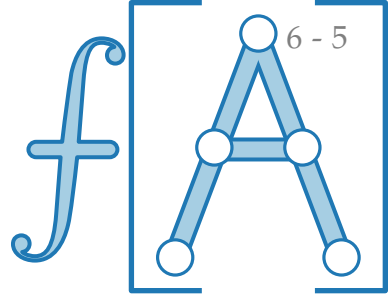
■ $n^4 > 1.2^n$ für $n \leq 100$



■ effiziente vs.
ineffiziente Algorithmen

■ Polynomialzeit vs.
Superpolynomialzeit

Motivation

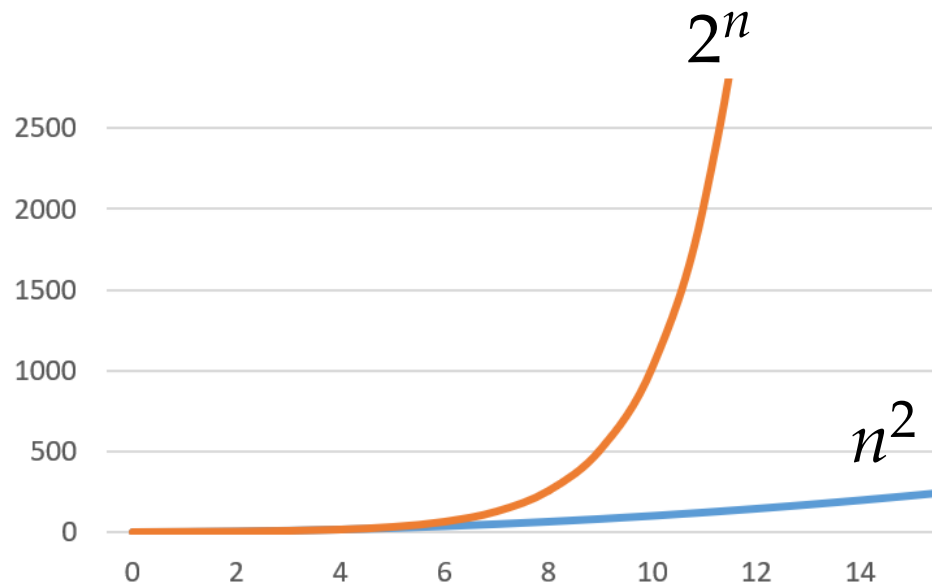


Exponentielle Laufzeit ... sollten wir einfach **aufgeben**?

■ ... können „*schnell*“ für mittelgroße Instanzen sein:

■ $n^4 > 1.2^n$ für $n \leq 100$

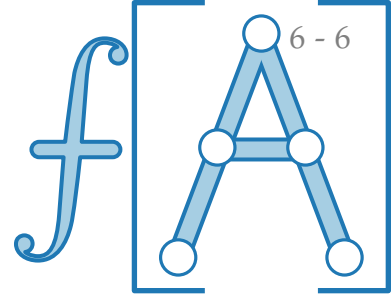
■ TSP exakt lösbar für $n \leq 2000$ und
für spezielle Instanzen mit $n \leq 85.900$



■ effiziente vs.
ineffiziente Algorithmen

■ Polynomialzeit vs.
Superpolynomialzeit

Motivation



Exponentielle Laufzeit ... sollten wir einfach **aufgeben**?

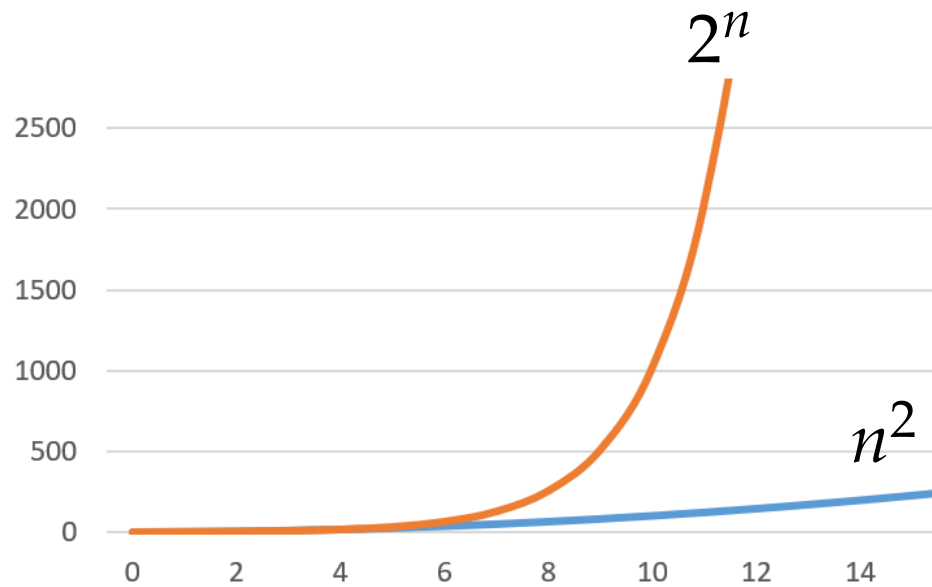
■ ... können „*schnell*“ für mittelgroße Instanzen sein:

■ $n^4 > 1.2^n$ für $n \leq 100$

■ TSP exakt lösbar für $n \leq 2000$ und für spezielle Instanzen mit $n \leq 85.900$

■ „versteckte“ Konstanten in Polynomialzeit Algorithmen:

$2^{100}n > 2^n$ für $n \leq 107$

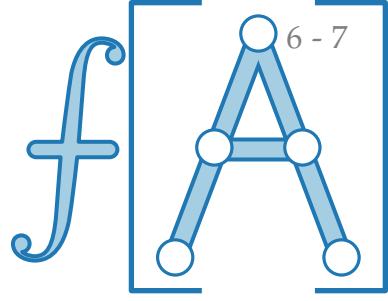


■ effiziente vs. ineffiziente Algorithmen

■ Polynomialzeit vs. Superpolynomialzeit

Motivation

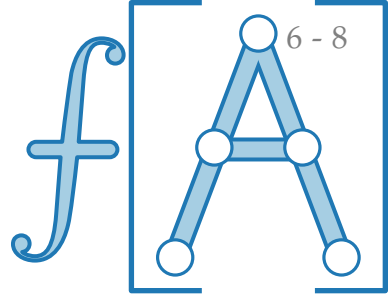
Exponentielle Laufzeit ... vielleicht brauchen wir nur **bessere Hardware?**



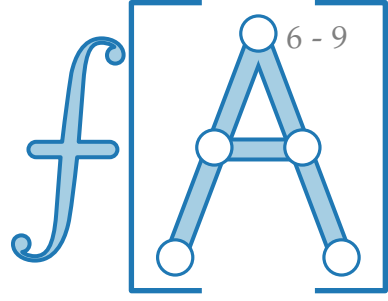
Motivation

Exponentielle Laufzeit ... vielleicht brauchen wir nur **bessere Hardware?**

- Angenommen, ein Algorithmus braucht a^n Schritte und kann Instanzen bis Größe n_0 in Zeit t lösen



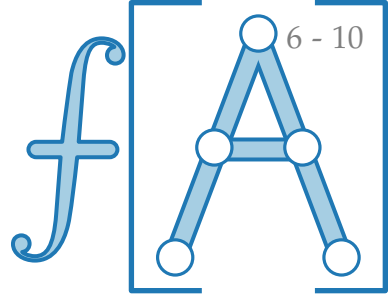
Motivation



Exponentielle Laufzeit ... vielleicht brauchen wir nur **bessere Hardware?**

- Angenommen, ein Algorithmus braucht a^n Schritte und kann Instanzen bis Größe n_0 in Zeit t lösen
- Verbesserung der Hardware durch einen konstanten Faktor c *addiert nur eine Konstante* (relativ zu c) zu n_0 :

Motivation

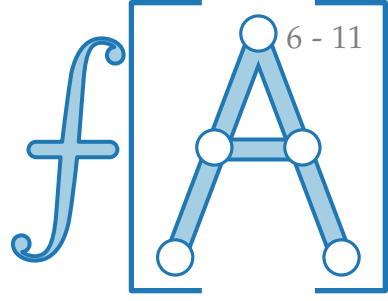


Exponentielle Laufzeit ... vielleicht brauchen wir nur **bessere Hardware?**

- Angenommen, ein Algorithmus braucht a^n Schritte und kann Instanzen bis Größe n_0 in Zeit t lösen
- Verbesserung der Hardware durch einen konstanten Faktor c *addiert nur eine Konstante* (relativ zu c) zu n_0 :

$$a^{n'_0} = c \cdot a^{n_0}$$

Motivation

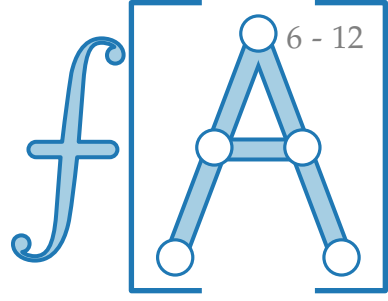


Exponentielle Laufzeit ... vielleicht brauchen wir nur **bessere Hardware**?

- Angenommen, ein Algorithmus braucht a^n Schritte und kann Instanzen bis Größe n_0 in Zeit t lösen
- Verbesserung der Hardware durch einen konstanten Faktor c *addiert nur eine Konstante* (relativ zu c) zu n_0 :

$$a^{n'_0} = c \cdot a^{n_0} \Rightarrow n'_0 = \log_a c + n_0$$

Motivation



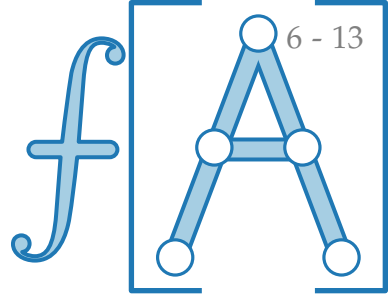
Exponentielle Laufzeit ... vielleicht brauchen wir nur **bessere Hardware?**

- Angenommen, ein Algorithmus braucht a^n Schritte und kann Instanzen bis Größe n_0 in Zeit t lösen
- Verbesserung der Hardware durch einen konstanten Faktor c *addiert nur eine Konstante* (relativ zu c) zu n_0 :

$$a^{n'_0} = c \cdot a^{n_0} \Rightarrow n'_0 = \log_a c + n_0$$

z.B. $\log_{1.5} 2 \approx 1.7$

Motivation



Exponentielle Laufzeit ... vielleicht brauchen wir nur **bessere Hardware?**

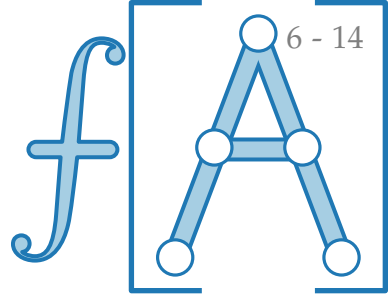
- Angenommen, ein Algorithmus braucht a^n Schritte und kann Instanzen bis Größe n_0 in Zeit t lösen
- Verbesserung der Hardware durch einen konstanten Faktor c *addiert nur eine Konstante* (relativ zu c) zu n_0 :

$$a^{n'_0} = c \cdot a^{n_0} \rightarrow n'_0 = \log_a c + n_0$$

z.B. $\log_{1.5} 2 \approx 1.7$

- Reduzieren des Basis der Laufzeit auf $b < a$ bedeutet eine *multiplikative* Erhöhung:

Motivation



Exponentielle Laufzeit ... vielleicht brauchen wir nur **bessere Hardware?**

- Angenommen, ein Algorithmus braucht a^n Schritte und kann Instanzen bis Größe n_0 in Zeit t lösen
- Verbesserung der Hardware durch einen konstanten Faktor c *addiert nur eine Konstante* (relativ zu c) zu n_0 :

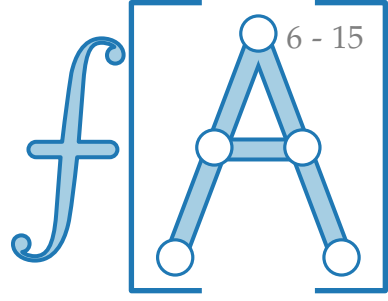
$$a^{n'_0} = c \cdot a^{n_0} \rightarrow n'_0 = \log_a c + n_0$$

z.B. $\log_{1.5} 2 \approx 1.7$

- Reduzieren des Basis der Laufzeit auf $b < a$ bedeutet eine *multiplikative* Erhöhung:

$$b^{n'_0} = a^{n_0}$$

Motivation



Exponentielle Laufzeit ... vielleicht brauchen wir nur **bessere Hardware?**

- Angenommen, ein Algorithmus braucht a^n Schritte und kann Instanzen bis Größe n_0 in Zeit t lösen
- Verbesserung der Hardware durch einen konstanten Faktor c *addiert nur eine Konstante* (relativ zu c) zu n_0 :

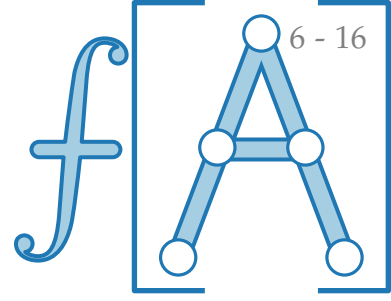
$$a^{n'_0} = c \cdot a^{n_0} \rightarrow n'_0 = \log_a c + n_0$$

z.B. $\log_{1.5} 2 \approx 1.7$

- Reduzieren der Basis der Laufzeit auf $b < a$ bedeutet eine *multiplikative* Erhöhung:

$$b^{n'_0} = a^{n_0} \rightarrow n'_0 = n_0 \cdot \log_b a$$

Motivation



Exponentielle Laufzeit ... vielleicht brauchen wir nur **bessere Hardware?**

- Angenommen, ein Algorithmus braucht a^n Schritte und kann Instanzen bis Größe n_0 in Zeit t lösen
- Verbesserung der Hardware durch einen konstanten Faktor c *addiert nur eine Konstante* (relativ zu c) zu n_0 :

$$a^{n'_0} = c \cdot a^{n_0} \rightarrow n'_0 = \log_a c + n_0$$

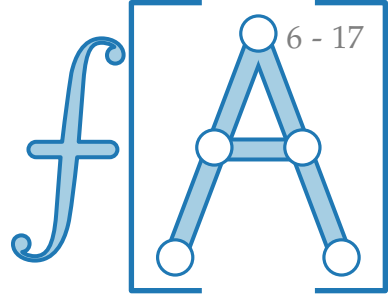
z.B. $\log_{1.5} 2 \approx 1.7$

- Reduzieren des Basis der Laufzeit auf $b < a$ bedeutet eine *multiplikative* Erhöhung:

$$b^{n'_0} = a^{n_0} \rightarrow n'_0 = n_0 \cdot \log_b a$$

z.B. $\log_{1.4} 1.5 \approx 1.2$

Motivation



Exponentielle Laufzeit ... vielleicht brauchen wir nur **bessere Hardware?**

- Angenommen, ein Algorithmus braucht a^n Schritte und kann Instanzen bis Größe n_0 in Zeit t lösen
- Verbesserung der Hardware durch einen konstanten Faktor c *addiert nur eine Konstante* (relativ zu c) zu n_0 :

$$a^{n'_0} = c \cdot a^{n_0} \rightarrow n'_0 = \log_a c + n_0$$

z.B. $\log_{1.5} 2 \approx 1.7$

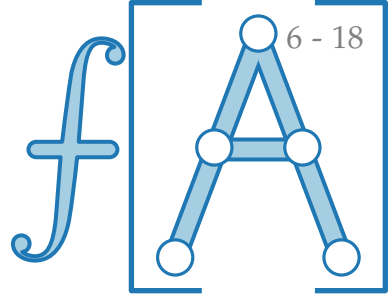
- Reduzieren des Basis der Laufzeit auf $b < a$ bedeutet eine *multiplikative* Erhöhung:

$$b^{n'_0} = a^{n_0} \rightarrow n'_0 = n_0 \cdot \log_b a$$

z.B. $\log_{1.4} 1.5 \approx 1.2$

- $\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$

Motivation



Exponentielle Laufzeit ... vielleicht brauchen wir nur **bessere Hardware?**

- Angenommen, ein Algorithmus braucht a^n Schritte und kann Instanzen bis Größe n_0 in Zeit t lösen
- Verbesserung der Hardware durch einen konstanten Faktor c *addiert nur eine Konstante* (relativ zu c) zu n_0 :

$$a^{n'_0} = c \cdot a^{n_0} \rightarrow n'_0 = \log_a c + n_0$$

z.B. $\log_{1.5} 2 \approx 1.7$

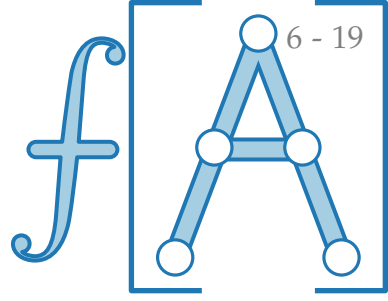
- Reduzieren des Basis der Laufzeit auf $b < a$ bedeutet eine *multiplikative* Erhöhung:

$$b^{n'_0} = a^{n_0} \rightarrow n'_0 = n_0 \cdot \log_b a$$

z.B. $\log_{1.4} 1.5 \approx 1.2$

- $\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$
- Polynomielle Faktoren vernachlässigbar

Motivation



Exponentielle Laufzeit ... vielleicht brauchen wir nur **bessere Hardware?**

- Angenommen, ein Algorithmus braucht a^n Schritte und kann Instanzen bis Größe n_0 in Zeit t lösen
- Verbesserung der Hardware durch einen konstanten Faktor c *addiert nur eine Konstante* (relativ zu c) zu n_0 :

$$a^{n'_0} = c \cdot a^{n_0} \rightarrow n'_0 = \log_a c + n_0$$

z.B. $\log_{1.5} 2 \approx 1.7$

- Reduzieren des Basis der Laufzeit auf $b < a$ bedeutet eine *multiplikative* Erhöhung:

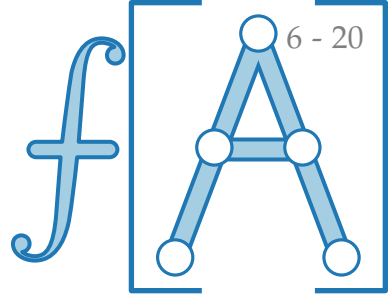
$$b^{n'_0} = a^{n_0} \rightarrow n'_0 = n_0 \cdot \log_b a$$

z.B. $\log_{1.4} 1.5 \approx 1.2$

- $\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$
- Polynomielle Faktoren vernachlässigbar
- Basis der exponentiellen Faktoren dominieren

Motivation

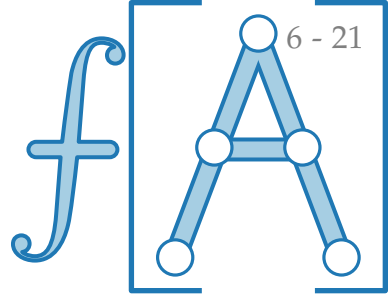
Exponentielle Laufzeit ... aber können wir wenigstens exakte Algorithmen finden, die schneller als **brute-force** Ansätze sind?



Motivation

Exponentielle Laufzeit ... aber können wir wenigstens exakte Algorithmen finden, die schneller als **brute-force** Ansätze sind?

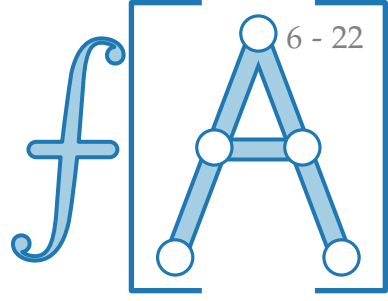
- TSP: Bellman-Held-Karp Algorithmus hat Laufzeit $\mathcal{O}(2^n n^2)$, verglichen mit einer $\mathcal{O}(n!n)$ -Zeit brute-force Suche.



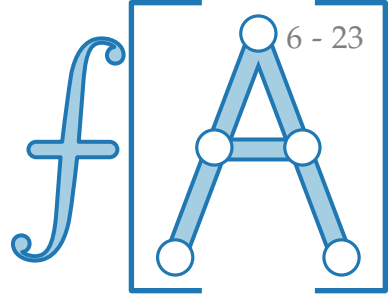
Motivation

Exponentielle Laufzeit ... aber können wir wenigstens exakte Algorithmen finden, die schneller als **brute-force** Ansätze sind?

- TSP: Bellman-Held-Karp Algorithmus hat Laufzeit $\mathcal{O}(2^n n^2)$, verglichen mit einer $\mathcal{O}(n!n)$ -Zeit brute-force Suche.
- MIS: Algorithmus von Tarjan & Trojanowski hat Laufzeit $\mathcal{O}(2^{n/3})$, verglichen mit einem trivialen $\mathcal{O}(n2^n)$ -Zeit Ansatz.



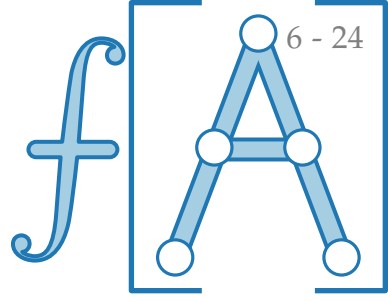
Motivation



Exponentielle Laufzeit ... aber können wir wenigstens exakte Algorithmen finden, die schneller als **brute-force** Ansätze sind?

- TSP: Bellman-Held-Karp Algorithmus hat Laufzeit $\mathcal{O}(2^n n^2)$, verglichen mit einer $\mathcal{O}(n!n)$ -Zeit brute-force Suche.
- MIS: Algorithmus von Tarjan & Trojanowski hat Laufzeit $\mathcal{O}(2^{n/3})$, verglichen mit einem trivialen $\mathcal{O}(n2^n)$ -Zeit Ansatz.
- COLORING: Lawlers Algorithmus hat Laufzeit $\mathcal{O}(n(1 + \sqrt[3]{3})^n)$, verglichen mit einer $\mathcal{O}(n^{n+1})$ -Zeit brute-force Suche.

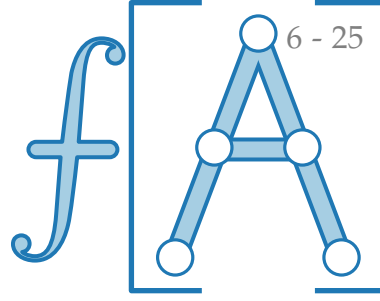
Motivation



Exponentielle Laufzeit ... aber können wir wenigstens exakte Algorithmen finden, die schneller als **brute-force** Ansätze sind?

- TSP: Bellman-Held-Karp Algorithmus hat Laufzeit $\mathcal{O}(2^n n^2)$, verglichen mit einer $\mathcal{O}(n!n)$ -Zeit brute-force Suche.
- MIS: Algorithmus von Tarjan & Trojanowski hat Laufzeit $\mathcal{O}(2^{n/3})$, verglichen mit einem trivialen $\mathcal{O}(n2^n)$ -Zeit Ansatz.
- COLORING: Lawlers Algorithmus hat Laufzeit $\mathcal{O}(n(1 + \sqrt[3]{3})^n)$, verglichen mit einer $\mathcal{O}(n^{n+1})$ -Zeit brute-force Suche.
- SAT: Es ist kein besserer Algorithmus als triviale brute-force Suche bekannt.

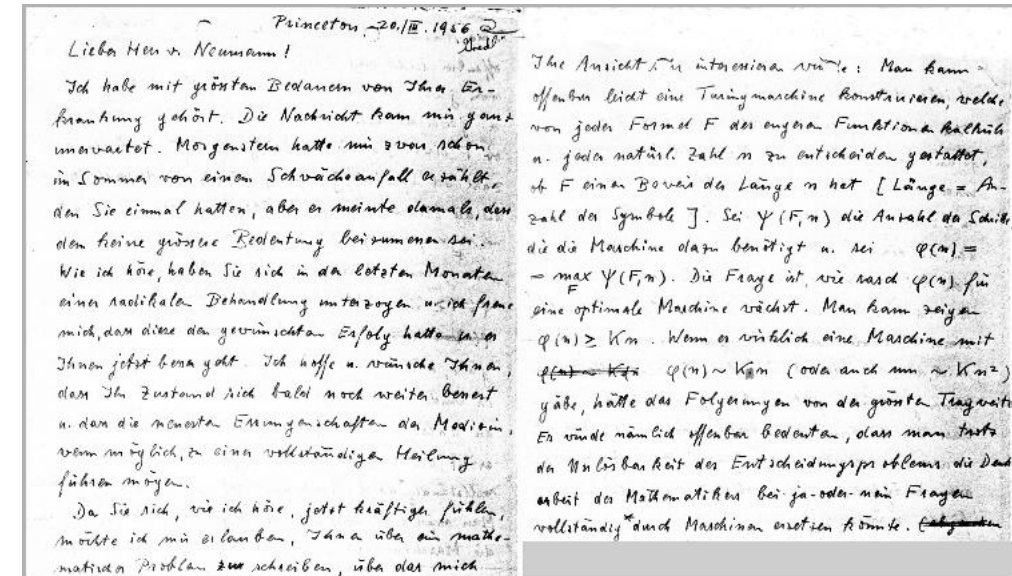
Motivation



Exponentielle Laufzeit ... aber können wir wenigstens exakte Algorithmen finden, die schneller als **brute-force** Ansätze sind?

- TSP: Bellman-Held-Karp Algorithmus hat Laufzeit $\mathcal{O}(2^n n^2)$, verglichen mit einer $\mathcal{O}(n!n)$ -Zeit brute-force Suche.
- MIS: Algorithmus von Tarjan & Trojanowski hat Laufzeit $\mathcal{O}(2^{n/3})$, verglichen mit einem trivialen $\mathcal{O}(n2^n)$ -Zeit Ansatz.
- COLORING: Lawlers Algorithmus hat Laufzeit $\mathcal{O}(n(1 + \sqrt[3]{3})^n)$, verglichen mit einer $\mathcal{O}(n^{n+1})$ -Zeit brute-force Suche.
- SAT: Es ist kein besserer Algorithmus als triviale brute-force Suche bekannt.
- „Es wäre interessant zu wissen, ... wie stark im allgemeinen bei finiten kombinatorischen Problemen die Anzahl der Schritte gegenüber dem blossen Probieren verringert werden kann.“
[Gödel's berühmter Brief zu von Neumann, 1956]

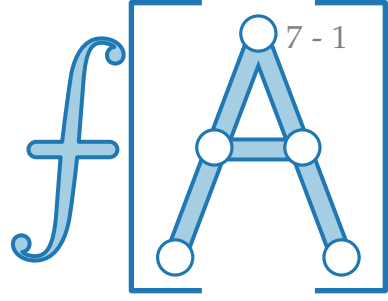
Kurt Gödel
*1906 Brünn, Österreich-Ungarn
†1978 Princeton, NJ



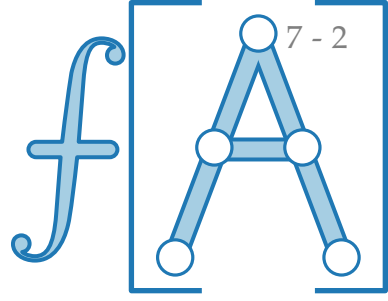
$\mathcal{O}^*$ -Notation

$$\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$$

- Polynomielle Faktoren vernachlässigbar
- Basis der exponentiellen Faktoren dominieren



$\mathcal{O}^*$ -Notation

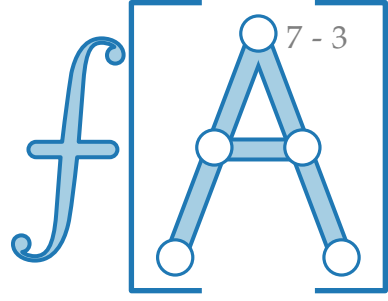


$$\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$$

- Polynomielle Faktoren vernachlässigbar
- Basis der exponentiellen Faktoren dominieren

Def. $f(n) \in \mathcal{O}^*(g(n))$

$\mathcal{O}^*$ -Notation

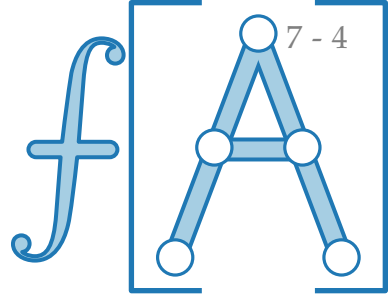


$$\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$$

- Polynomielle Faktoren vernachlässigbar
- Basis der exponentiellen Faktoren dominieren

Def. $f(n) \in \mathcal{O}^*(g(n))$
 $\Leftrightarrow \exists$ Polynom $p(n)$

$\mathcal{O}^*$ -Notation

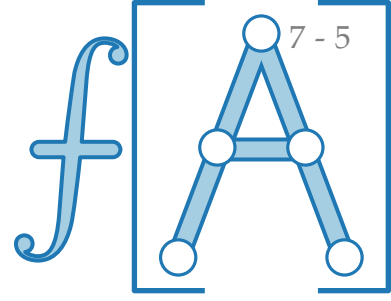


$$\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$$

- Polynomielle Faktoren vernachlässigbar
- Basis der exponentiellen Faktoren dominieren

Def. $f(n) \in \mathcal{O}^*(g(n))$
 $\Leftrightarrow \exists$ Polynom $p(n)$ mit $f(n) \in \mathcal{O}(g(n) \cdot p(n))$

$\mathcal{O}^*$ -Notation



$$\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$$

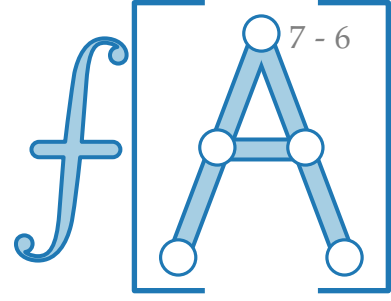
- Polynomielle Faktoren vernachlässigbar
- Basis der exponentiellen Faktoren dominieren

Def. $f(n) \in \mathcal{O}^*(g(n))$
 $\Leftrightarrow \exists$ Polynom $p(n)$ mit $f(n) \in \mathcal{O}(g(n) \cdot p(n))$

■ Typische Resultate

Ansatz	Laufzeit in $\mathcal{O}$ -Notation	$\mathcal{O}^*$ -Notation
Brute-Force		
Algorithmus A		
Algorithmus B		

$\mathcal{O}^*$ -Notation



$$\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$$

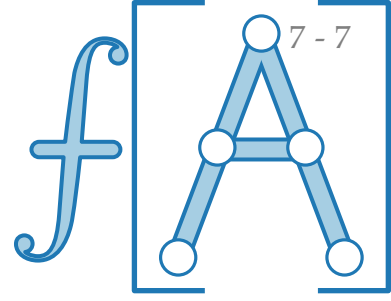
- Polynomielle Faktoren vernachlässigbar
- Basis der exponentiellen Faktoren dominieren

Def. $f(n) \in \mathcal{O}^*(g(n))$
 $\Leftrightarrow \exists$ Polynom $p(n)$ mit $f(n) \in \mathcal{O}(g(n) \cdot p(n))$

■ Typische Resultate

Ansatz	Laufzeit in $\mathcal{O}$ -Notation	$\mathcal{O}^*$ -Notation
Brute-Force	$\mathcal{O}(2^n)$	
Algorithmus A		
Algorithmus B		

$\mathcal{O}^*$ -Notation



$$\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$$

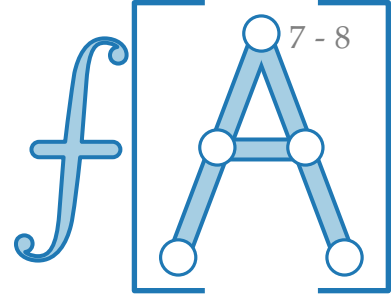
- Polynomielle Faktoren vernachlässigbar
- Basis der exponentiellen Faktoren dominieren

Def. $f(n) \in \mathcal{O}^*(g(n))$
 $\Leftrightarrow \exists$ Polynom $p(n)$ mit $f(n) \in \mathcal{O}(g(n) \cdot p(n))$

■ Typische Resultate

Ansatz	Laufzeit in $\mathcal{O}$ -Notation	$\mathcal{O}^*$ -Notation
Brute-Force	$\mathcal{O}(2^n)$	$\mathcal{O}^*(2^n)$
Algorithmus A		
Algorithmus B		

$\mathcal{O}^*$ -Notation



$$\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$$

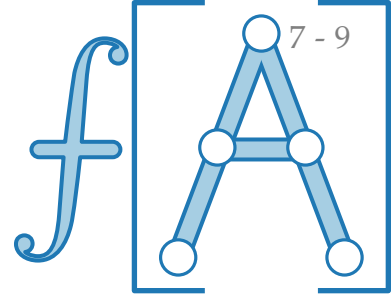
- Polynomielle Faktoren vernachlässigbar
- Basis der exponentiellen Faktoren dominieren

Def. $f(n) \in \mathcal{O}^*(g(n))$
 $\Leftrightarrow \exists$ Polynom $p(n)$ mit $f(n) \in \mathcal{O}(g(n) \cdot p(n))$

■ Typische Resultate

Ansatz	Laufzeit in $\mathcal{O}$ -Notation	$\mathcal{O}^*$ -Notation
Brute-Force	$\mathcal{O}(2^n)$	$\mathcal{O}^*(2^n)$
Algorithmus A	$\mathcal{O}(1.5^n \cdot n^2)$	
Algorithmus B		

$\mathcal{O}^*$ -Notation



$$\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$$

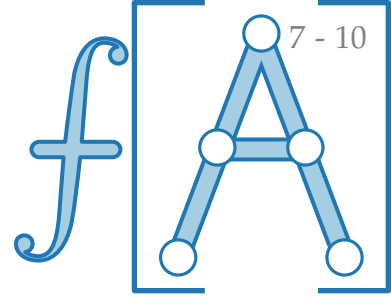
- Polynomielle Faktoren vernachlässigbar
- Basis der exponentiellen Faktoren dominieren

Def. $f(n) \in \mathcal{O}^*(g(n))$
 $\Leftrightarrow \exists$ Polynom $p(n)$ mit $f(n) \in \mathcal{O}(g(n) \cdot p(n))$

■ Typische Resultate

Ansatz	Laufzeit in $\mathcal{O}$ -Notation	$\mathcal{O}^*$ -Notation
Brute-Force	$\mathcal{O}(2^n)$	$\mathcal{O}^*(2^n)$
Algorithmus A	$\mathcal{O}(1.5^n \cdot n^2)$	$\mathcal{O}^*(1.5^n)$
Algorithmus B		

$\mathcal{O}^*$ -Notation



$$\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$$

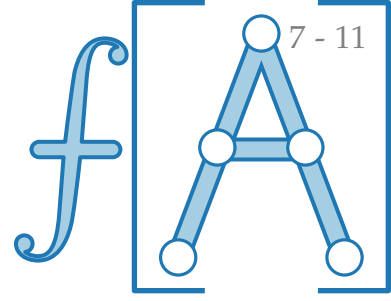
- Polynomielle Faktoren vernachlässigbar
- Basis der exponentiellen Faktoren dominieren

Def. $f(n) \in \mathcal{O}^*(g(n))$
 $\Leftrightarrow \exists$ Polynom $p(n)$ mit $f(n) \in \mathcal{O}(g(n) \cdot p(n))$

■ Typische Resultate

Ansatz	Laufzeit in $\mathcal{O}$ -Notation	$\mathcal{O}^*$ -Notation
Brute-Force	$\mathcal{O}(2^n)$	$\mathcal{O}^*(2^n)$
Algorithmus A	$\mathcal{O}(1.5^n \cdot n^2)$	$\mathcal{O}^*(1.5^n)$
Algorithmus B	$\mathcal{O}(1.4^n \cdot n^{10})$	

$\mathcal{O}^*$ -Notation



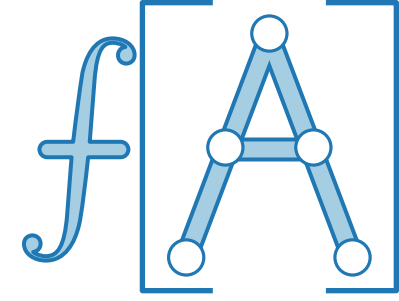
$$\mathcal{O}(1.4^n \cdot n^{10}) \subsetneq \mathcal{O}(1.5^n \cdot n^2) \subsetneq \mathcal{O}(2^n) \subsetneq \mathcal{O}(4^n)$$

- Polynomielle Faktoren vernachlässigbar
- Basis der exponentiellen Faktoren dominieren

Def. $f(n) \in \mathcal{O}^*(g(n))$
 $\Leftrightarrow \exists$ Polynom $p(n)$ mit $f(n) \in \mathcal{O}(g(n) \cdot p(n))$

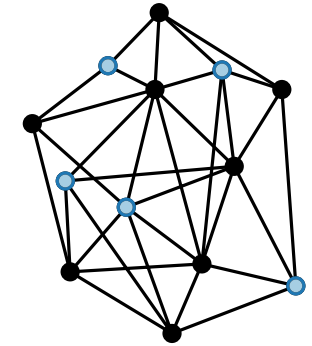
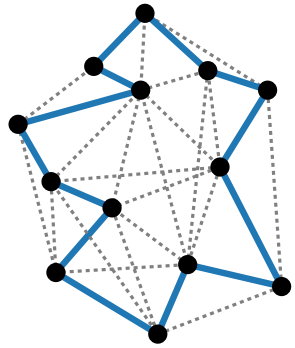
■ Typische Resultate

Ansatz	Laufzeit in $\mathcal{O}$ -Notation	$\mathcal{O}^*$ -Notation
Brute-Force	$\mathcal{O}(2^n)$	$\mathcal{O}^*(2^n)$
Algorithmus A	$\mathcal{O}(1.5^n \cdot n^2)$	$\mathcal{O}^*(1.5^n)$
Algorithmus B	$\mathcal{O}(1.4^n \cdot n^{10})$	$\mathcal{O}^*(1.4^n)$

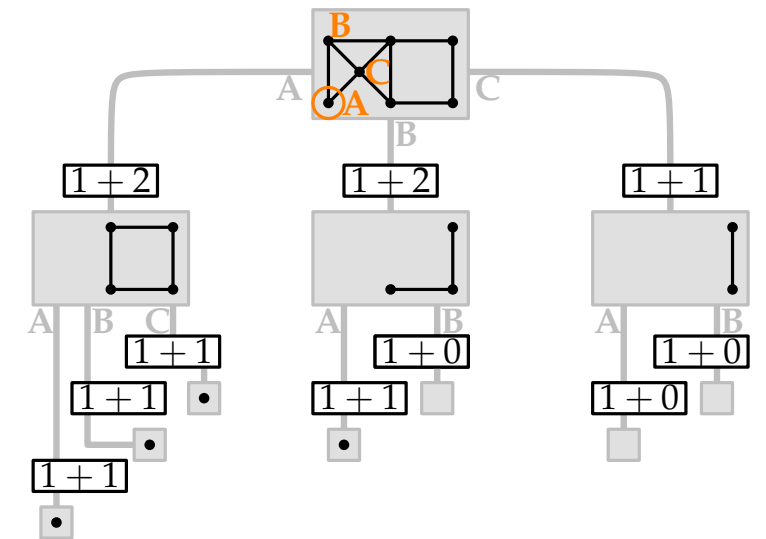
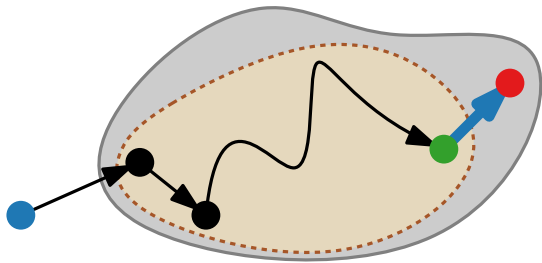


Fortgeschrittene Algorithmen

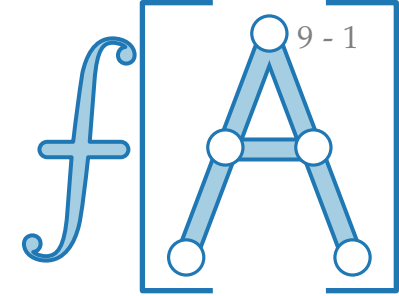
19. Vorlesung Exakte Algorithmen (für NP-schwere Probleme)



Teil II: Problem des Handlungsreisenden (TSP)



Problem des Handlungsreisenden (TSP)



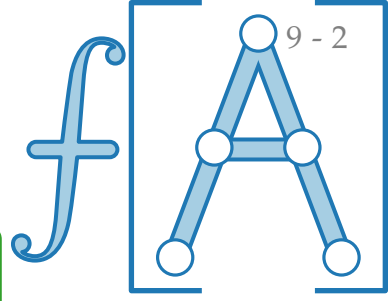
Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

THE TRAVELLING SALESMAN PROBLEM

WHAT'S THE SHORTEST ROUTE TO VISIT ALL LOCATIONS AND RETURN?

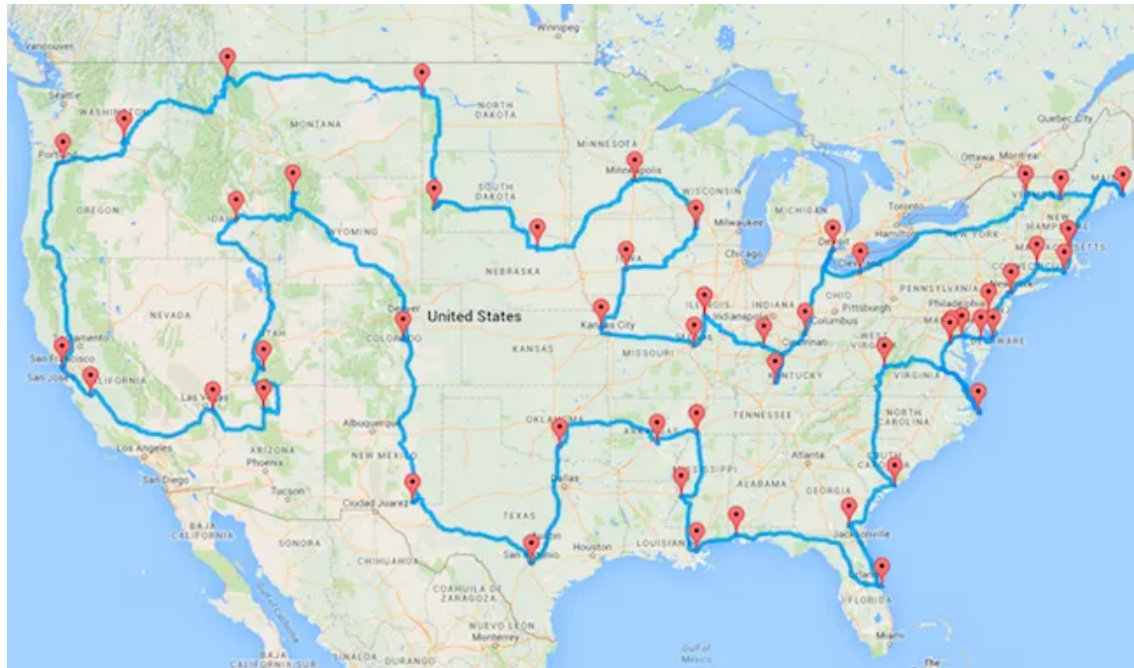
ADDING MORE STOPS TAKES LONGER AND LONGER AND LONGER TO FIGURE IT OUT

Problem des Handlungsreisenden (TSP)



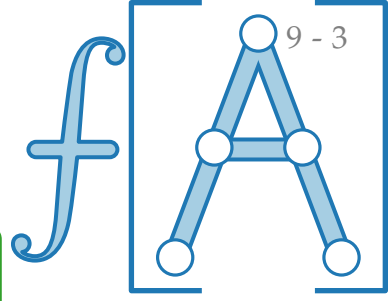
Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$



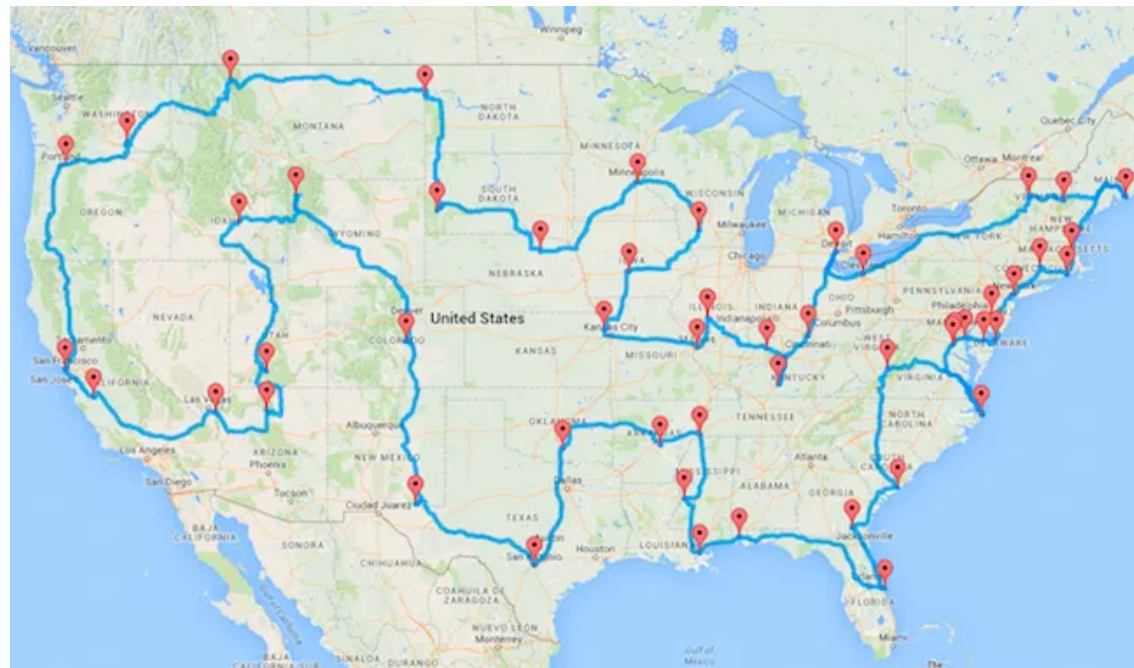
Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

Problem des Handlungsreisenden (TSP)



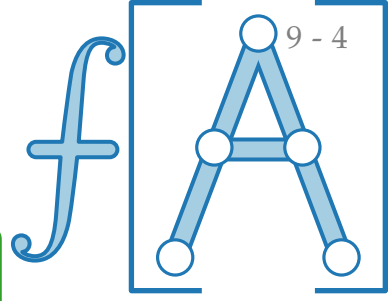
Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d



Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

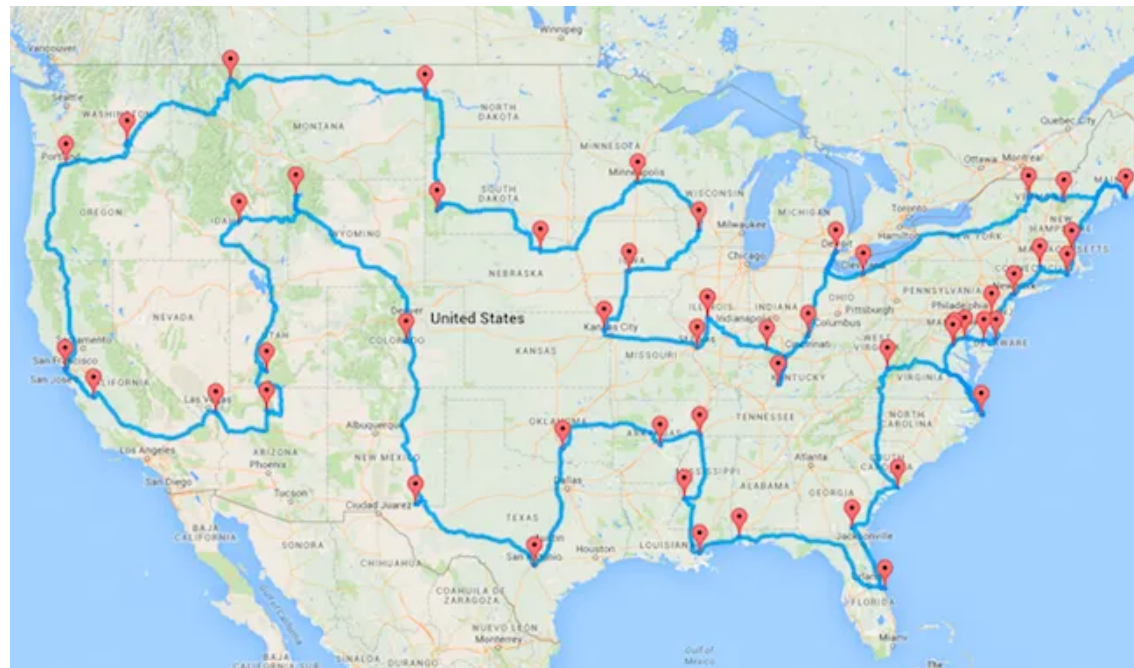
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

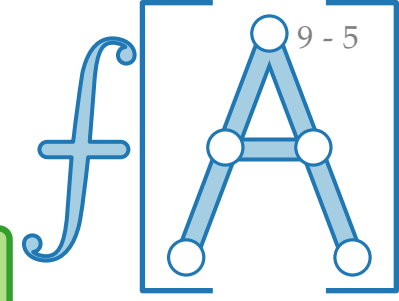
Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

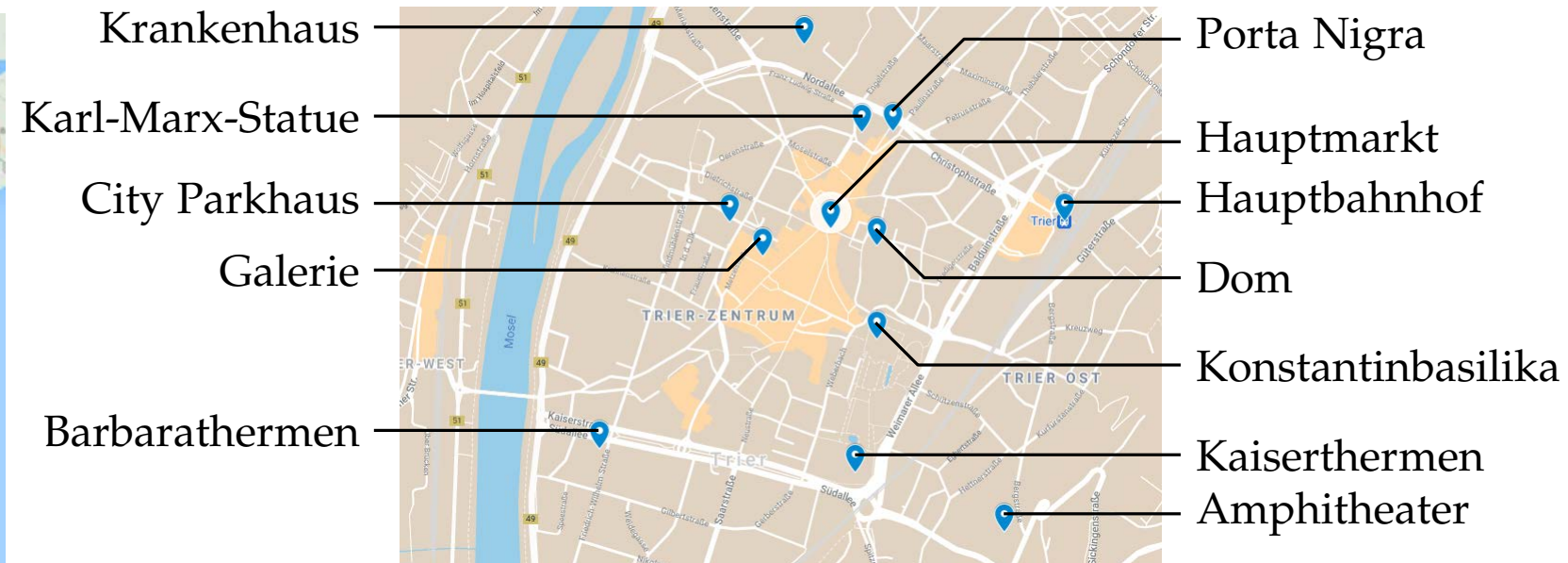
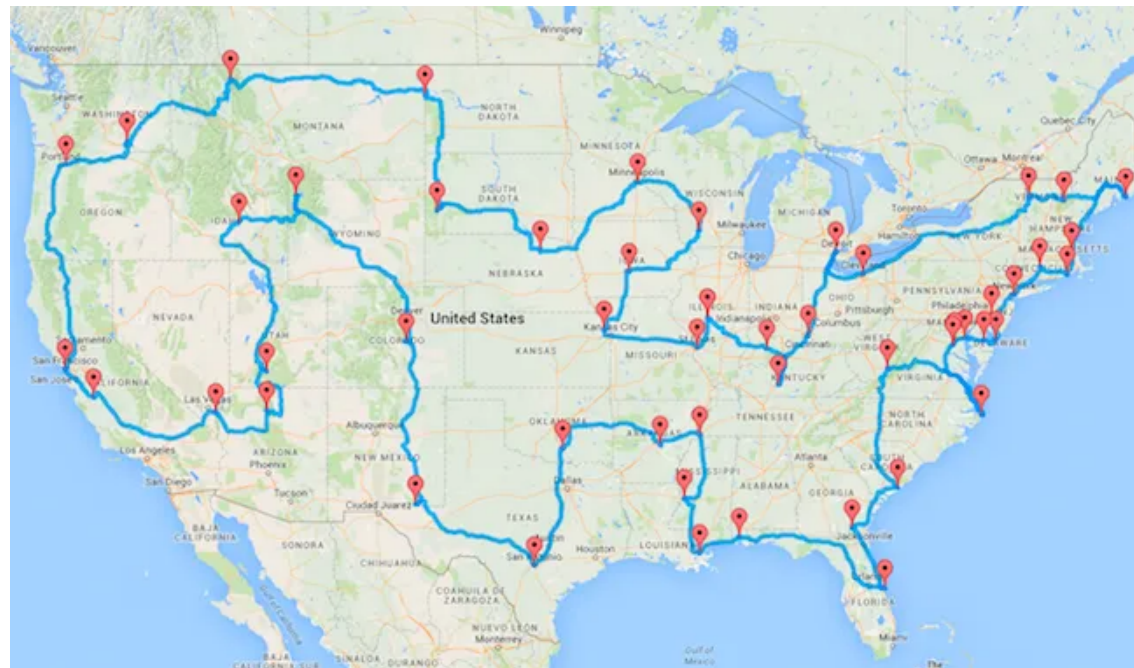
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

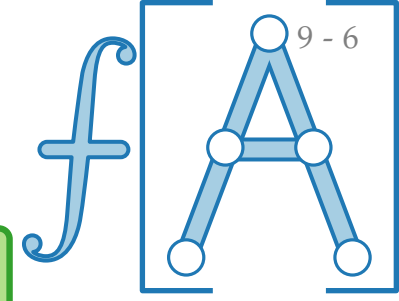
Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

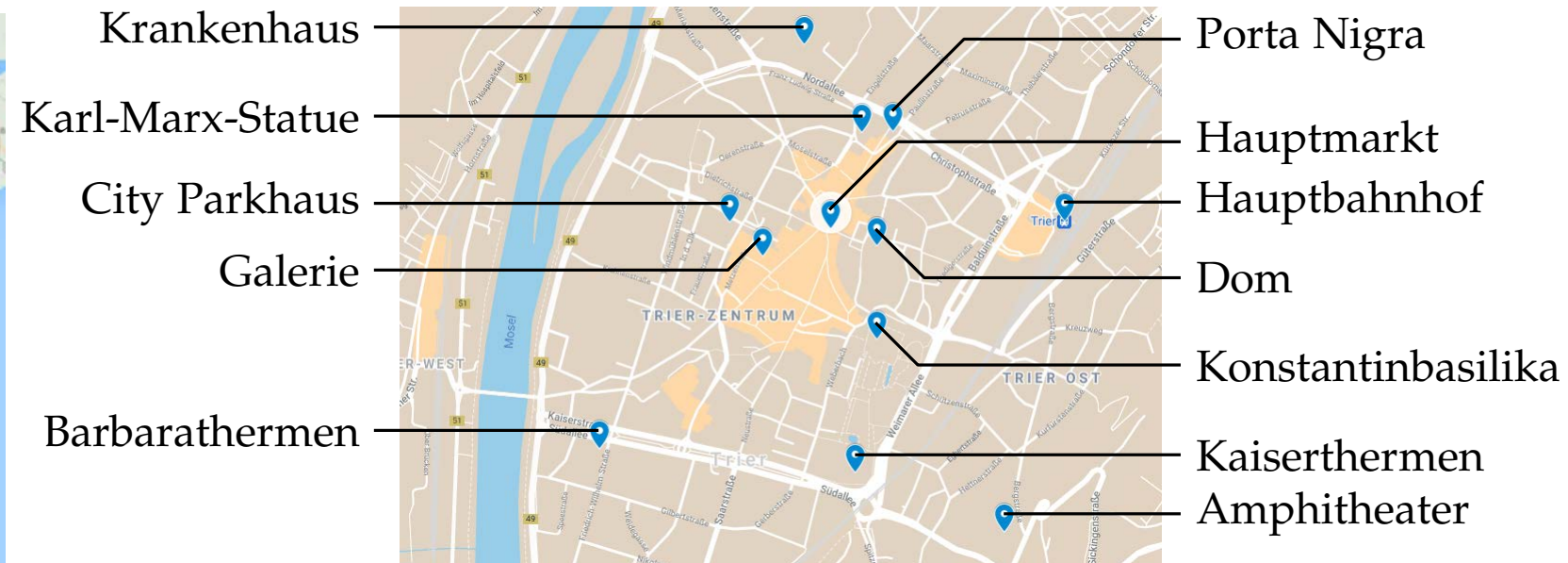
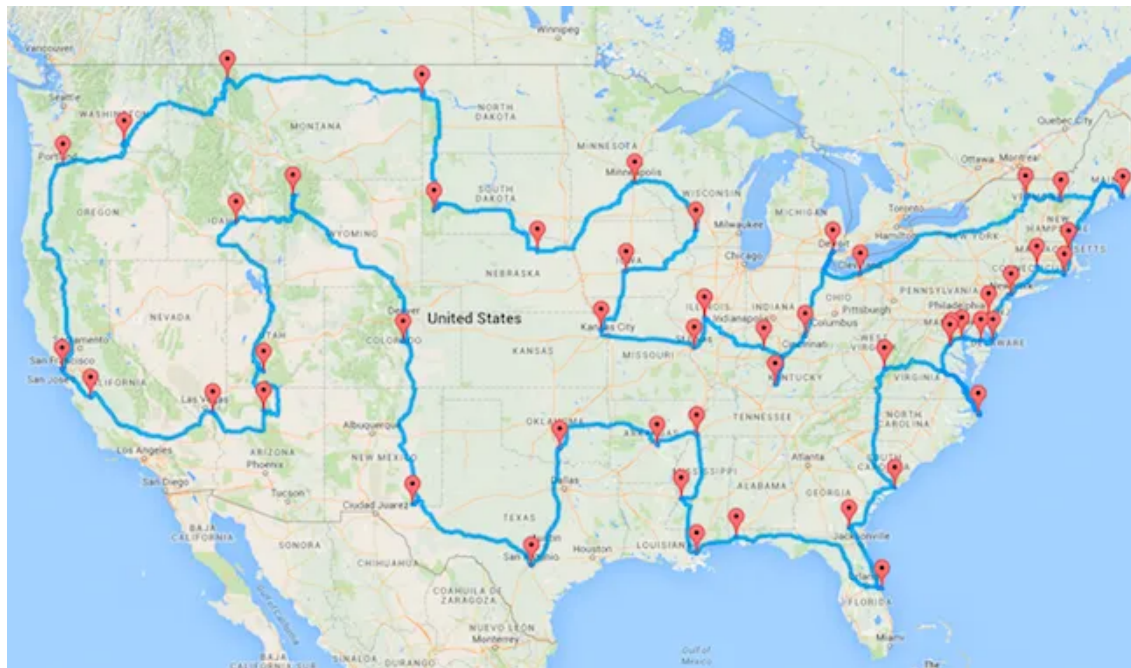
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in Q_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

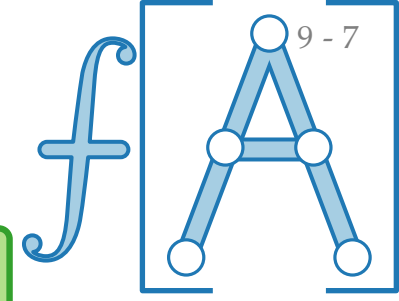
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

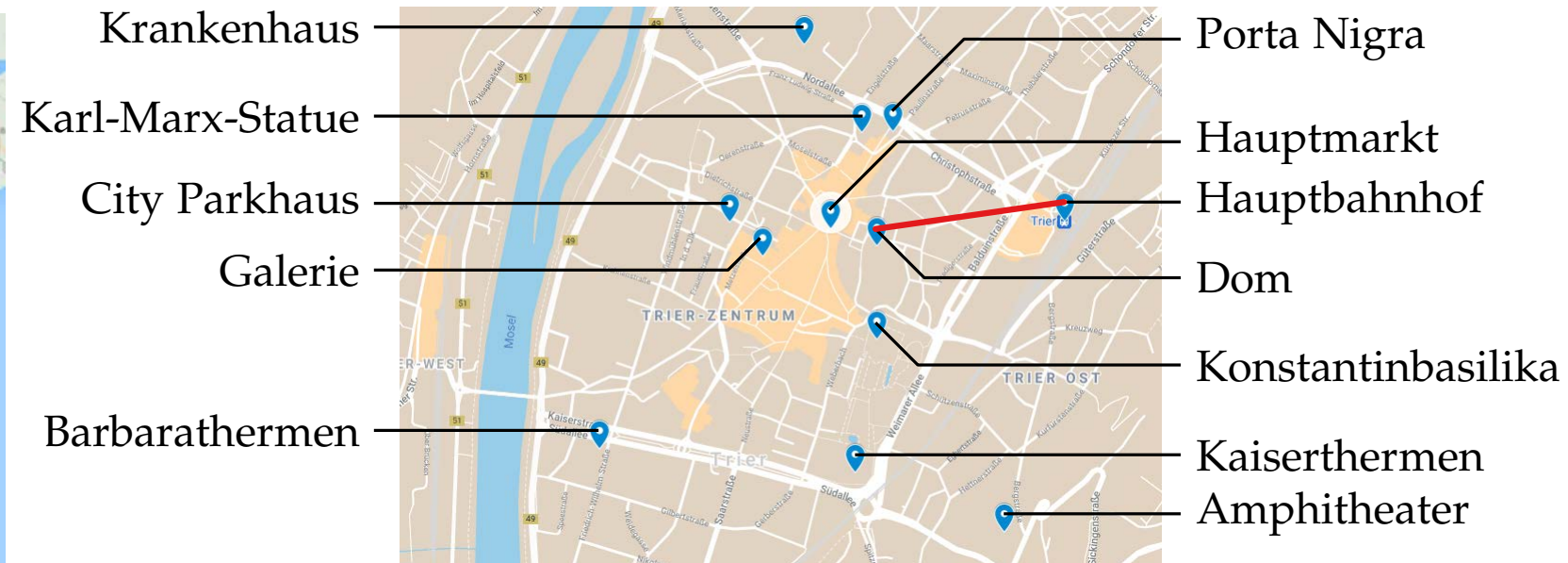
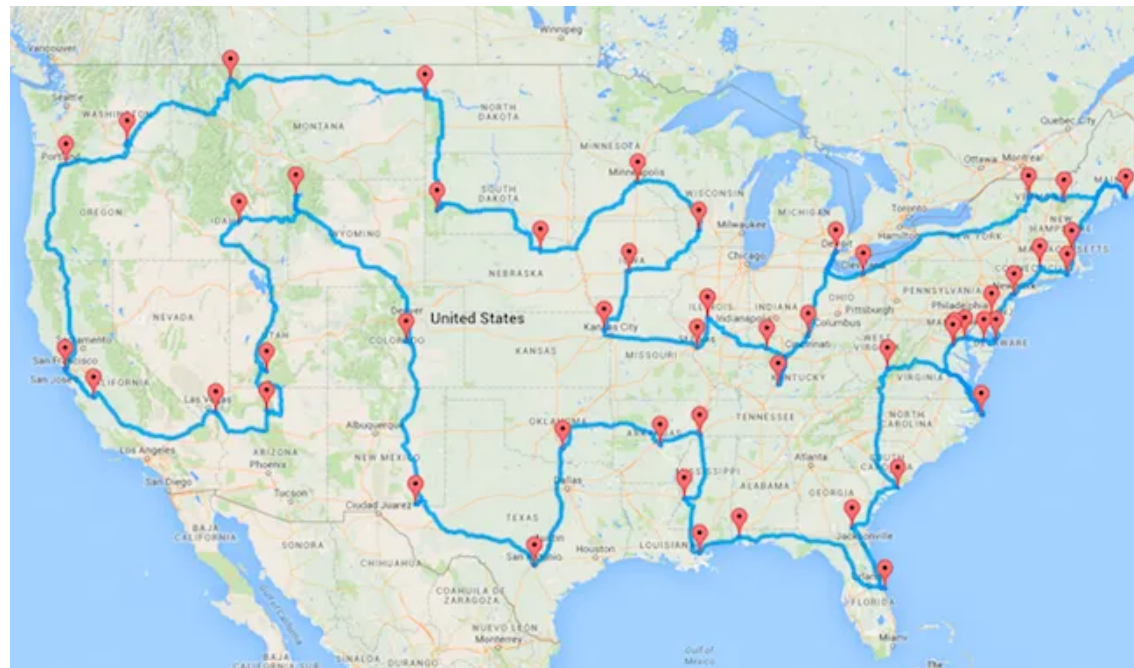
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

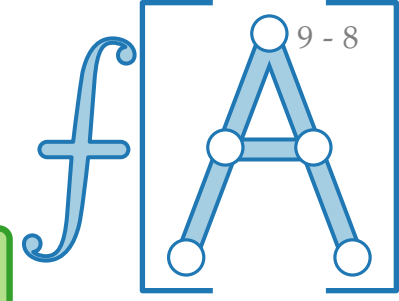
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

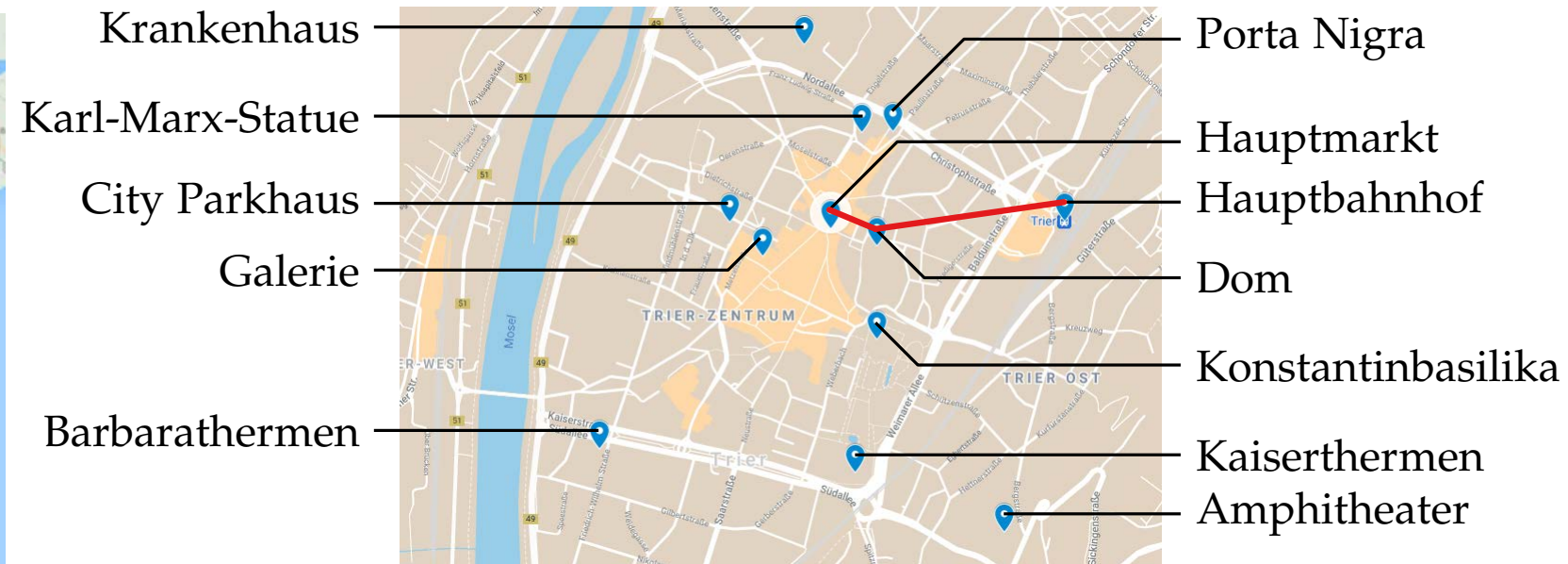
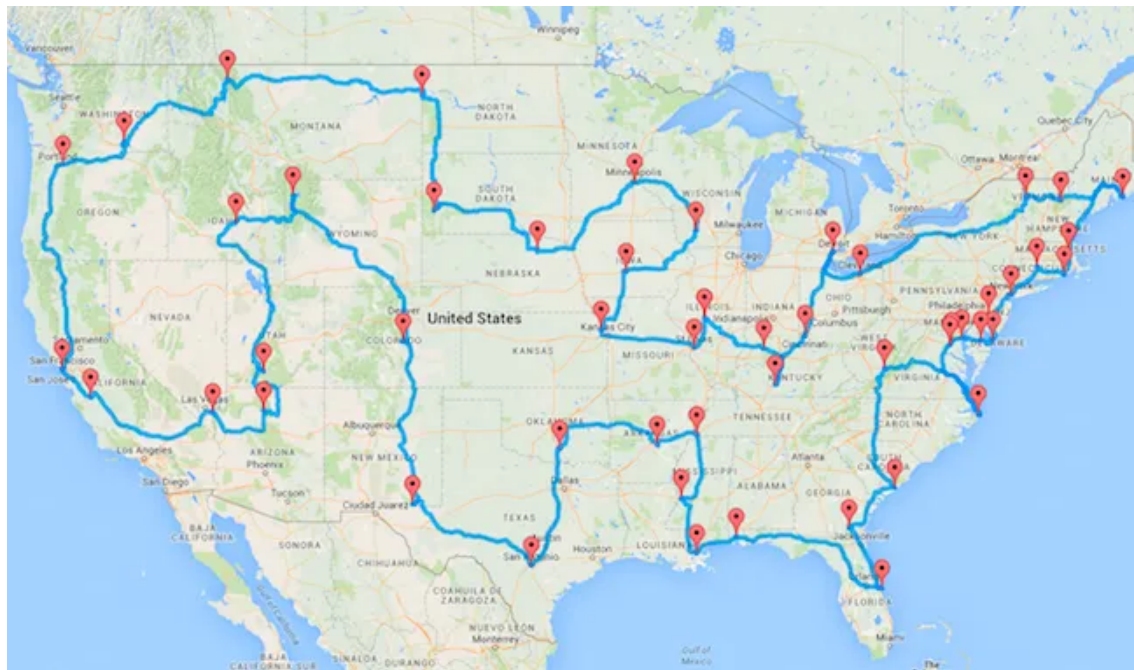
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

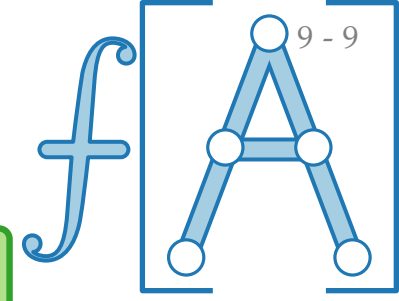
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

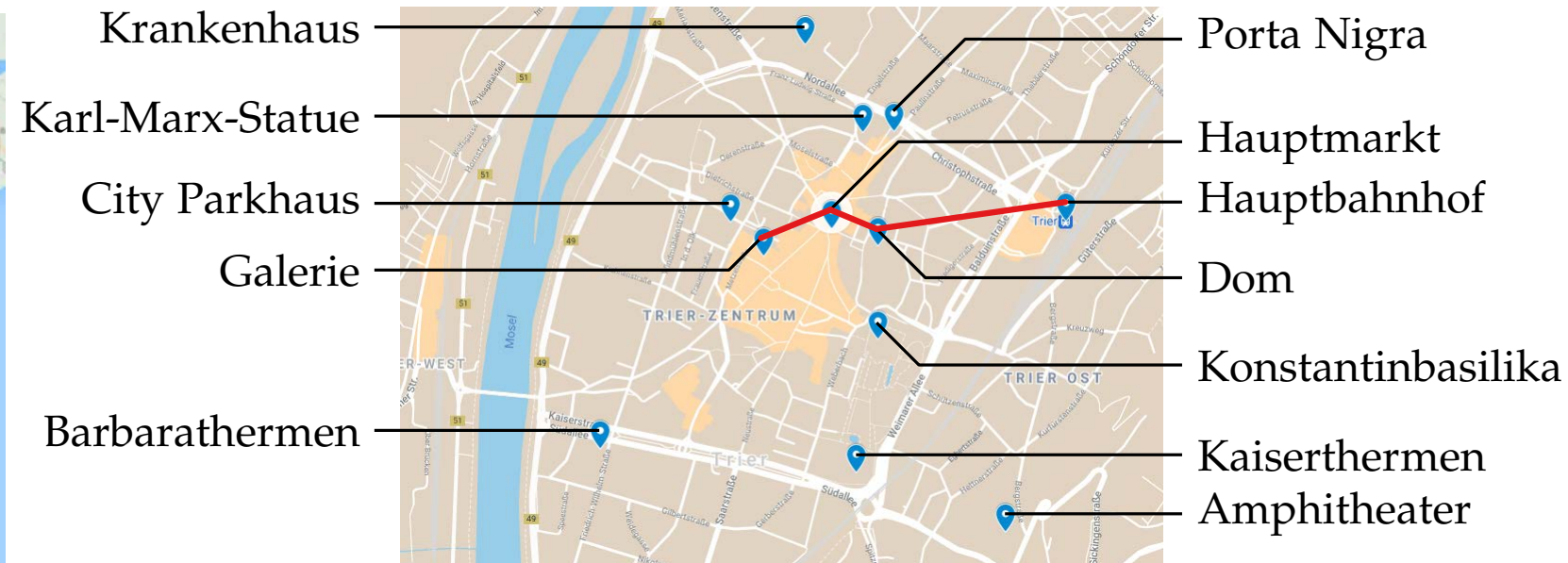
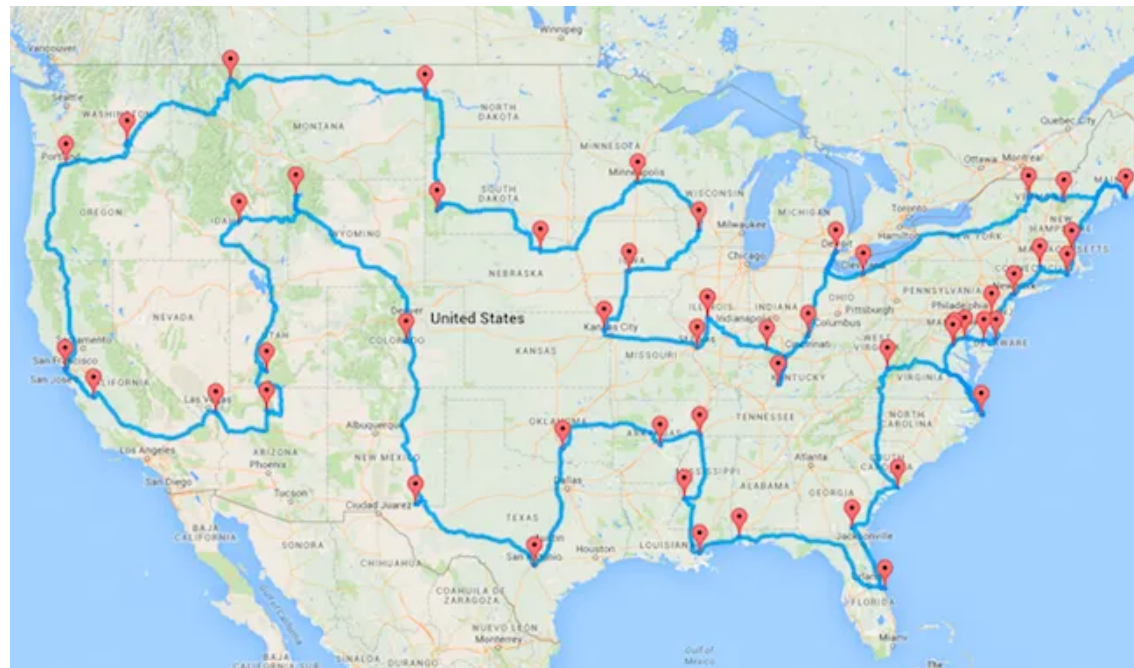
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

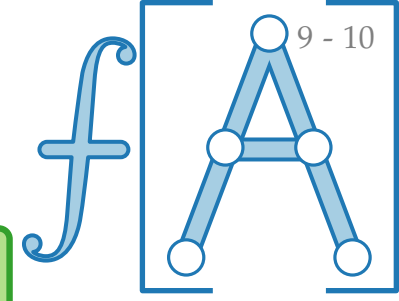
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

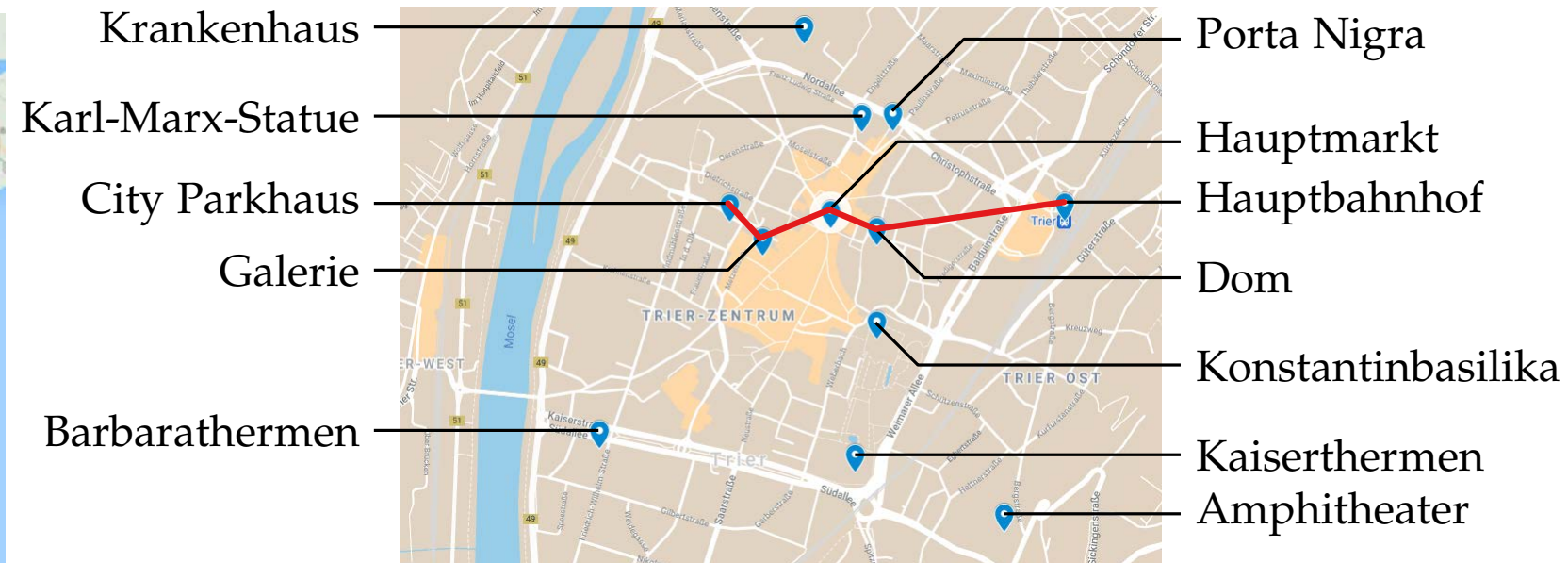
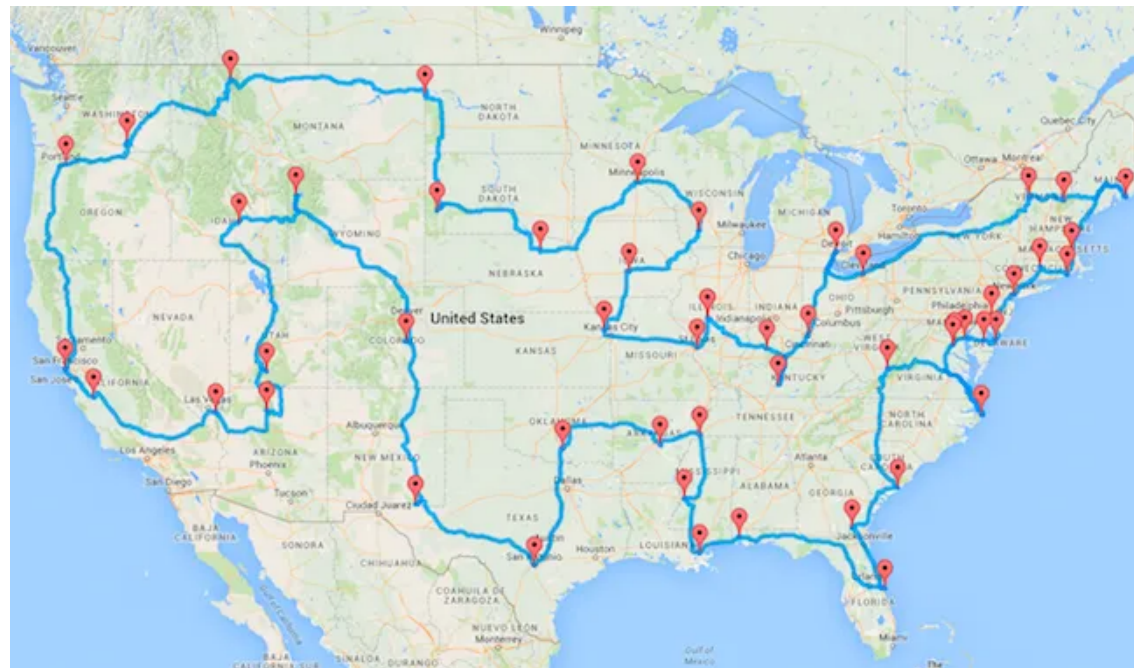
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

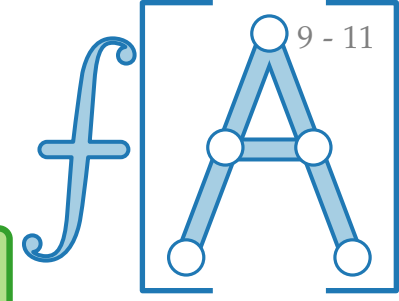
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

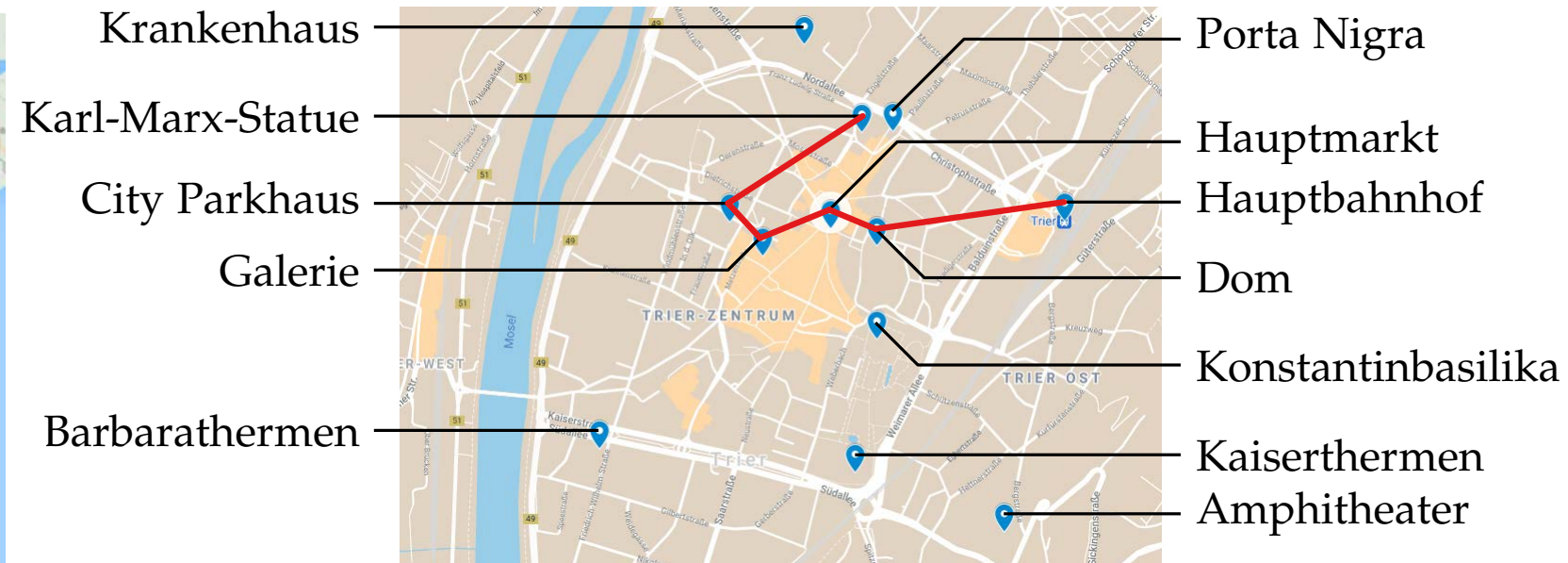
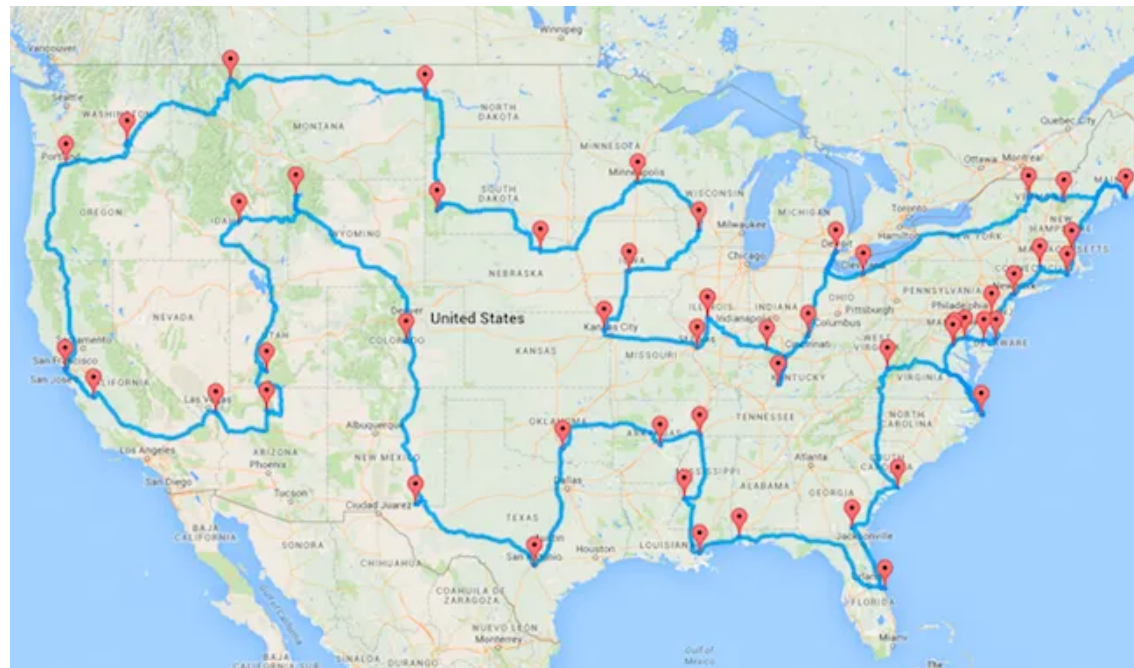
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

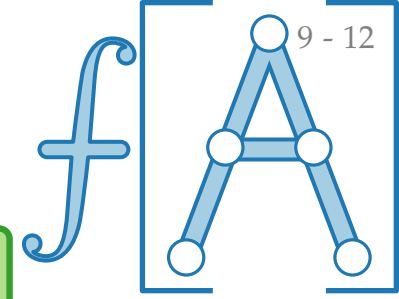
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

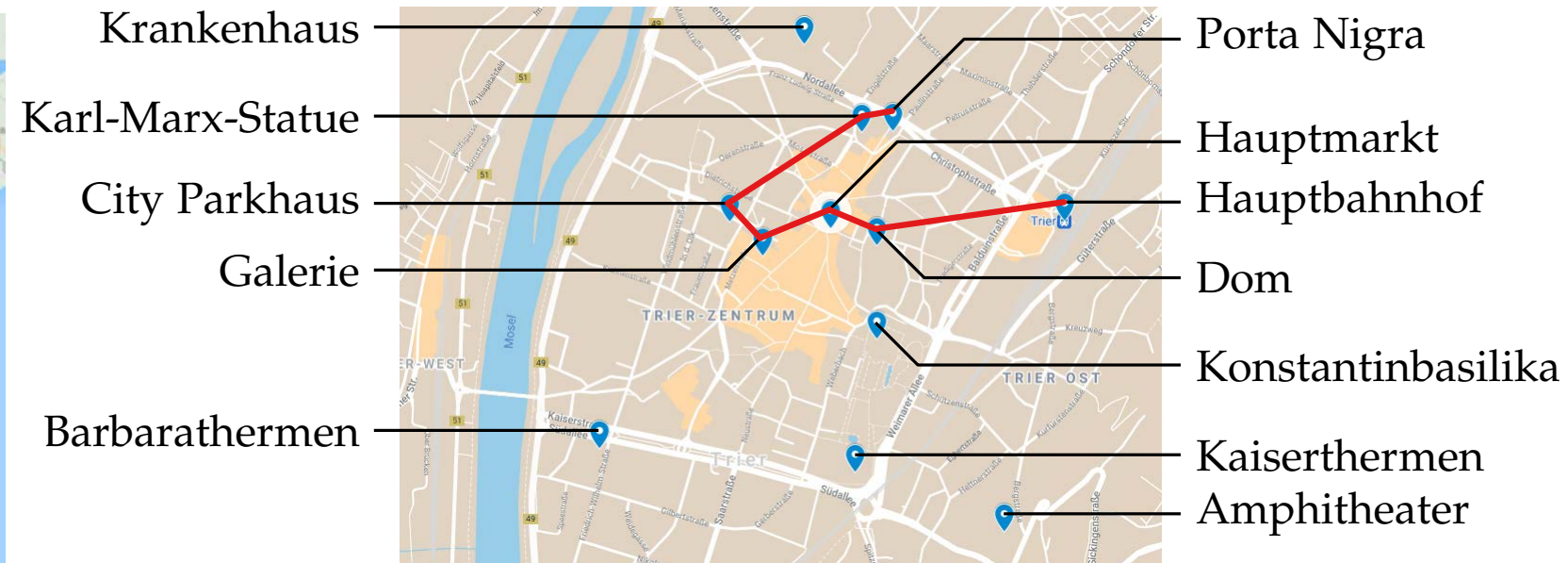
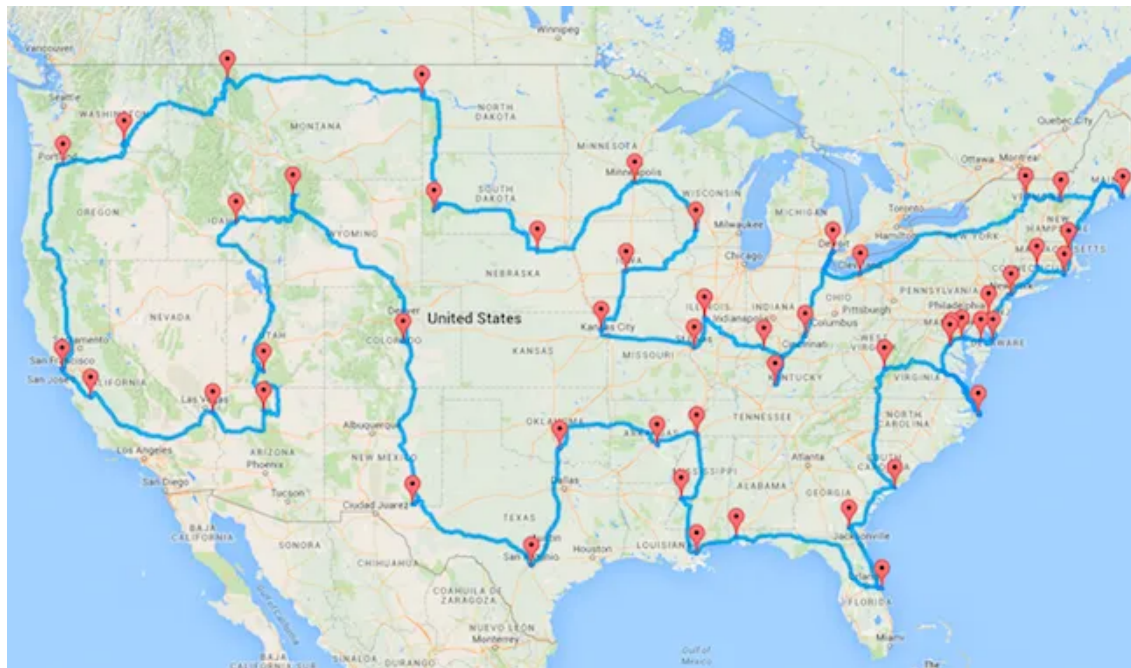
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

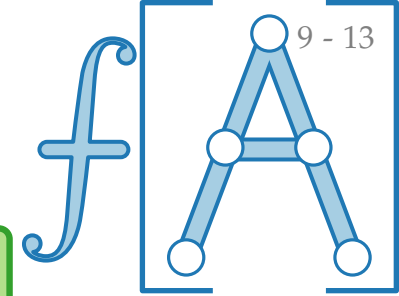
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

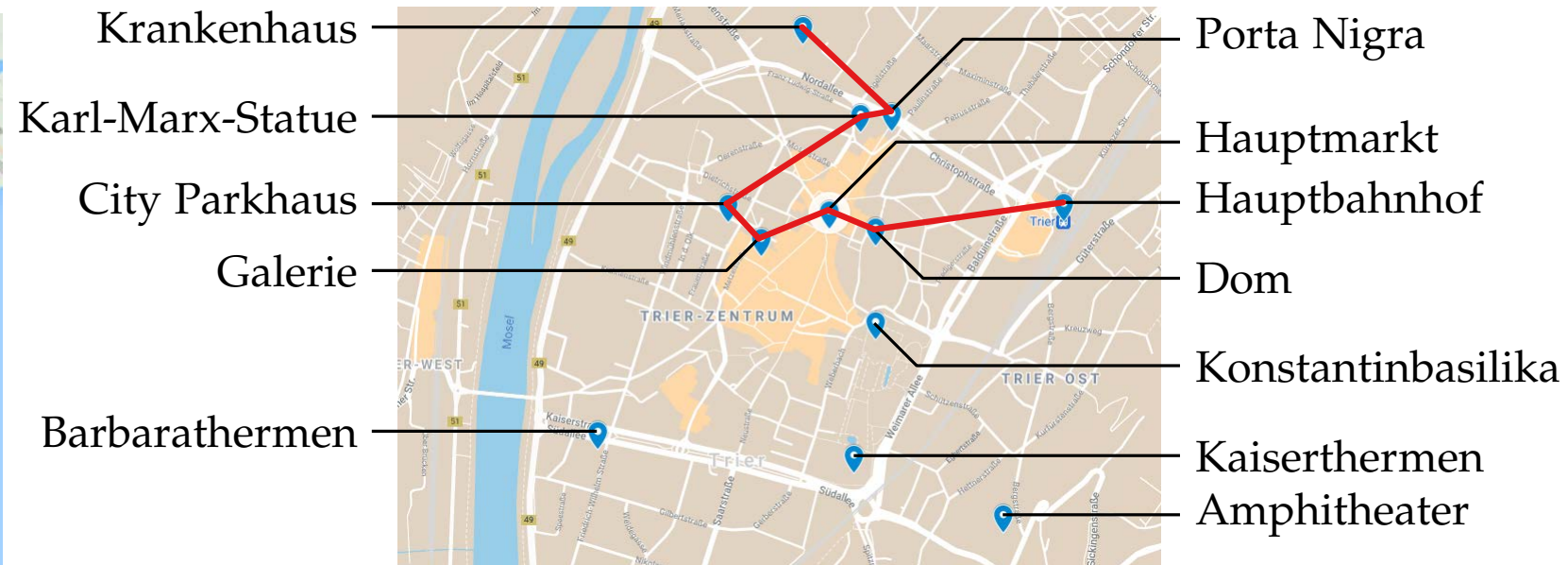
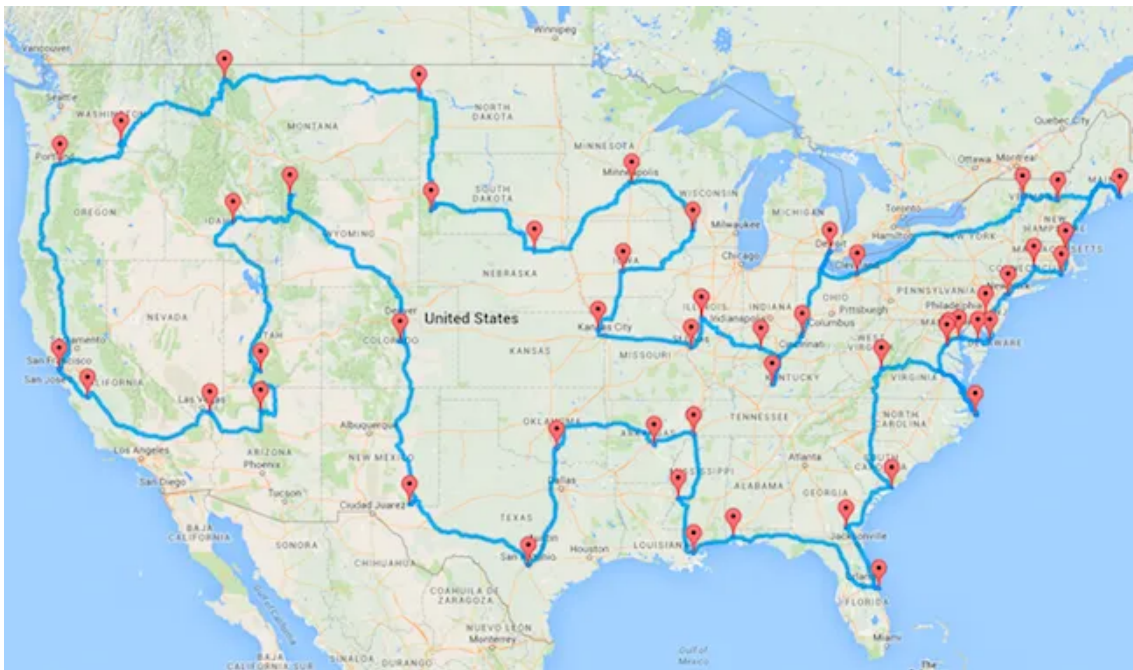
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

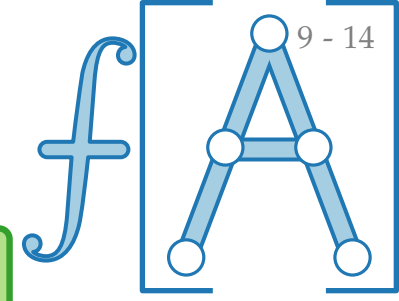
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

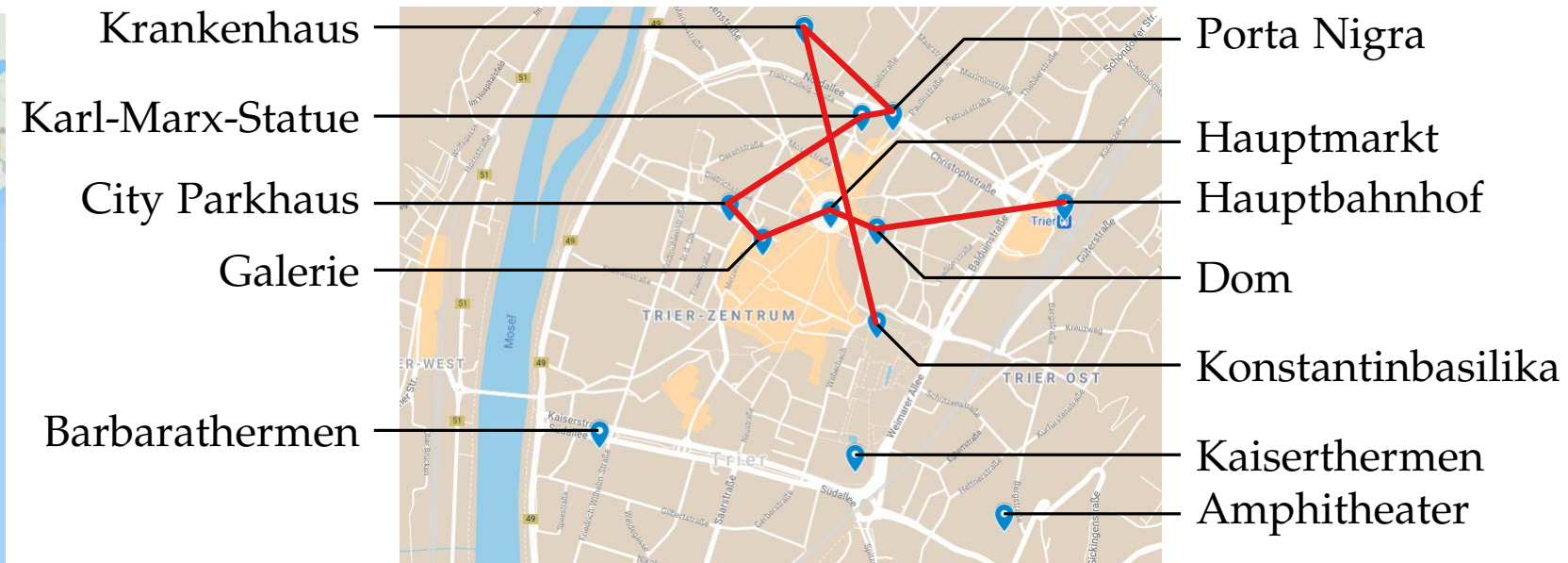
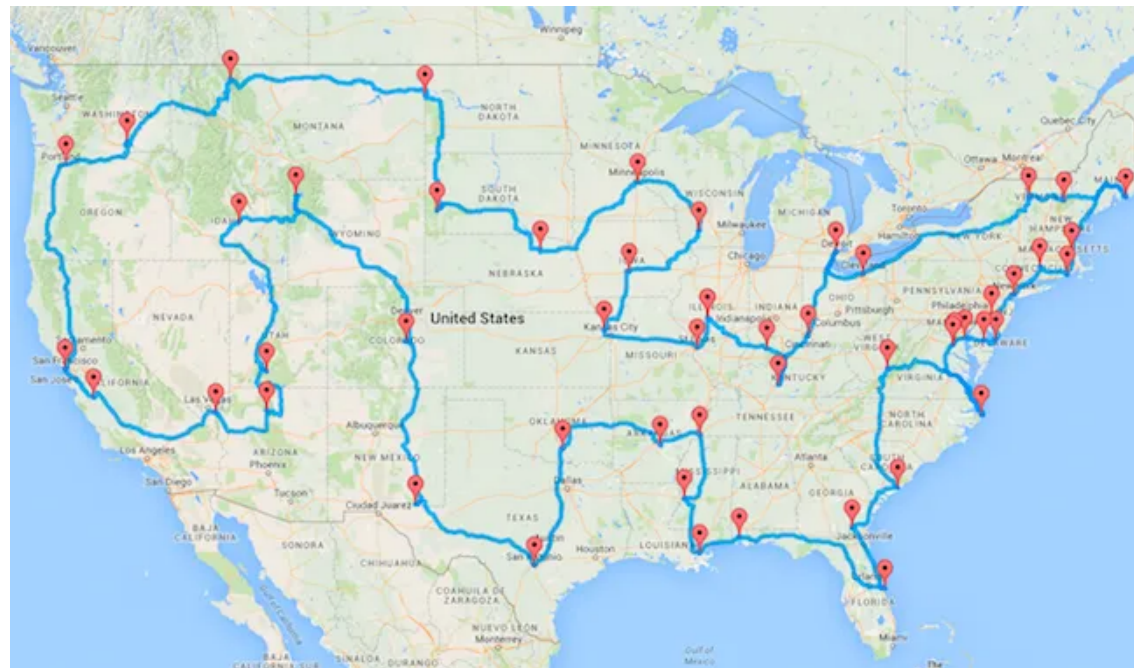
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

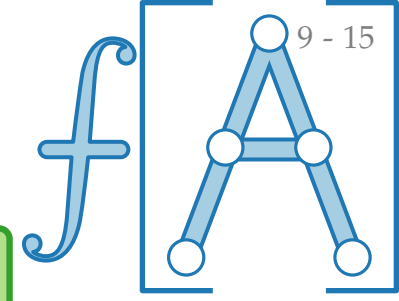
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

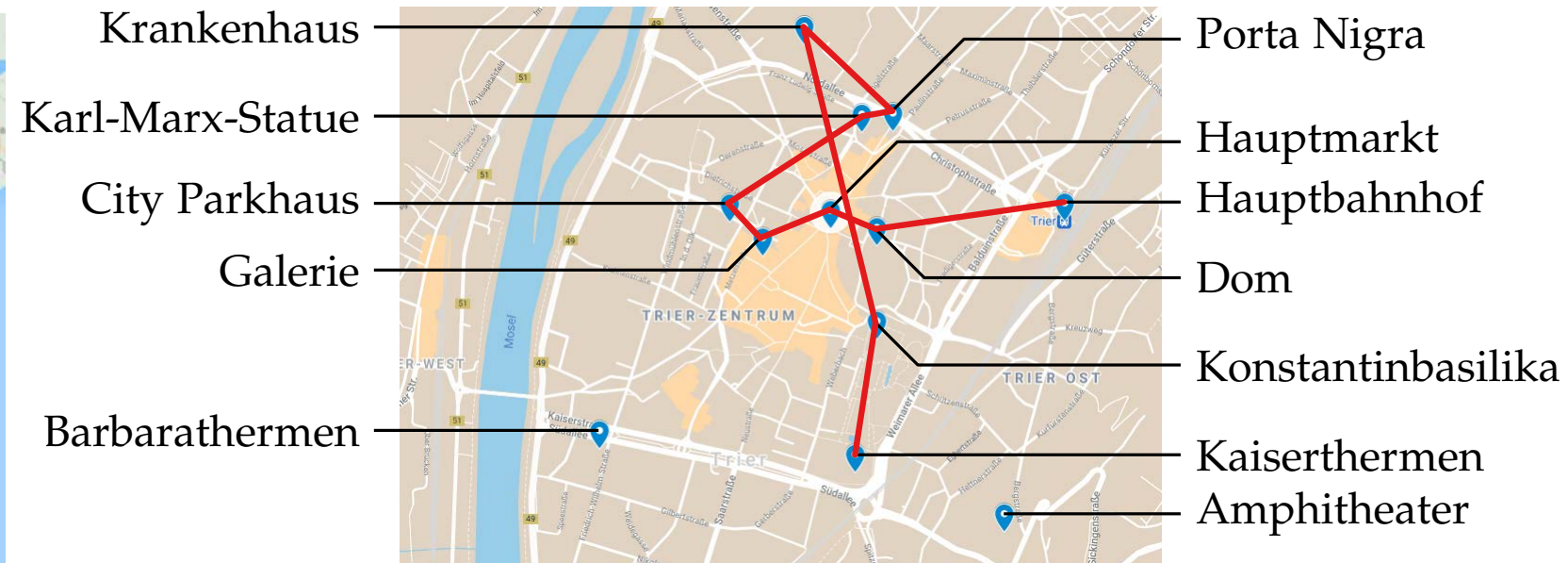
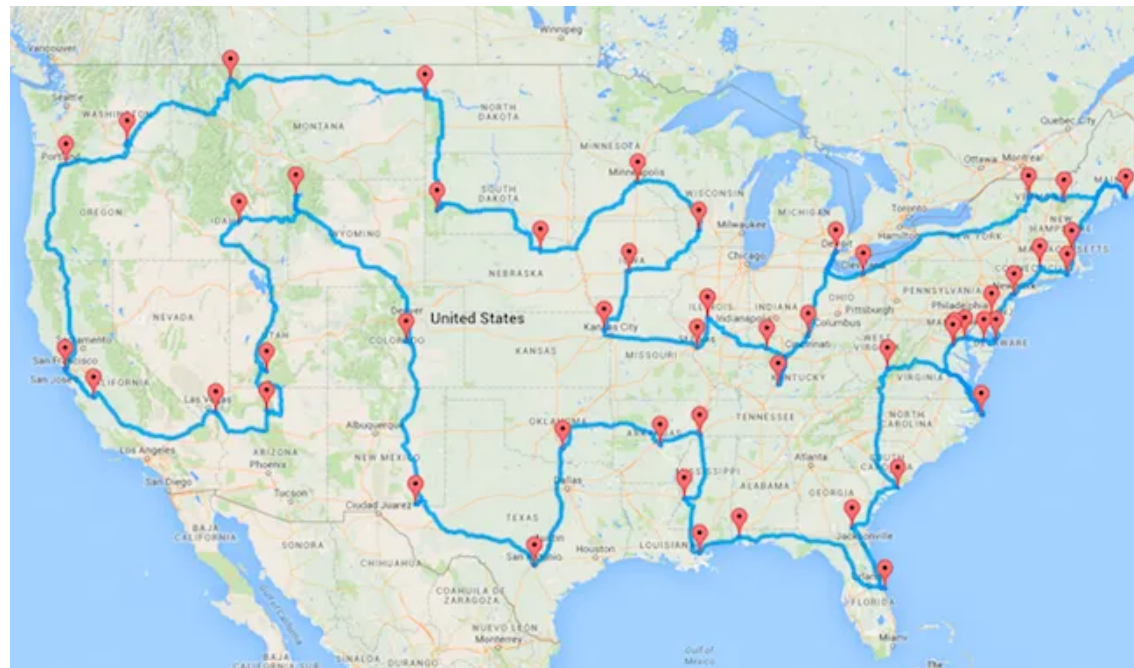
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

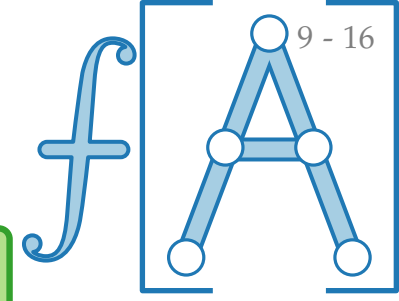
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

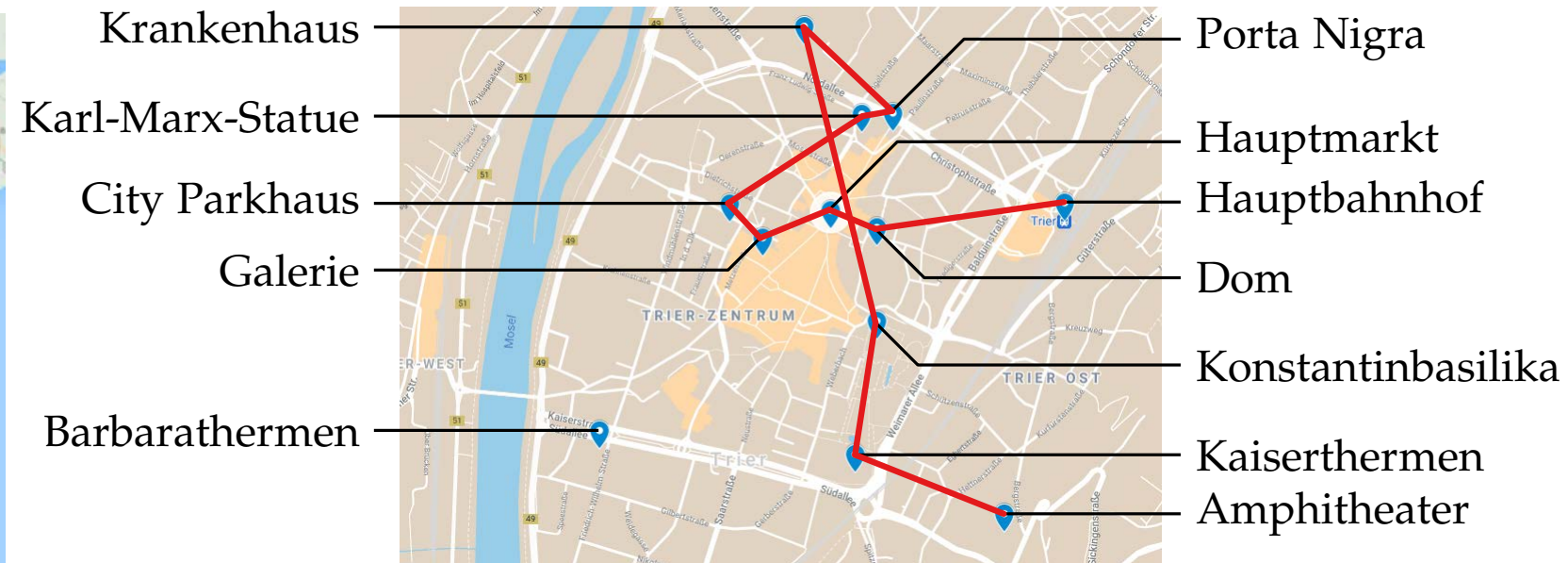
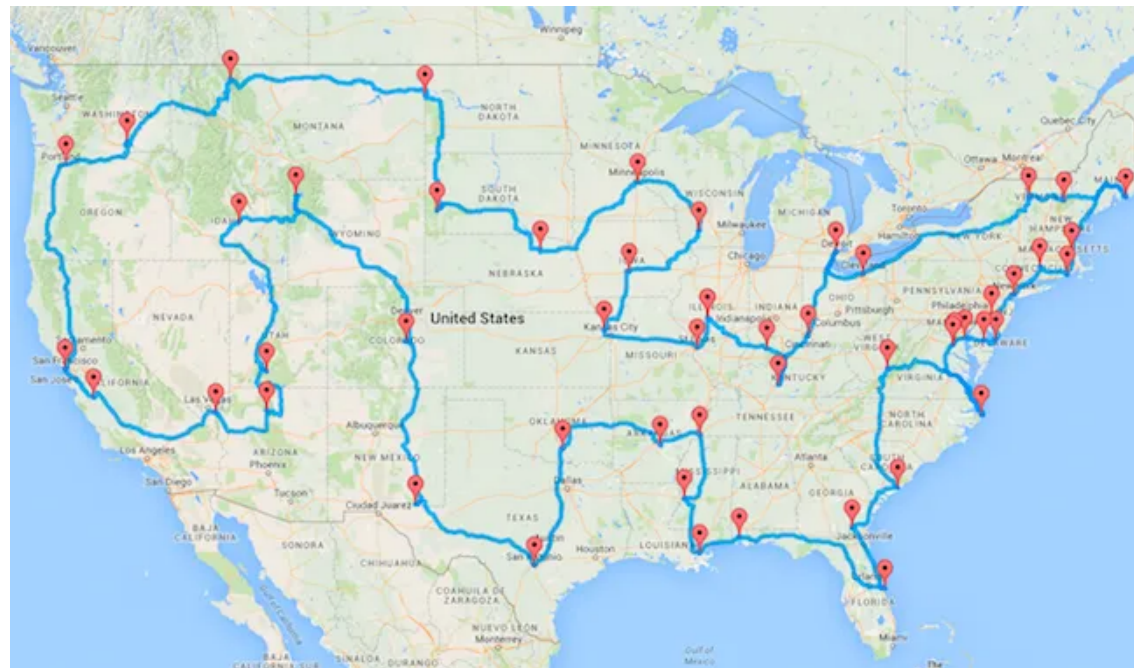
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

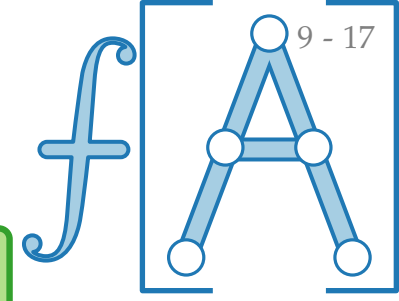
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

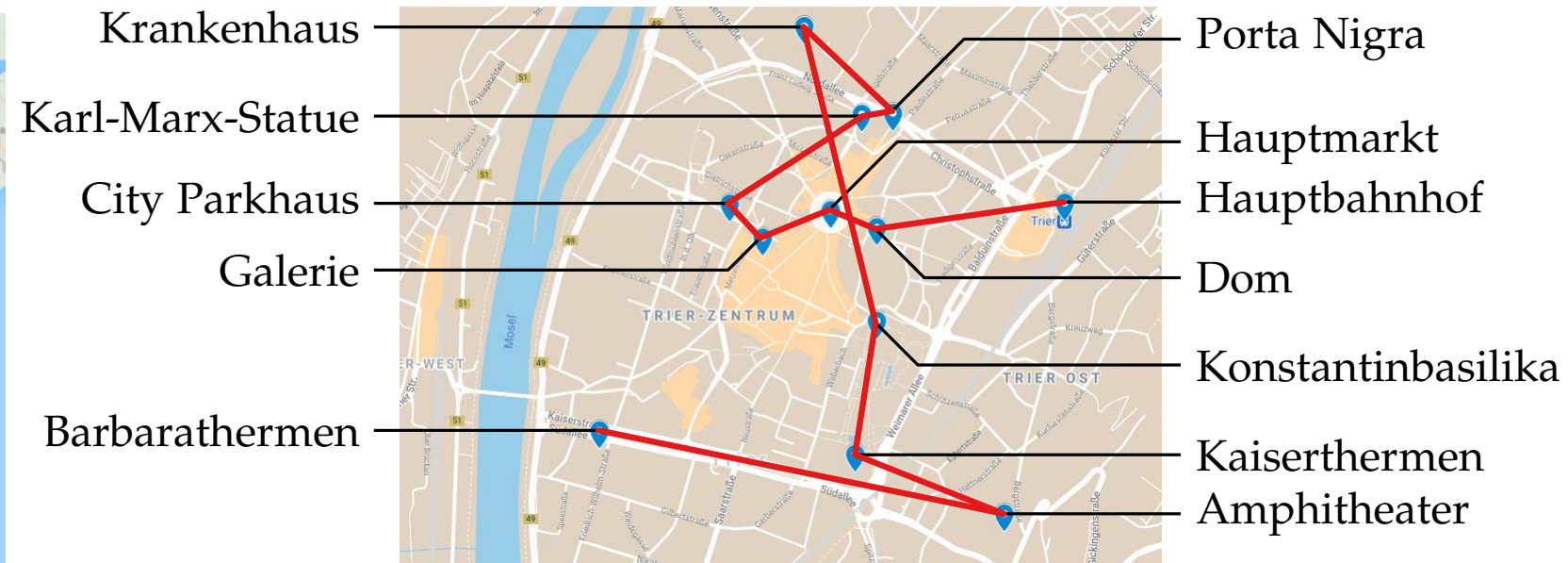
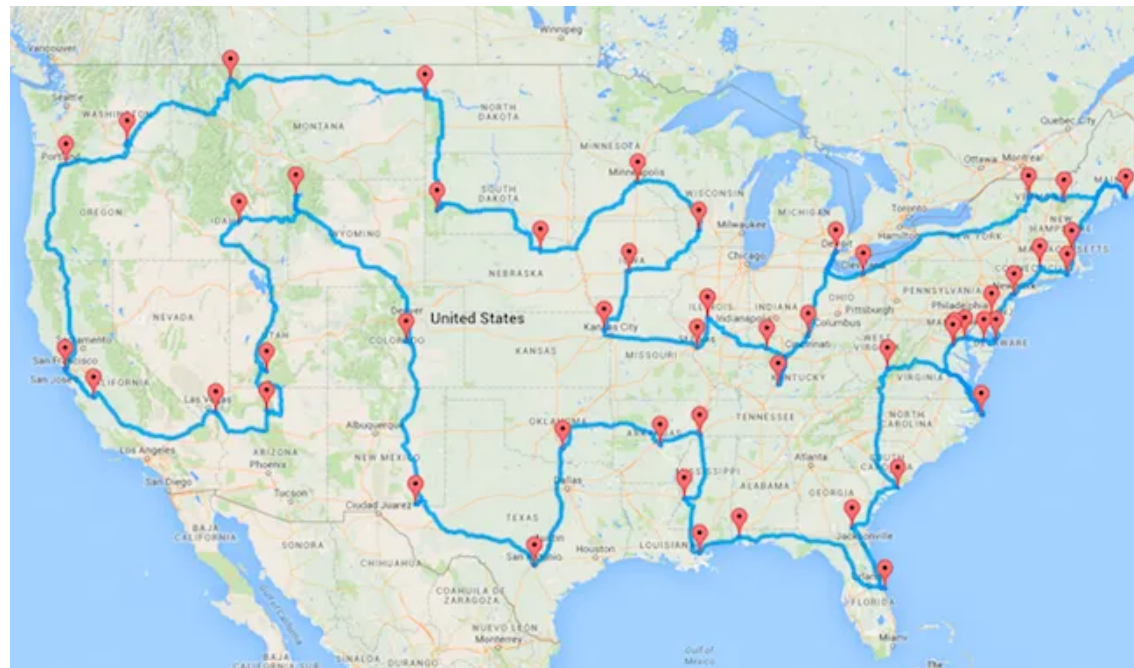
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

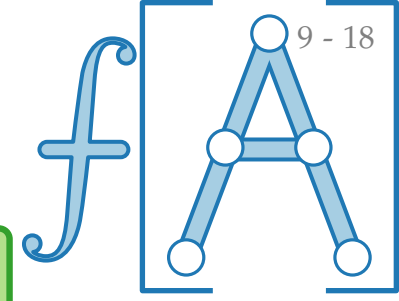
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

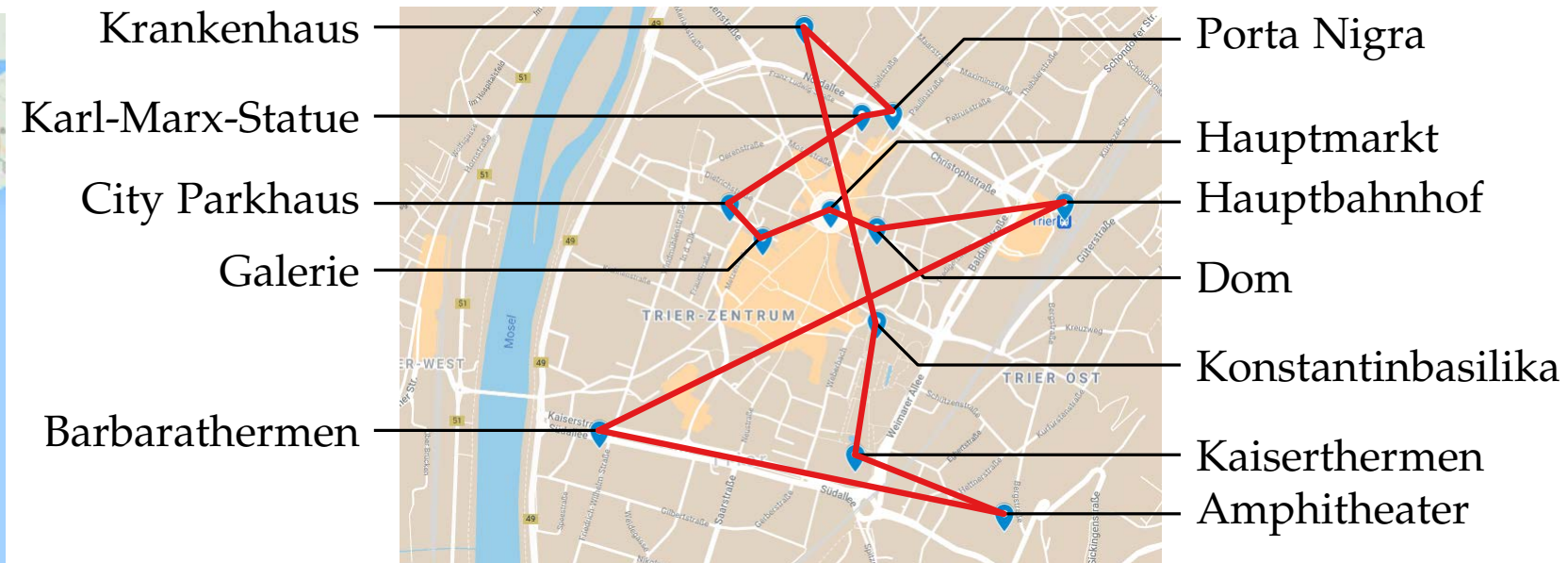
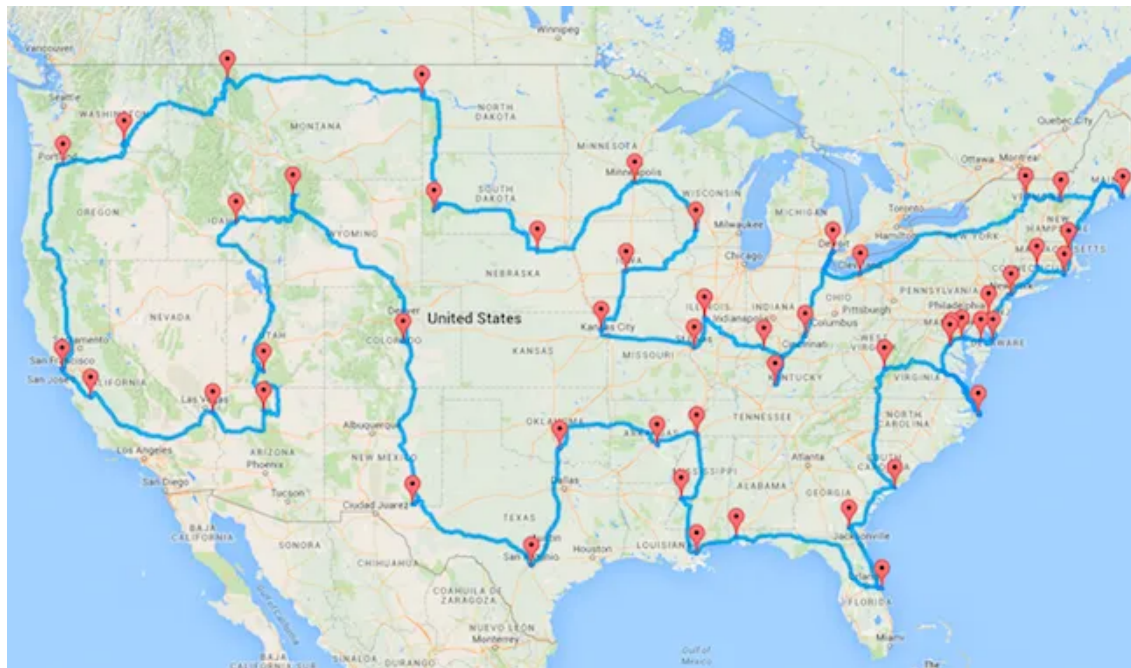
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

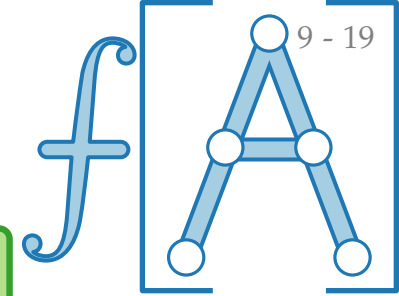
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

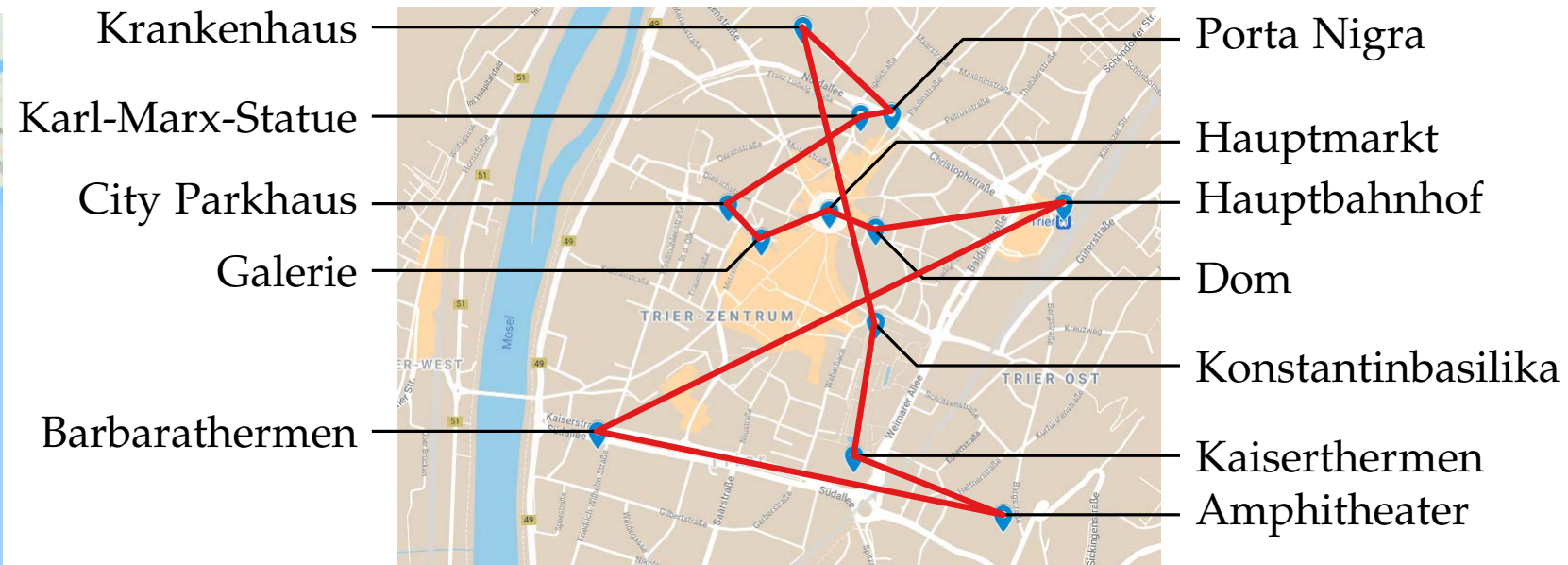
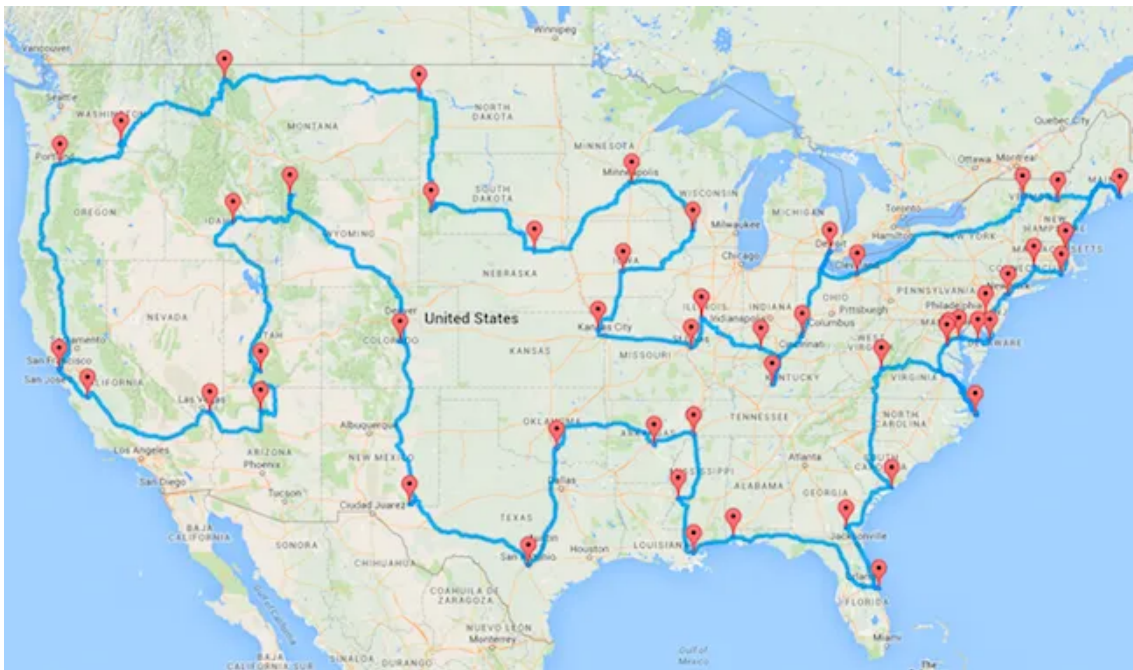
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

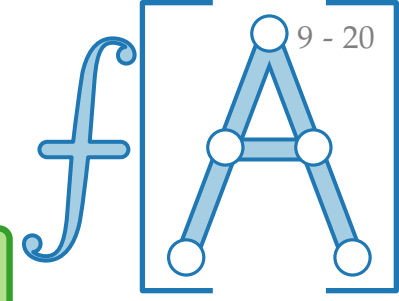
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?
Nein ...

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

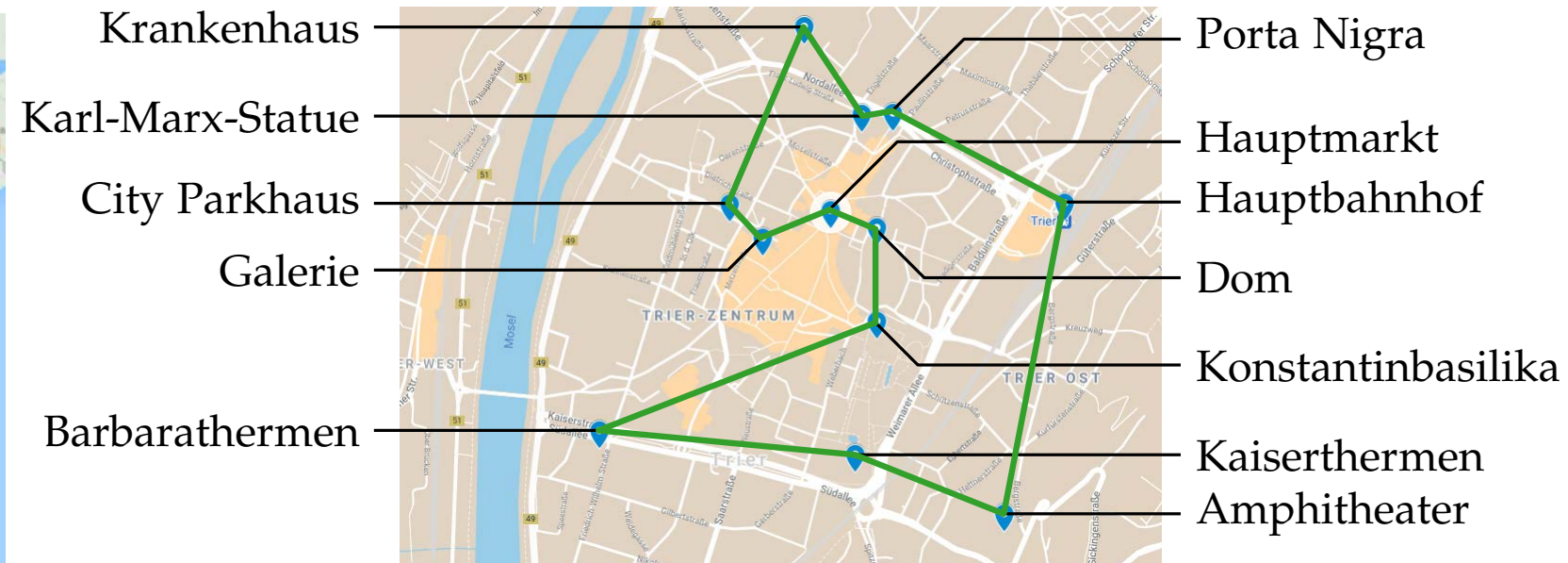
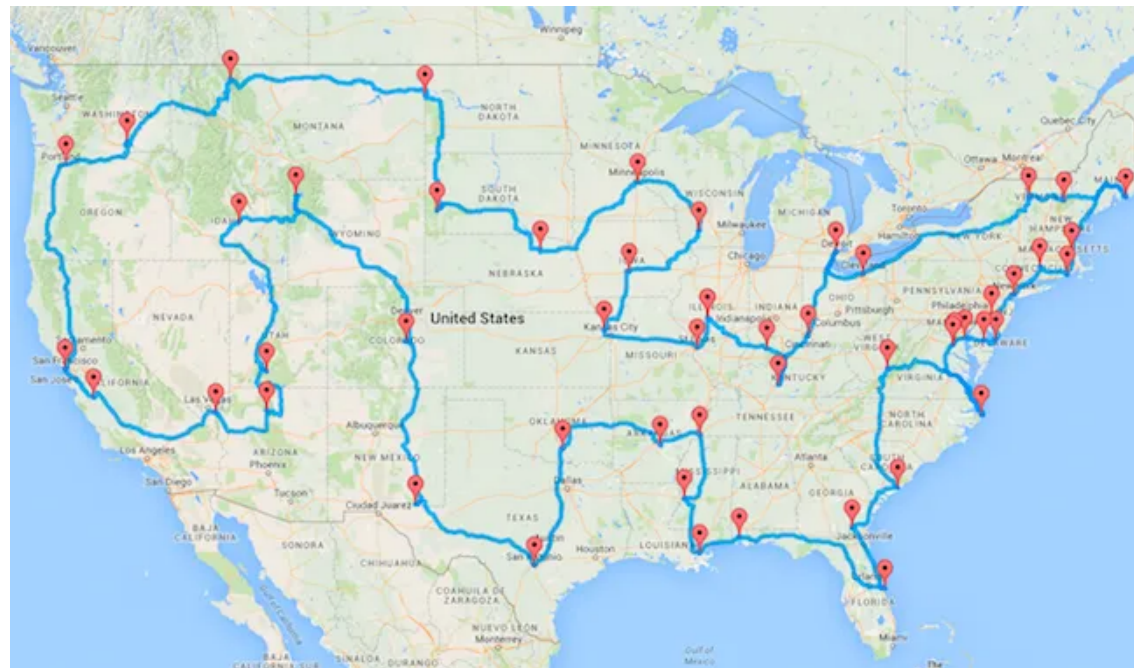
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

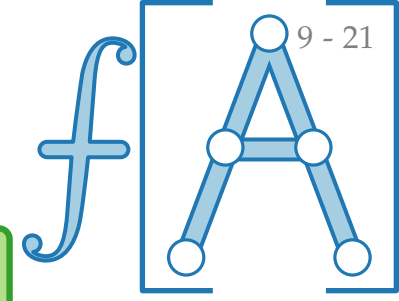
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?
Nein ...

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

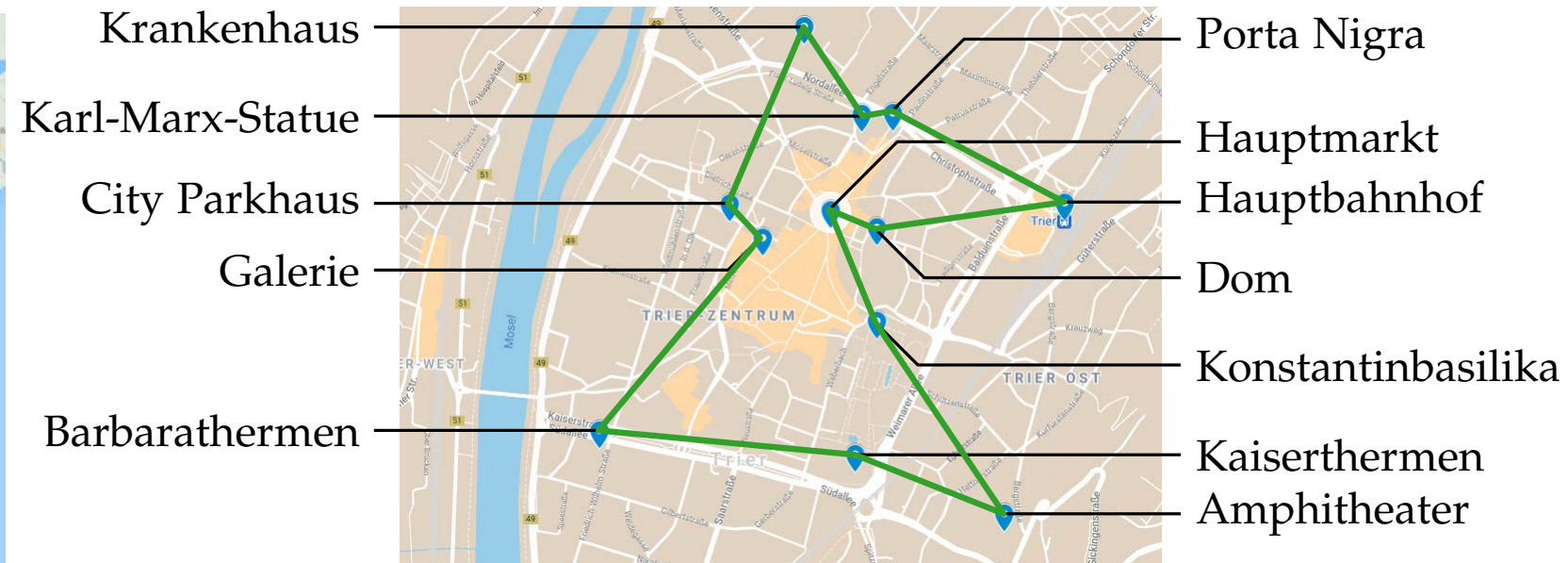
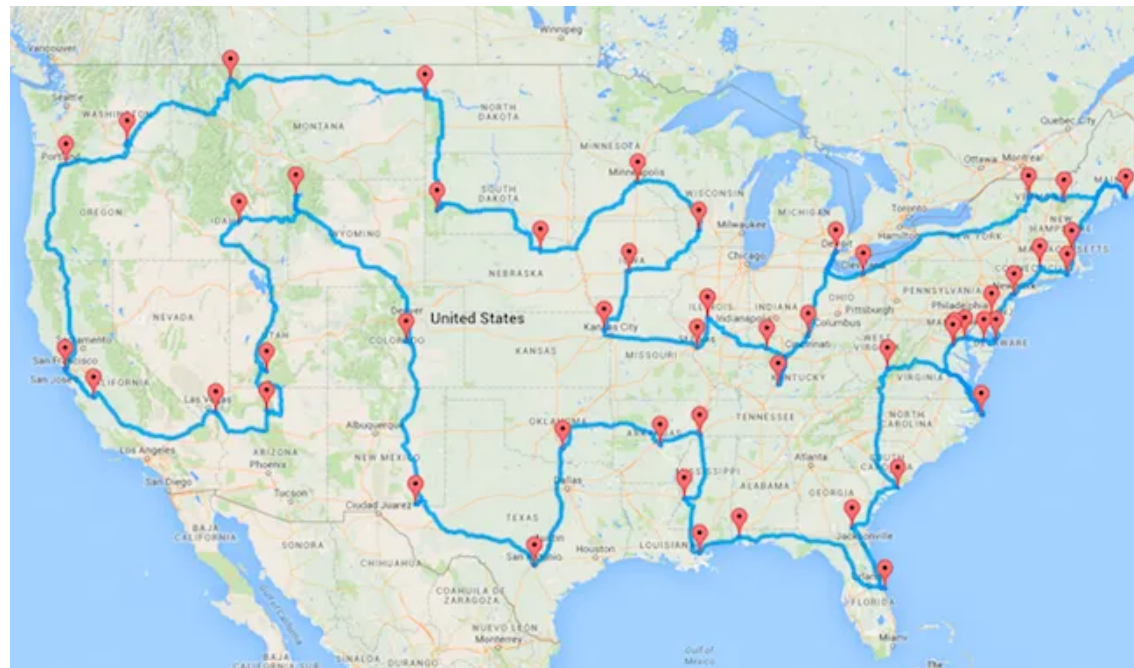
Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

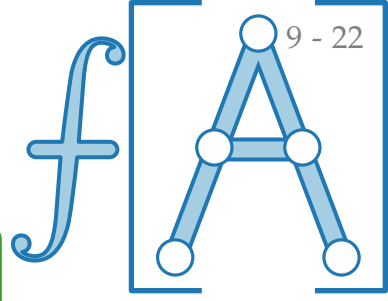
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Gehe immer zum nächsten Ort?
Nein ...

Quelle: <https://medium.com/@mrabetahmed/solving-travelling-salesman-problem-with-variable-neighborhood-search-ba89e4e7507>

Problem des Handlungsreisenden (TSP)

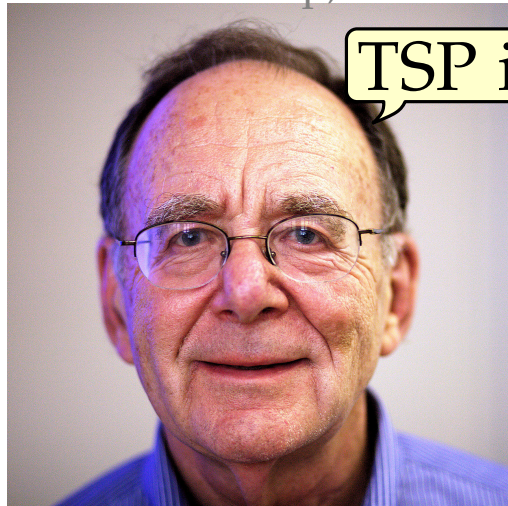


Problem. TRAVELINGSALESMANPROBLEM

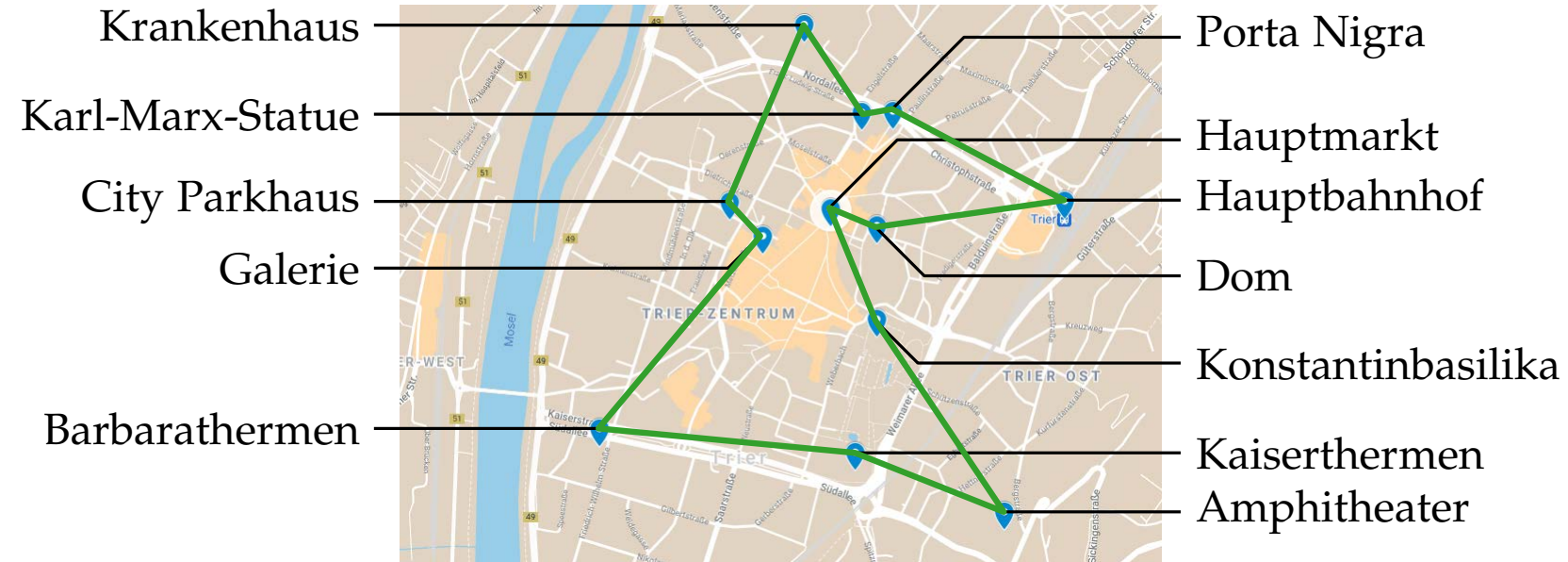
Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.

Richard M. Karp, 1972

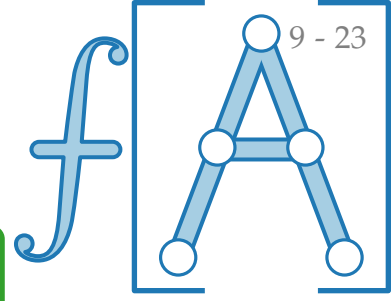


TSP ist NP-schwer.



Gehe immer zum nächsten Ort?
Nein ...

Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

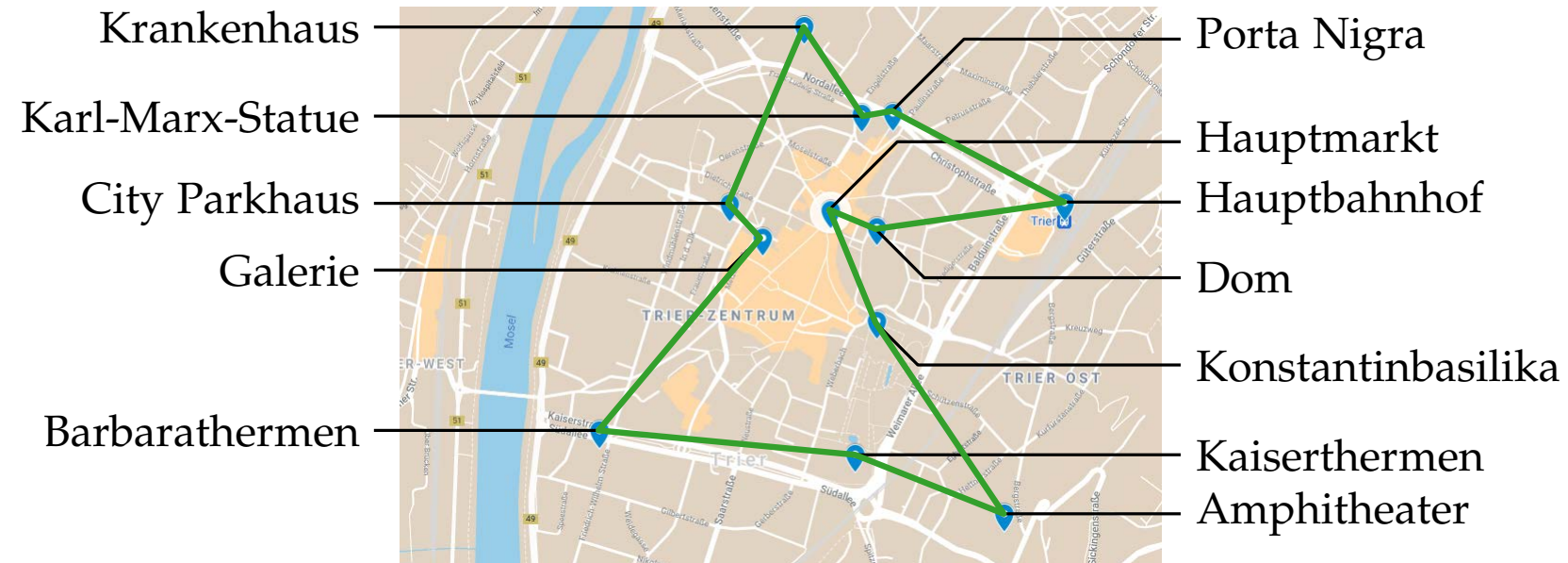
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.

Richard M. Karp, 1972



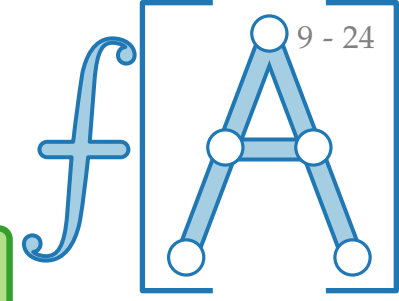
TSP ist NP-schwer.

Es ist sogar NP-schwer,
eine Lösung zu finden, die
„nur“ 1.000.000 mal so lang
ist wie die kürzeste.



Gehe immer zum nächsten Ort?
Nein ...

Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.

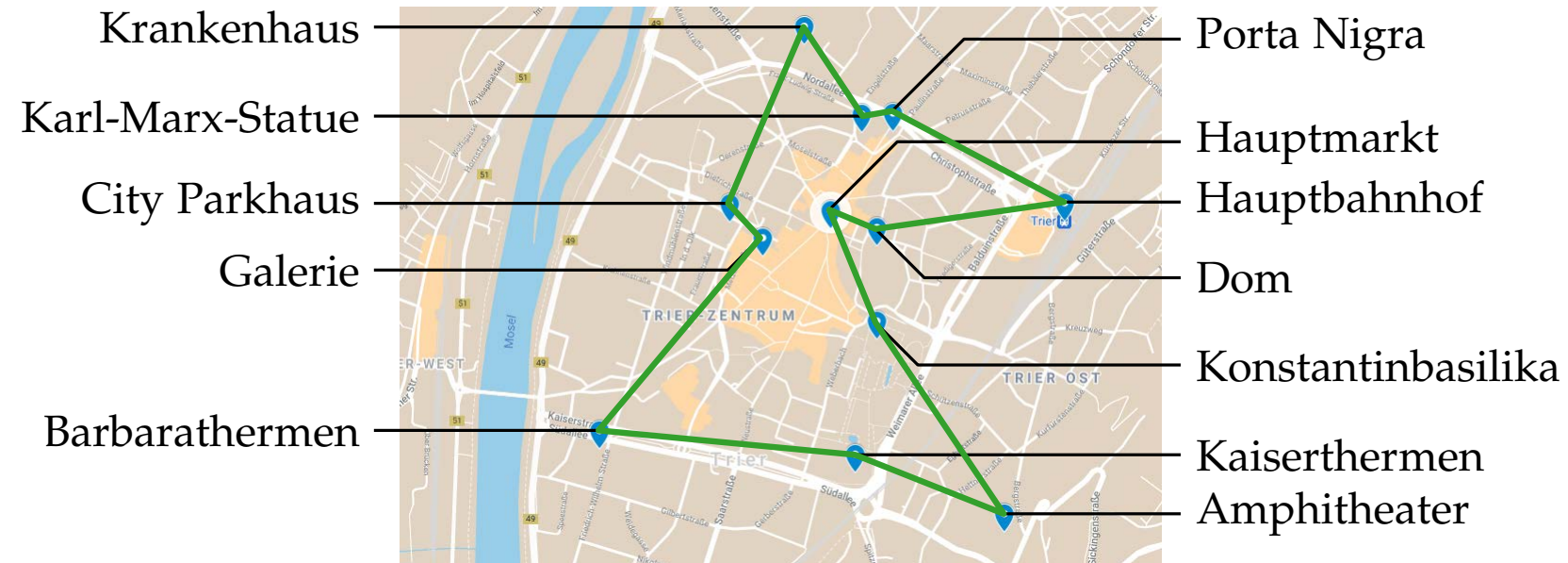
Richard M. Karp, 1972



TSP ist NP-schwer.

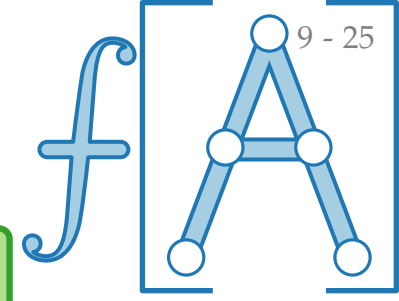
Es ist sogar NP-schwer, eine Lösung zu finden, die „nur“ 1.000.000 mal so lang ist wie die kürzeste.

Brute-force.



Gehe immer zum nächsten Ort?
Nein ...

Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.

Richard M. Karp, 1972

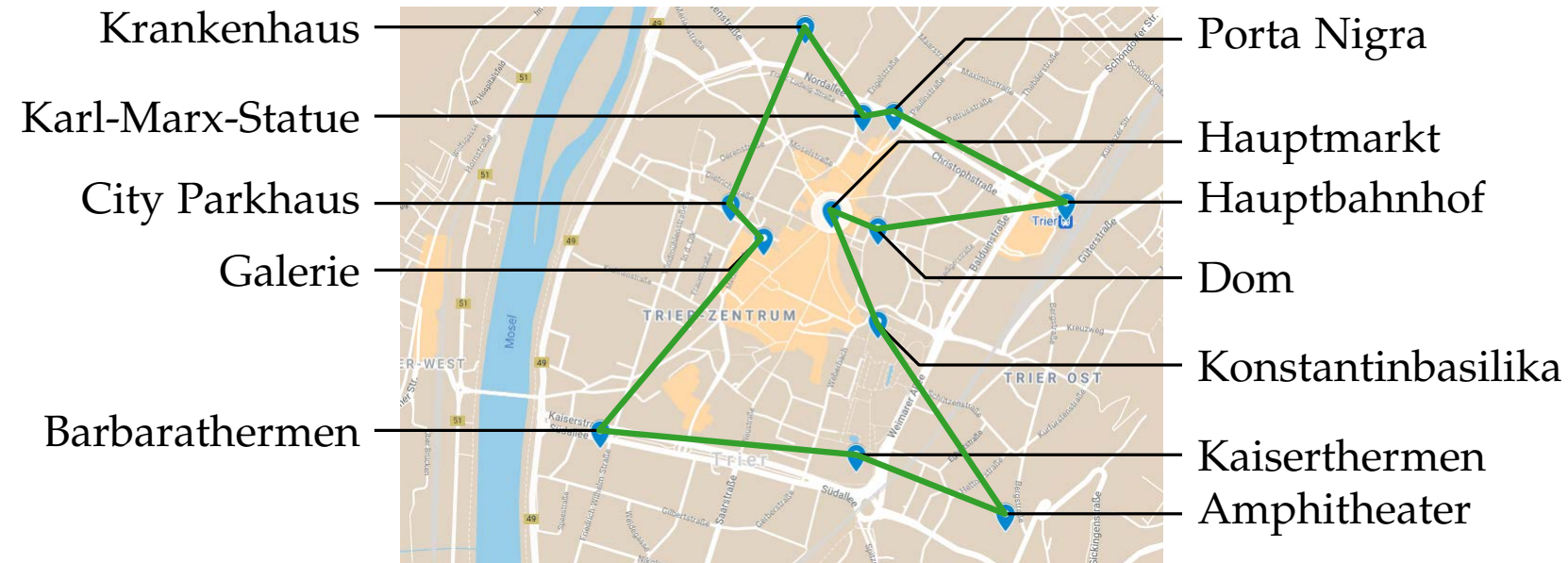


TSP ist NP-schwer.

Es ist sogar NP-schwer, eine Lösung zu finden, die „nur“ 1.000.000 mal so lang ist wie die kürzeste.

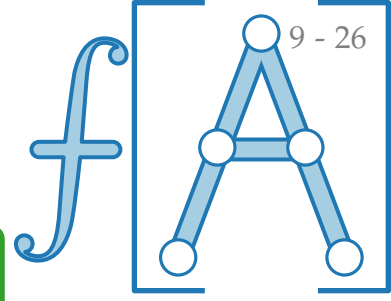
Brute-force.

- Probiere alle Permutationen und wähle eine mit kleinstem Gewicht.
- Laufzeit:



Gehe immer zum nächsten Ort?
Nein ...

Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.

Richard M. Karp, 1972

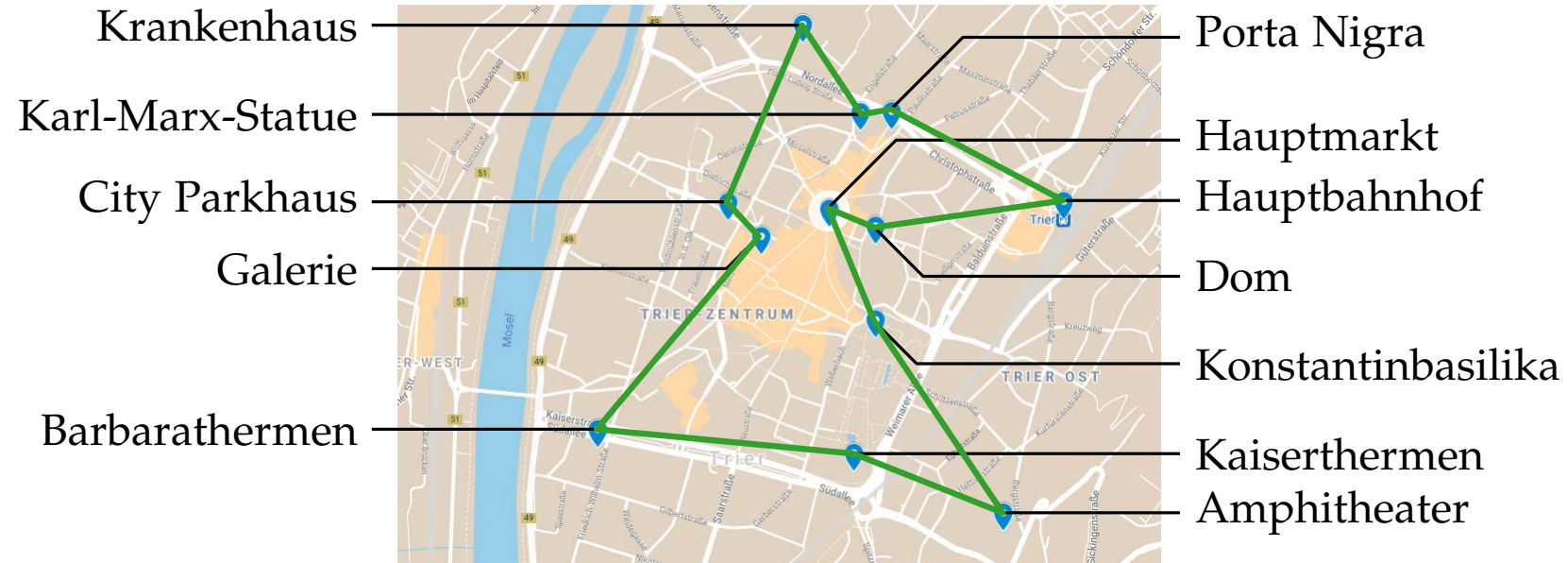


TSP ist NP-schwer.

Es ist sogar NP-schwer, eine Lösung zu finden, die „nur“ 1.000.000 mal so lang ist wie die kürzeste.

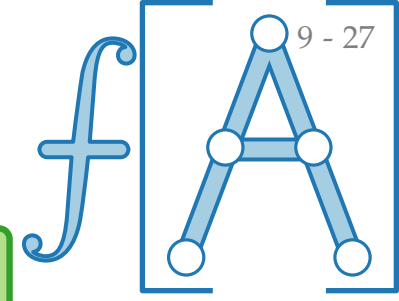
Brute-force.

- Probiere alle Permutationen und wähle eine mit kleinstem Gewicht.
- Laufzeit: $\Theta(n! \cdot n)$



Gehe immer zum nächsten Ort?
Nein ...

Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.

Richard M. Karp, 1972

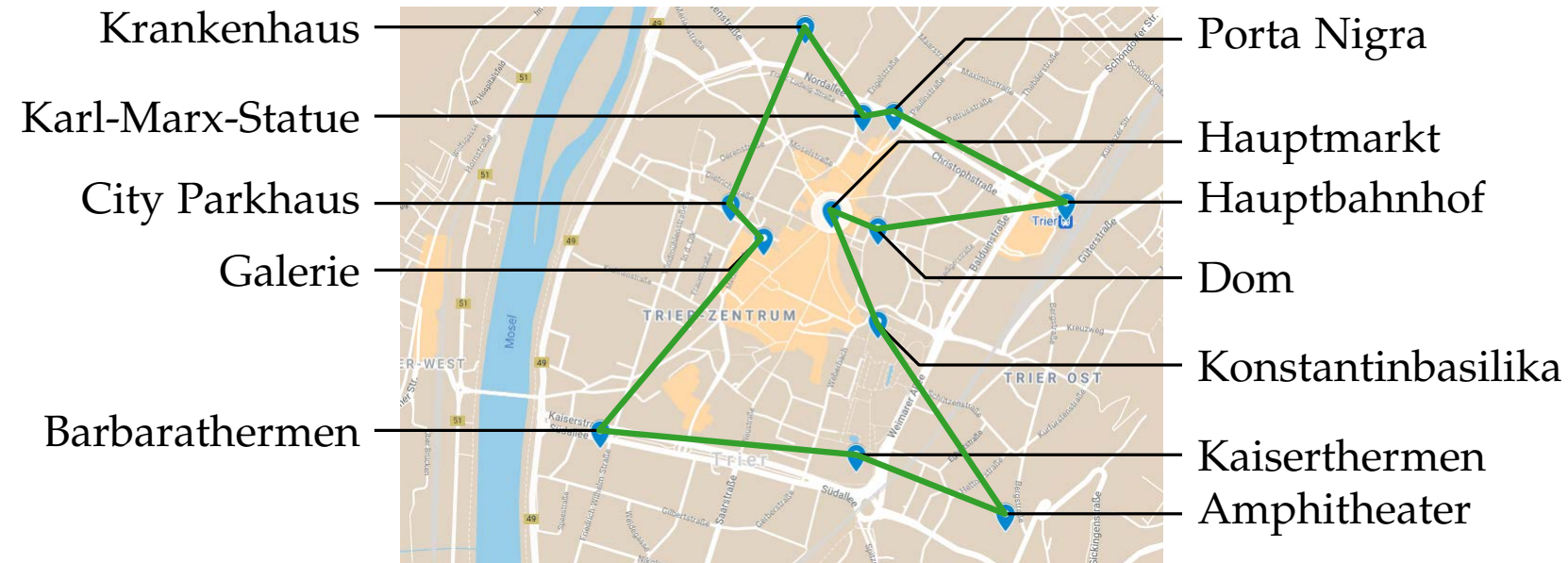


TSP ist NP-schwer.

Es ist sogar NP-schwer, eine Lösung zu finden, die „nur“ 1.000.000 mal so lang ist wie die kürzeste.

Brute-force.

- Probiere alle Permutationen und wähle eine mit kleinstem Gewicht.
- Laufzeit: $\Theta(n! \cdot n) = n \cdot 2^{\Theta(n \log n)}$

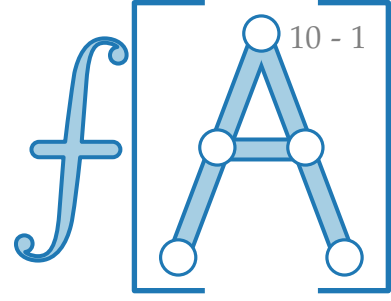


Gehe immer zum nächsten Ort?
Nein ...

Dynamisches Program – Idee

Idee.

(Bellman-Held-Karp
Algorithmus)



Richard M. Karp
1935 Boston, MA

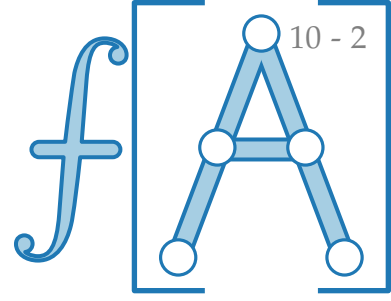


Richard E. Bellman
*1920 Brooklyn, NY
†1984 Los Angeles, CA



Dynamisches Program – Idee

(Bellman-Held-Karp
Algorithmus)



Idee.

- Verwende optimale Teilstrukturen mit dynamischer Programmierung wieder.

Richard M. Karp
1935 Boston, MA

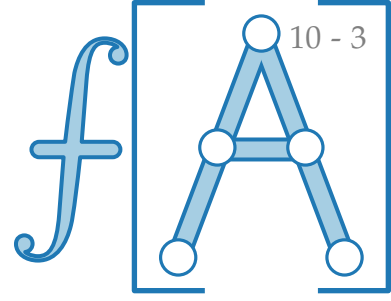


Richard E. Bellman
*1920 Brooklyn, NY
†1984 Los Angeles, CA



Dynamisches Program – Idee

(Bellman-Held-Karp
Algorithmus)



Idee.

- Verwende optimale Teilstrukturen mit dynamischer Programmierung wieder.
- Wähle einen Startknoten $s \in V$.

Richard M. Karp
1935 Boston, MA



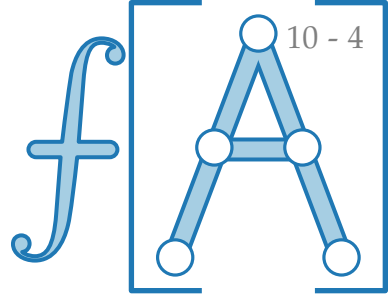
Richard E. Bellman
*1920 Brooklyn, NY
†1984 Los Angeles, CA



○
s

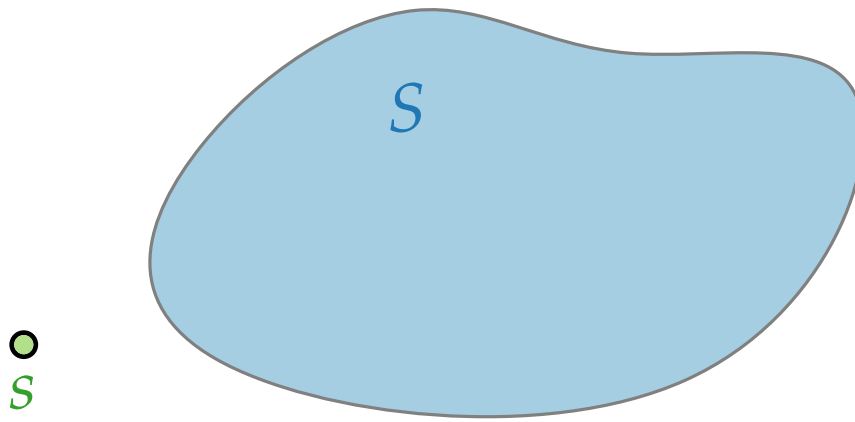
Dynamisches Program – Idee

(Bellman-Held-Karp
Algorithmus)



Idee.

- Verwende optimale Teilstrukturen mit dynamischer Programmierung wieder.
- Wähle einen Startknoten $s \in V$.
- Für jedes $S \subseteq V - s$



Richard M. Karp
1935 Boston, MA

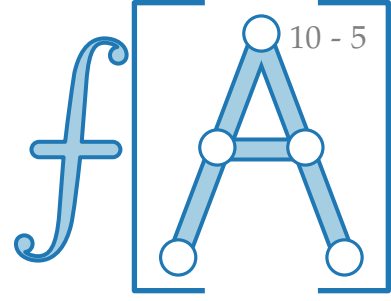


Richard E. Bellman
*1920 Brooklyn, NY
†1984 Los Angeles, CA



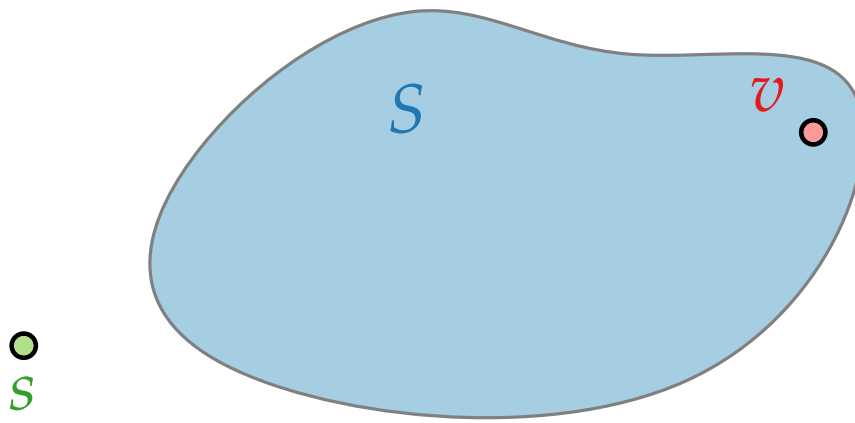
Dynamisches Program – Idee

(Bellman-Held-Karp
Algorithmus)



Idee.

- Verwende optimale Teilstrukturen mit dynamischer Programmierung wieder.
- Wähle einen Startknoten $s \in V$.
- Für jedes $S \subseteq V - s$ und $v \in S$ sei:



Richard M. Karp
1935 Boston, MA

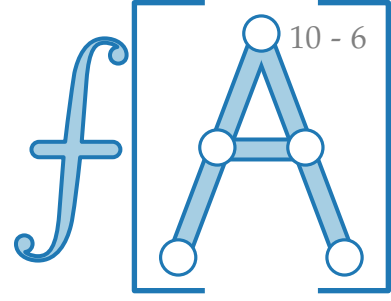


Richard E. Bellman
*1920 Brooklyn, NY
†1984 Los Angeles, CA



Dynamisches Program – Idee

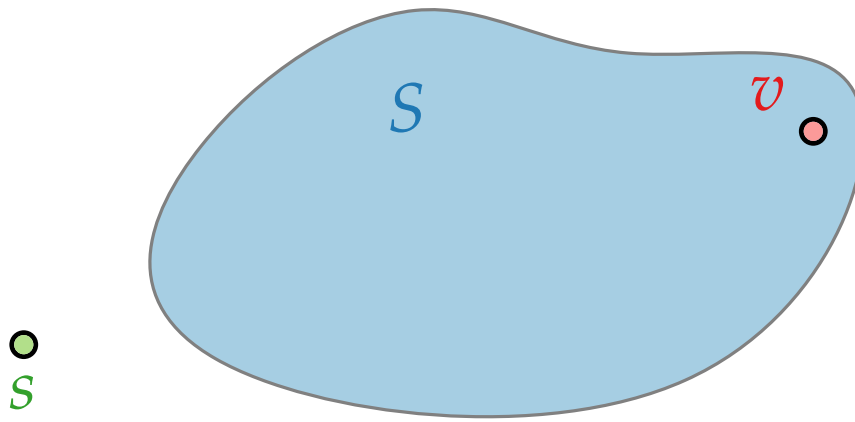
(Bellman-Held-Karp
Algorithmus)



Idee.

- Verwende optimale Teilstrukturen mit dynamischer Programmierung wieder.
- Wähle einen Startknoten $s \in V$.
- Für jedes $S \subseteq V - s$ und $v \in S$ sei:

$\text{OPT}[S, v]$ = Länge eines kürzesten s - v -Wegs,
der genau alle Knoten in $S \cup \{s\}$ besucht.



Richard M. Karp
1935 Boston, MA

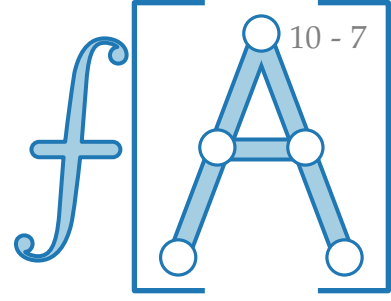


Richard E. Bellman
*1920 Brooklyn, NY
†1984 Los Angeles, CA



Dynamisches Program – Idee

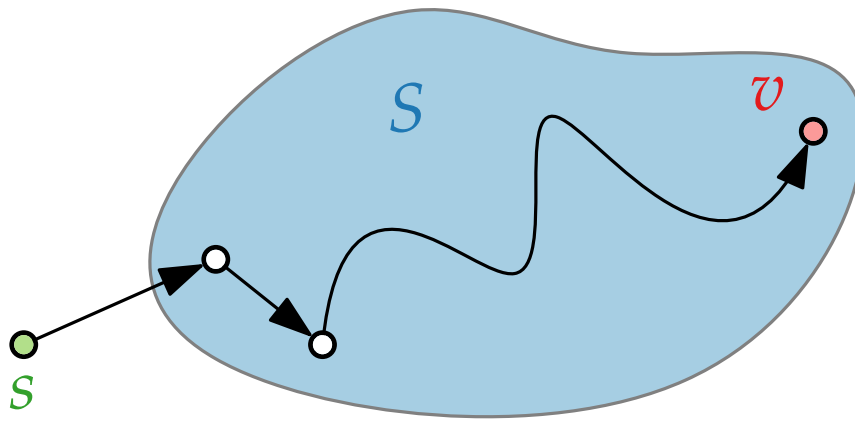
(Bellman-Held-Karp
Algorithmus)



Idee.

- Verwende optimale Teilstrukturen mit dynamischer Programmierung wieder.
- Wähle einen Startknoten $s \in V$.
- Für jedes $S \subseteq V - s$ und $v \in S$ sei:

$\text{OPT}[S, v]$ = Länge eines kürzesten s - v -Wegs,
der genau alle Knoten in $S \cup \{s\}$ besucht.



Richard M. Karp
1935 Boston, MA

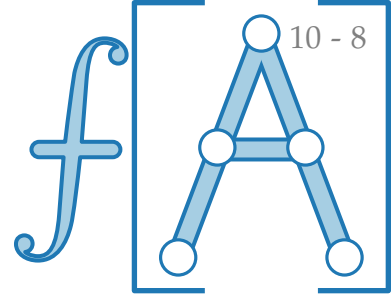


Richard E. Bellman
*1920 Brooklyn, NY
†1984 Los Angeles, CA



Dynamisches Programm – Idee

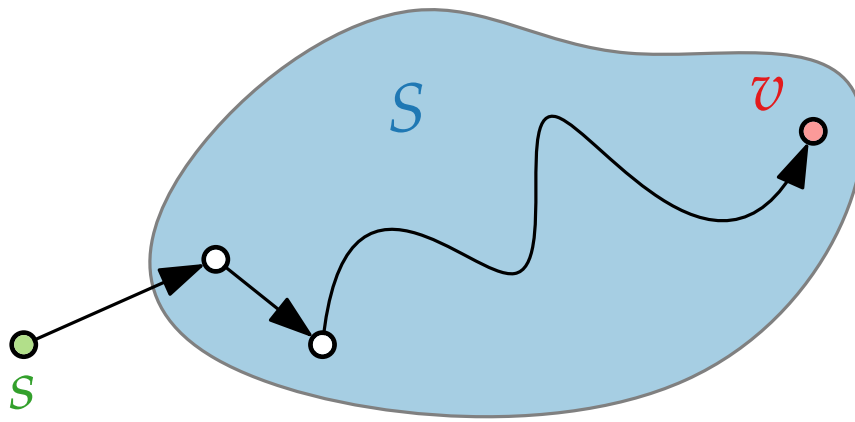
(Bellman-Held-Karp
Algorithmus)



Idee.

- Verwende optimale Teilstrukturen mit dynamischer Programmierung wieder.
- Wähle einen Startknoten $s \in V$.
- Für jedes $S \subseteq V - s$ und $v \in S$ sei:

$\text{OPT}[S, v]$ = Länge eines kürzesten s - v -Wegs,
der genau alle Knoten in $S \cup \{s\}$ besucht.



- Benutze $\text{OPT}[S - v, u]$, um $\text{OPT}[S, v]$ zu berechnen.

Richard M. Karp
1935 Boston, MA



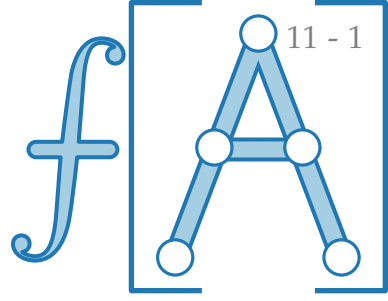
Richard E. Bellman
*1920 Brooklyn, NY
†1984 Los Angeles, CA



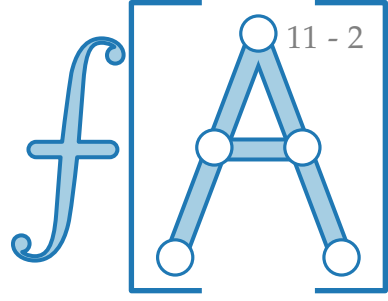
Dynamisches Program – Details

Details.

- Der Basisfall $S = \{v\}$ ist einfach: $\text{OPT}[\{v\}, v] = d(s, v)$.

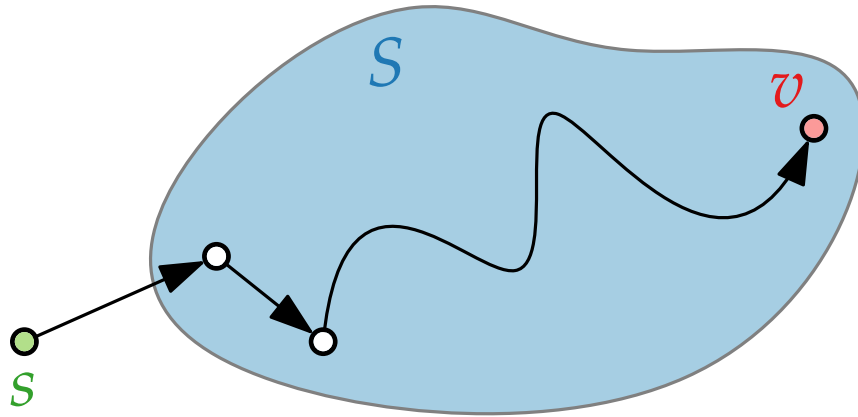


Dynamisches Program – Details

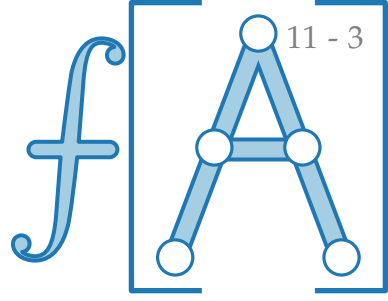


Details.

- Der Basisfall $S = \{v\}$ ist einfach: $\text{OPT}[\{v\}, v] = d(s, v)$.
- Wenn $|S| \geq 2$, dann berechne $\text{OPT}[S, v]$ rekursiv:



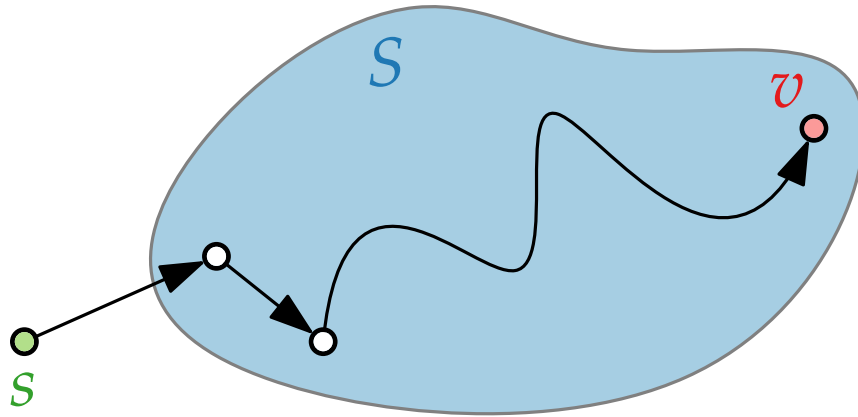
Dynamisches Program – Details



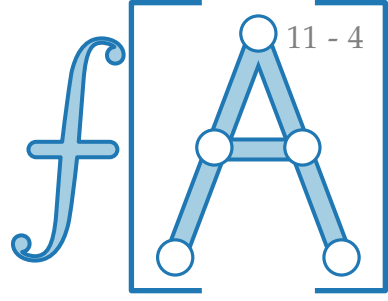
Details.

- Der Basisfall $S = \{v\}$ ist einfach: $\text{OPT}[\{v\}, v] = d(s, v)$.
- Wenn $|S| \geq 2$, dann berechne $\text{OPT}[S, v]$ rekursiv:

$$\text{OPT}[S, v] = \min\{ \quad \quad \quad \}$$



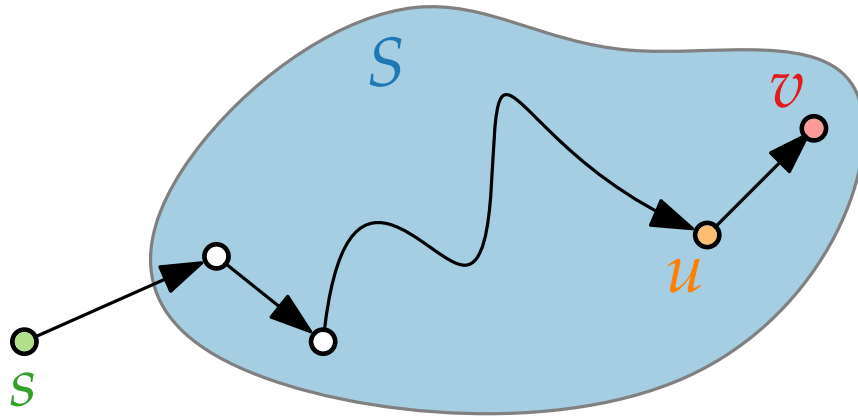
Dynamisches Programm – Details



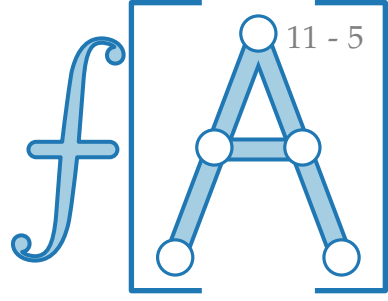
Details.

- Der Basisfall $S = \{v\}$ ist einfach: $\text{OPT}[\{v\}, v] = d(s, v)$.
- Wenn $|S| \geq 2$, dann berechne $\text{OPT}[S, v]$ rekursiv:

$$\text{OPT}[S, v] = \min_{u \in S - v} \{$$



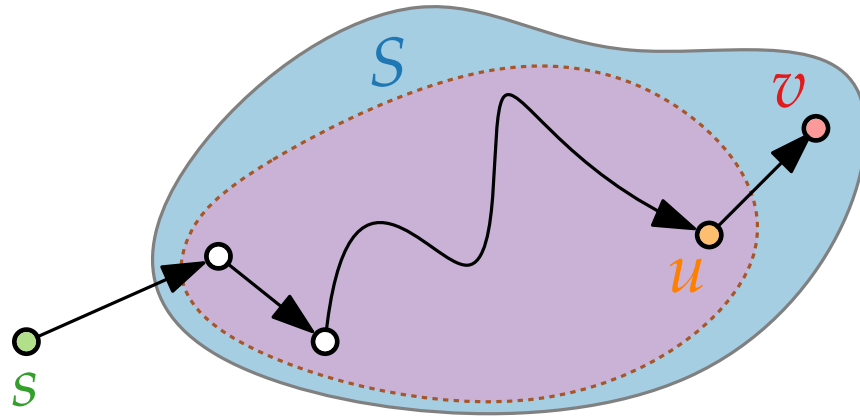
Dynamisches Programm – Details



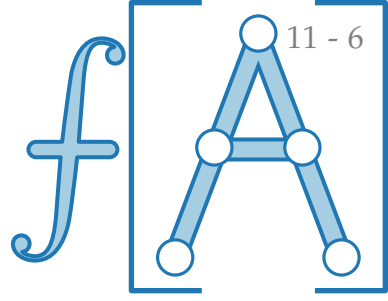
Details.

- Der Basisfall $S = \{v\}$ ist einfach: $\text{OPT}[\{v\}, v] = d(s, v)$.
- Wenn $|S| \geq 2$, dann berechne $\text{OPT}[S, v]$ rekursiv:

$$\text{OPT}[S, v] = \min\{\text{OPT}[S - v, u] \mid u \in S - v\}$$



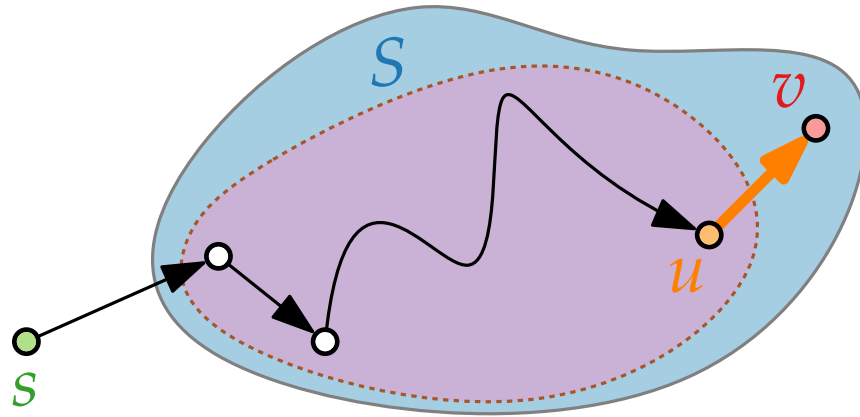
Dynamisches Program – Details



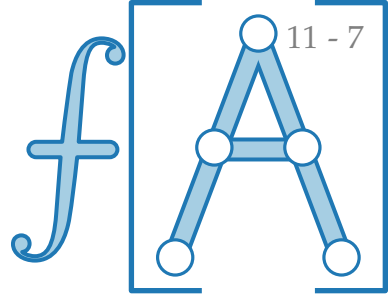
Details.

- Der Basisfall $S = \{v\}$ ist einfach: $\text{OPT}[\{v\}, v] = d(s, v)$.
- Wenn $|S| \geq 2$, dann berechne $\text{OPT}[S, v]$ rekursiv:

$$\text{OPT}[S, v] = \min\{\text{OPT}[S - v, u] + d(u, v) \mid u \in S - v\}$$



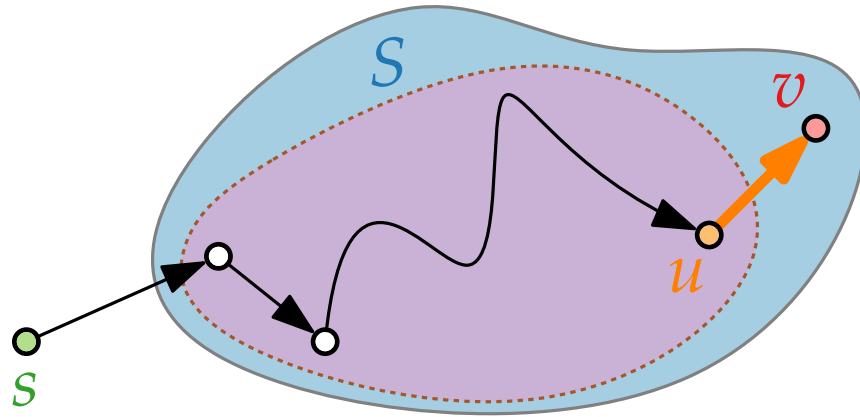
Dynamisches Programm – Details



Details.

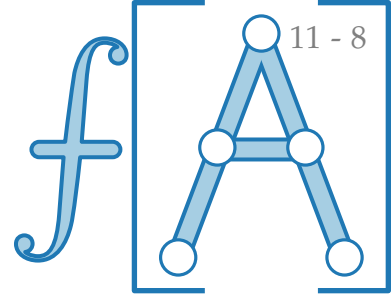
- Der Basisfall $S = \{v\}$ ist einfach: $\text{OPT}[\{v\}, v] = d(s, v)$.
- Wenn $|S| \geq 2$, dann berechne $\text{OPT}[S, v]$ rekursiv:

$$\text{OPT}[S, v] = \min\{\text{OPT}[S - v, u] + d(u, v) \mid u \in S - v\}$$



- Nach dem Berechnen von $\text{OPT}[S, v]$ für jedes $S \subseteq V - s$ und jedes $v \in V - s$ kann man die optimale Lösung so finden:

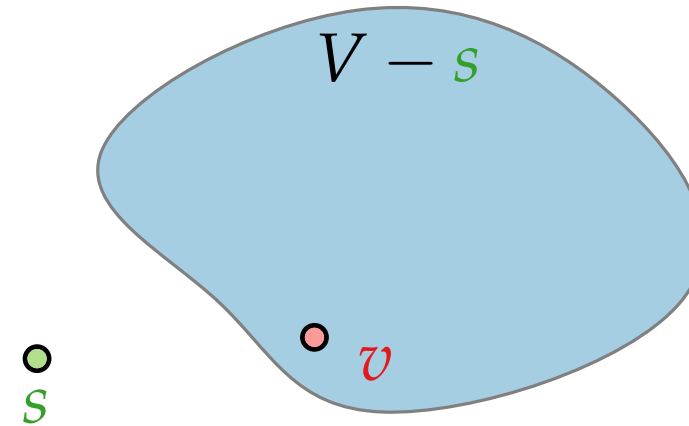
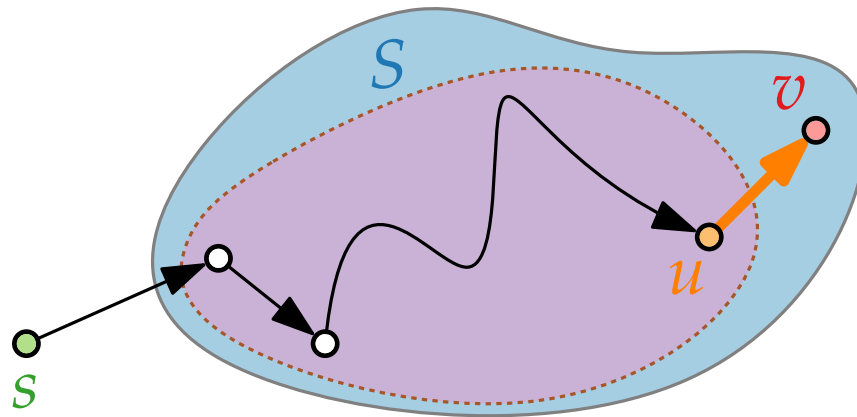
Dynamisches Programm – Details



Details.

- Der Basisfall $S = \{v\}$ ist einfach: $\text{OPT}[\{v\}, v] = d(s, v)$.
- Wenn $|S| \geq 2$, dann berechne $\text{OPT}[S, v]$ rekursiv:

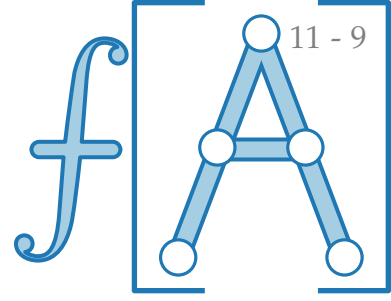
$$\text{OPT}[S, v] = \min\{\text{OPT}[S - v, u] + d(u, v) \mid u \in S - v\}$$



- Nach dem Berechnen von $\text{OPT}[S, v]$ für jedes $S \subseteq V - s$ und jedes $v \in V - s$ kann man die optimale Lösung so finden:

$$\text{OPT} = \min\{ \quad \mid v \in V - s \}$$

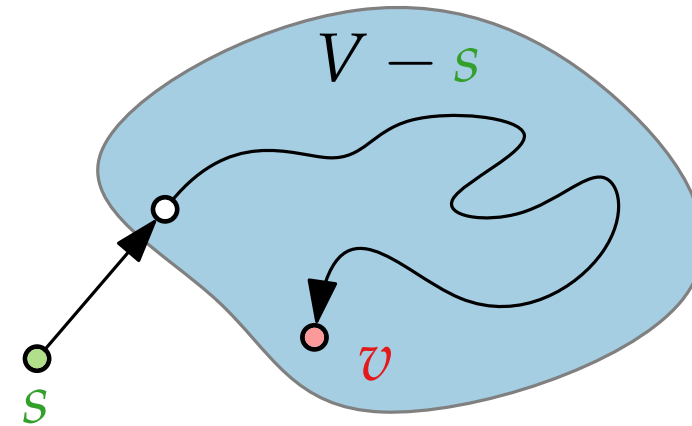
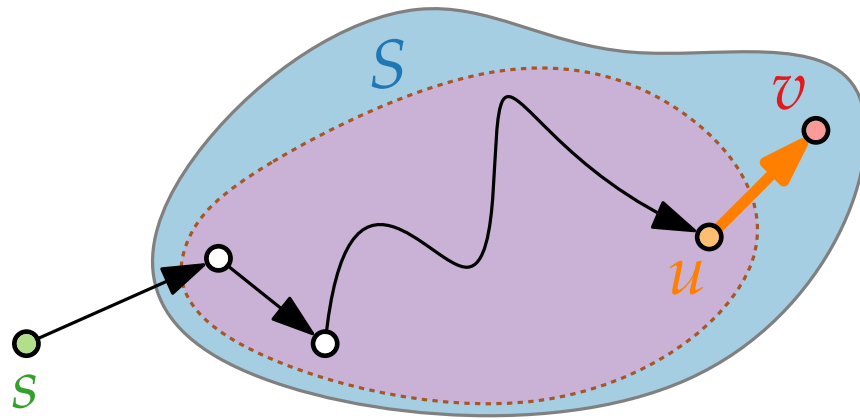
Dynamisches Programm – Details



Details.

- Der Basisfall $S = \{v\}$ ist einfach: $\text{OPT}[\{v\}, v] = d(s, v)$.
- Wenn $|S| \geq 2$, dann berechne $\text{OPT}[S, v]$ rekursiv:

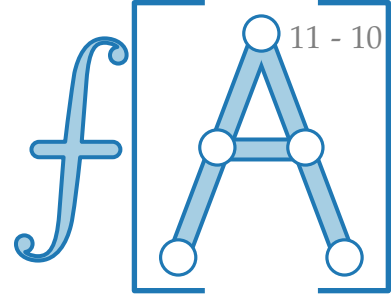
$$\text{OPT}[S, v] = \min\{\text{OPT}[S - v, u] + d(u, v) \mid u \in S - v\}$$



- Nach dem Berechnen von $\text{OPT}[S, v]$ für jedes $S \subseteq V - s$ und jedes $v \in V - s$ kann man die optimale Lösung so finden:

$$\text{OPT} = \min\{\text{OPT}[V - s, v] \mid v \in V - s\}$$

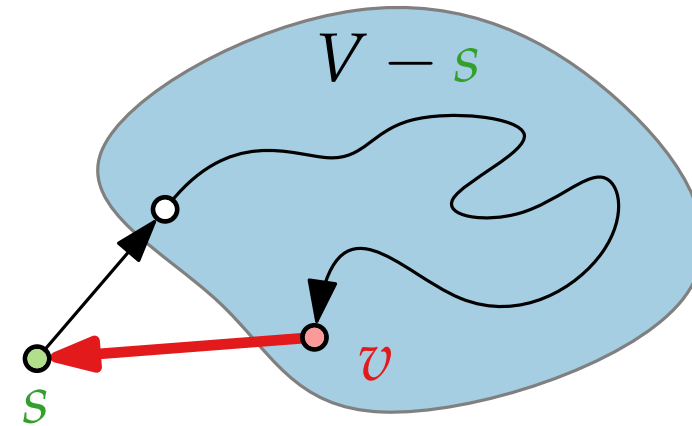
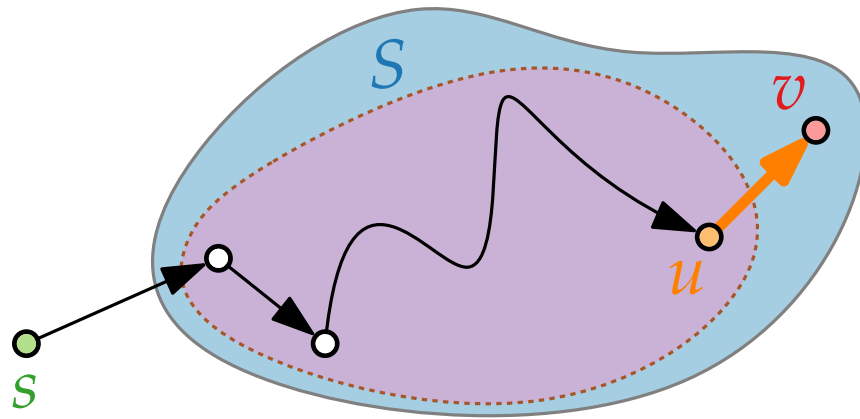
Dynamisches Programm – Details



Details.

- Der Basisfall $S = \{v\}$ ist einfach: $\text{OPT}[\{v\}, v] = d(s, v)$.
- Wenn $|S| \geq 2$, dann berechne $\text{OPT}[S, v]$ rekursiv:

$$\text{OPT}[S, v] = \min\{\text{OPT}[S - v, u] + d(u, v) \mid u \in S - v\}$$

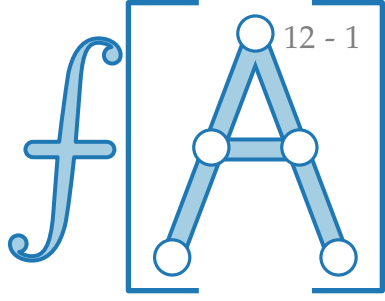
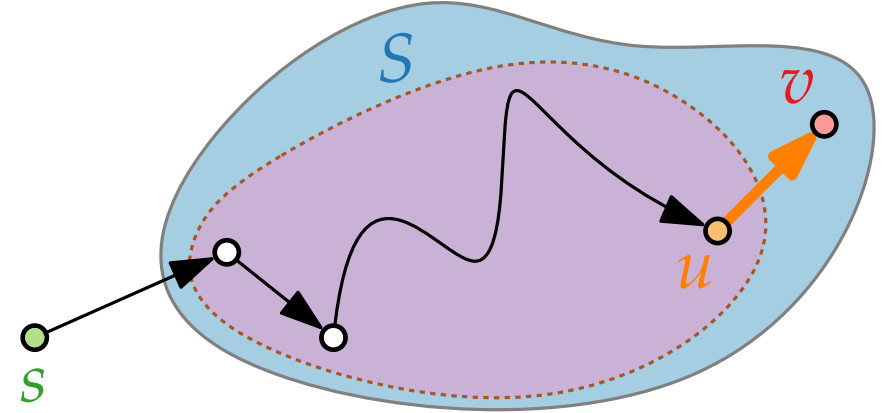


- Nach dem Berechnen von $\text{OPT}[S, v]$ für jedes $S \subseteq V - s$ und jedes $v \in V - s$ kann man die optimale Lösung so finden:

$$\text{OPT} = \min\{\text{OPT}[V - s, v] + d(v, s) \mid v \in V - s\}$$

Dynamisches Program – Analyse

BELLMANNHELDKARP(G, d)

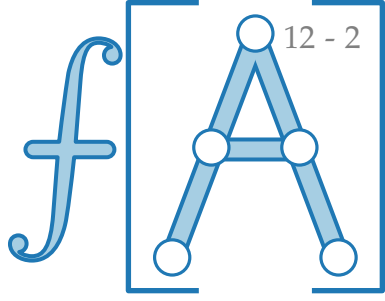
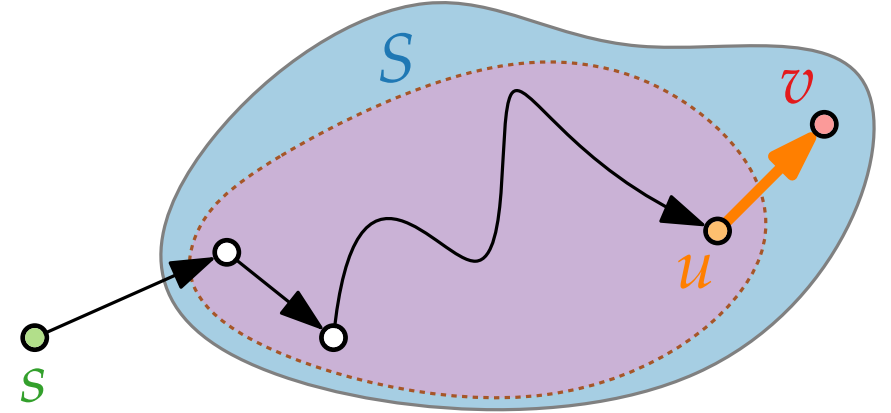


Dynamisches Program – Analyse

BELLMANNHELDKARP(G, d)

 foreach $v \in V - s$ do

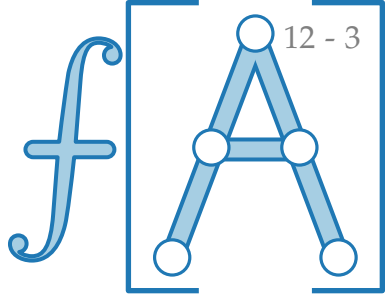
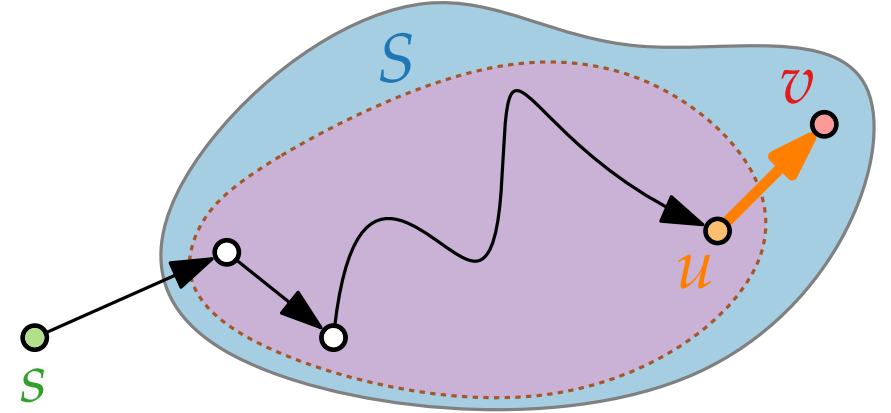
 └



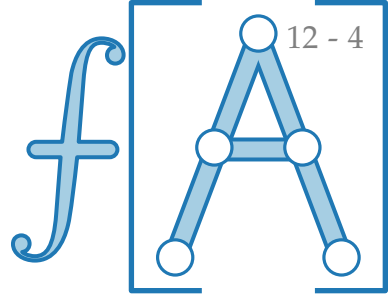
Dynamisches Program – Analyse

BELLMANNHELDKARP(G, d)

foreach $v \in V - s$ **do**
 $\text{OPT}[\{v\}, v] = d(s, v)$



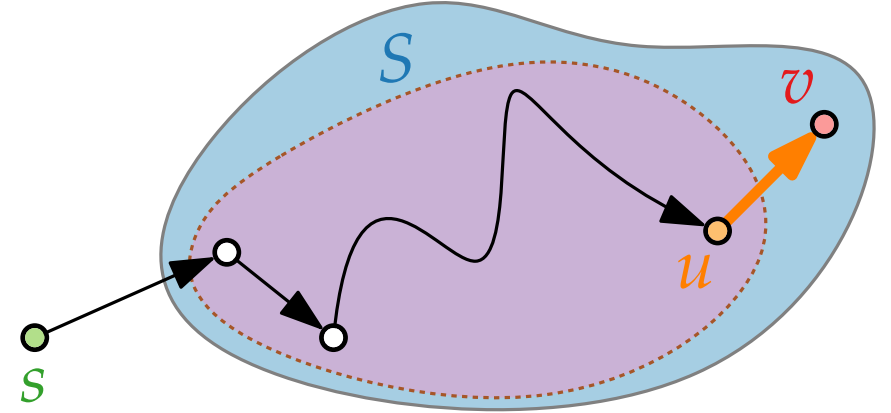
Dynamisches Program – Analyse



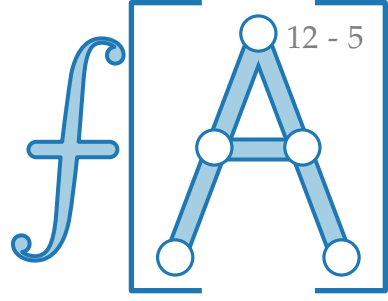
BELLMANNHELDKARP(G, d)

foreach $v \in V - s$ **do**
 $\text{OPT}[\{v\}, v] = d(s, v)$

for $j = 2$ **to** $n - 1$ **do**



Dynamisches Program – Analyse



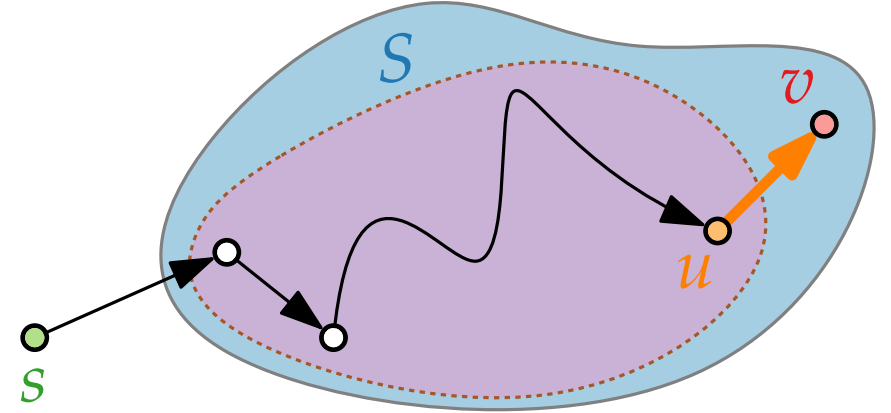
BELLMANNHELDKARP(G, d)

foreach $v \in V - s$ do

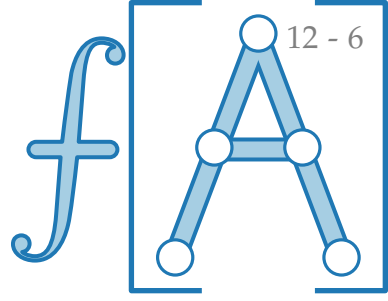
└ OPT[{ v }, v] = $d(s, v)$

for $j = 2$ to $n - 1$ do

└ foreach $S \subseteq V - s$ with $|S| = j$ do



Dynamisches Program – Analyse



BELLMANNHELDKARP(G, d)

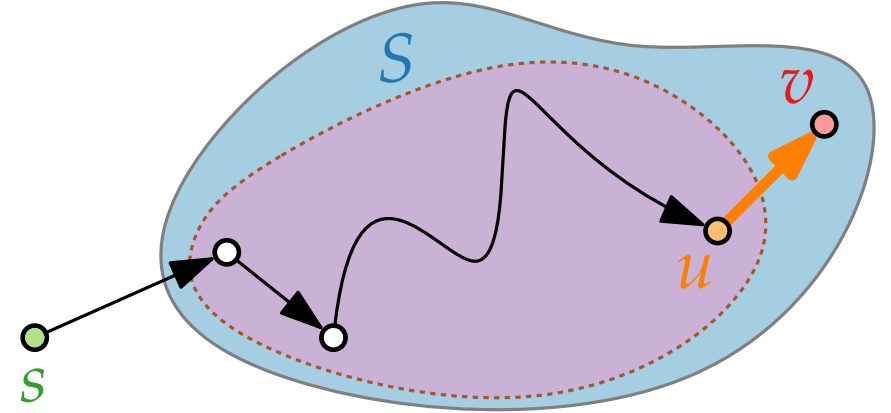
foreach $v \in V - s$ do

└ OPT[$\{v\}, v$] = $d(s, v)$

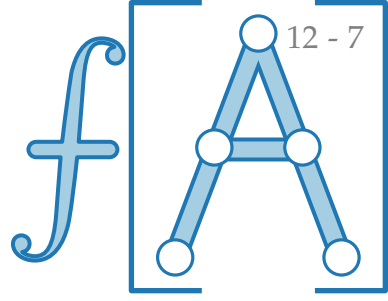
for $j = 2$ to $n - 1$ do

└ foreach $S \subseteq V - s$ with $|S| = j$ do

└└ foreach $v \in S$ do



Dynamisches Program – Analyse



BELLMANNHELDKARP(G, d)

foreach $v \in V - s$ **do**

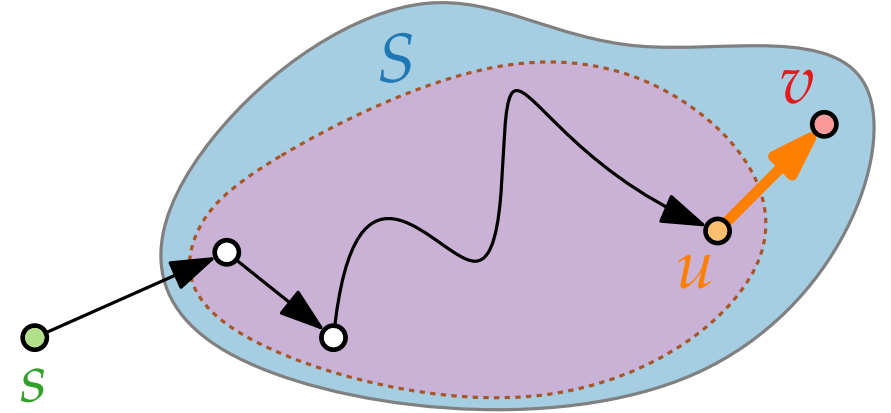
└ $\text{OPT}[\{v\}, v] = d(s, v)$

for $j = 2$ **to** $n - 1$ **do**

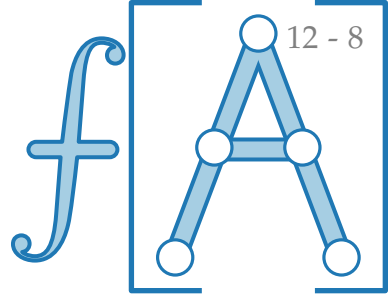
└ **foreach** $S \subseteq V - s$ with $|S| = j$ **do**

└ **foreach** $v \in S$ **do**

└ $\text{OPT}[S, v] = \min \{ \text{OPT}[S - v, u] + d(u, v) \mid u \in S - v \}$



Dynamisches Program – Analyse



BELLMANNHELDKARP(G, d)

foreach $v \in V - s$ **do**

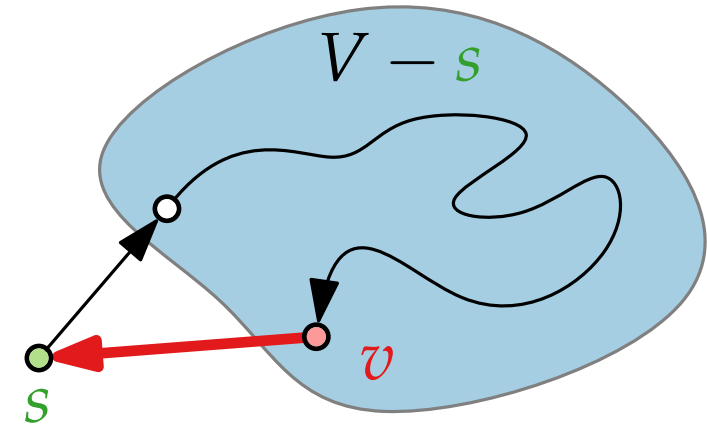
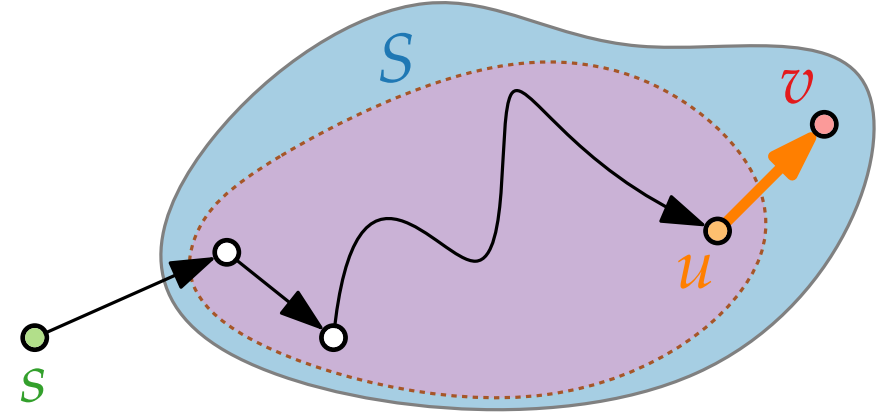
└ $\text{OPT}[\{v\}, v] = d(s, v)$

for $j = 2$ **to** $n - 1$ **do**

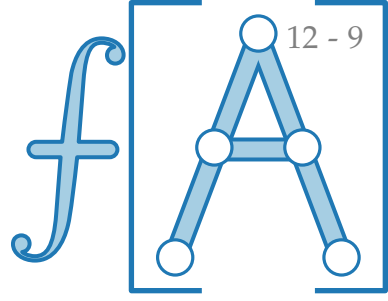
└ **foreach** $S \subseteq V - s$ with $|S| = j$ **do**

└ **foreach** $v \in S$ **do**

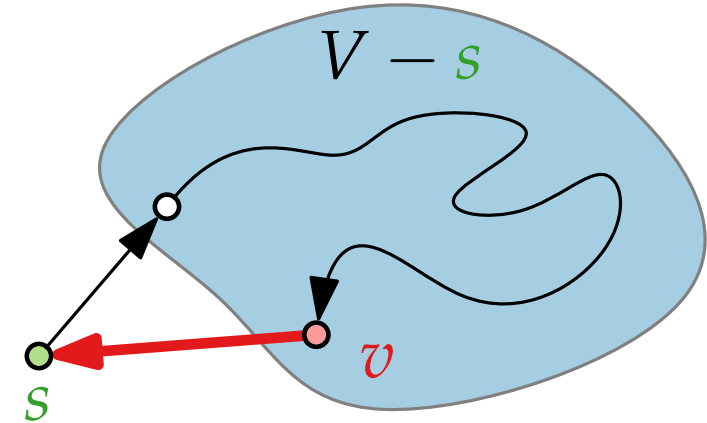
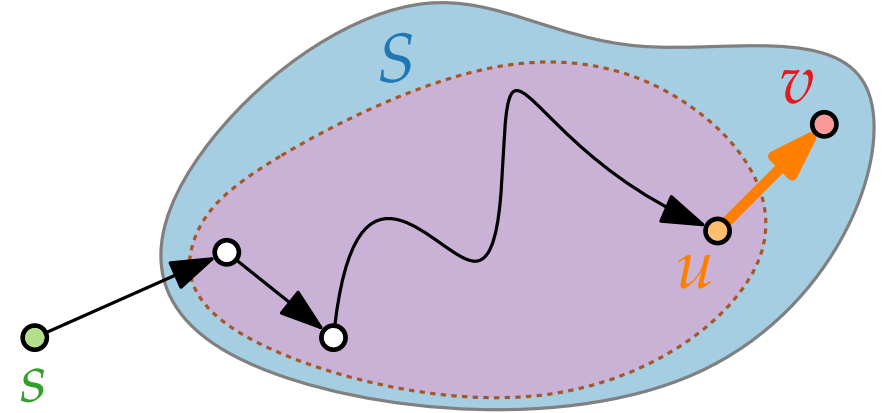
└ $\text{OPT}[S, v] = \min \{ \text{OPT}[S - v, u] + d(u, v) \mid u \in S - v \}$



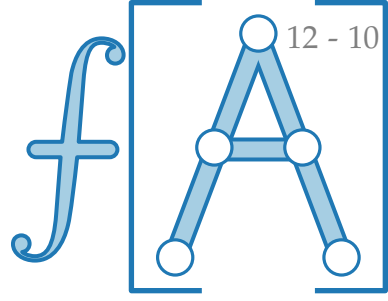
Dynamisches Program – Analyse



```
BELLMANNHELDKARP( $G, d$ )  
  foreach  $v \in V - s$  do  
     $\text{OPT}[\{v\}, v] = d(s, v)$   
  for  $j = 2$  to  $n - 1$  do  
    foreach  $S \subseteq V - s$  with  $|S| = j$  do  
      foreach  $v \in S$  do  
         $\text{OPT}[S, v] = \min\{ \text{OPT}[S - v, u] + d(u, v) \mid u \in S - v \}$   
  return  $\min\{ \text{OPT}[V - s, v] + d(v, s) \mid v \in V - s \}$ 
```

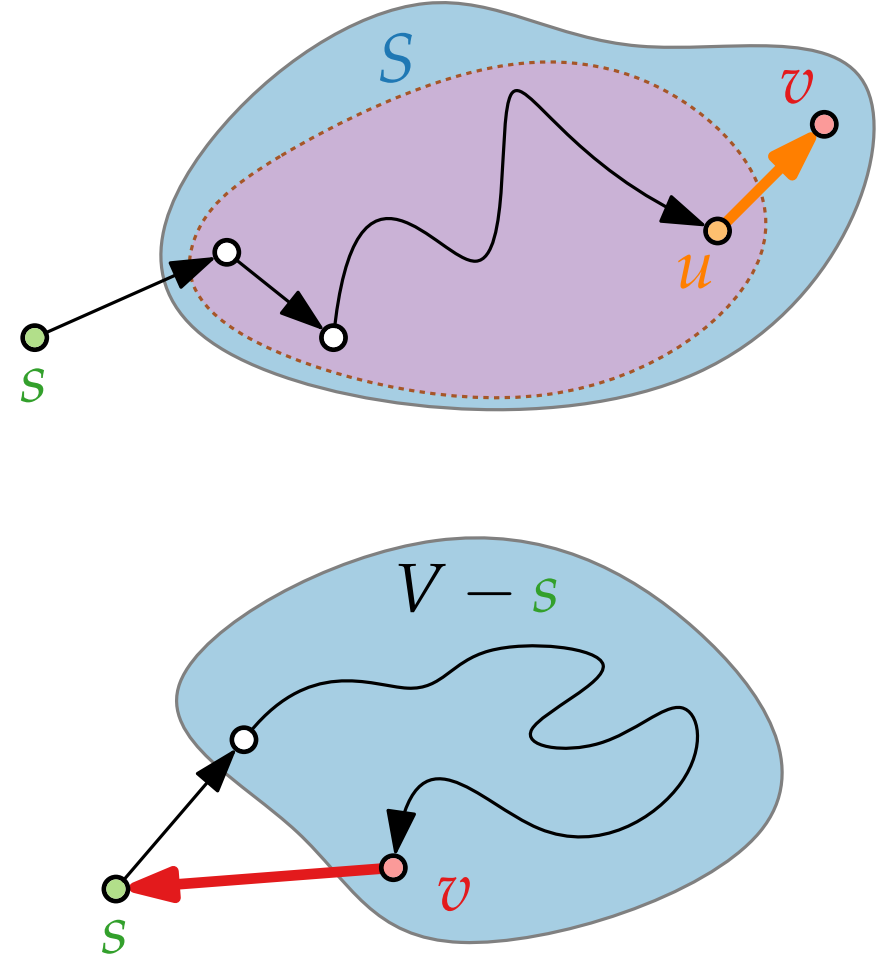


Dynamisches Programm – Analyse

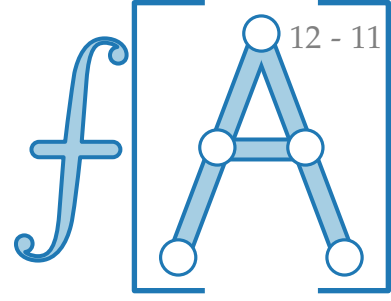


```
BELLMANNHELDKARP( $G, d$ )  
  foreach  $v \in V - s$  do  
     $\text{OPT}[\{v\}, v] = d(s, v)$   
  for  $j = 2$  to  $n - 1$  do  
    foreach  $S \subseteq V - s$  with  $|S| = j$  do  
      foreach  $v \in S$  do  
         $\text{OPT}[S, v] = \min\{ \text{OPT}[S - v, u] + d(u, v) \mid u \in S - v \}$   
  return  $\min\{ \text{OPT}[V - s, v] + d(v, s) \mid v \in V - s \}$ 
```

- Eine kürzeste Tour kann durch Backtracking der DP Tabelle ausgegeben werden (wie üblich).



Dynamisches Program – Analyse

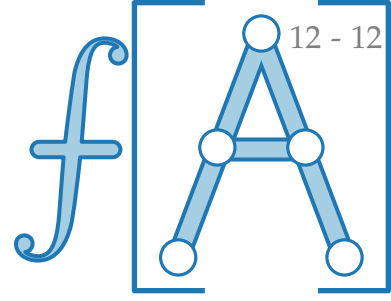


Analyse.

```
BELLMANNHELDKARP( $G, d$ )  
  foreach  $v \in V - s$  do  
     $\text{OPT}[\{v\}, v] = d(s, v)$   
  for  $j = 2$  to  $n - 1$  do  
    foreach  $S \subseteq V - s$  with  $|S| = j$  do  
      foreach  $v \in S$  do  
         $\text{OPT}[S, v] = \min\{ \text{OPT}[S - v, u]$   
           $+ d(u, v) \mid u \in S - v \}$   
  return  $\min\{ \text{OPT}[V - s, v] + d(v, s) \mid v \in V - s \}$ 
```

- Eine kürzeste Tour kann durch Backtracking der DP Tabelle ausgegeben werden (wie üblich).

Dynamisches Program – Analyse

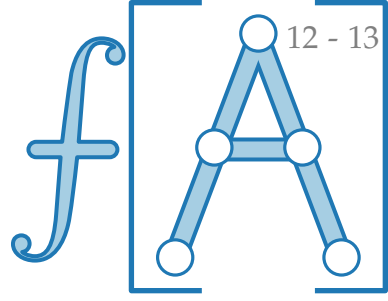


Analyse.

```
BELLMANNHELDKARP( $G, d$ )  
  foreach  $v \in V - s$  do  
     $\text{OPT}[\{v\}, v] = d(s, v)$   
  for  $j = 2$  to  $n - 1$  do  
    foreach  $S \subseteq V - s$  with  $|S| = j$  do  
      foreach  $v \in S$  do  
         $\text{OPT}[S, v] = \min\{ \text{OPT}[S - v, u]$   
           $+ d(u, v) \mid u \in S - v \}$   
  return  $\min\{ \text{OPT}[V - s, v] + d(v, s) \mid v \in V - s \}$ 
```

- Eine kürzeste Tour kann durch Backtracking der DP Tabelle ausgegeben werden (wie üblich).

Dynamisches Program – Analyse

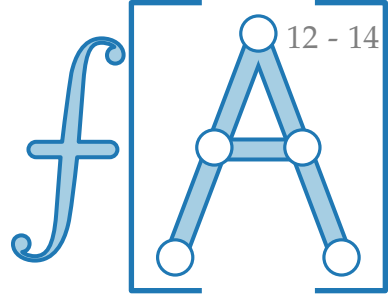


Analyse.

```
BELLMANNHELDKARP( $G, d$ )  
  foreach  $v \in V - s$  do  
     $\text{OPT}[\{v\}, v] = d(s, v)$   
  for  $j = 2$  to  $n - 1$  do  
    foreach  $S \subseteq V - s$  with  $|S| = j$  do  
      foreach  $v \in S$  do  
         $\text{OPT}[S, v] = \min\{ \text{OPT}[S - v, u]$   
           $+ d(u, v) \mid u \in S - v \}$   
      }  $\mathcal{O}(n)$   
  return  $\min\{ \text{OPT}[V - s, v] + d(v, s) \mid v \in V - s \}$ 
```

- Eine kürzeste Tour kann durch Backtracking der DP Tabelle ausgegeben werden (wie üblich).

Dynamisches Program – Analyse

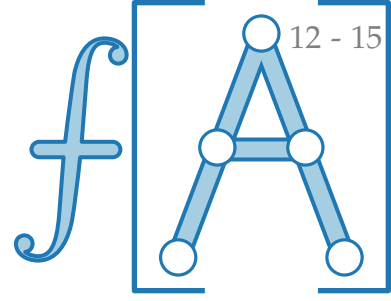


Analyse.

```
BELLMANNHELDKARP( $G, d$ )  
  foreach  $v \in V - s$  do  
     $\text{OPT}[\{v\}, v] = d(s, v)$   
  for  $j = 2$  to  $n - 1$  do  
    foreach  $S \subseteq V - s$  with  $|S| = j$  do  
      foreach  $v \in S$  do  
         $\text{OPT}[S, v] = \min\{ \text{OPT}[S - v, u]$   
           $+ d(u, v) \mid u \in S - v \}$   
      }  $\mathcal{O}(n)$   
  return  $\min\{ \text{OPT}[V - s, v] + d(v, s) \mid v \in V - s \}$ 
```

- Eine kürzeste Tour kann durch Backtracking der DP Tabelle ausgegeben werden (wie üblich).

Dynamisches Programm – Analyse



Analyse.

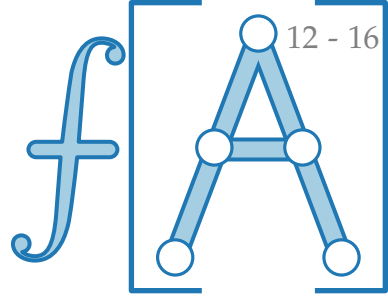
```
BELLMANHELDKARP( $G, d$ )  
  foreach  $v \in V - s$  do  
     $\text{OPT}[\{v\}, v] = d(s, v)$   
  for  $j = 2$  to  $n - 1$  do  
    foreach  $S \subseteq V - s$  with  $|S| = j$  do  
      foreach  $v \in S$  do  
         $\text{OPT}[S, v] = \min\{ \text{OPT}[S - v, u] + d(u, v) \mid u \in S - v \}$   
  return  $\min\{ \text{OPT}[V - s, v] + d(v, s) \mid v \in V - s \}$ 
```

$\mathcal{O}(2^n)$

$\mathcal{O}(n)$

- Eine kürzeste Tour kann durch Backtracking der DP Tabelle ausgegeben werden (wie üblich).

Dynamisches Program – Analyse



```
BELLMANNHELDKARP( $G, d$ )
```

```
  foreach  $v \in V - s$  do
```

```
    OPT[ $\{v\}, v$ ] =  $d(s, v)$ 
```

```
  for  $j = 2$  to  $n - 1$  do
```

```
    foreach  $S \subseteq V - s$  with  $|S| = j$  do
```

```
      foreach  $v \in S$  do
```

```
        OPT[ $S, v$ ] = min{ OPT[ $S - v, u$ ]  
                        +  $d(u, v)$  |  $u \in S - v$  }
```

```
  return min{ OPT[ $V - s, v$ ] +  $d(v, s)$  |  $v \in V - s$  }
```

} $\mathcal{O}(2^n)$

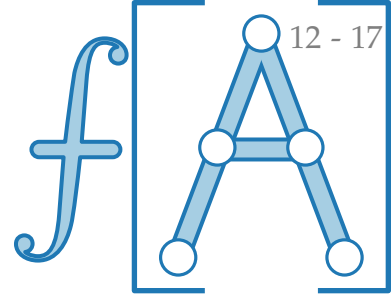
} $\mathcal{O}(n)$

Analyse.

- Innerste Schleife wird $\mathcal{O}(2^n \cdot n)$ mal ausgeführt

- Eine kürzeste Tour kann durch Backtracking der DP Tabelle ausgegeben werden (wie üblich).

Dynamisches Program – Analyse



```
BELLMANNHELDKARP( $G, d$ )
```

```
  foreach  $v \in V - s$  do
```

```
    OPT[{ $v$ },  $v$ ] =  $d(s, v)$ 
```

```
  for  $j = 2$  to  $n - 1$  do
```

```
    foreach  $S \subseteq V - s$  with  $|S| = j$  do
```

```
      foreach  $v \in S$  do
```

```
        OPT[ $S, v$ ] = min{ OPT[ $S - v, u$ ]  
                        +  $d(u, v)$  |  $u \in S - v$  }
```

```
  return min{ OPT[ $V - s, v$ ] +  $d(v, s)$  |  $v \in V - s$  }
```

} $\mathcal{O}(2^n)$

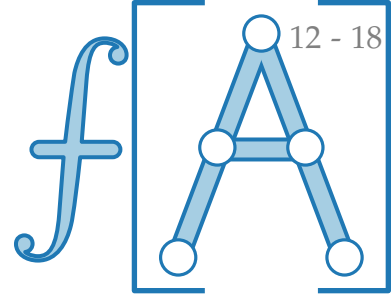
} $\mathcal{O}(n)$

Analyse.

- Innerste Schleife wird $\mathcal{O}(2^n \cdot n)$ mal ausgeführt
- Jede Ausführung braucht $\mathcal{O}(n)$ Zeit

- Eine kürzeste Tour kann durch Backtracking der DP Tabelle ausgegeben werden (wie üblich).

Dynamisches Program – Analyse



BELLMANNHELDKARP(G, d)

 foreach $v \in V - s$ do

$\lfloor$ OPT[$\{v\}, v$] = $d(s, v)$

 for $j = 2$ to $n - 1$ do

 foreach $S \subseteq V - s$ with $|S| = j$ do

 foreach $v \in S$ do

$\lfloor$ OPT[S, v] = min{ OPT[$S - v, u$]
 + $d(u, v)$ | $u \in S - v$ }

 return min{ OPT[$V - s, v$] + $d(v, s)$ | $v \in V - s$ }

} $\mathcal{O}(2^n)$

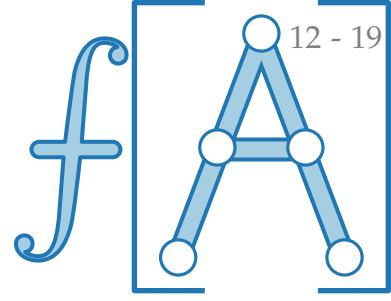
} $\mathcal{O}(n)$

Analyse.

- Innerste Schleife wird $\mathcal{O}(2^n \cdot n)$ mal ausgeführt
- Jede Ausführung braucht $\mathcal{O}(n)$ Zeit
- Gesamtzeit $\mathcal{O}(2^n n^2) = \mathcal{O}^*(2^n)$

- Eine kürzeste Tour kann durch Backtracking der DP Tabelle ausgegeben werden (wie üblich).

Dynamisches Program – Analyse



```
BELLMANNHELDKARP( $G, d$ )
```

```
  foreach  $v \in V - s$  do
```

```
    OPT[{ $v$ },  $v$ ] =  $d(s, v)$ 
```

```
  for  $j = 2$  to  $n - 1$  do
```

```
    foreach  $S \subseteq V - s$  with  $|S| = j$  do
```

```
      foreach  $v \in S$  do
```

```
        OPT[ $S, v$ ] = min{ OPT[ $S - v, u$ ]  
                        +  $d(u, v)$  |  $u \in S - v$  }
```

```
  return min{ OPT[ $V - s, v$ ] +  $d(v, s)$  |  $v \in V - s$  }
```

} $\mathcal{O}(2^n)$

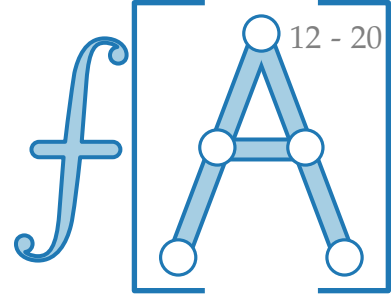
} $\mathcal{O}(n)$

Analyse.

- Innerste Schleife wird $\mathcal{O}(2^n \cdot n)$ mal ausgeführt
- Jede Ausführung braucht $\mathcal{O}(n)$ Zeit
- Gesamtzeit $\mathcal{O}(2^n n^2) = \mathcal{O}^*(2^n)$
- Speicherplatz in $\Theta(2^n \cdot n)$

- Eine kürzeste Tour kann durch Backtracking der DP Tabelle ausgegeben werden (wie üblich).

Dynamisches Program – Analyse



```
BELLMANNHELDKARP( $G, d$ )
```

```
  foreach  $v \in V - s$  do
```

```
    OPT[ $\{v\}, v$ ] =  $d(s, v)$ 
```

```
  for  $j = 2$  to  $n - 1$  do
```

```
    foreach  $S \subseteq V - s$  with  $|S| = j$  do
```

```
      foreach  $v \in S$  do
```

```
        OPT[ $S, v$ ] = min{ OPT[ $S - v, u$ ]  
                        +  $d(u, v)$  |  $u \in S - v$  }
```

```
  return min{ OPT[ $V - s, v$ ] +  $d(v, s)$  |  $v \in V - s$  }
```

} $\mathcal{O}(2^n)$

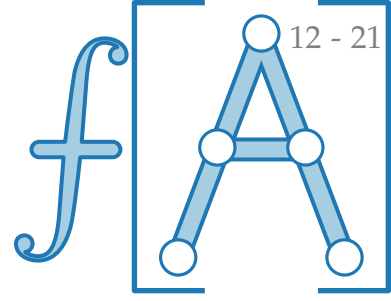
} $\mathcal{O}(n)$

Analyse.

- Innerste Schleife wird $\mathcal{O}(2^n \cdot n)$ mal ausgeführt
- Jede Ausführung braucht $\mathcal{O}(n)$ Zeit
- Gesamtzeit $\mathcal{O}(2^n n^2) = \mathcal{O}^*(2^n)$
- Speicherplatz in $\Theta(2^n \cdot n)$
- oder sogar besser? Welche Tabelleneinträge müssen wir speichern?

- Eine kürzeste Tour kann durch Backtracking der DP Tabelle ausgegeben werden (wie üblich).

Dynamisches Program – Analyse



```
BELLMANNHELDKARP( $G, d$ )
```

```
  foreach  $v \in V - s$  do
```

```
    OPT[ $\{v\}, v$ ] =  $d(s, v)$ 
```

```
  for  $j = 2$  to  $n - 1$  do
```

```
    foreach  $S \subseteq V - s$  with  $|S| = j$  do
```

```
      foreach  $v \in S$  do
```

```
        OPT[ $S, v$ ] = min{ OPT[ $S - v, u$ ]  
                        +  $d(u, v)$  |  $u \in S - v$  }
```

```
  return min{ OPT[ $V - s, v$ ] +  $d(v, s)$  |  $v \in V - s$  }
```

} $\mathcal{O}(2^n)$

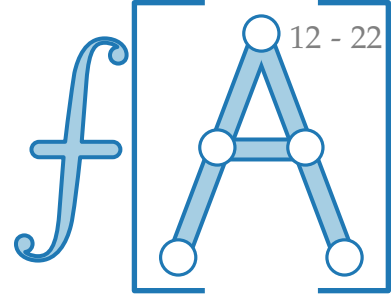
} $\mathcal{O}(n)$

- Eine kürzeste Tour kann durch Backtracking der DP Tabelle ausgegeben werden (wie üblich).

Analyse.

- Innerste Schleife wird $\mathcal{O}(2^n \cdot n)$ mal ausgeführt
- Jede Ausführung braucht $\mathcal{O}(n)$ Zeit
- Gesamtzeit $\mathcal{O}(2^n n^2) = \mathcal{O}^*(2^n)$
- Speicherplatz in $\Theta(2^n \cdot n)$
- oder sogar besser? Welche Tabelleneinträge müssen wir speichern?
- Wenn wir nur die Länge finden wollen, reicht es Einträge der Länge $j - 1$ zu speichern

Dynamisches Program – Analyse



```
BELLMANNHELDKARP( $G, d$ )
```

```
  foreach  $v \in V - s$  do
```

```
    OPT[ $\{v\}, v$ ] =  $d(s, v)$ 
```

```
  for  $j = 2$  to  $n - 1$  do
```

```
    foreach  $S \subseteq V - s$  with  $|S| = j$  do
```

```
      foreach  $v \in S$  do
```

```
        OPT[ $S, v$ ] = min{ OPT[ $S - v, u$ ]  
                        +  $d(u, v)$  |  $u \in S - v$  }
```

```
  return min{ OPT[ $V - s, v$ ] +  $d(v, s)$  |  $v \in V - s$  }
```

} $\mathcal{O}(2^n)$

} $\mathcal{O}(n)$

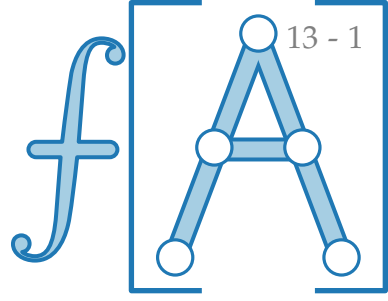
- Eine kürzeste Tour kann durch Backtracking der DP Tabelle ausgegeben werden (wie üblich).

Analyse.

- Innerste Schleife wird $\mathcal{O}(2^n \cdot n)$ mal ausgeführt
- Jede Ausführung braucht $\mathcal{O}(n)$ Zeit
- Gesamtzeit $\mathcal{O}(2^n n^2) = \mathcal{O}^*(2^n)$
- Speicherplatz in $\Theta(2^n \cdot n)$
- oder sogar besser? Welche Tabelleneinträge müssen wir speichern?
- Wenn wir nur die Länge finden wollen, reicht es Einträge der Länge $j - 1$ zu speichern
- ➔ Speicherplatz in $\Theta(2^n \cdot \sqrt{n})$

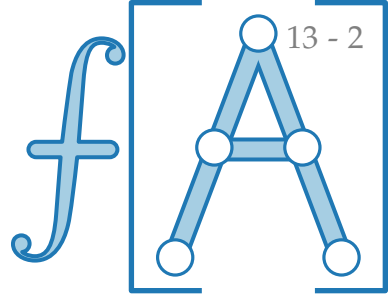
Diskussion

- DP Algorithmus läuft in $\mathcal{O}^*(2^n)$ Zeit mit $\mathcal{O}(2^n \cdot \sqrt{n})$ Platz



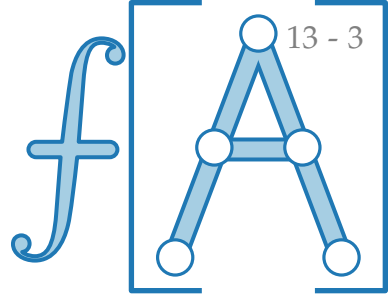
Diskussion

- DP Algorithmus läuft in $\mathcal{O}^*(2^n)$ Zeit mit $\mathcal{O}(2^n \cdot \sqrt{n})$ Platz
- Brute-force braucht $2^{\mathcal{O}(n \log n)}$ Zeit
 - ➔ Opfere Platz für Geschwindigkeit



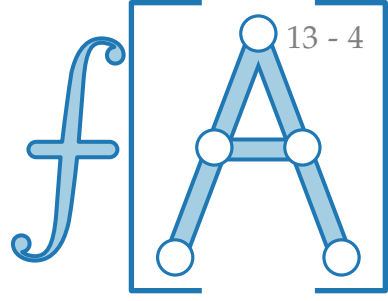
Diskussion

- DP Algorithmus läuft in $\mathcal{O}^*(2^n)$ Zeit mit $\mathcal{O}(2^n \cdot \sqrt{n})$ Platz
- Brute-force braucht $2^{\mathcal{O}(n \log n)}$ Zeit
 - ➔ Opfere Platz für Geschwindigkeit
- Viele Varianten von TSP: symmetrisch, asymmetrisch, metrisch, Vehicle Routing Problem, ...

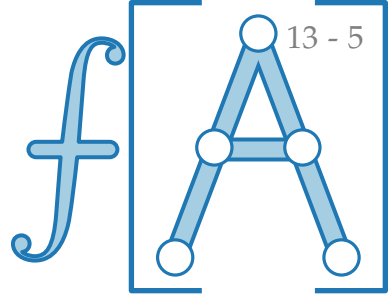


Diskussion

- DP Algorithmus läuft in $\mathcal{O}^*(2^n)$ Zeit mit $\mathcal{O}(2^n \cdot \sqrt{n})$ Platz
- Brute-force braucht $2^{\mathcal{O}(n \log n)}$ Zeit
 - ➔ Opfere Platz für Geschwindigkeit
- Viele Varianten von TSP: symmetrisch, asymmetrisch, metrisch, Vehicle Routing Problem, ...
- Metrisches TSP hat 2-Approximation

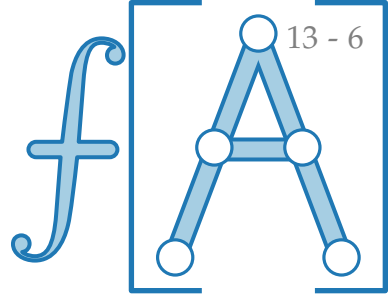


Diskussion



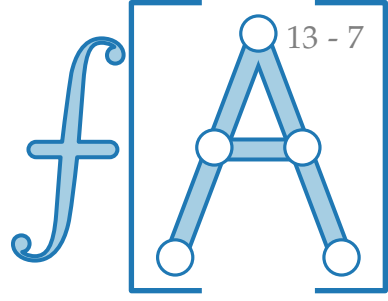
- DP Algorithmus läuft in $\mathcal{O}^*(2^n)$ Zeit mit $\mathcal{O}(2^n \cdot \sqrt{n})$ Platz
- Brute-force braucht $2^{\mathcal{O}(n \log n)}$ Zeit
 - ➔ Opfere Platz für Geschwindigkeit
- Viele Varianten von TSP: symmetrisch, asymmetrisch, metrisch, Vehicle Routing Problem, ...
- Metrisches TSP hat 2-Approximation
- Euklidisches TSP hat $(1 + \varepsilon)$ -Approximation für jedes $\varepsilon > 0$

Diskussion

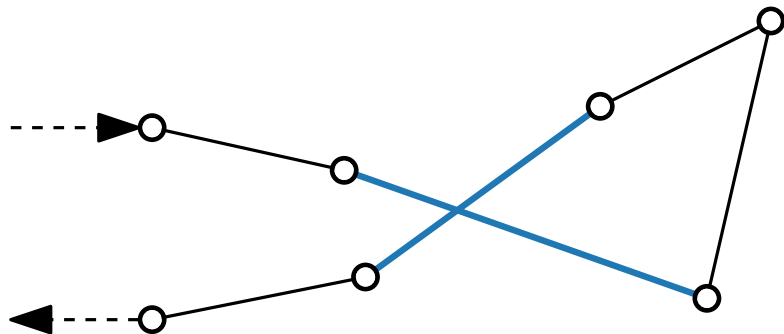


- DP Algorithmus läuft in $\mathcal{O}^*(2^n)$ Zeit mit $\mathcal{O}(2^n \cdot \sqrt{n})$ Platz
- Brute-force braucht $2^{\mathcal{O}(n \log n)}$ Zeit
 - ➔ Opfere Platz für Geschwindigkeit
- Viele Varianten von TSP: symmetrisch, asymmetrisch, metrisch, Vehicle Routing Problem, ...
- Metrisches TSP hat 2-Approximation
- Euklidisches TSP hat $(1 + \varepsilon)$ -Approximation für jedes $\varepsilon > 0$
- Eine gute praktische Heuristik ist es, mit einem Greedy Hamiltonian Cycle anzufangen und die Lösung dann mit 2-OPT und 3-OPT Austauschen iterativ zu verbessern.

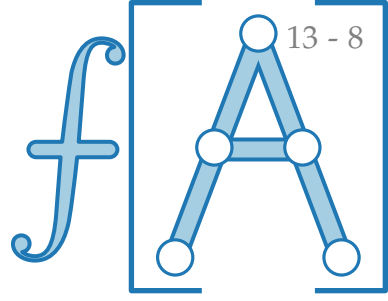
Diskussion



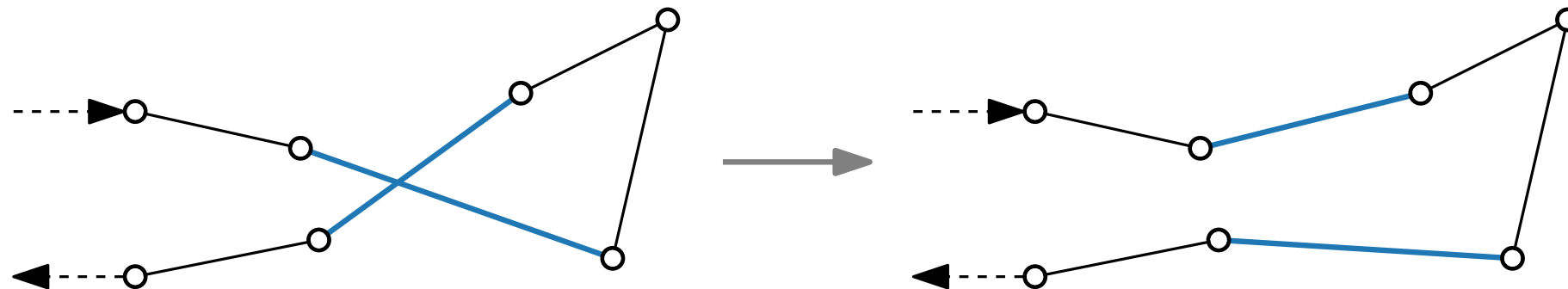
- DP Algorithmus läuft in $\mathcal{O}^*(2^n)$ Zeit mit $\mathcal{O}(2^n \cdot \sqrt{n})$ Platz
- Brute-force braucht $2^{\mathcal{O}(n \log n)}$ Zeit
 - ➔ Opfere Platz für Geschwindigkeit
- Viele Varianten von TSP: symmetrisch, asymmetrisch, metrisch, Vehicle Routing Problem, ...
- Metrisches TSP hat 2-Approximation
- Euklidisches TSP hat $(1 + \varepsilon)$ -Approximation für jedes $\varepsilon > 0$
- Eine gute praktische Heuristik ist es, mit einem Greedy Hamiltonian Cycle anzufangen und die Lösung dann mit 2-OPT und 3-OPT Austauschen iterativ zu verbessern.

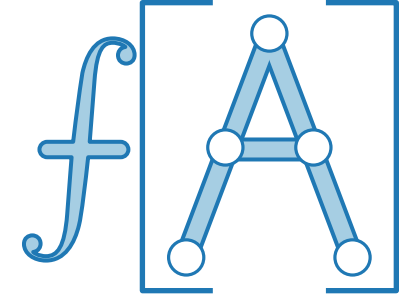


Diskussion



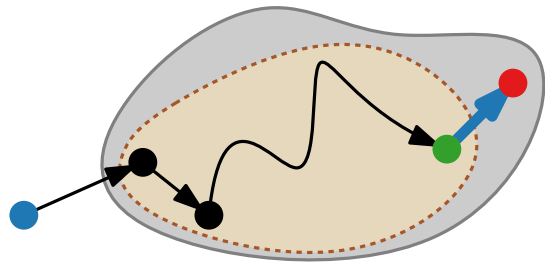
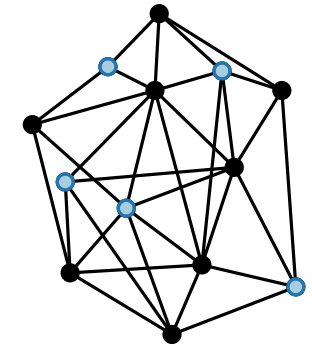
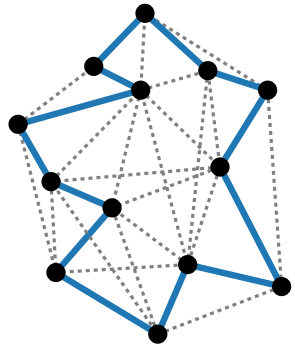
- DP Algorithmus läuft in $\mathcal{O}^*(2^n)$ Zeit mit $\mathcal{O}(2^n \cdot \sqrt{n})$ Platz
- Brute-force braucht $2^{\mathcal{O}(n \log n)}$ Zeit
 - ➔ Opfere Platz für Geschwindigkeit
- Viele Varianten von TSP: symmetrisch, asymmetrisch, metrisch, Vehicle Routing Problem, ...
- Metrisches TSP hat 2-Approximation
- Euklidisches TSP hat $(1 + \varepsilon)$ -Approximation für jedes $\varepsilon > 0$
- Eine gute praktische Heuristik ist es, mit einem Greedy Hamiltonian Cycle anzufangen und die Lösung dann mit 2-OPT und 3-OPT Austauschen iterativ zu verbessern.



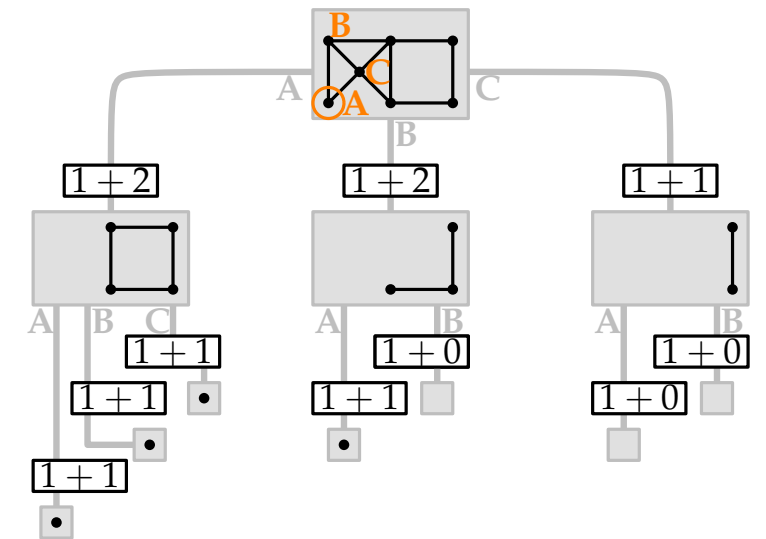


Fortgeschrittene Algorithmen

19. Vorlesung Exakte Algorithmen (für NP-schwere Probleme)

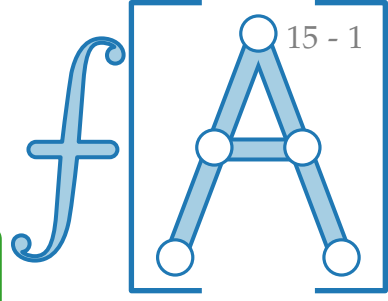


Teil III: Größte unabhängige Menge (MIS)



Größte unabhängige Menge (MIS)

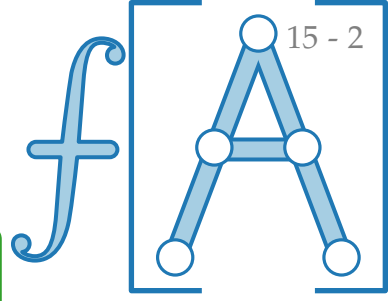
Problem. MAXIMUMINDEPENDENTSET



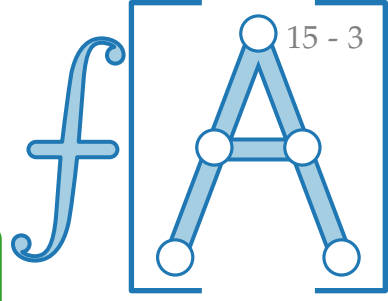
Größte unabhängige Menge (MIS)

Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

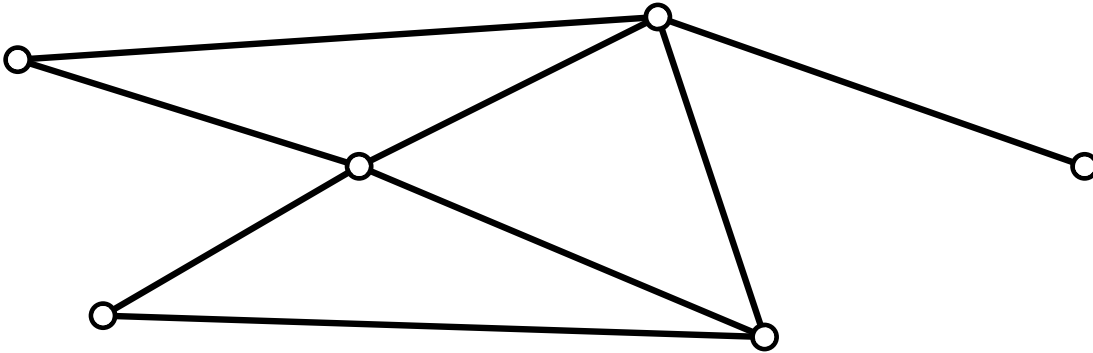


Größte unabhängige Menge (MIS)

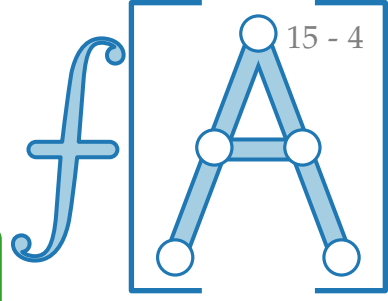


Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten



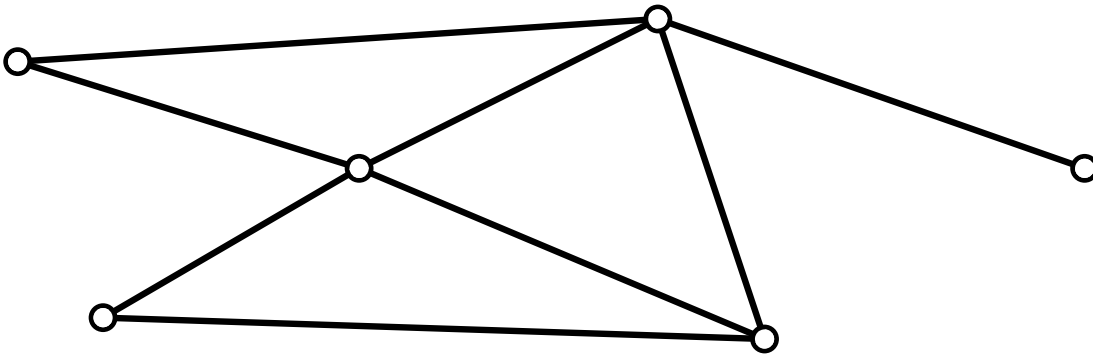
Größte unabhängige Menge (MIS)



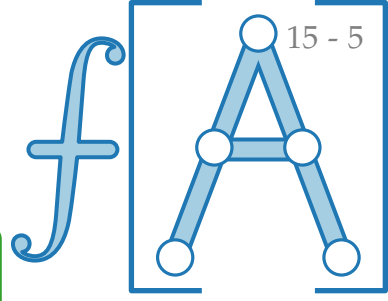
Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



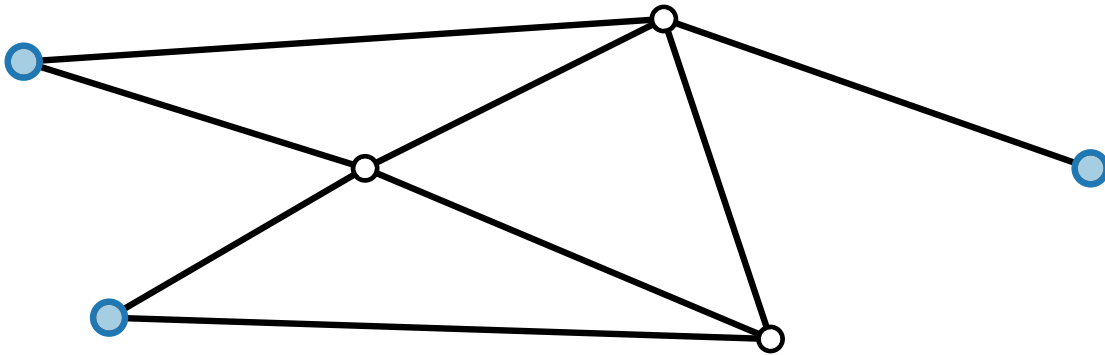
Größte unabhängige Menge (MIS)



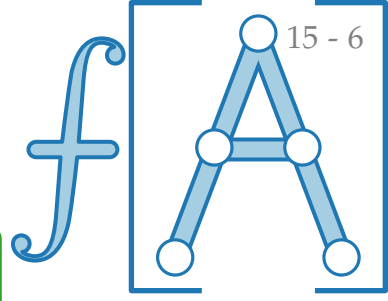
Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



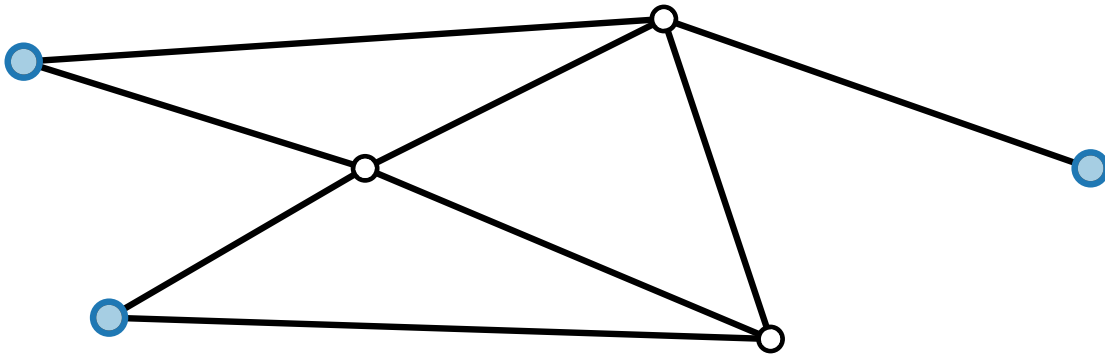
Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

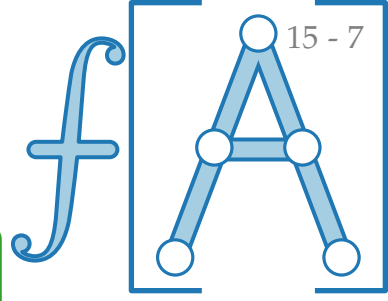
Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



Brute-force.

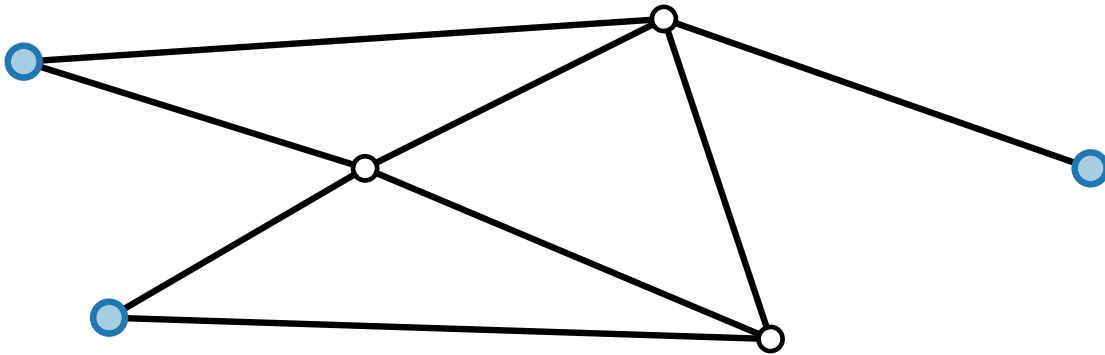
Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

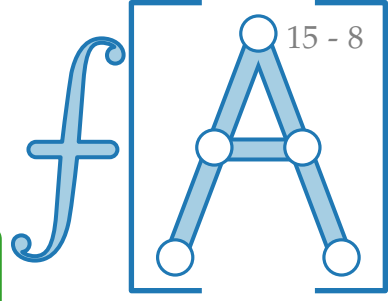
Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



Brute-force.

- Probiere alle Teilmengen von V aus

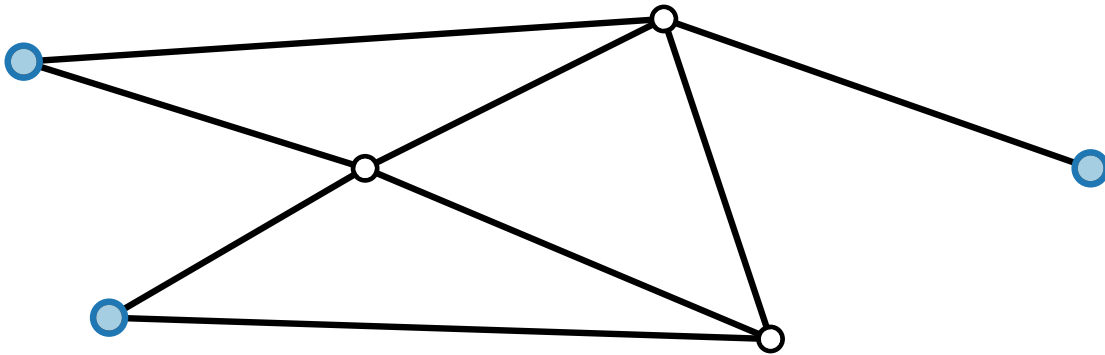
Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

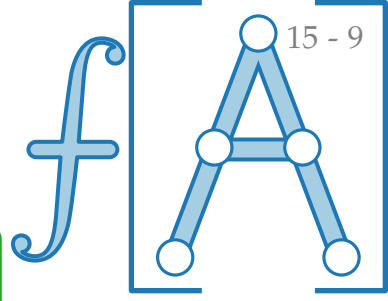
Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



Brute-force.

- Probiere alle Teilmengen von V aus
- Laufzeit:

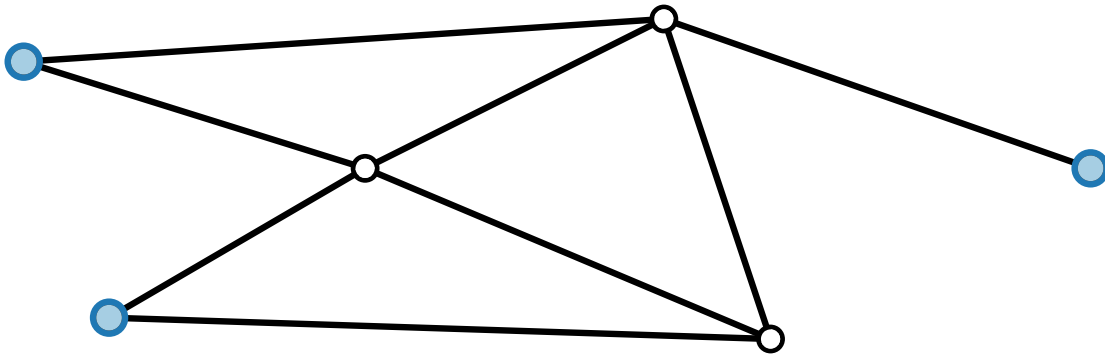
Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

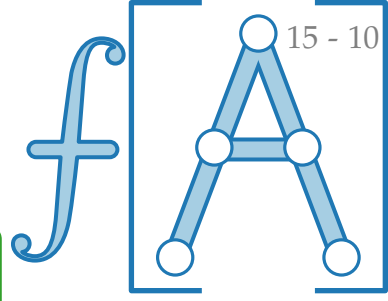
Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



Brute-force.

- Probiere alle Teilmengen von V aus
- Laufzeit: $\mathcal{O}(2^n \cdot n)$

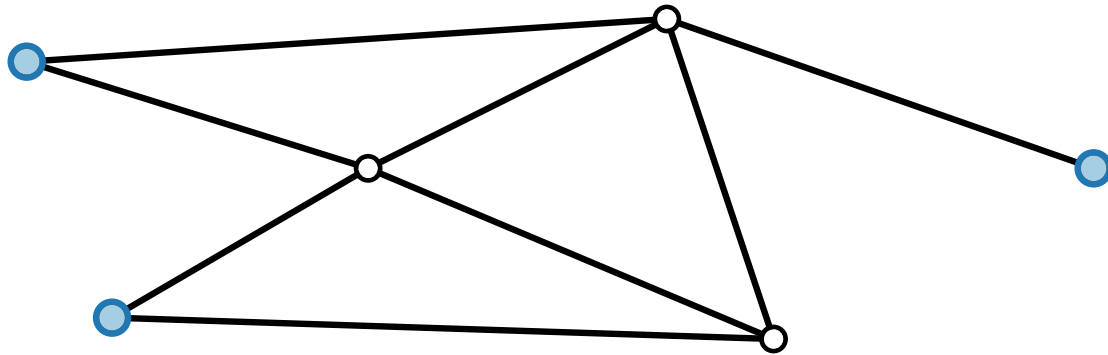
Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.

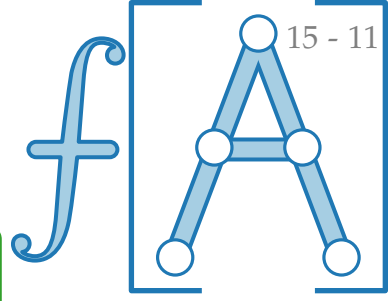


Naives MIS Branching.

Brute-force.

- Probiere alle Teilmengen von V aus
- Laufzeit: $\mathcal{O}(2^n \cdot n)$

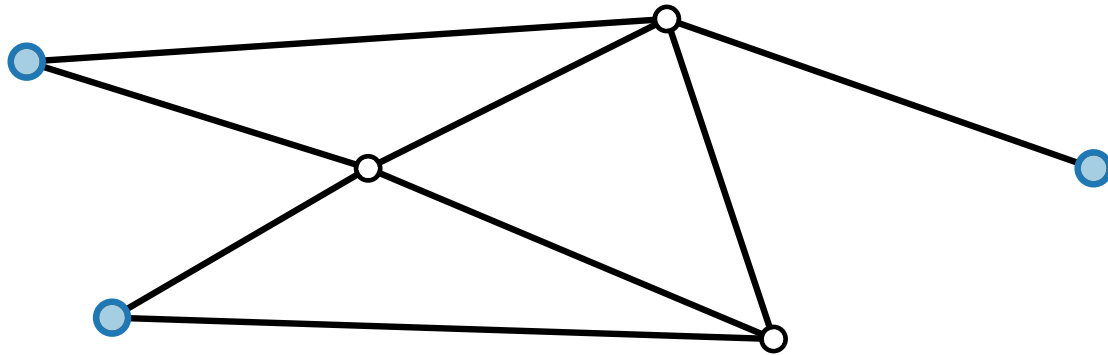
Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



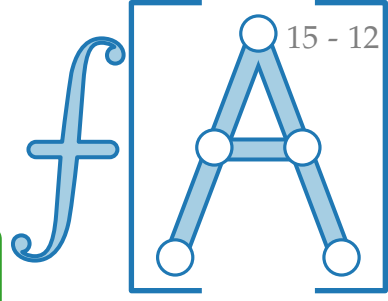
Naives MIS Branching.

- Entweder wähle Knoten v oder nicht.

Brute-force.

- Probiere alle Teilmengen von V aus
- Laufzeit: $\mathcal{O}(2^n \cdot n)$

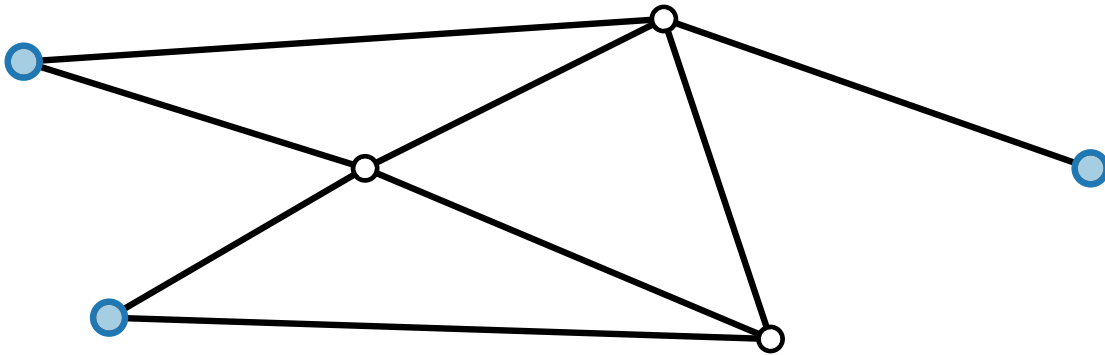
Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



Naives MIS Branching.

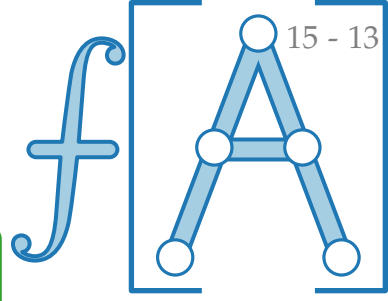
$$N[v] := N(v) \cup \{v\}$$

- Entweder wähle Knoten v oder nicht.

Brute-force.

- Probiere alle Teilmengen von V aus
- Laufzeit: $\mathcal{O}(2^n \cdot n)$

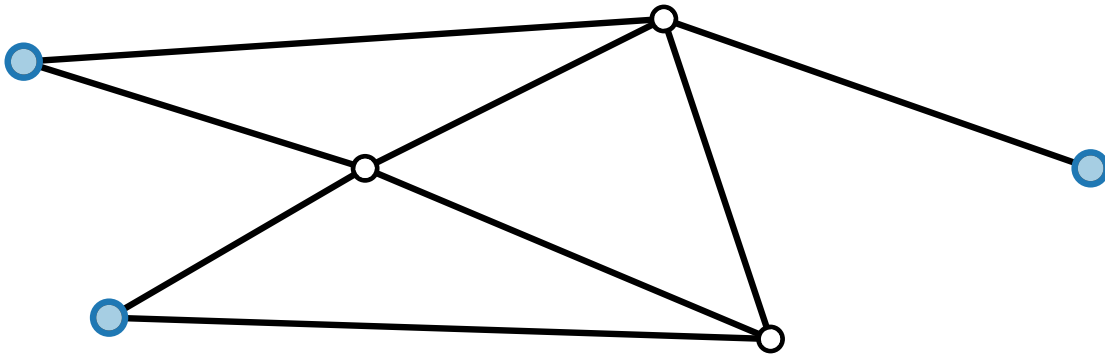
Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



Naives MIS Branching.

$$N[v] := N(v) \cup \{v\}$$

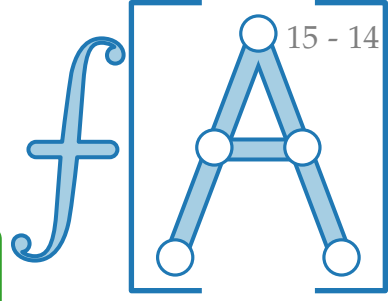
■ Entweder wähle Knoten v oder nicht.

NAIVEMIS(G)

Brute-force.

- Probiere alle Teilmengen von V aus
- Laufzeit: $\mathcal{O}(2^n \cdot n)$

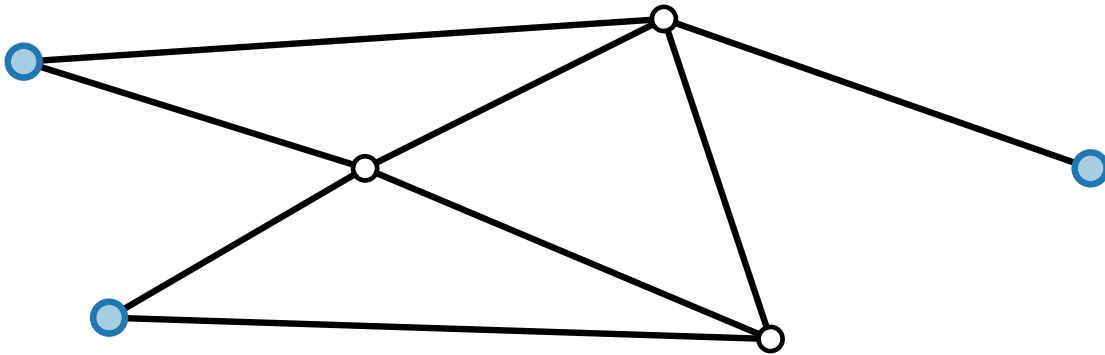
Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



Naives MIS Branching.

$$N[v] := N(v) \cup \{v\}$$

■ Entweder wähle Knoten v oder nicht.

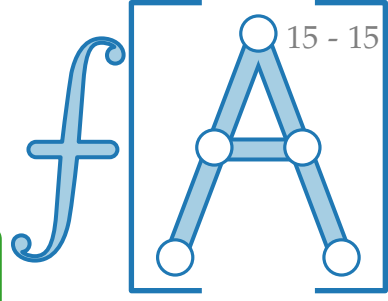
Brute-force.

- Probiere alle Teilmengen von V aus
- Laufzeit: $\mathcal{O}(2^n \cdot n)$

NAIVEMIS(G)

```
if  $V = \emptyset$  then  
  return 0
```

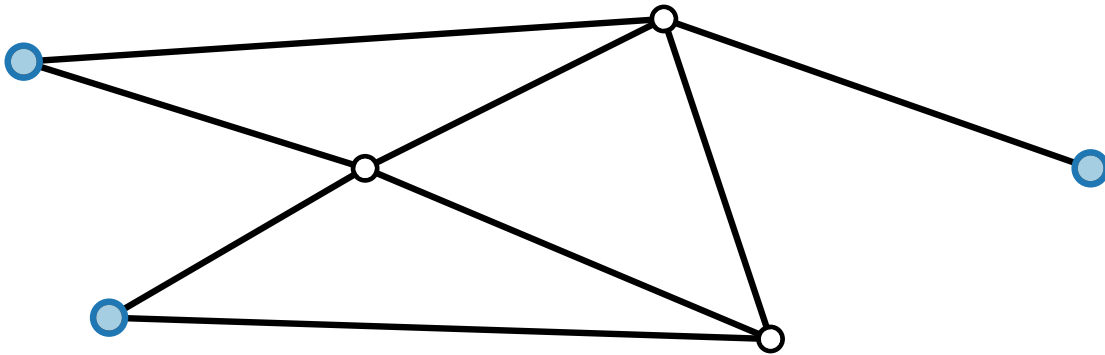

Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



Brute-force.

- Probiere alle Teilmengen von V aus
- Laufzeit: $\mathcal{O}(2^n \cdot n)$

Naives MIS Branching.

$$N[v] := N(v) \cup \{v\}$$

- Entweder wähle Knoten v oder nicht.

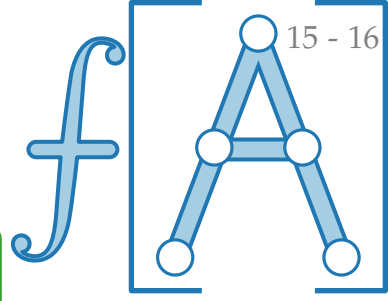
NAIVEMIS(G)

if $V = \emptyset$ **then**

└ **return** 0

v = Beliebiger Knoten in V

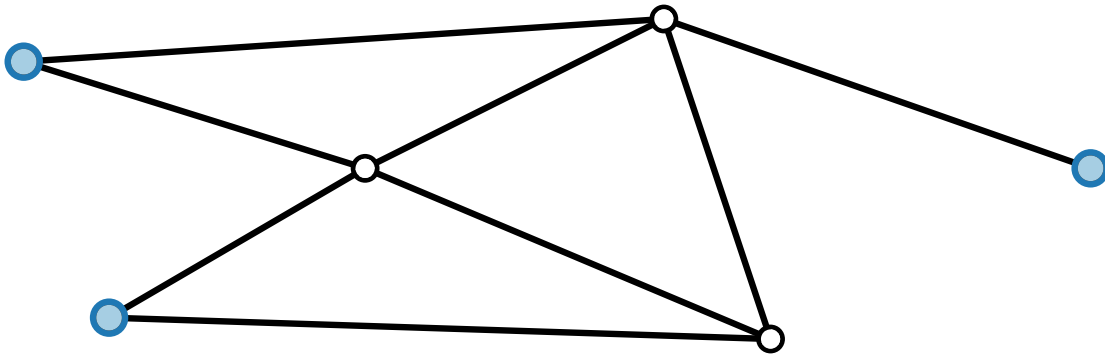
Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



Brute-force.

- Probiere alle Teilmengen von V aus
- Laufzeit: $\mathcal{O}(2^n \cdot n)$

Naives MIS Branching.

$$N[v] := N(v) \cup \{v\}$$

- Entweder wähle Knoten v oder nicht.

NAIVEMIS(G)

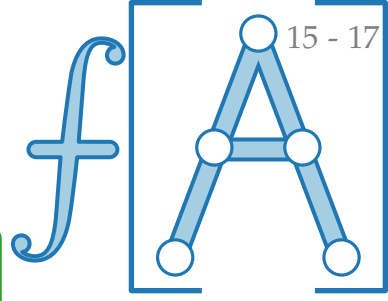
if $V = \emptyset$ **then**

└ **return** 0

v = Beliebiger Knoten in V

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$

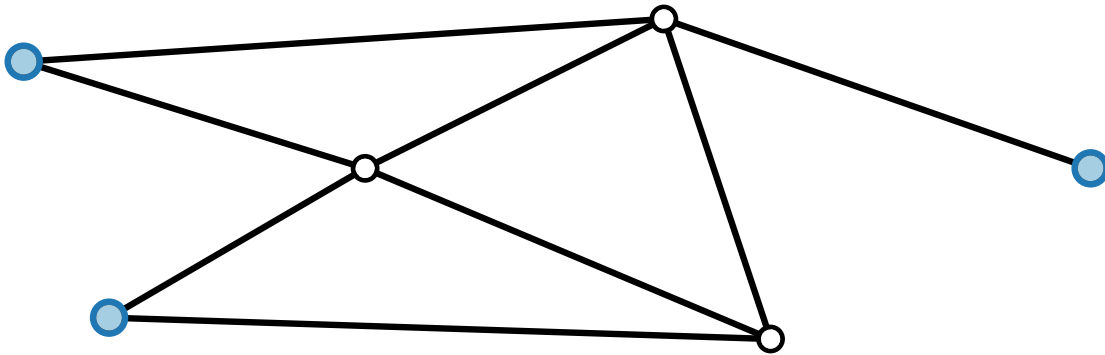
Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



Brute-force.

- Probiere alle Teilmengen von V aus
- Laufzeit: $\mathcal{O}(2^n \cdot n)$

Naives MIS Branching.

$$N[v] := N(v) \cup \{v\}$$

- Entweder wähle Knoten v oder nicht.

NAIVEMIS(G)

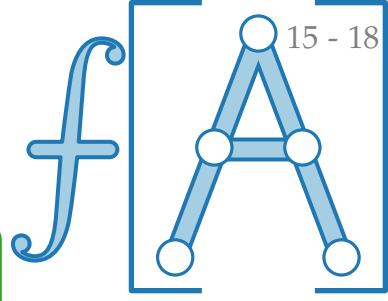
if $V = \emptyset$ **then**

└ **return** 0

v = Beliebiger Knoten in V wähle v

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$

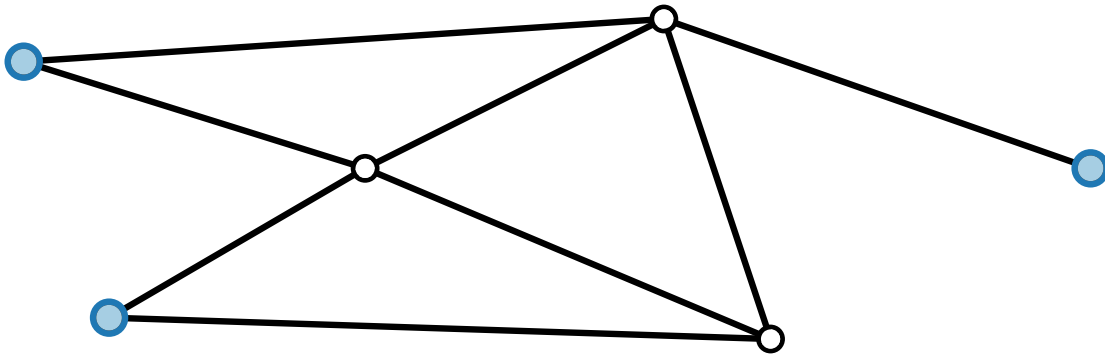
Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



Brute-force.

- Probiere alle Teilmengen von V aus
- Laufzeit: $\mathcal{O}(2^n \cdot n)$

Naives MIS Branching.

$$N[v] := N(v) \cup \{v\}$$

- Entweder wähle Knoten v oder nicht.

NAIVEMIS(G)

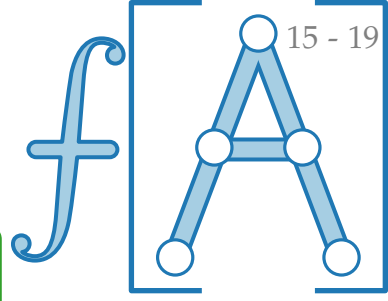
if $V = \emptyset$ **then**

└ **return** 0

v = Beliebiger Knoten in V wähle v

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$

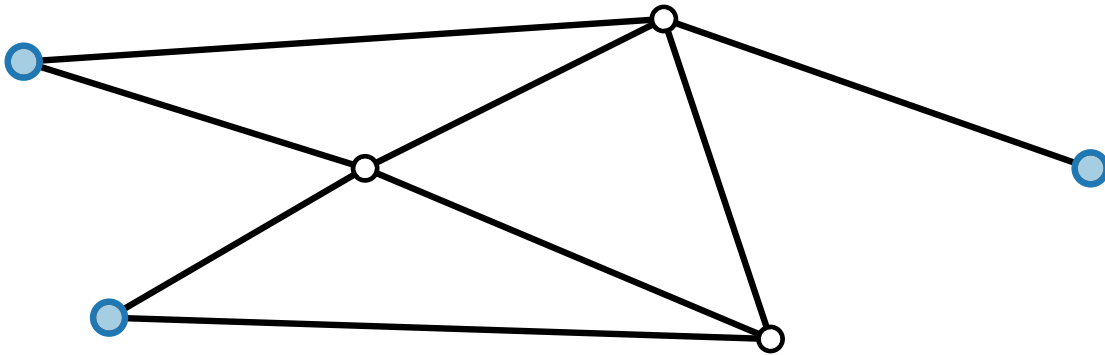
Größte unabhängige Menge (MIS)



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.



Brute-force.

- Probiere alle Teilmengen von V aus
- Laufzeit: $\mathcal{O}(2^n \cdot n)$

Naives MIS Branching.

$$N[v] := N(v) \cup \{v\}$$

- Entweder wähle Knoten v oder nicht.

NAIVEMIS(G)

if $V = \emptyset$ **then**

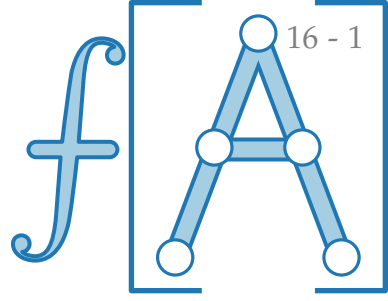
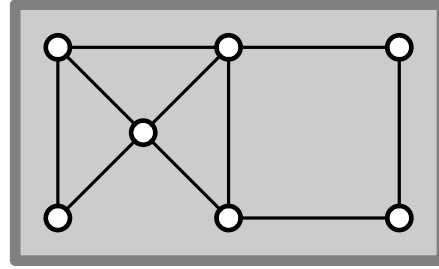
└ **return** 0

v = Beliebiger Knoten in V wähle v

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$

wähle v nicht

Beispiel



NAIVEMIS(G)

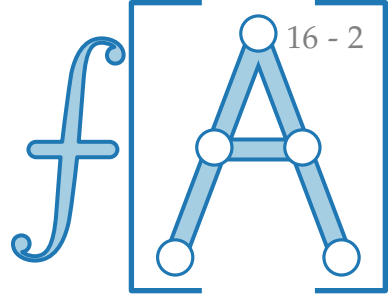
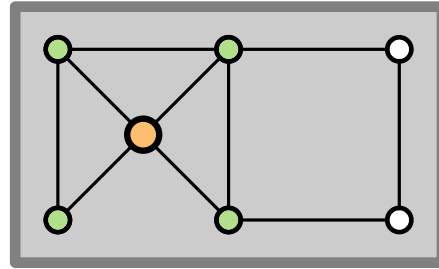
if $V = \emptyset$ **then**

return 0

v = Beliebiger Knoten in V

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$

Beispiel



NAIVEMIS(G)

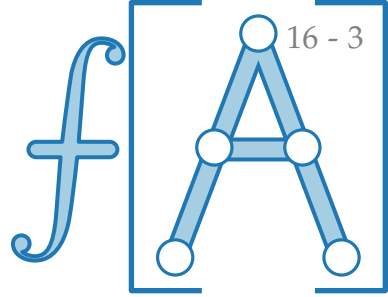
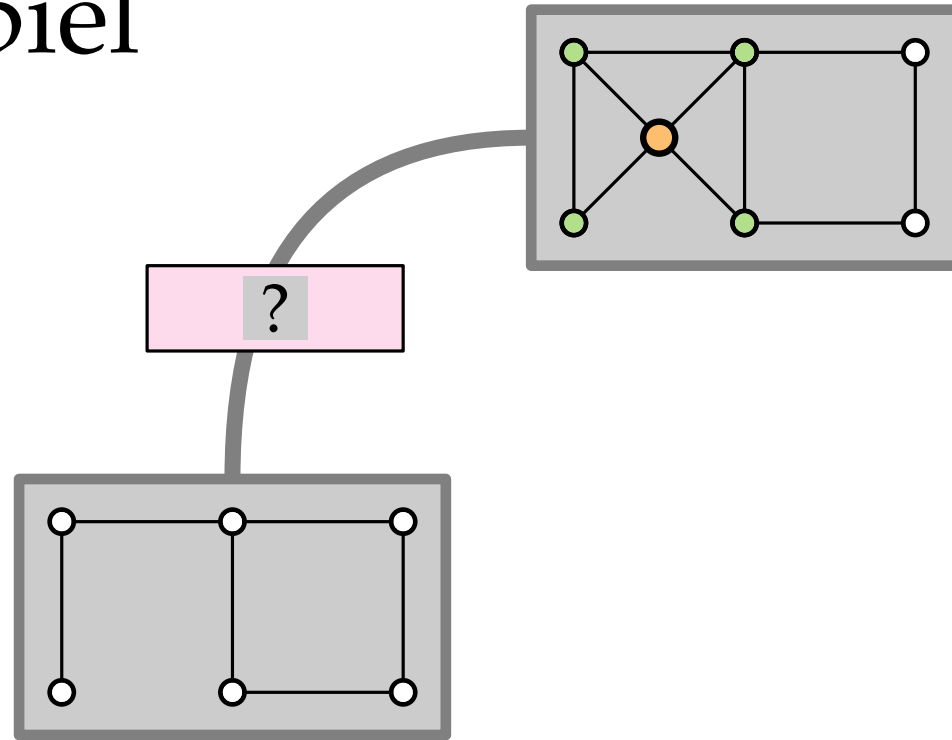
if $V = \emptyset$ **then**

return 0

v = Beliebiger Knoten in V

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$

Beispiel



NAIVEMIS(G)

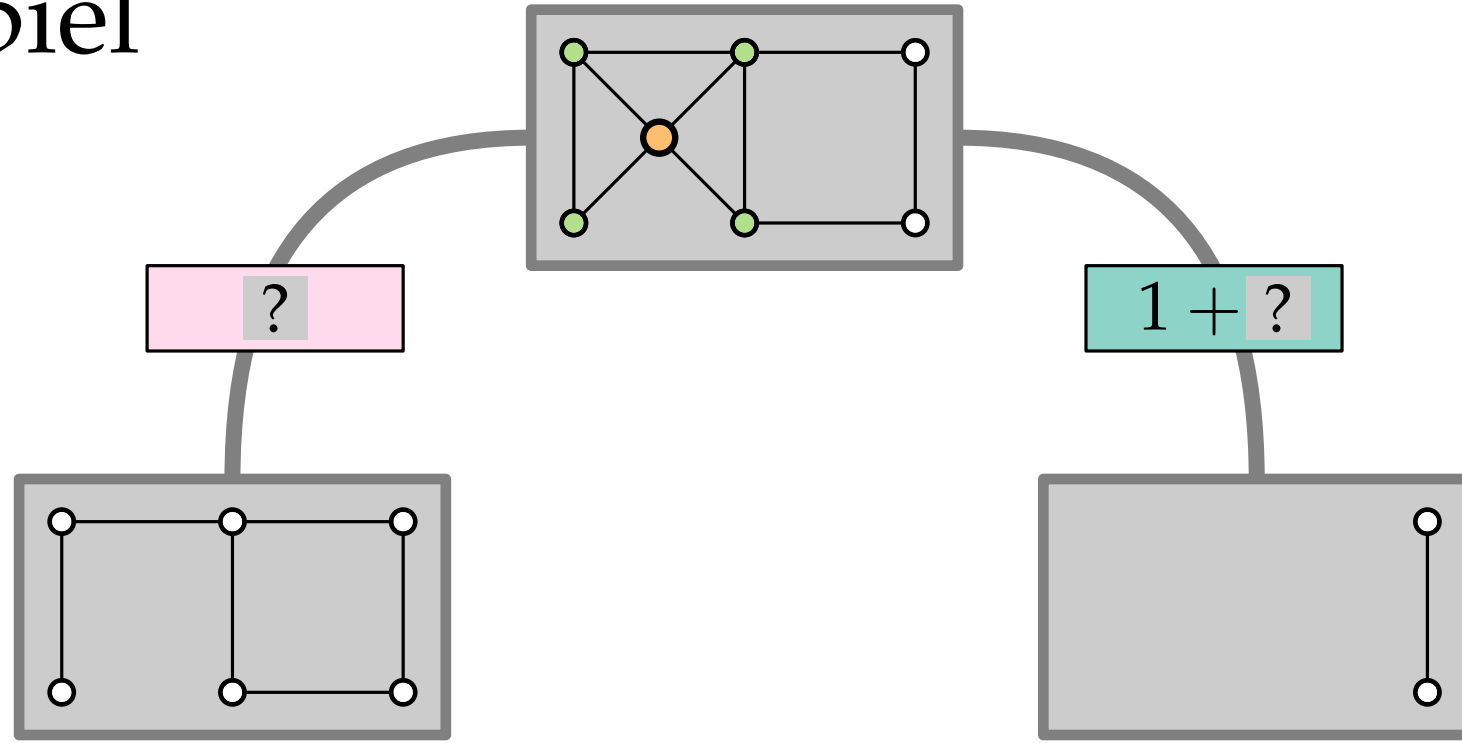
if $V = \emptyset$ **then**

└ **return** 0

v = Beliebiger Knoten in V

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$

Beispiel



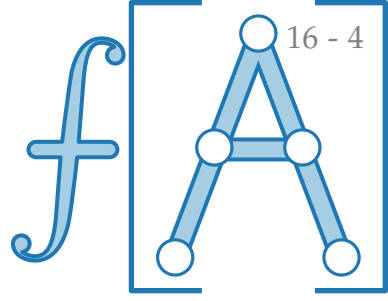
NAIVEMIS(G)

if $V = \emptyset$ **then**

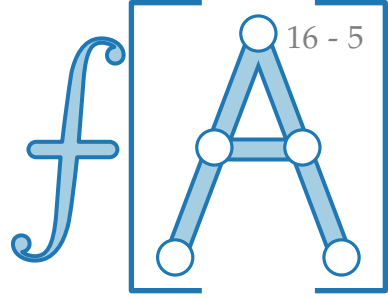
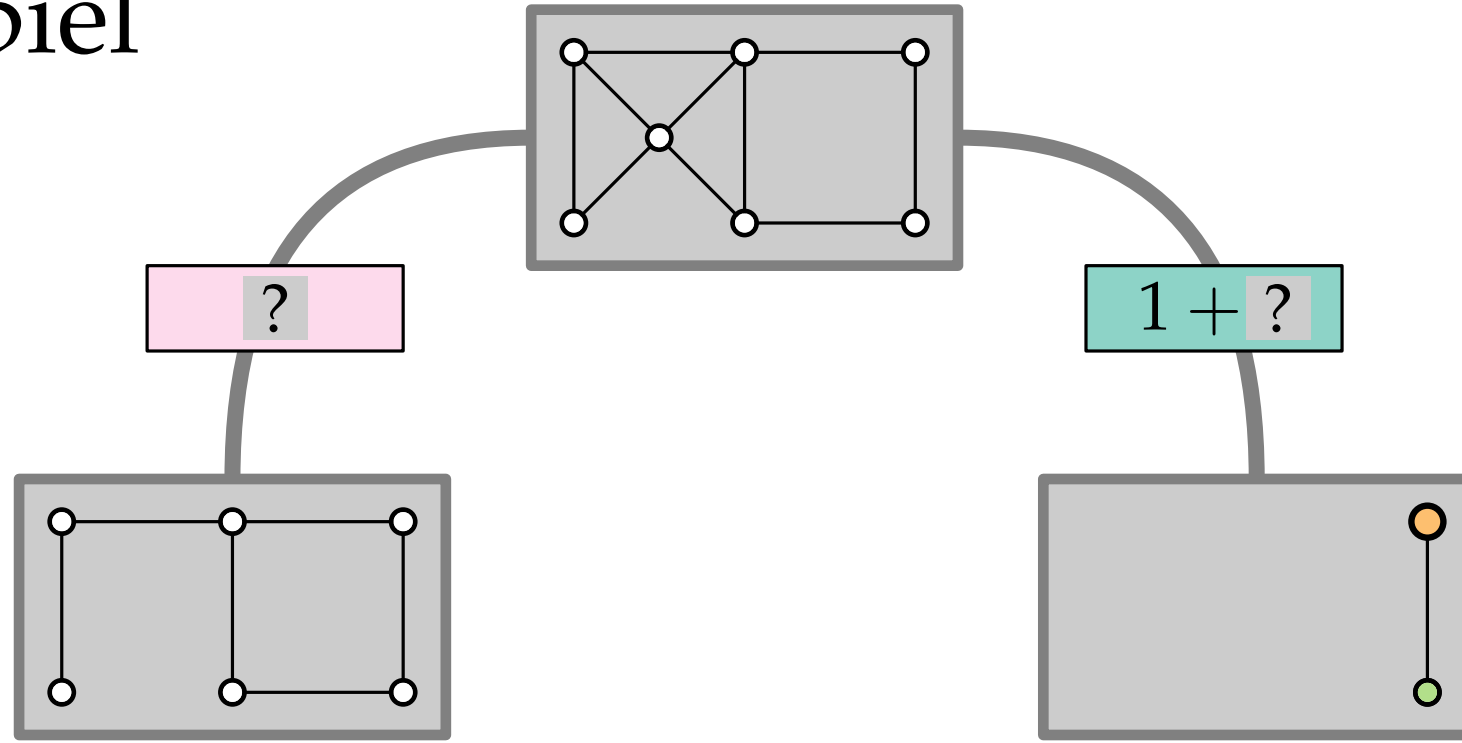
└ **return** 0

v = Beliebiger Knoten in V

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$



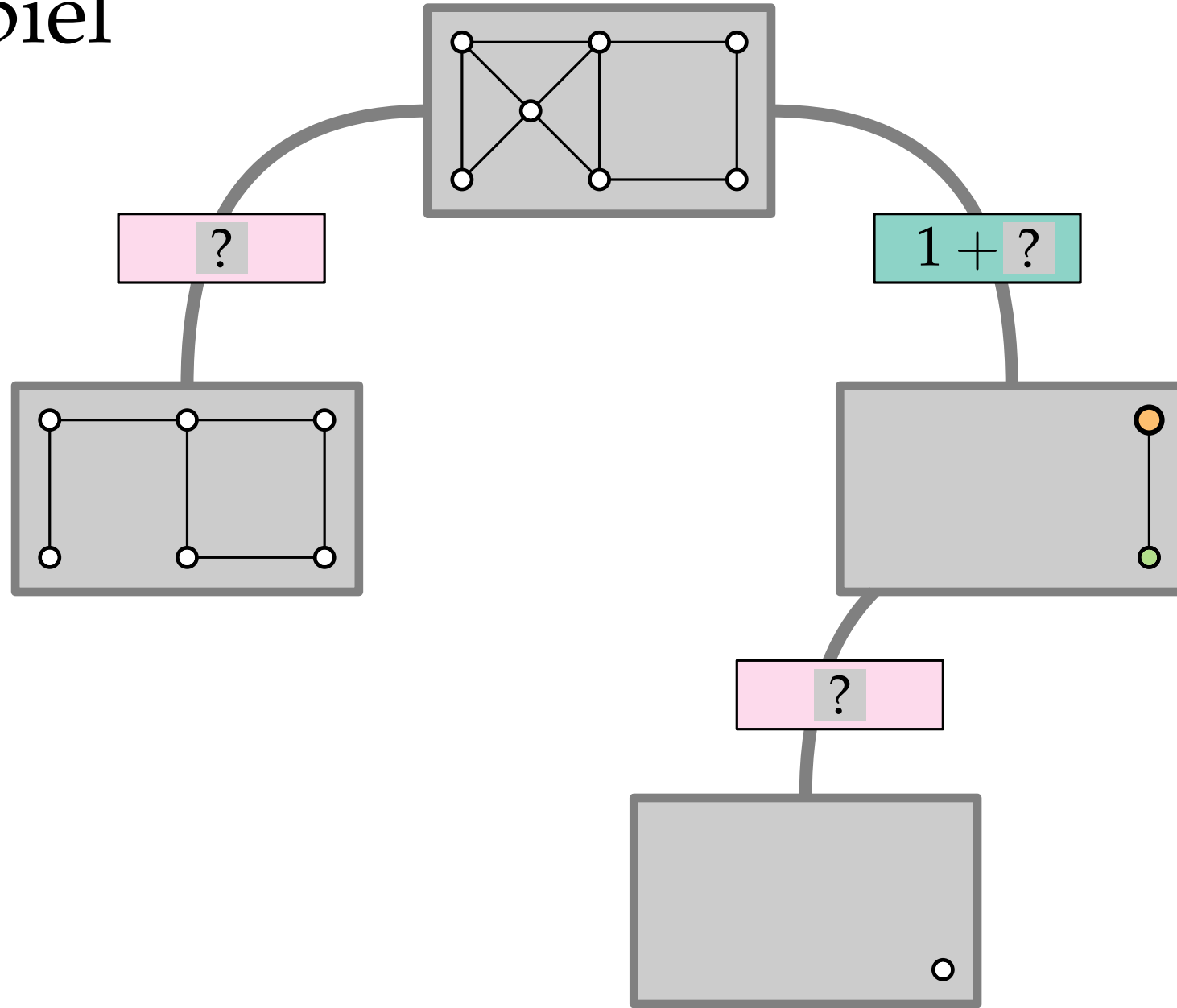
Beispiel



```

NAIVEMIS( $G$ )
  if  $V = \emptyset$  then
    return 0
   $v =$  Beliebiger Knoten in  $V$ 
  return  $\max\{1 + \text{NAIVE}$ MIS( $G - N[v]$ ),
     $\text{NAIVE}$ MIS( $G - \{v\}$ )\}
    
```

Beispiel



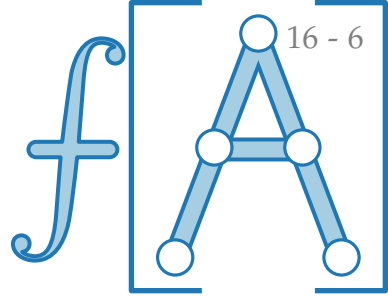
NAIVEMIS(G)

if $V = \emptyset$ **then**

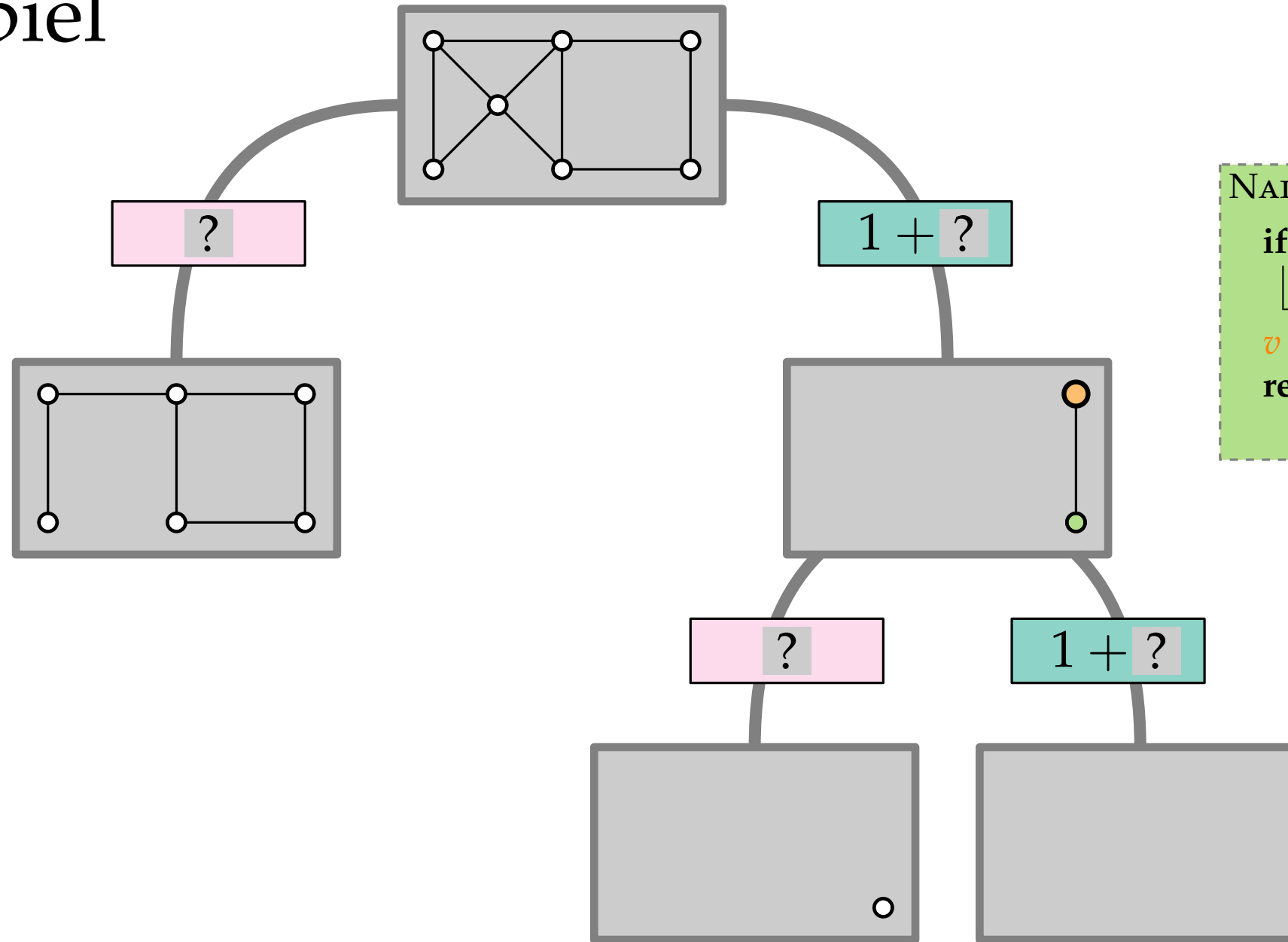
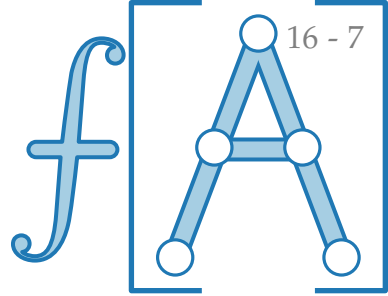
return 0

v = Beliebiger Knoten in V

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$



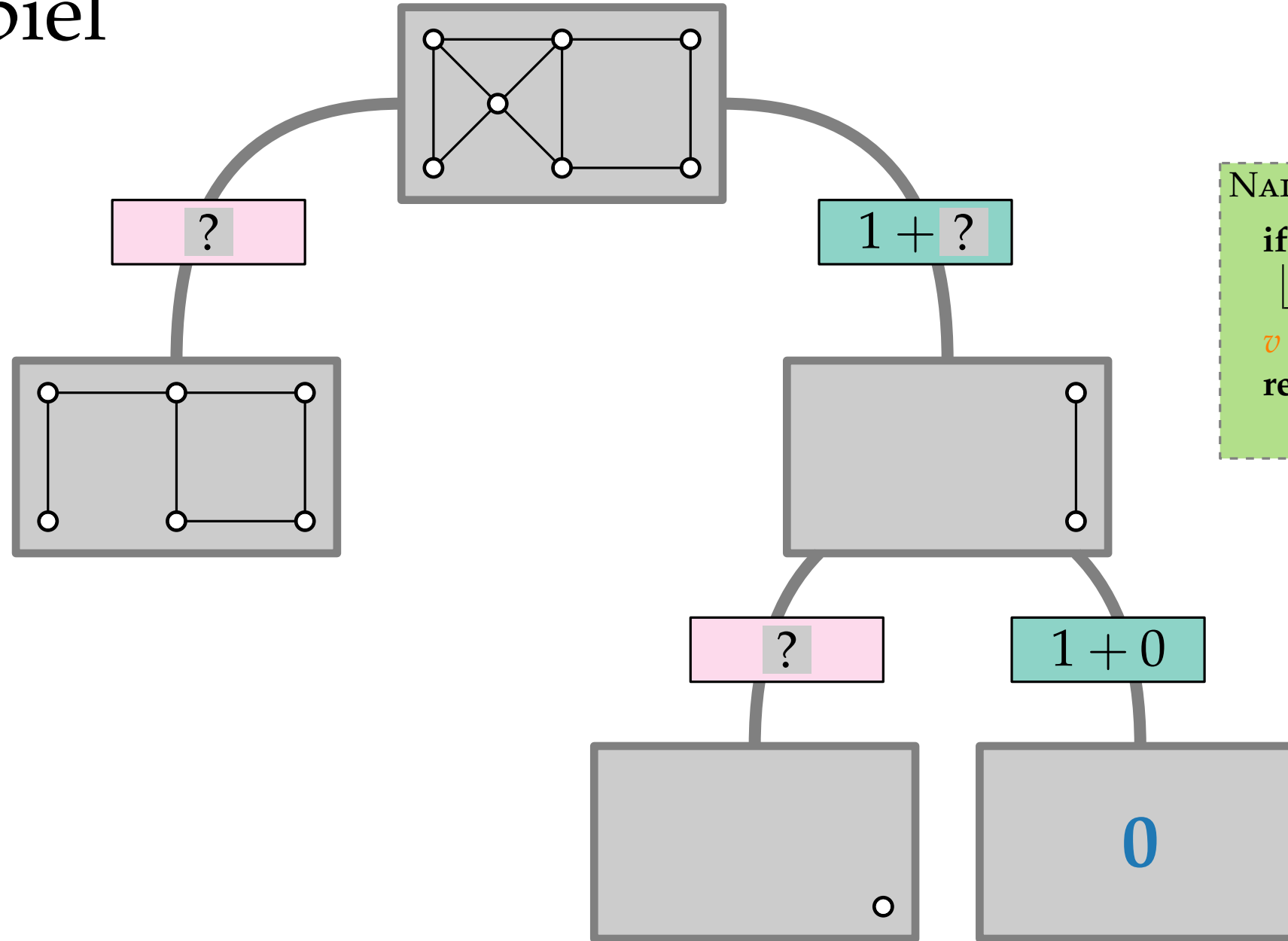
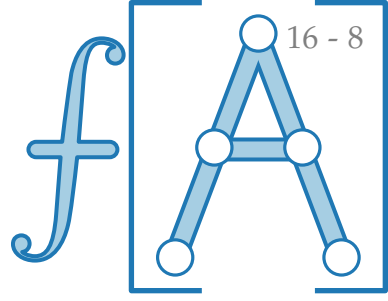
Beispiel



```

NaiveMIS( $G$ )
  if  $V = \emptyset$  then
    return 0
   $v =$  Beliebiger Knoten in  $V$ 
  return  $\max\{1 + \text{NaiveMIS}(G - N[v]),$ 
     $\text{NaiveMIS}(G - \{v\})\}$ 
  
```

Beispiel



NAIVEMIS(G)

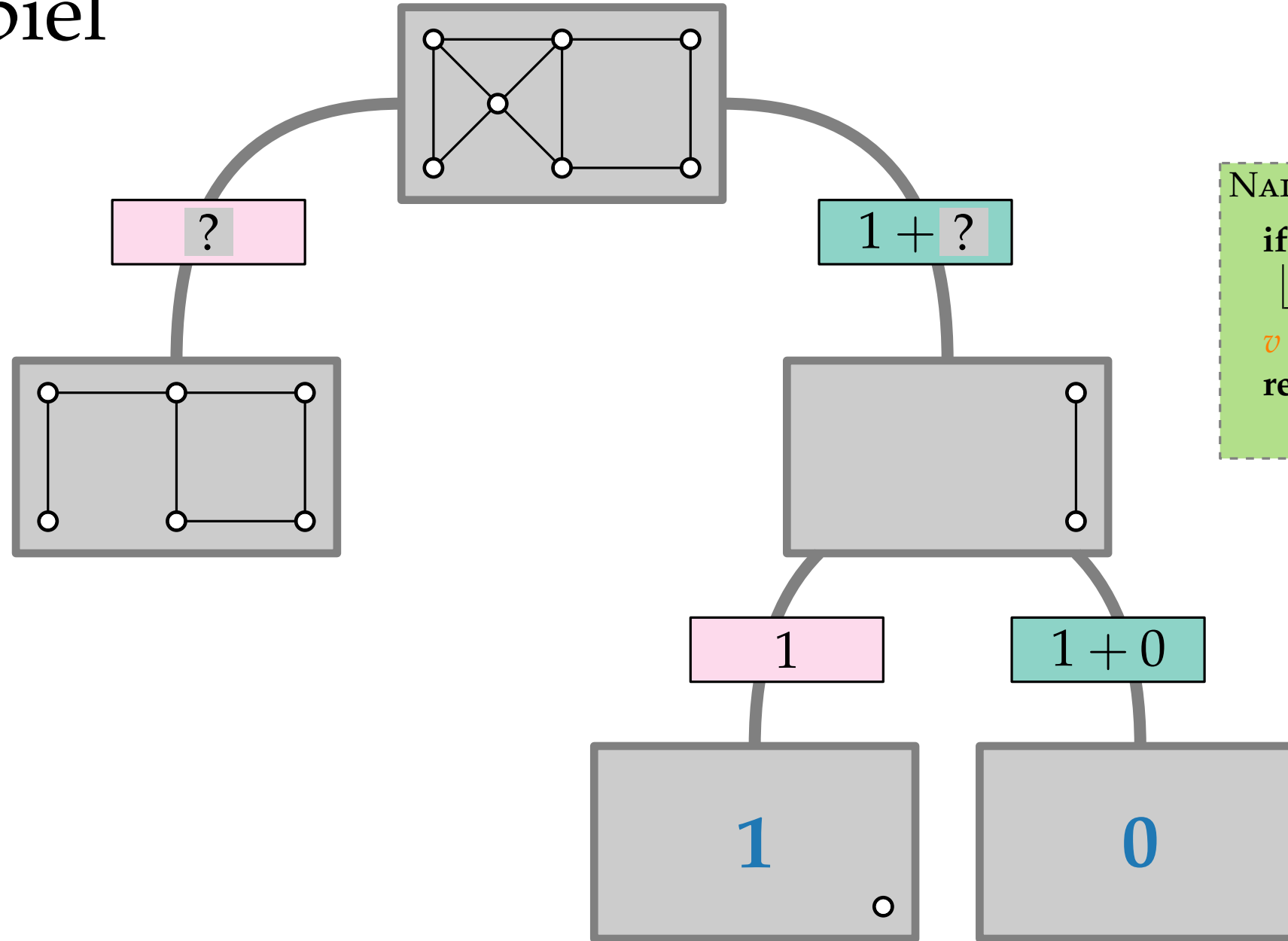
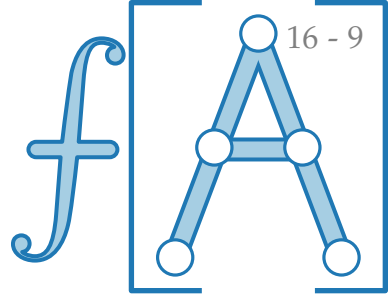
if $V = \emptyset$ **then**

└ **return** 0

v = Beliebiger Knoten in V

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$

Beispiel



NAIVEMIS(G)

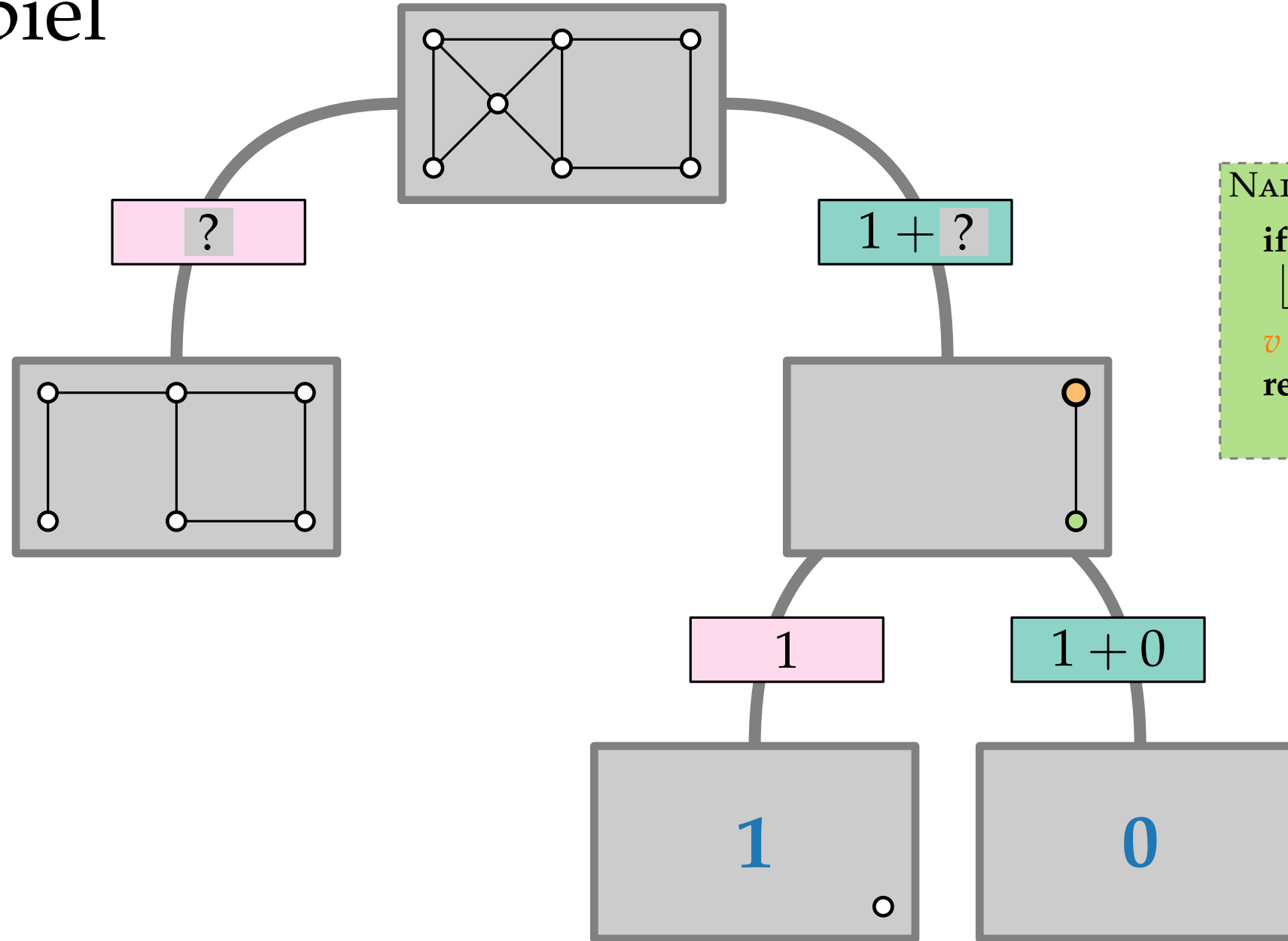
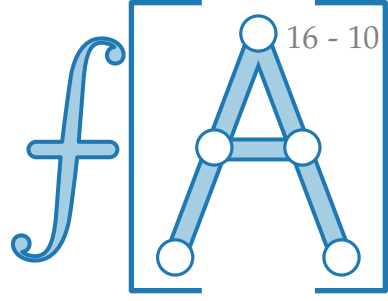
if $V = \emptyset$ **then**

└ **return** 0

v = Beliebiger Knoten in V

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$

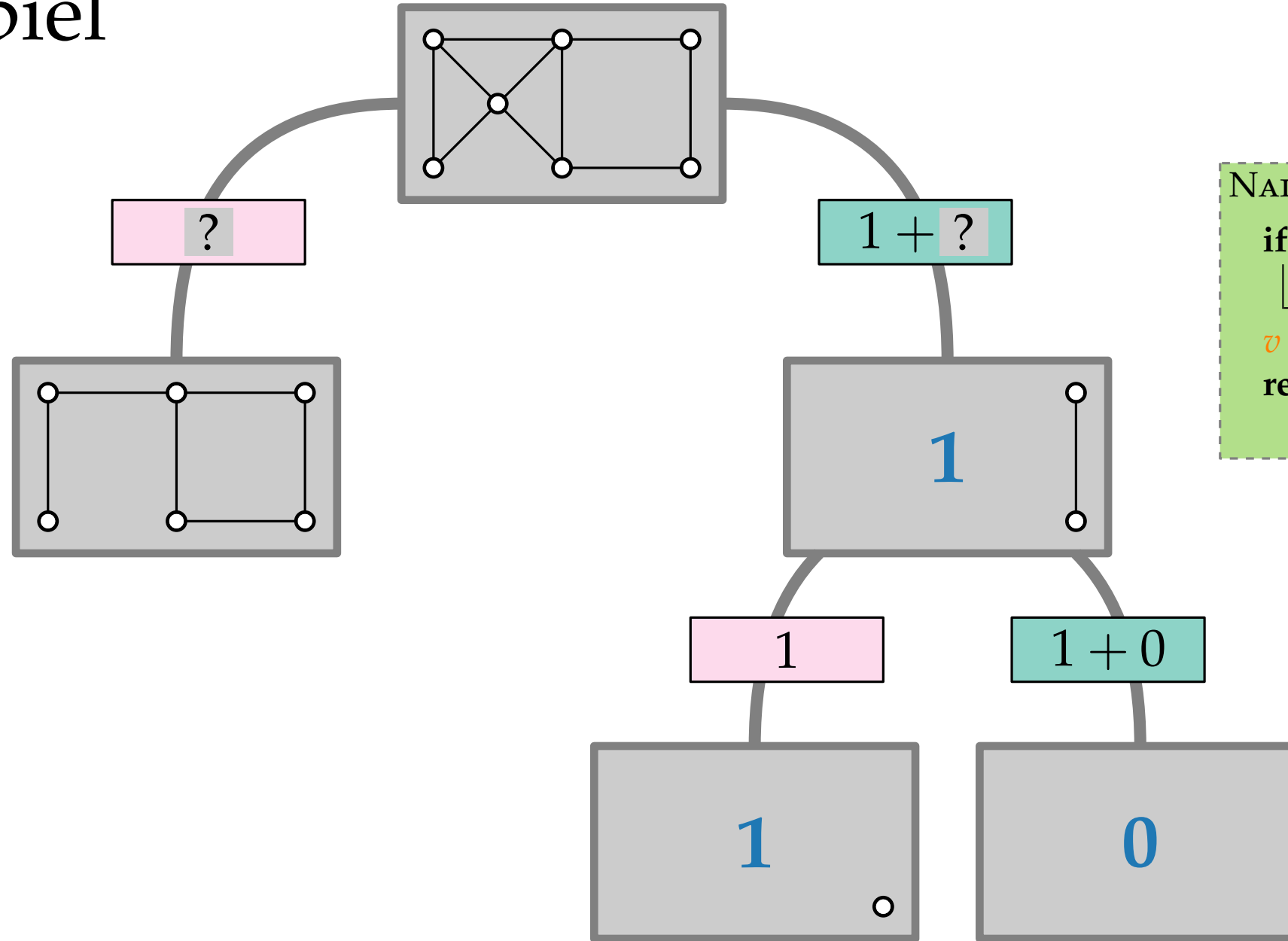
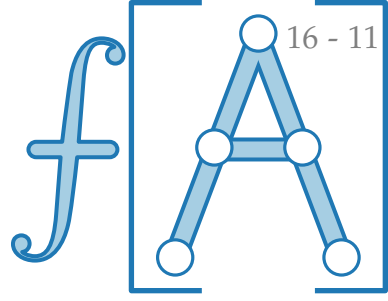
Beispiel



```

NaiveMIS( $G$ )
  if  $V = \emptyset$  then
    return 0
   $v =$  Beliebiger Knoten in  $V$ 
  return  $\max\{1 + \text{NaiveMIS}(G - N[v]),$ 
     $\text{NaiveMIS}(G - \{v\})\}$ 
  
```

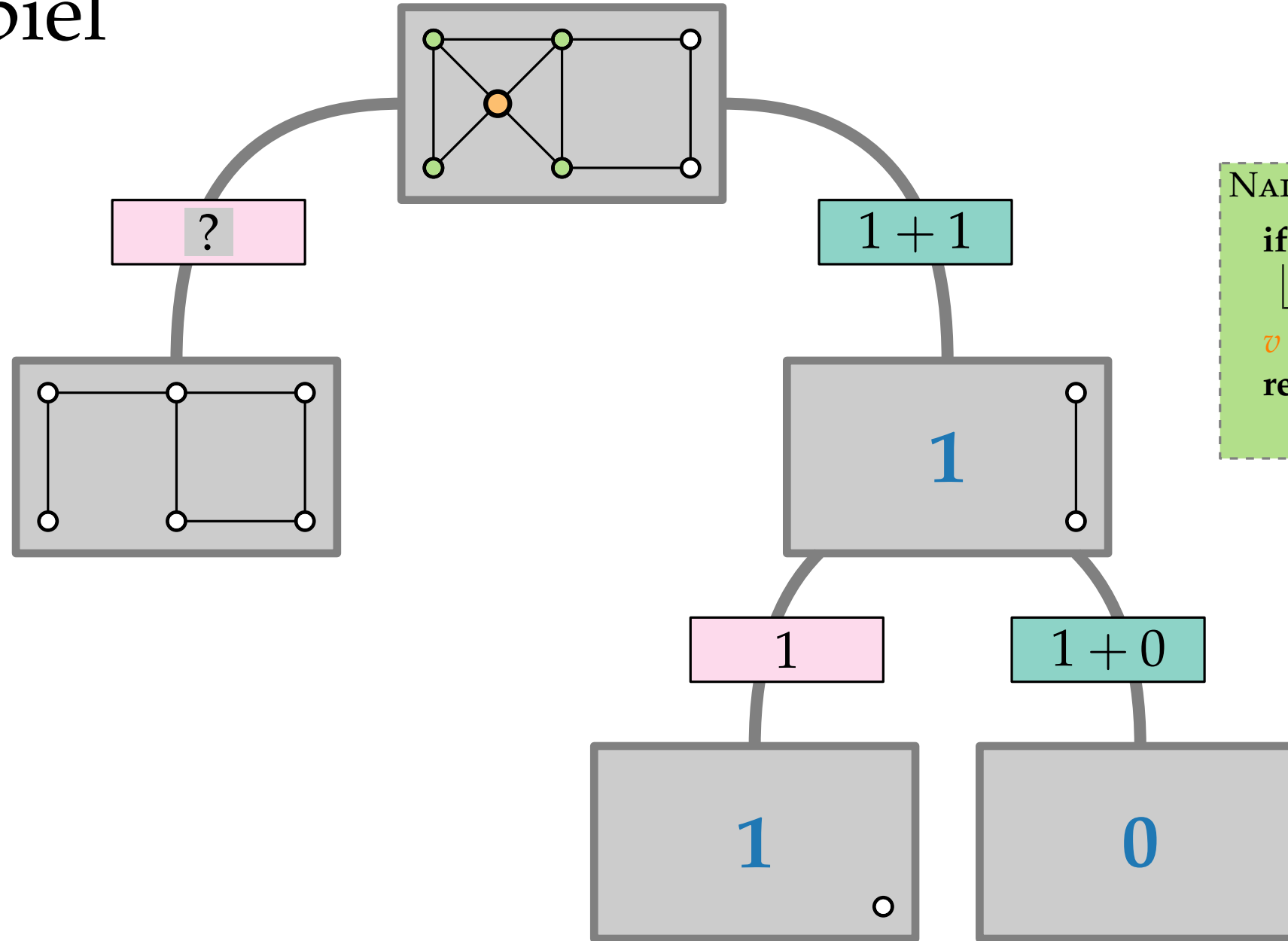
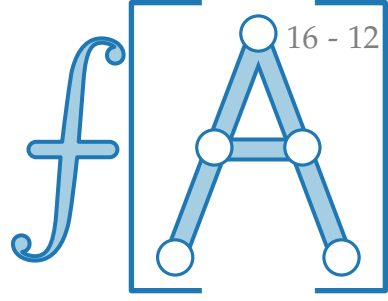
Beispiel



```

NaiveMIS( $G$ )
  if  $V = \emptyset$  then
    return 0
   $v$  = Beliebiger Knoten in  $V$ 
  return  $\max\{1 + \text{NaiveMIS}(G - N[v]),$ 
     $\text{NaiveMIS}(G - \{v\})\}$ 
    
```

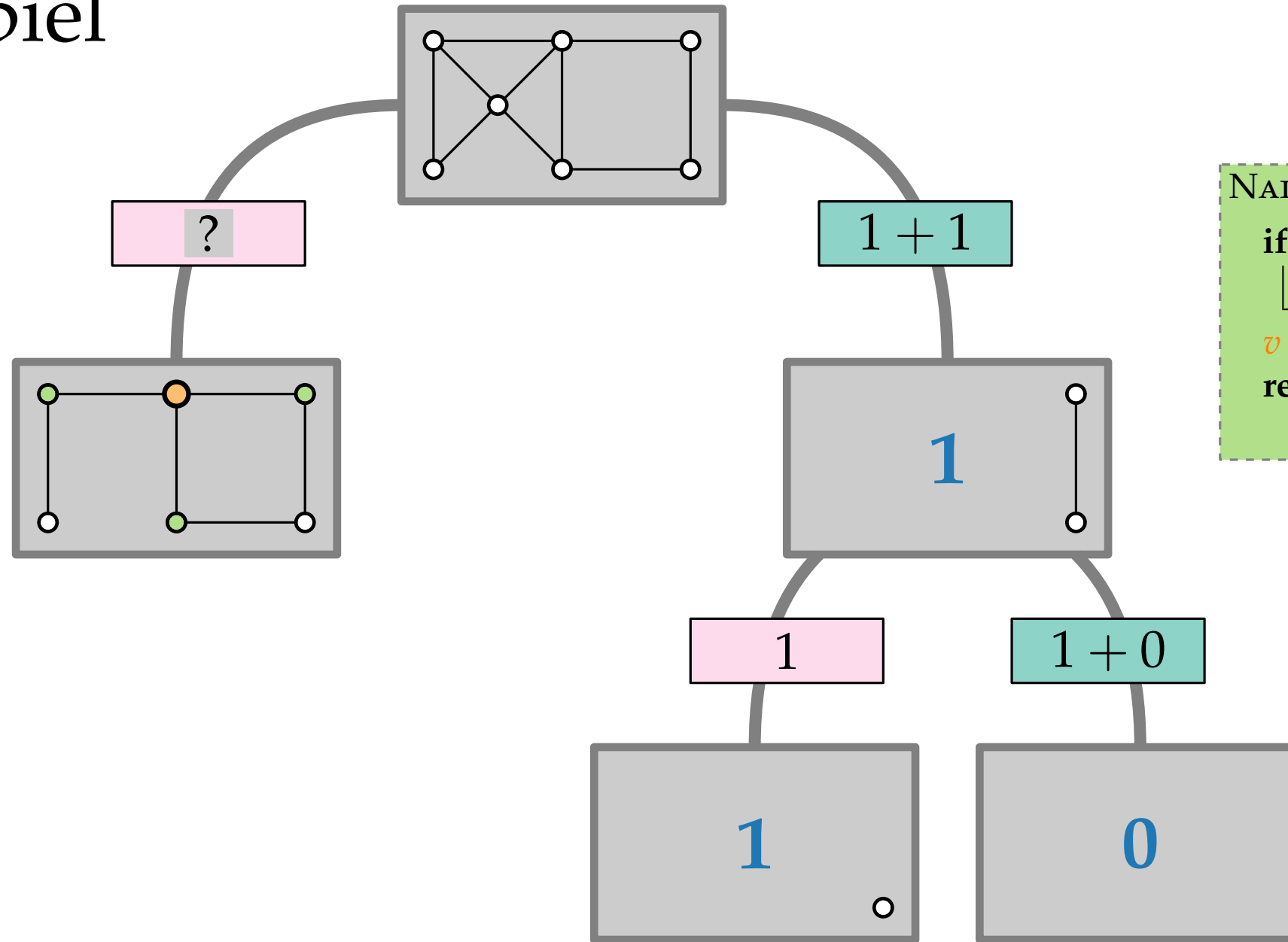
Beispiel



```

NaiveMIS( $G$ )
  if  $V = \emptyset$  then
    return 0
   $v$  = Beliebiger Knoten in  $V$ 
  return  $\max\{1 + \text{NaiveMIS}(G - N[v]),$ 
     $\text{NaiveMIS}(G - \{v\})\}$ 
    
```

Beispiel

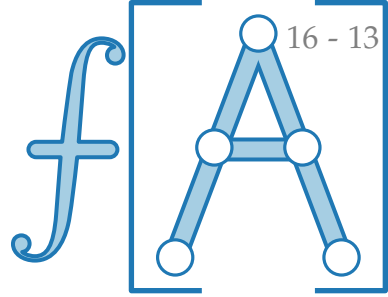

$$\text{NAIVEMIS}(G)$$

if $V = \emptyset$ then

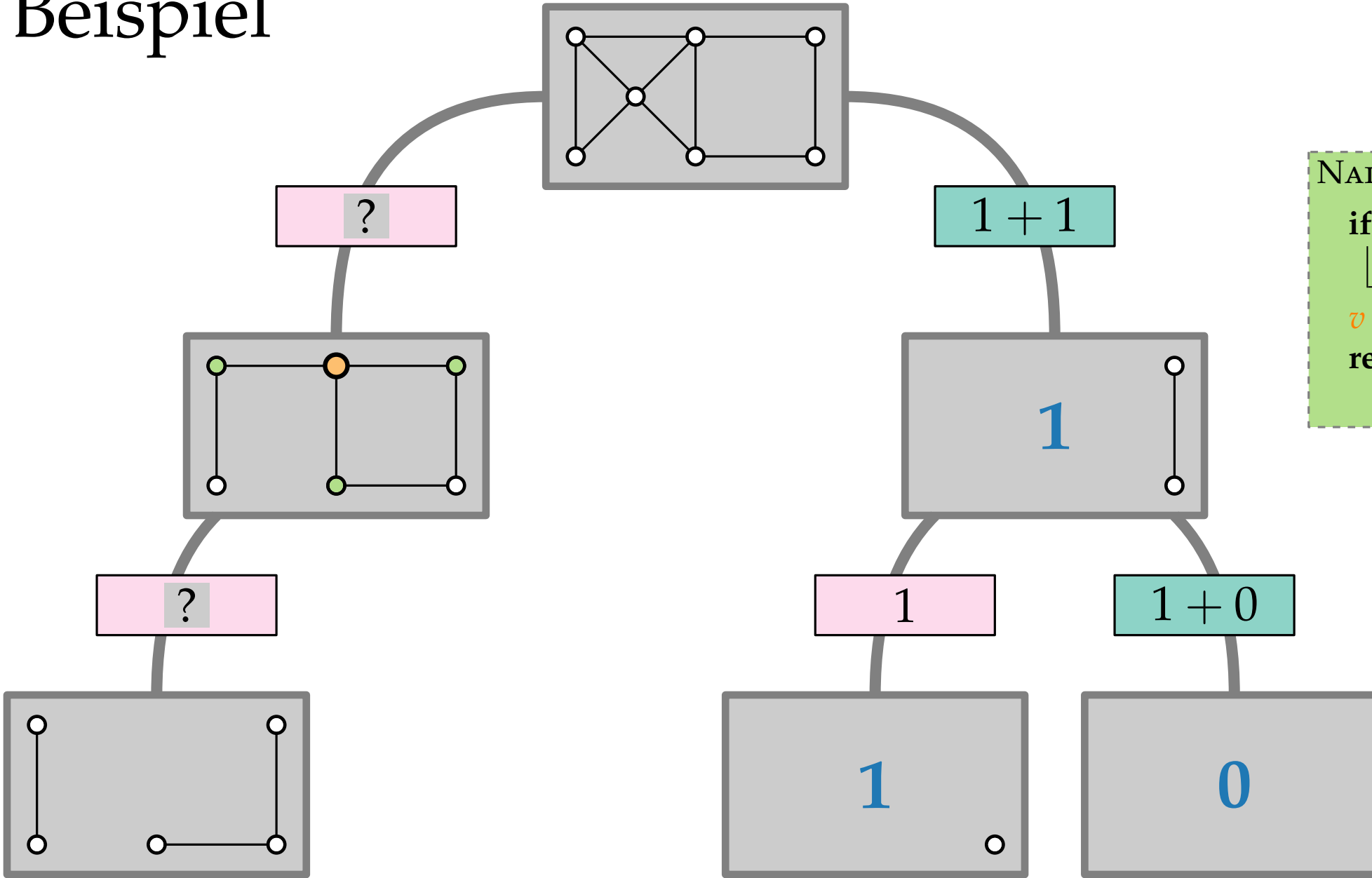
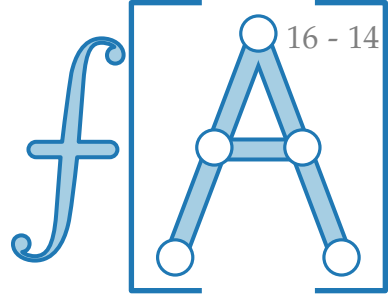
```
return 0
```

v = Beliebiger Knoten in V

```
return max{1 + NaiveMIS( $G - N[v]$ ),  
          NaiveMIS( $G - \{v\}$ )}
```



Beispiel



NAIVEMIS(G)

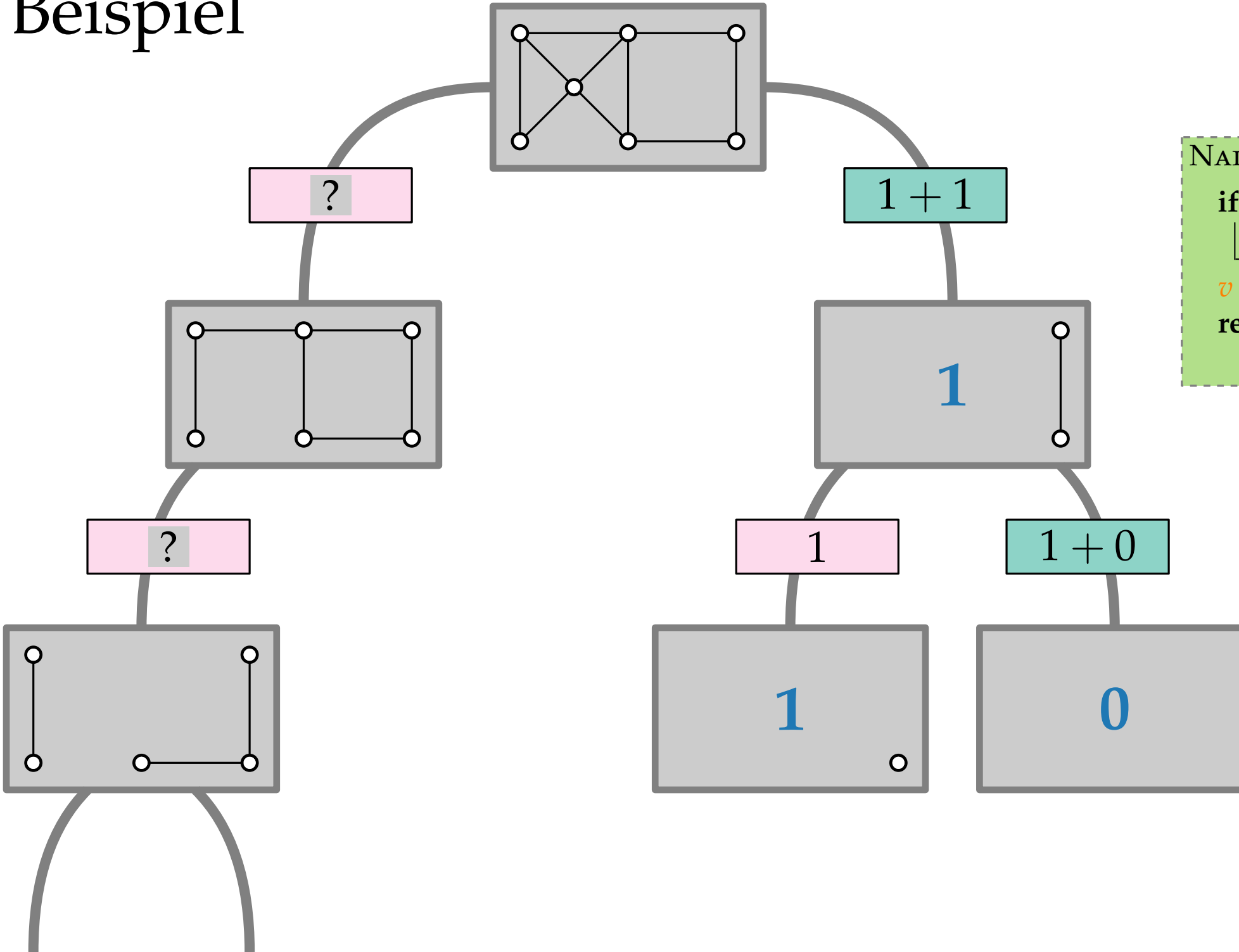
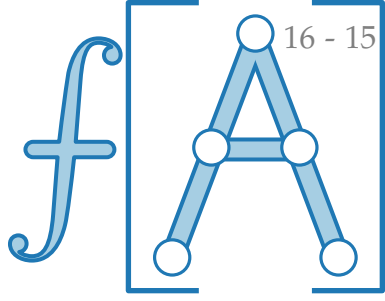
if $V = \emptyset$ **then**

return 0

v = Beliebiger Knoten in V

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$

Beispiel



NAIVEMIS(G)

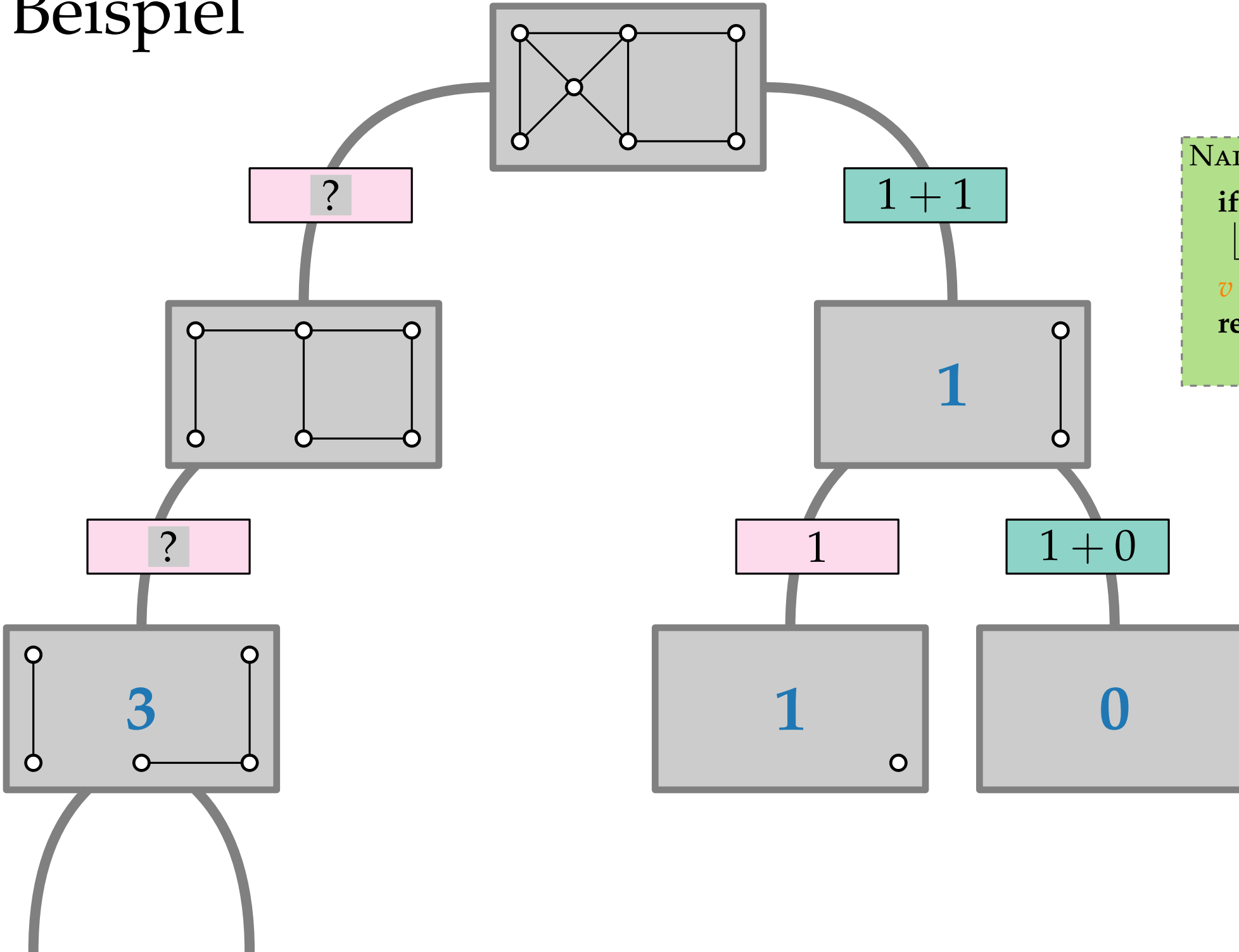
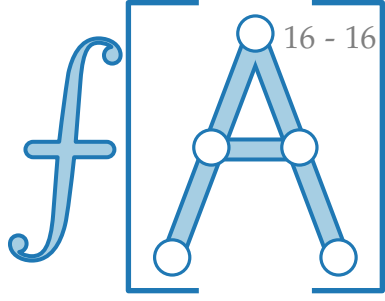
if $V = \emptyset$ **then**

└ **return** 0

v = Beliebiger Knoten in V

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$

Beispiel



$\text{NaiveMIS}(G)$

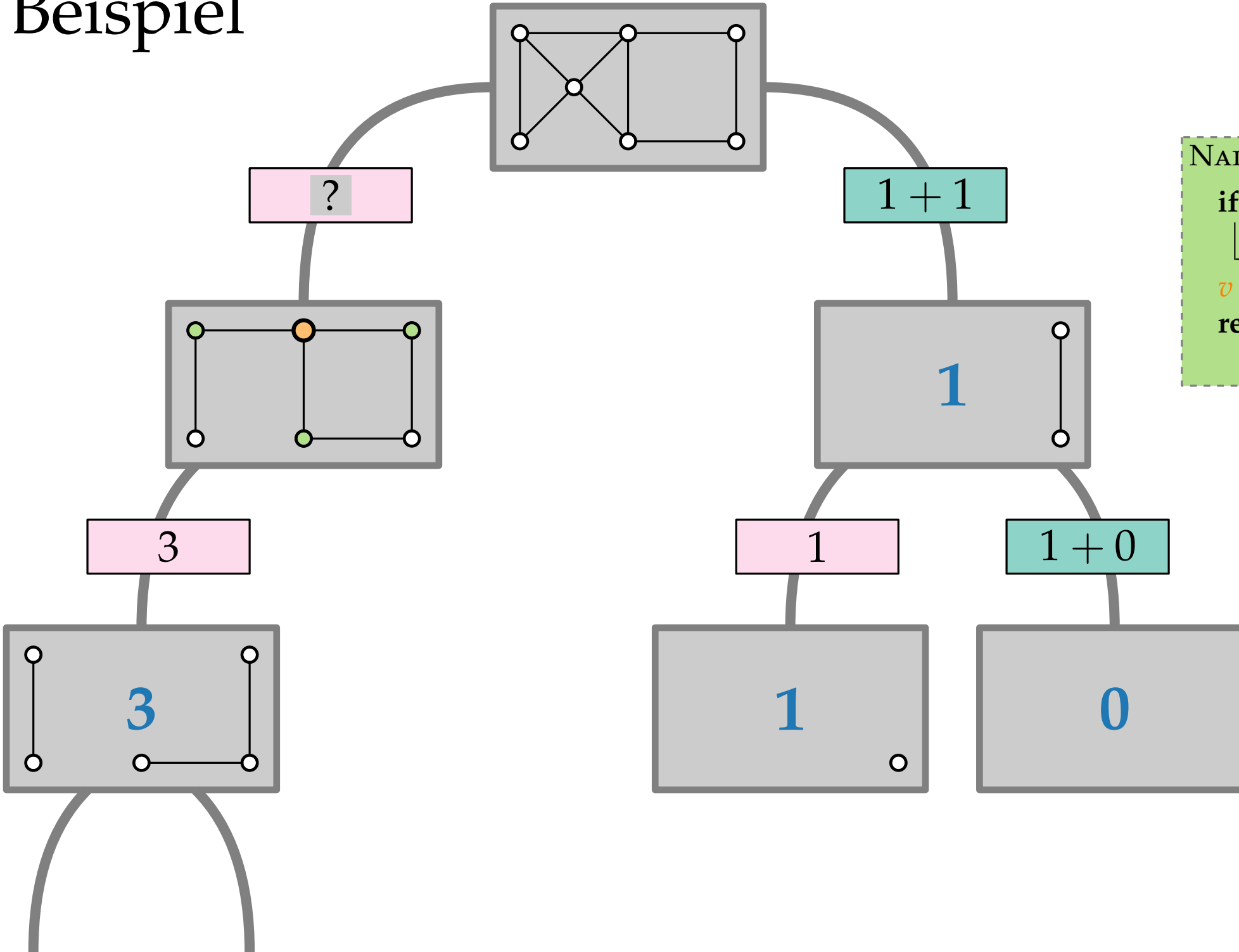
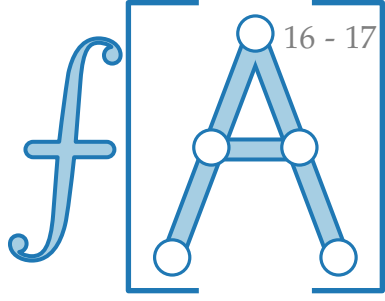
if $V = \emptyset$ **then**

return 0

v = Beliebiger Knoten in V

return $\max\{1 + \text{NaiveMIS}(G - N[v]),$
 $\text{NaiveMIS}(G - \{v\})\}$

Beispiel



$\text{NAIVEMIS}(G)$

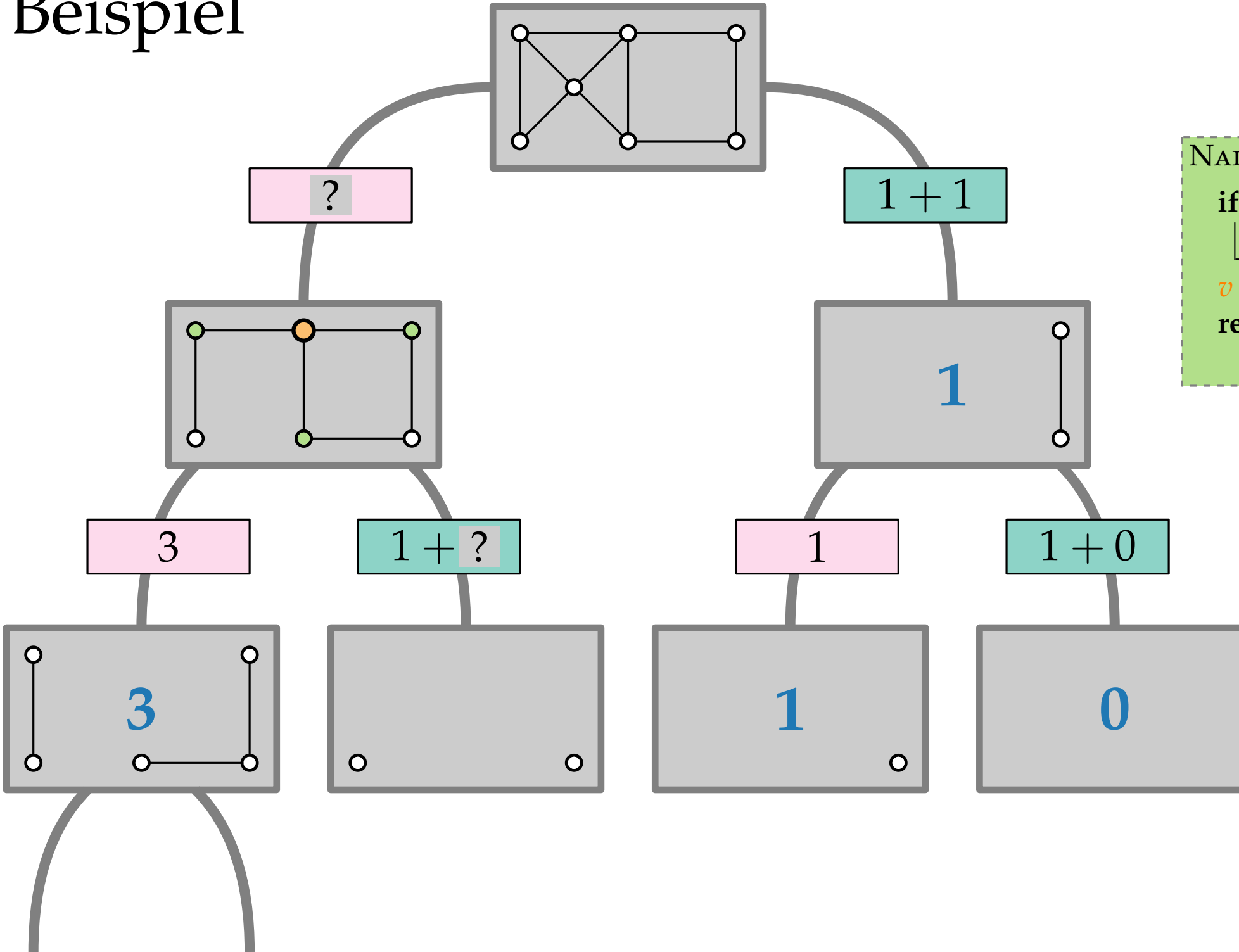
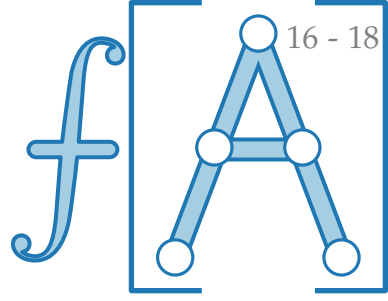
if $V = \emptyset$ **then**

return 0

v = Beliebiger Knoten in V

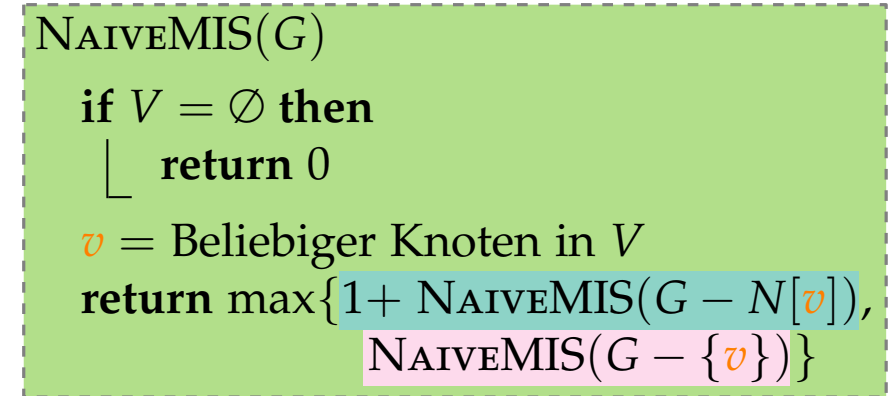
return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$

Beispiel

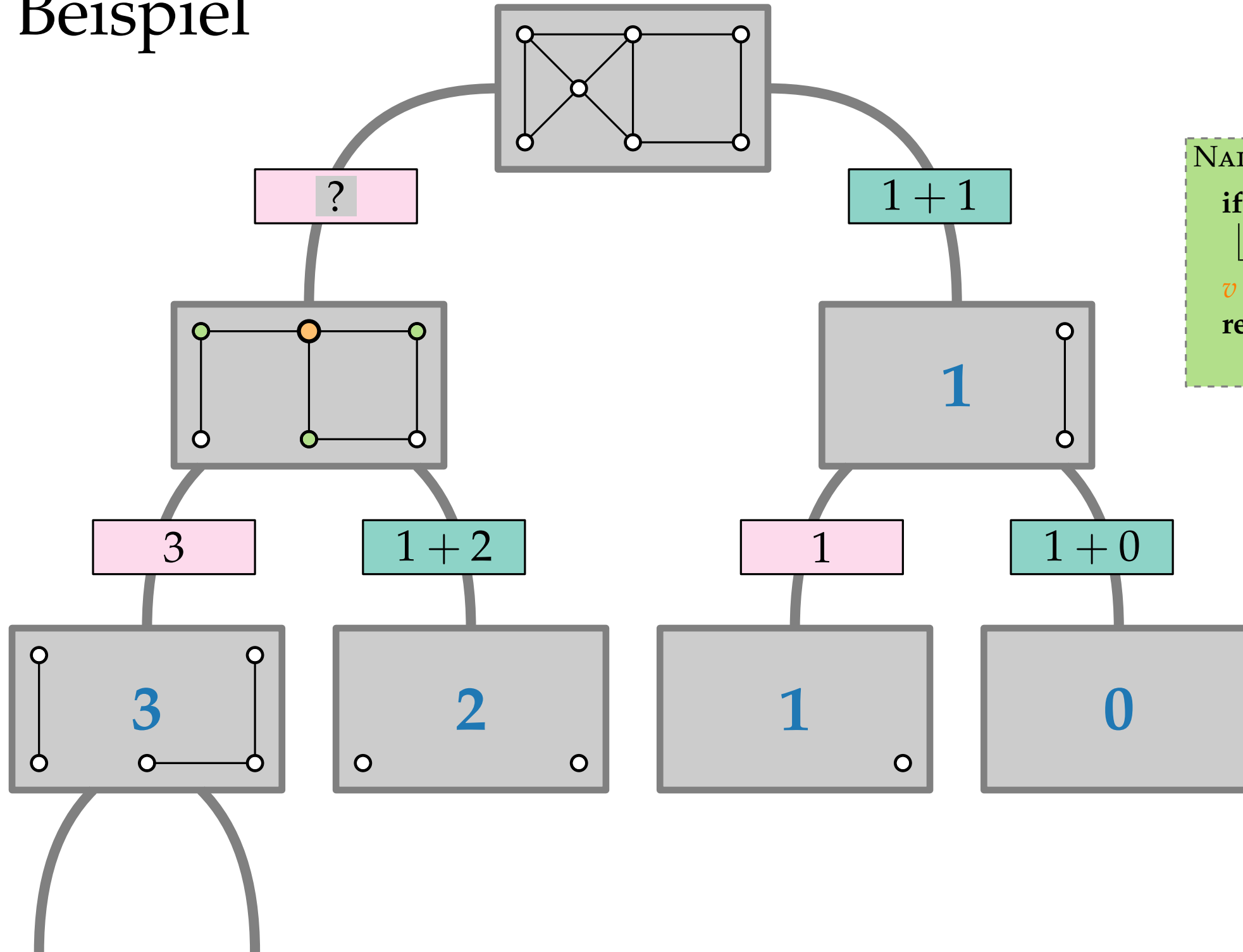
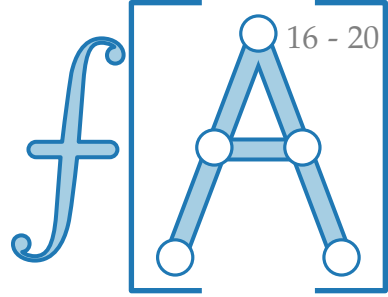


```

NaiveMIS( $G$ )
  if  $V = \emptyset$  then
    return 0
   $v$  = Beliebiger Knoten in  $V$ 
  return  $\max\{1 + \text{NaiveMIS}(G - N[v]),$ 
     $\text{NaiveMIS}(G - \{v\})\}$ 
  
```



Beispiel


$$\text{NAIVEMIS}(G)$$

if $V = \emptyset$ then

```

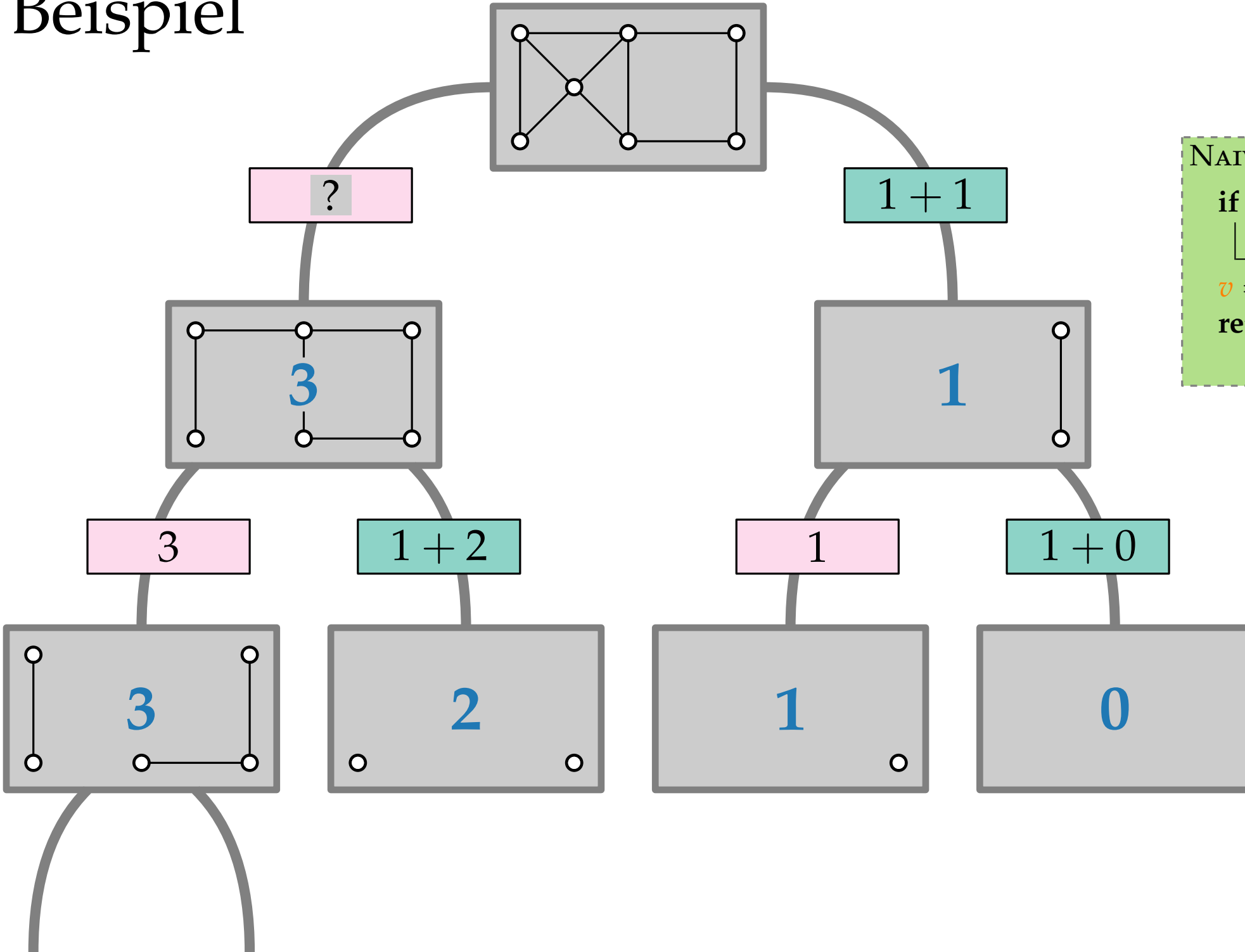
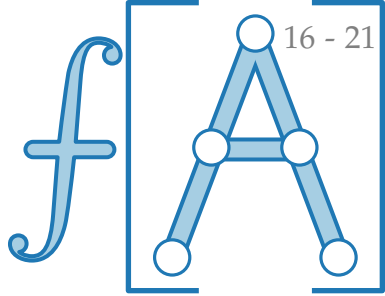
|         return 0

```

v = Beliebiger Knoten in V

```
return max{1 + NaiveMIS( $G - N[v]$ ),  
          NaiveMIS( $G - \{v\}$ )}
```

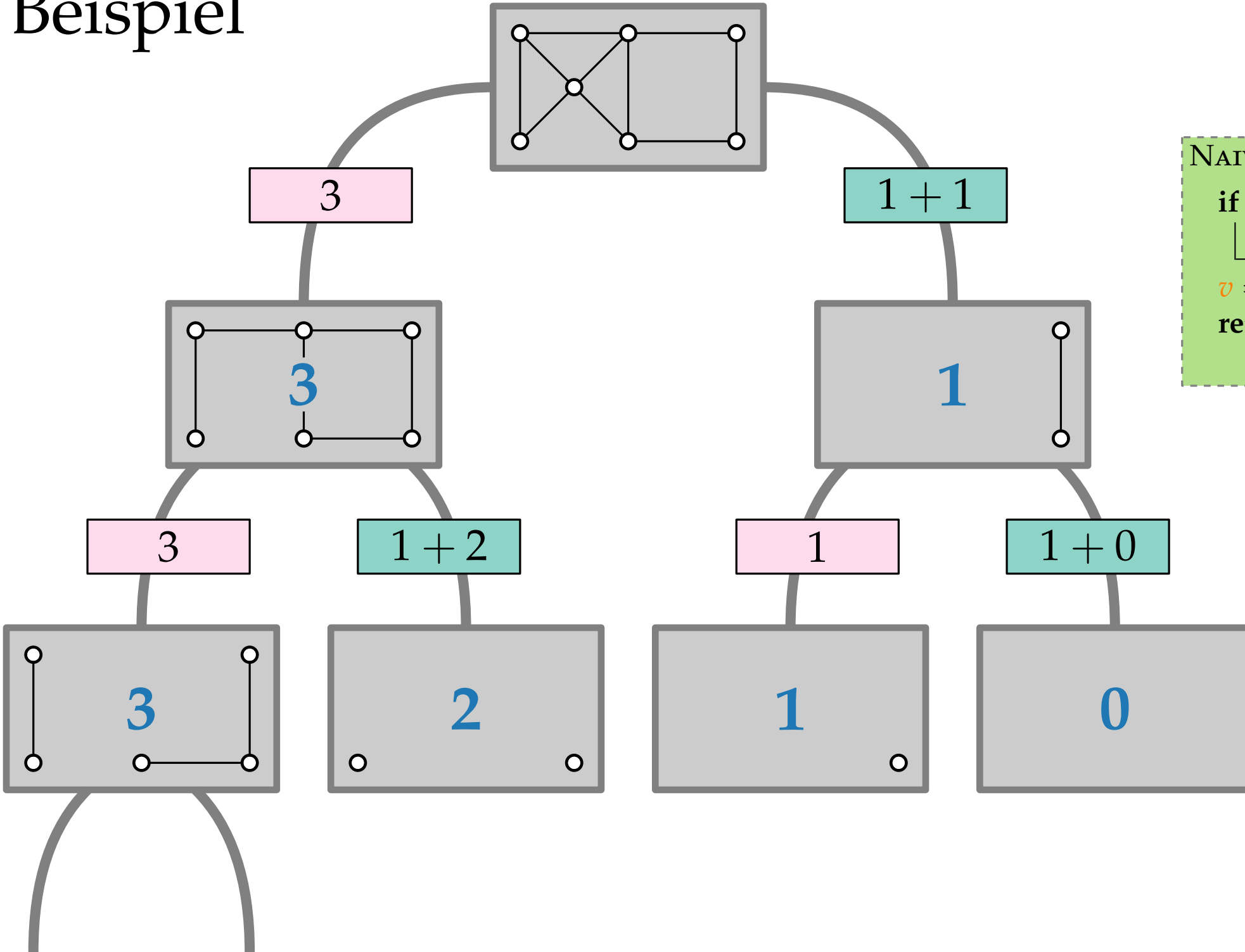
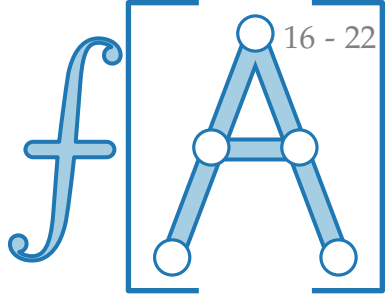
Beispiel



```

NaiveMIS(G)
  if  $V = \emptyset$  then
    return 0
   $v$  = Beliebiger Knoten in  $V$ 
  return  $\max\{1 + \text{NaiveMIS}(G - N[v]),$ 
     $\text{NaiveMIS}(G - \{v\})\}$ 
    
```

Beispiel



NAIVEMIS(G)

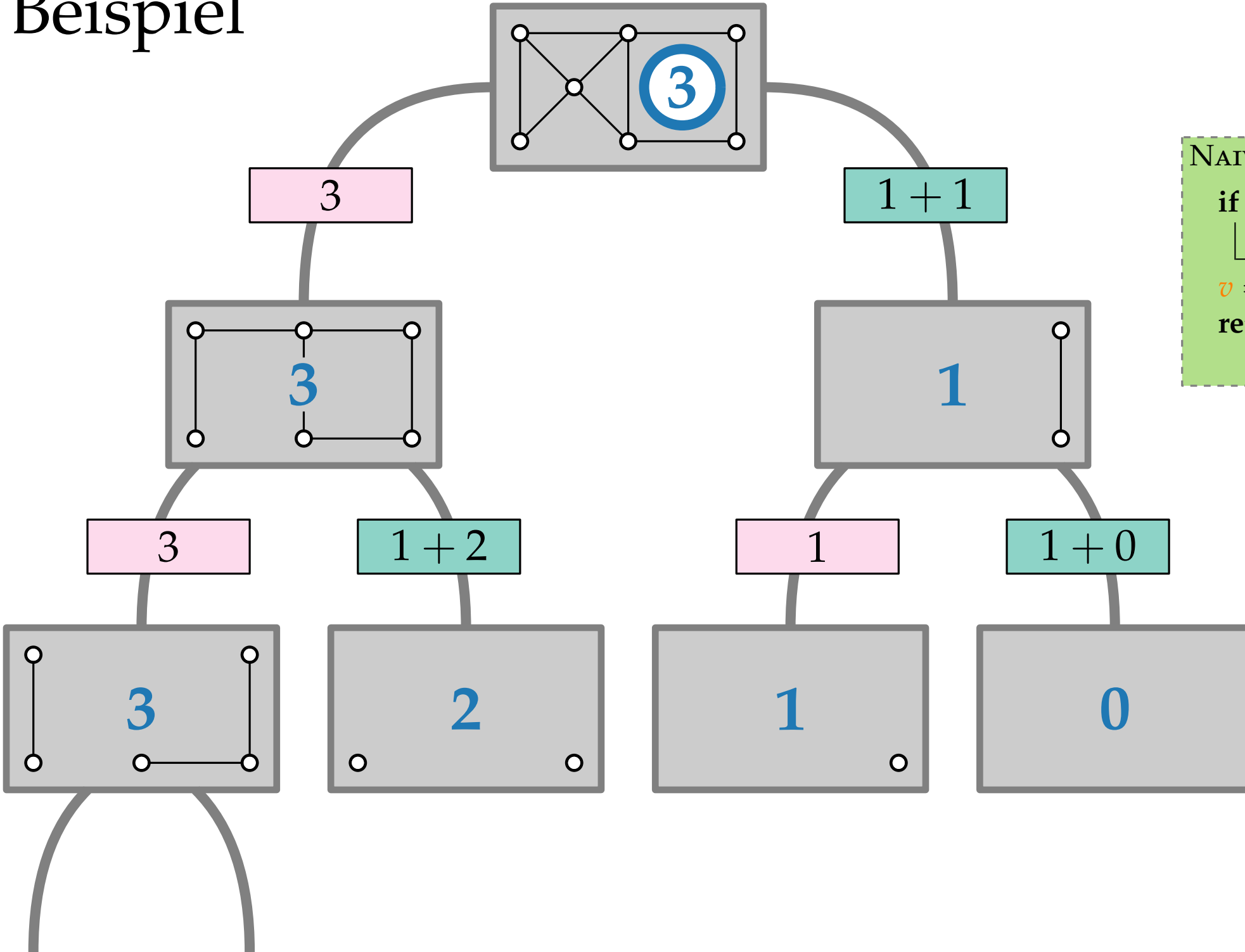
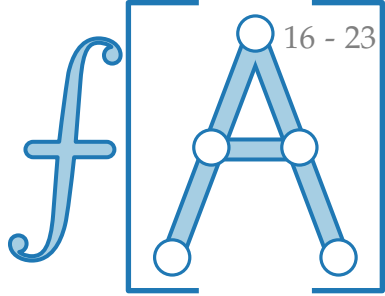
if $V = \emptyset$ **then**

└ **return** 0

v = Beliebiger Knoten in V

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$

Beispiel



NAIVEMIS(G)

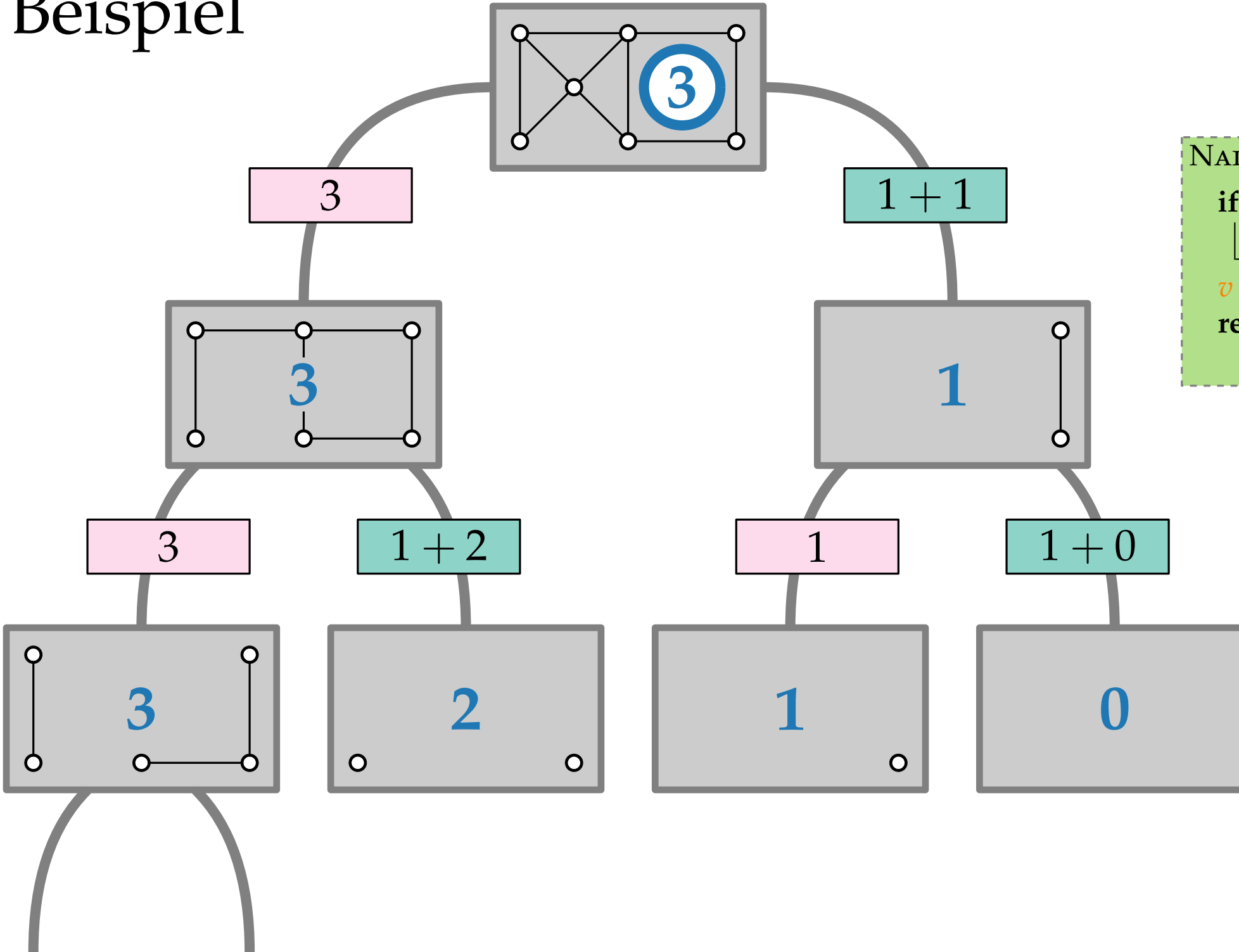
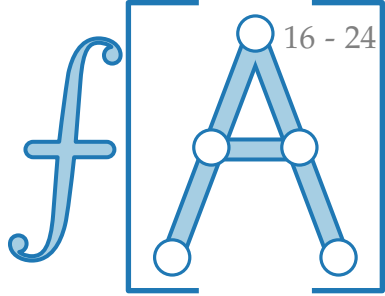
if $V = \emptyset$ **then**

return 0

v = Beliebiger Knoten in V

return $\max\{1 + \text{NAIVEMIS}(G - N[v]),$
 $\text{NAIVEMIS}(G - \{v\})\}$

Beispiel

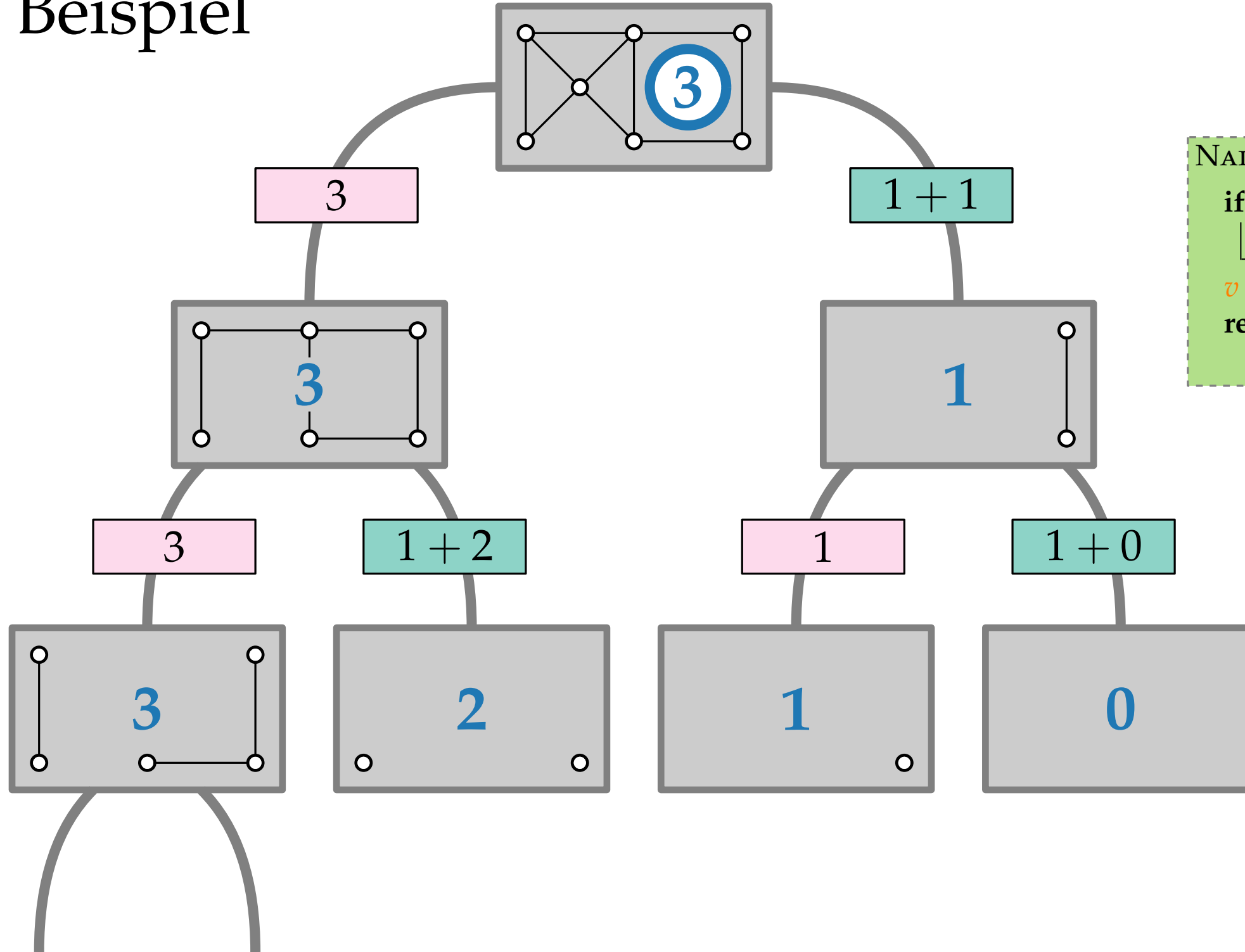
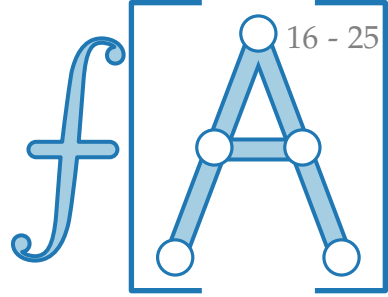


```

NaiveMIS( $G$ )
  if  $V = \emptyset$  then
    return 0
   $v$  = Beliebiger Knoten in  $V$ 
  return  $\max\{1 + \text{NaiveMIS}(G - N[v]),$ 
     $\text{NaiveMIS}(G - \{v\})\}$ 
    
```

Laufzeit?

Beispiel

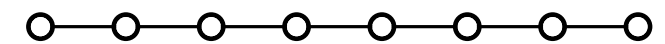


```

NAIVEMIS( $G$ )
  if  $V = \emptyset$  then
    return 0
   $v$  = Beliebiger Knoten in  $V$ 
  return  $\max\{1 + \text{NAIVEMIS}(G - N[v]),$ 
     $\text{NAIVEMIS}(G - \{v\})\}$ 
    
```

Laufzeit?

Immer noch $\mathcal{O}^*(2^n)$

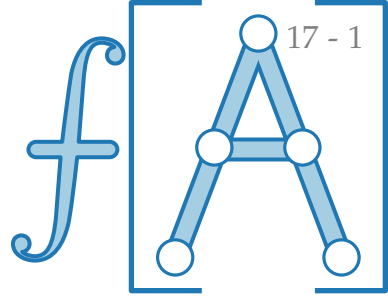


Schlaueres Branching

$$N[v] := N(v) \cup \{v\}$$

Lemma.

Sei U eine größte unabhängige Menge in G .

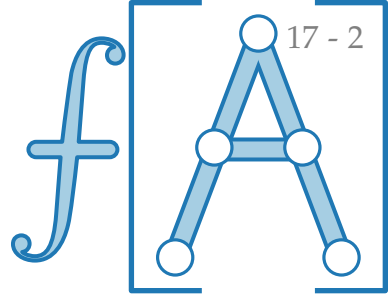


Schlaueres Branching

$$N[v] := N(v) \cup \{v\}$$

Lemma.

Sei U eine größte unabhängige Menge in G .
Dann gilt für jeden Knoten $v \in V$:



• v

Schlaueres Branching

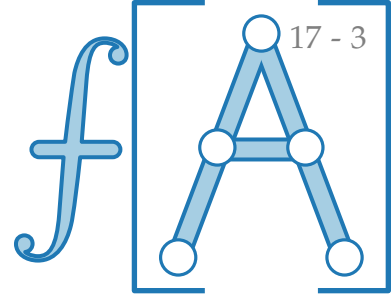
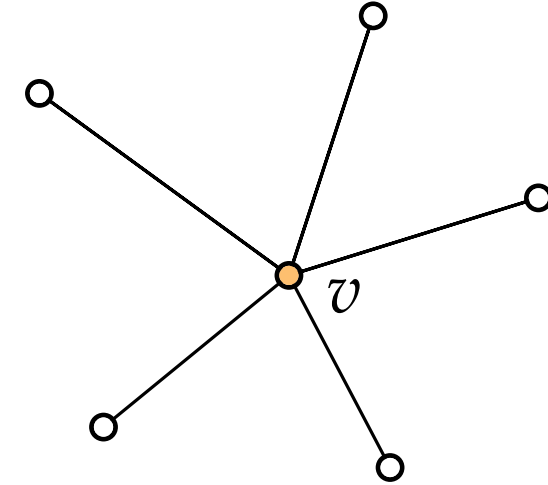
$$N[v] := N(v) \cup \{v\}$$

Lemma.

Sei U eine größte unabhängige Menge in G .

Dann gilt für jeden Knoten $v \in V$:

1. $v \in U \Rightarrow N(v) \cap U = \emptyset$



Schlauerer Branching

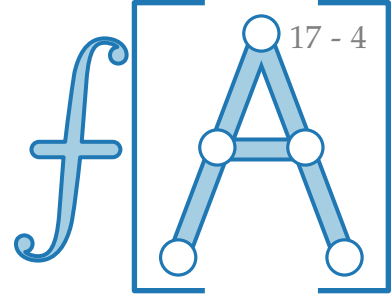
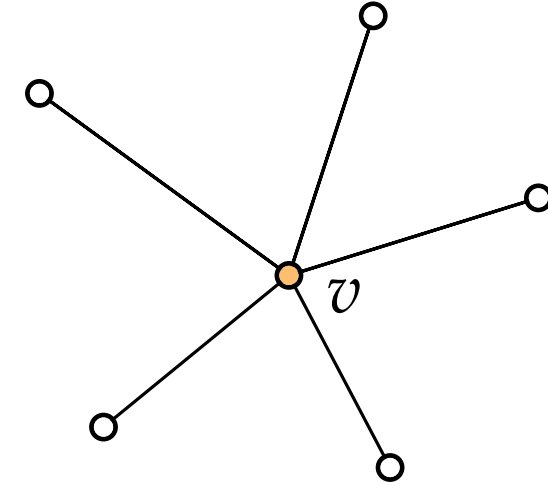
$$N[v] := N(v) \cup \{v\}$$

Lemma.

Sei U eine größte unabhängige Menge in G .

Dann gilt für jeden Knoten $v \in V$:

1. $v \in U \Rightarrow N(v) \cap U = \emptyset$
2. $v \notin U \Rightarrow |N(v) \cap U| \geq 1$



Schlaueres Branching

$$N[v] := N(v) \cup \{v\}$$

Lemma.

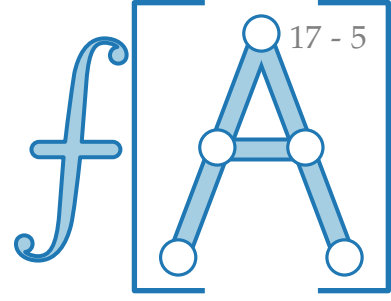
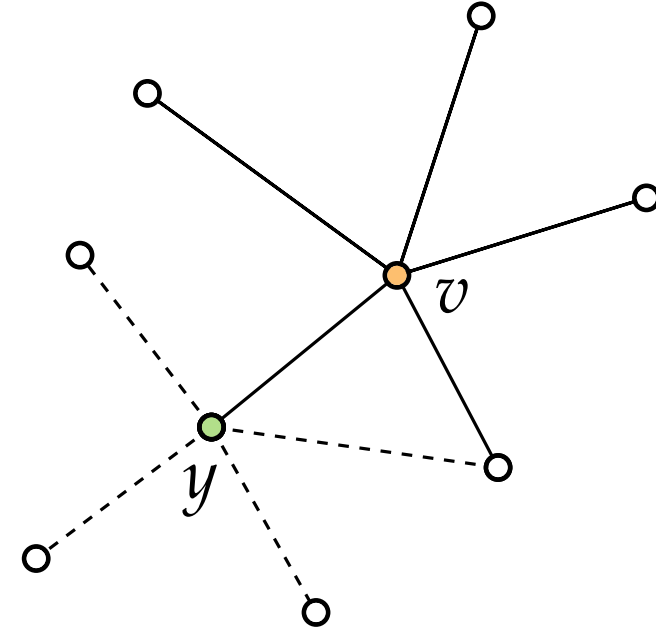
Sei U eine größte unabhängige Menge in G .

Dann gilt für jeden Knoten $v \in V$:

1. $v \in U \Rightarrow N(v) \cap U = \emptyset$

2. $v \notin U \Rightarrow |N(v) \cap U| \geq 1$

Also enthält $N[v]$ ein $y \in U$ und kein anderer Knoten von $N[y]$ ist in U .



Schlaueres Branching

$$N[v] := N(v) \cup \{v\}$$

Lemma.

Sei U eine größte unabhängige Menge in G .

Dann gilt für jeden Knoten $v \in V$:

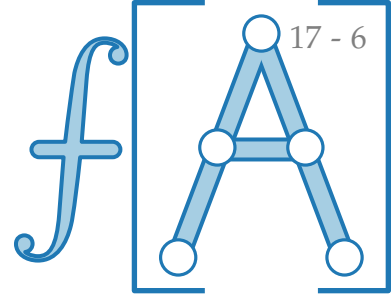
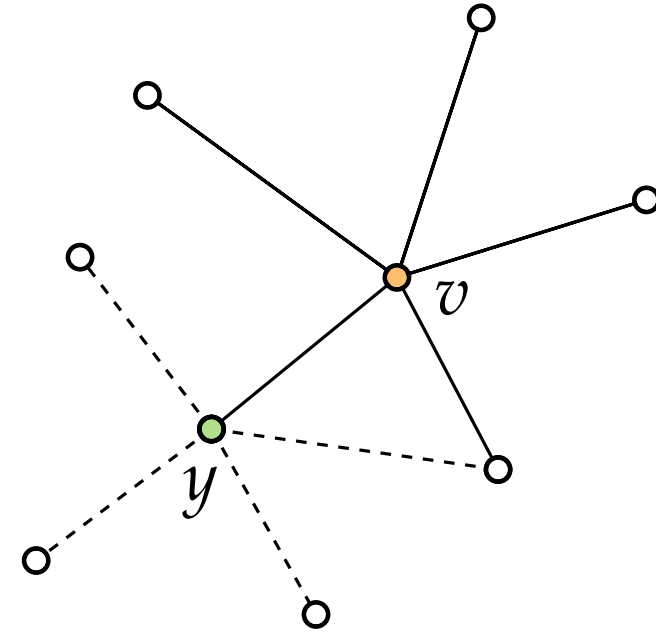
1. $v \in U \Rightarrow N(v) \cap U = \emptyset$

2. $v \notin U \Rightarrow |N(v) \cap U| \geq 1$

Also enthält $N[v]$ ein $y \in U$ und kein anderer Knoten von $N[y]$ ist in U .

Schlaueres MIS Branching.

- Für einen Knoten v , branche auf Knoten in $N[v]$.



Schlaueres Branching

$$N[v] := N(v) \cup \{v\}$$

Lemma.

Sei U eine größte unabhängige Menge in G .

Dann gilt für jeden Knoten $v \in V$:

1. $v \in U \Rightarrow N(v) \cap U = \emptyset$

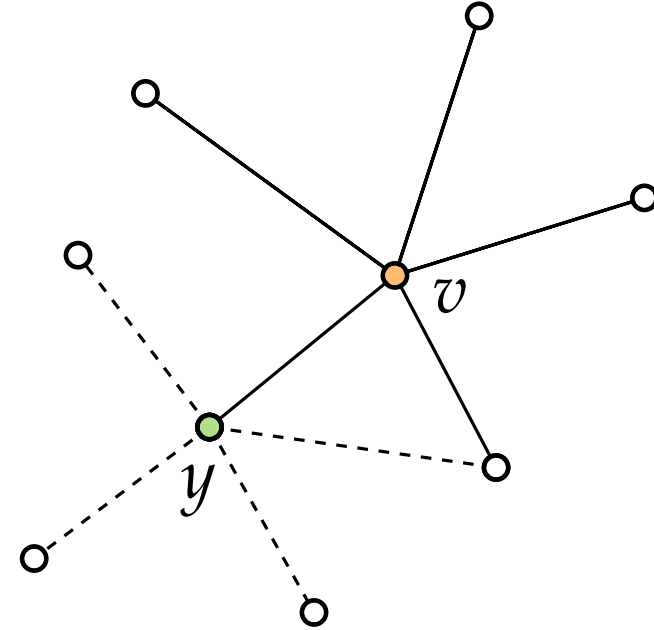
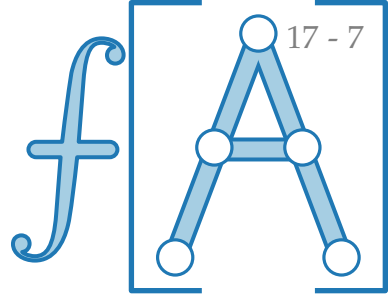
2. $v \notin U \Rightarrow |N(v) \cap U| \geq 1$

Also enthält $N[v]$ ein $y \in U$ und kein anderer Knoten von $N[y]$ ist in U .

Schlaueres MIS Branching.

- Für einen Knoten v , branche auf Knoten in $N[v]$.

SMARTERMIS(G)



Schlaueres Branching

$$N[v] := N(v) \cup \{v\}$$

Lemma.

Sei U eine größte unabhängige Menge in G .

Dann gilt für jeden Knoten $v \in V$:

1. $v \in U \Rightarrow N(v) \cap U = \emptyset$

2. $v \notin U \Rightarrow |N(v) \cap U| \geq 1$

Also enthält $N[v]$ ein $y \in U$ und kein anderer Knoten von $N[y]$ ist in U .

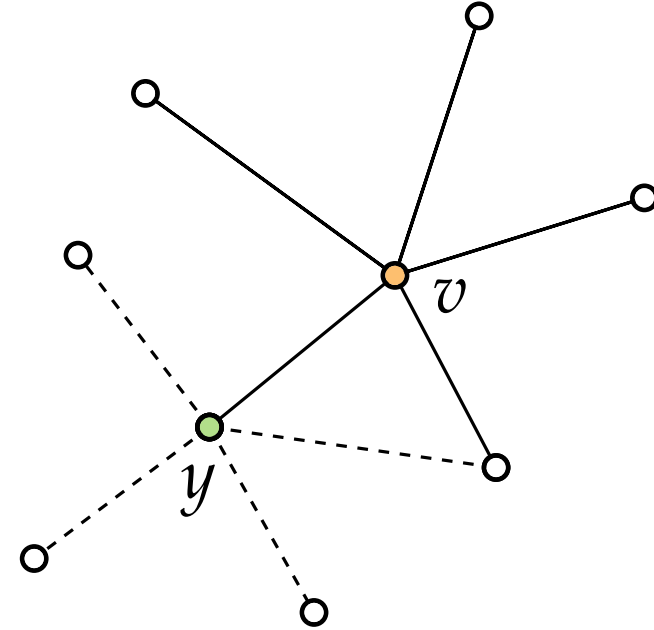
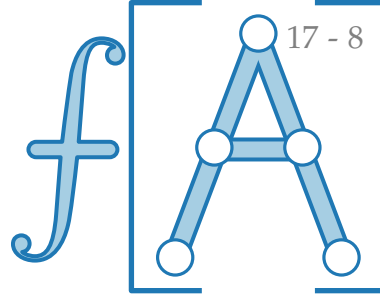
Schlaueres MIS Branching.

- Für einen Knoten v , branche auf Knoten in $N[v]$.

SMARTERMIS(G)

if $V = \emptyset$ then

└ return 0



Schlaueres Branching

$$N[v] := N(v) \cup \{v\}$$

Lemma.

Sei U eine größte unabhängige Menge in G .

Dann gilt für jeden Knoten $v \in V$:

1. $v \in U \Rightarrow N(v) \cap U = \emptyset$

2. $v \notin U \Rightarrow |N(v) \cap U| \geq 1$

Also enthält $N[v]$ ein $y \in U$ und kein anderer Knoten von $N[y]$ ist in U .

Schlaueres MIS Branching.

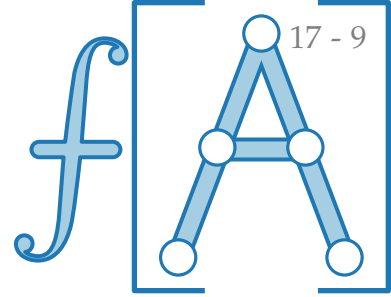
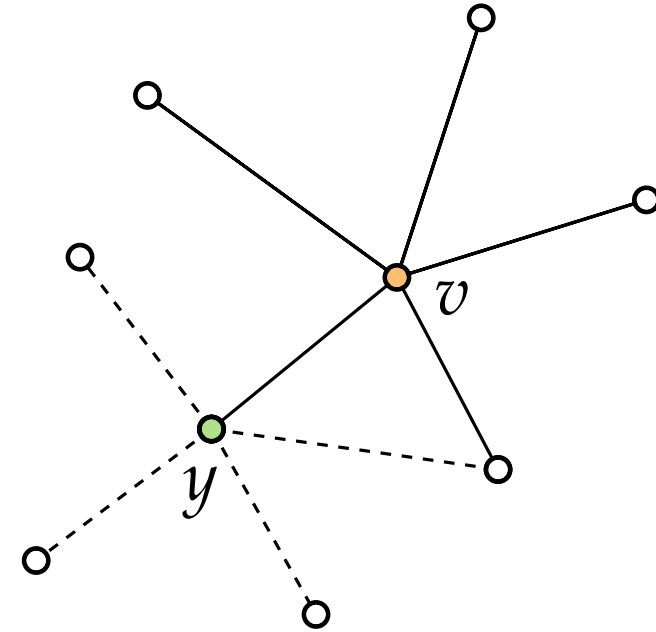
- Für einen Knoten v , branche auf Knoten in $N[v]$.

SMARTERMIS(G)

if $V = \emptyset$ then

└ return 0

v = Knoten mit kleinstem Grad in $V(G)$



Schlaueres Branching

$$N[v] := N(v) \cup \{v\}$$

Lemma.

Sei U eine größte unabhängige Menge in G .

Dann gilt für jeden Knoten $v \in V$:

1. $v \in U \Rightarrow N(v) \cap U = \emptyset$

2. $v \notin U \Rightarrow |N(v) \cap U| \geq 1$

Also enthält $N[v]$ ein $y \in U$ und kein anderer Knoten von $N[y]$ ist in U .

Schlaueres MIS Branching.

■ Für einen Knoten v , branche auf Knoten in $N[v]$.

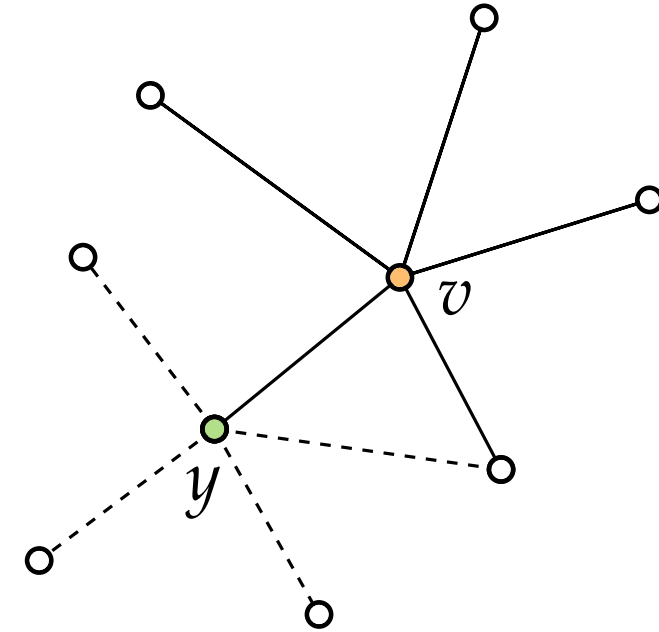
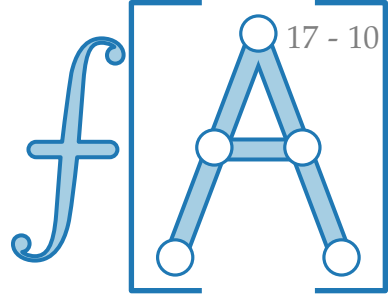
SMARTERMIS(G)

if $V = \emptyset$ then

└ return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{$



Schlaueres Branching

$$N[v] := N(v) \cup \{v\}$$

Lemma.

Sei U eine größte unabhängige Menge in G .

Dann gilt für jeden Knoten $v \in V$:

1. $v \in U \Rightarrow N(v) \cap U = \emptyset$

2. $v \notin U \Rightarrow |N(v) \cap U| \geq 1$

Also enthält $N[v]$ ein $y \in U$ und kein anderer Knoten von $N[y]$ ist in U .

Schlaueres MIS Branching.

■ Für einen Knoten v , branche auf Knoten in $N[v]$.

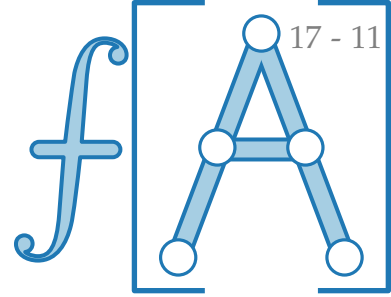
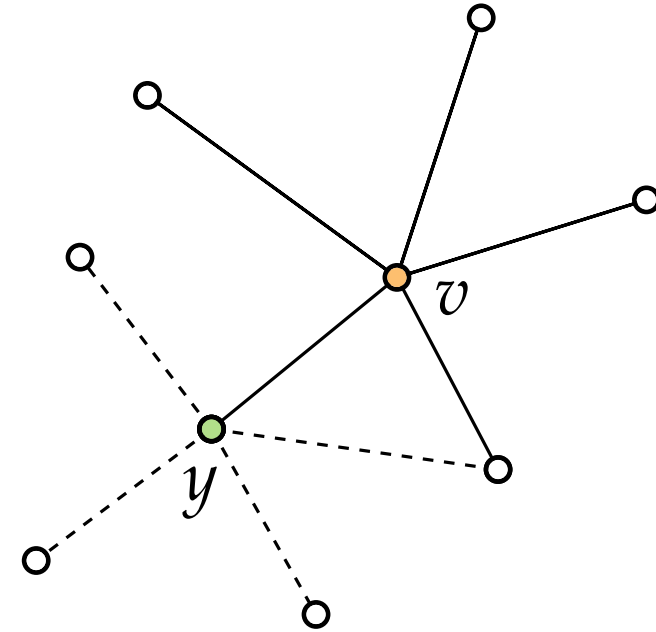
SMARTERMIS(G)

if $V = \emptyset$ then

└ return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{ \quad \mid y \in N[v] \}$



Schlaueres Branching

$$N[v] := N(v) \cup \{v\}$$

Lemma.

Sei U eine größte unabhängige Menge in G .

Dann gilt für jeden Knoten $v \in V$:

1. $v \in U \Rightarrow N(v) \cap U = \emptyset$

2. $v \notin U \Rightarrow |N(v) \cap U| \geq 1$

Also enthält $N[v]$ ein $y \in U$ und kein anderer Knoten von $N[y]$ ist in U .

Schlaueres MIS Branching.

■ Für einen Knoten v , branche auf Knoten in $N[v]$.

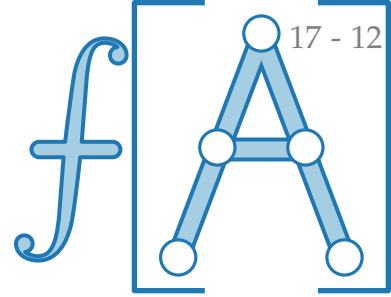
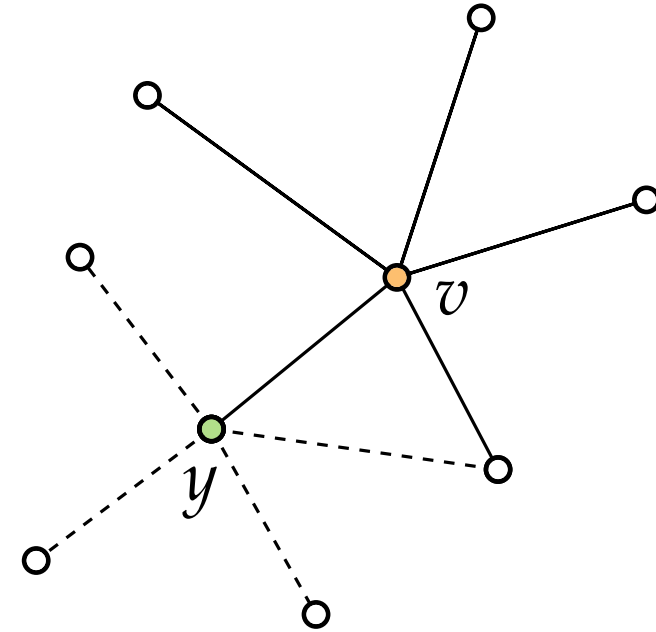
SMARTERMIS(G)

if $V = \emptyset$ then

└ return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$



Schlaueres Branching

$$N[v] := N(v) \cup \{v\}$$

Lemma.

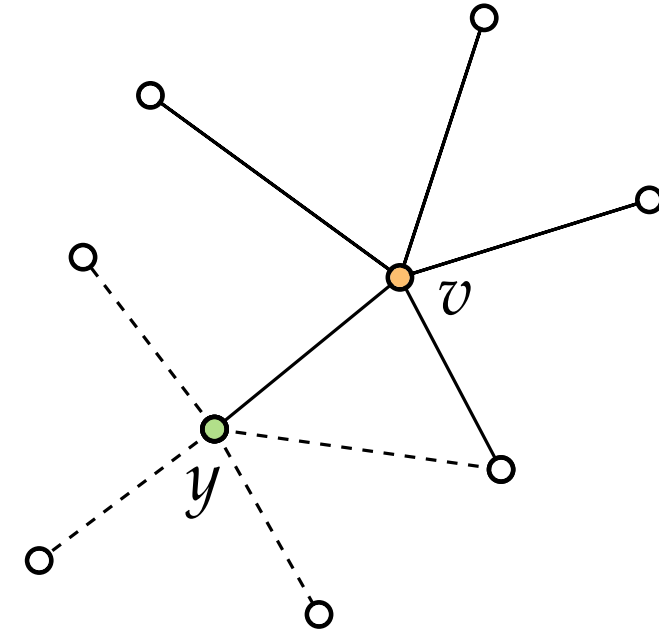
Sei U eine größte unabhängige Menge in G .

Dann gilt für jeden Knoten $v \in V$:

1. $v \in U \Rightarrow N(v) \cap U = \emptyset$

2. $v \notin U \Rightarrow |N(v) \cap U| \geq 1$

Also enthält $N[v]$ ein $y \in U$ und kein anderer Knoten von $N[y]$ ist in U .



Schlaueres MIS Branching.

- Für einen Knoten v , branche auf Knoten in $N[v]$.

SMARTERMIS(G)

if $V = \emptyset$ then

└ return 0

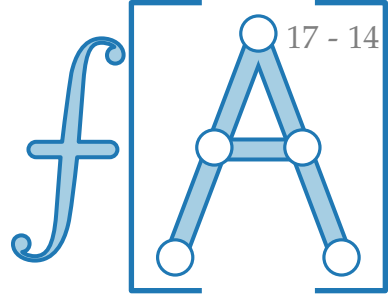
v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

- Korrektheit folgt aus Lemma

Schlaueres Branching

$$N[v] := N(v) \cup \{v\}$$



Lemma.

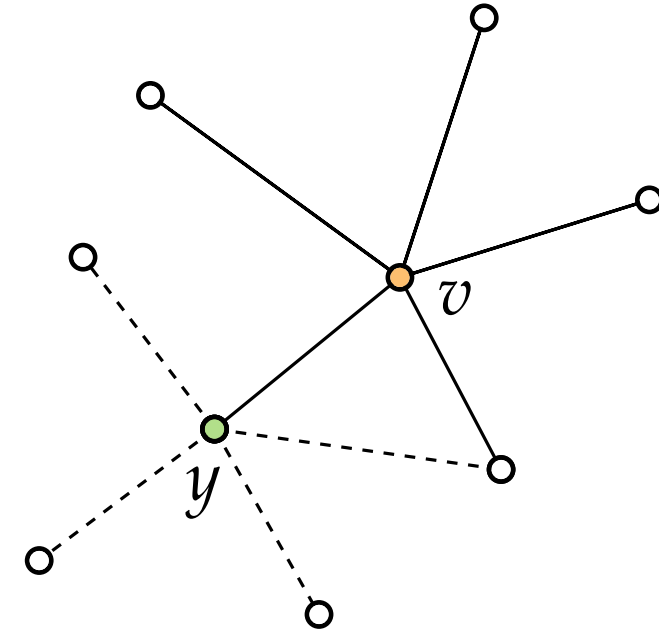
Sei U eine größte unabhängige Menge in G .

Dann gilt für jeden Knoten $v \in V$:

1. $v \in U \Rightarrow N(v) \cap U = \emptyset$

2. $v \notin U \Rightarrow |N(v) \cap U| \geq 1$

Also enthält $N[v]$ ein $y \in U$ und kein anderer Knoten von $N[y]$ ist in U .



Schlaueres MIS Branching.

- Für einen Knoten v , branche auf Knoten in $N[v]$.

SMARTERMIS(G)

if $V = \emptyset$ then

└ return 0

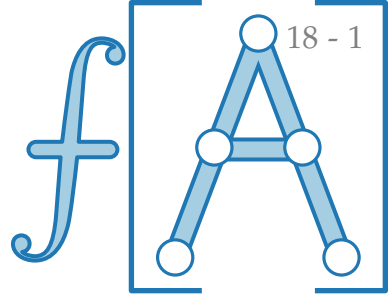
v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

- Korrektheit folgt aus Lemma
- Wir beweisen Laufzeit in $\mathcal{O}^*(3^{n/3}) \subset \mathcal{O}^*(1.4423^n)$

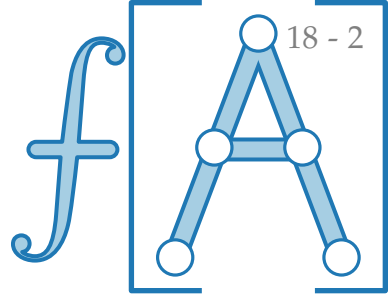
Branching Analyse

Ausführung entspricht einem **Suchbaum**, dessen Knoten mit dem Input des rekursiven Aufrufs beschriftet sind.



Branching Analyse

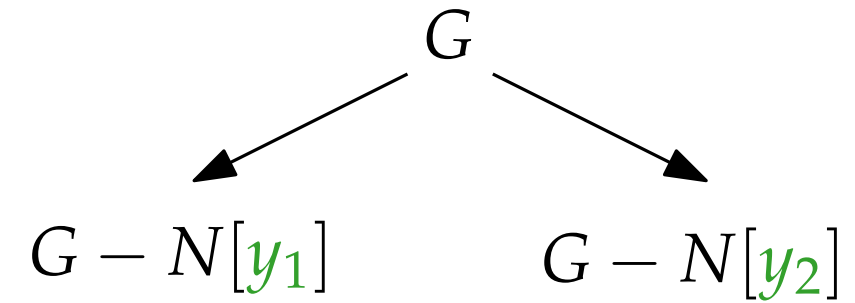
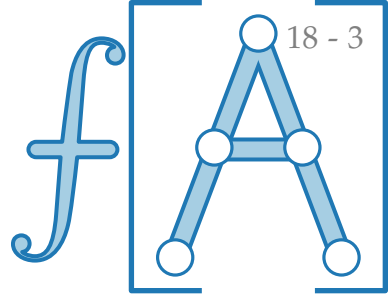
Ausführung entspricht einem **Suchbaum**, dessen Knoten mit dem Input des rekursiven Aufrufs beschriftet sind.



G

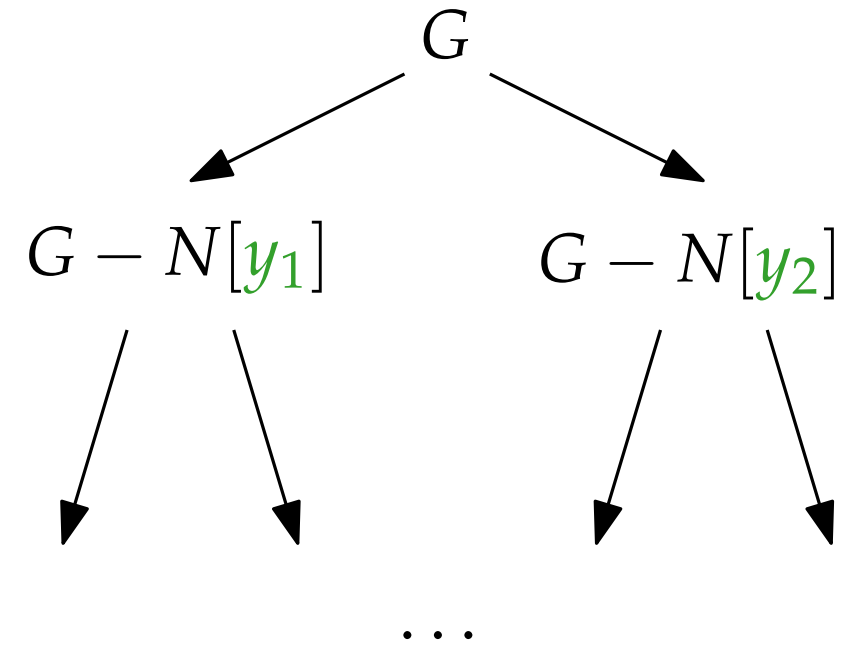
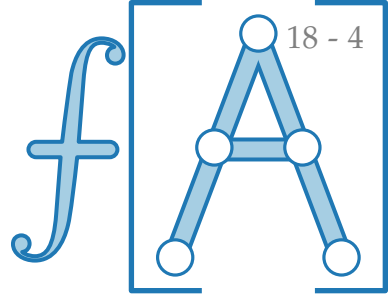
Branching Analyse

Ausführung entspricht einem **Suchbaum**, dessen Knoten mit dem Input des rekursiven Aufrufs beschriftet sind.



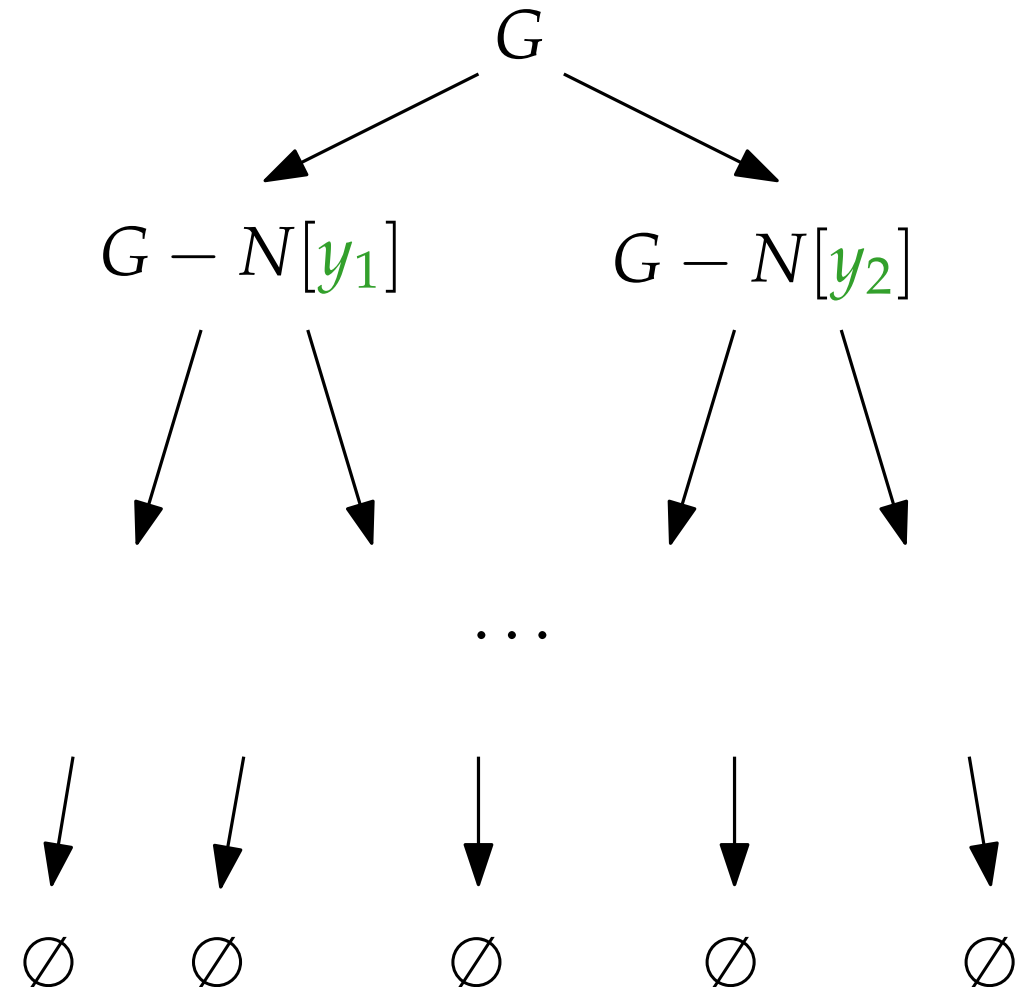
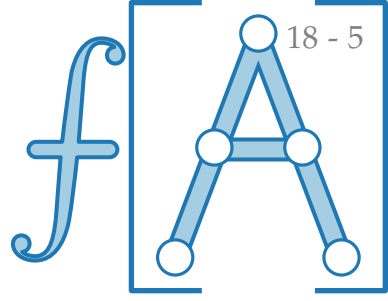
Branching Analyse

Ausführung entspricht einem **Suchbaum**, dessen Knoten mit dem Input des rekursiven Aufrufs beschriftet sind.

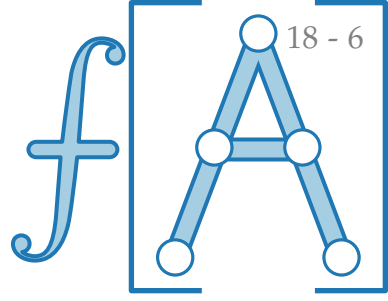


Branching Analyse

Ausführung entspricht einem **Suchbaum**, dessen Knoten mit dem Input des rekursiven Aufrufs beschriftet sind.

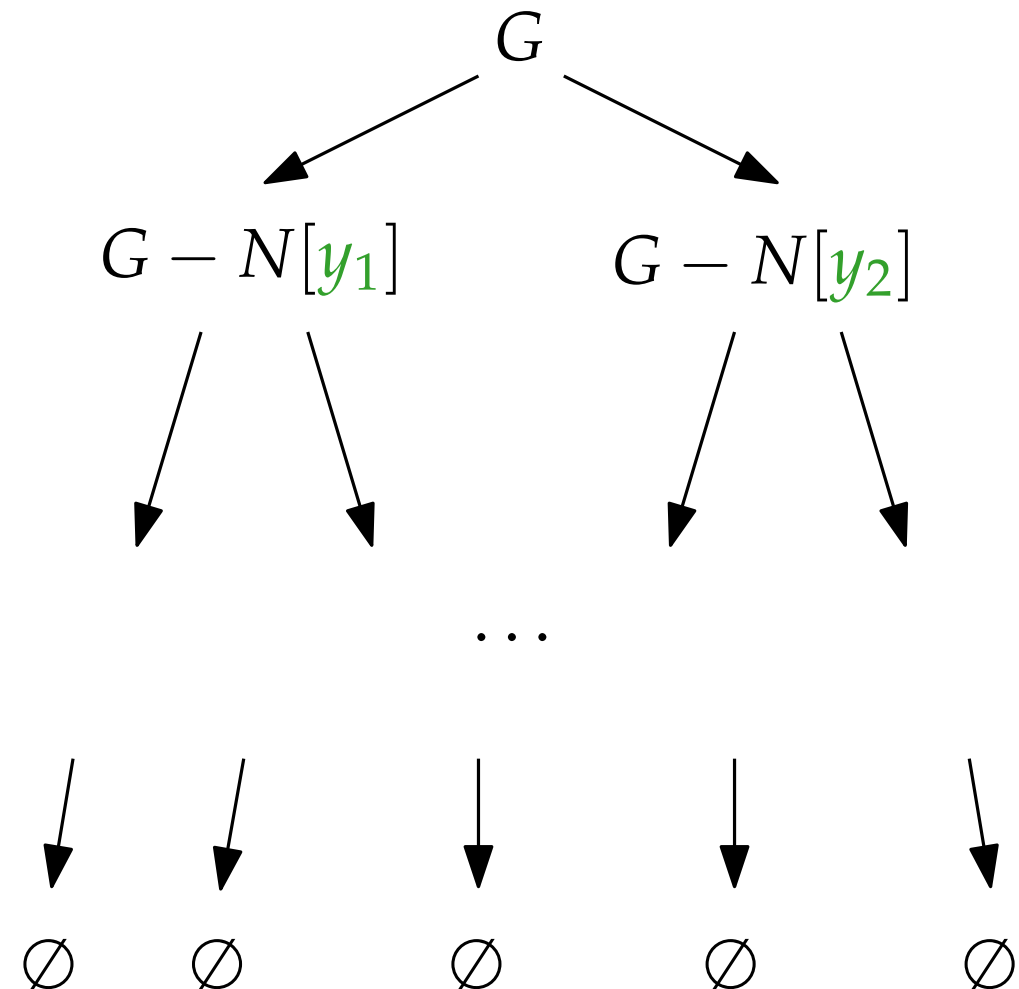


Branching Analyse

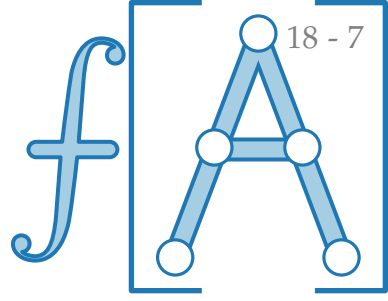


Ausführung entspricht einem **Suchbaum**, dessen Knoten mit dem Input des rekursiven Aufrufs beschriftet sind.

- Sei $B(n)$ die größte Anzahl der Blätter in einem Suchbaum für einen Graph mit n Knoten.

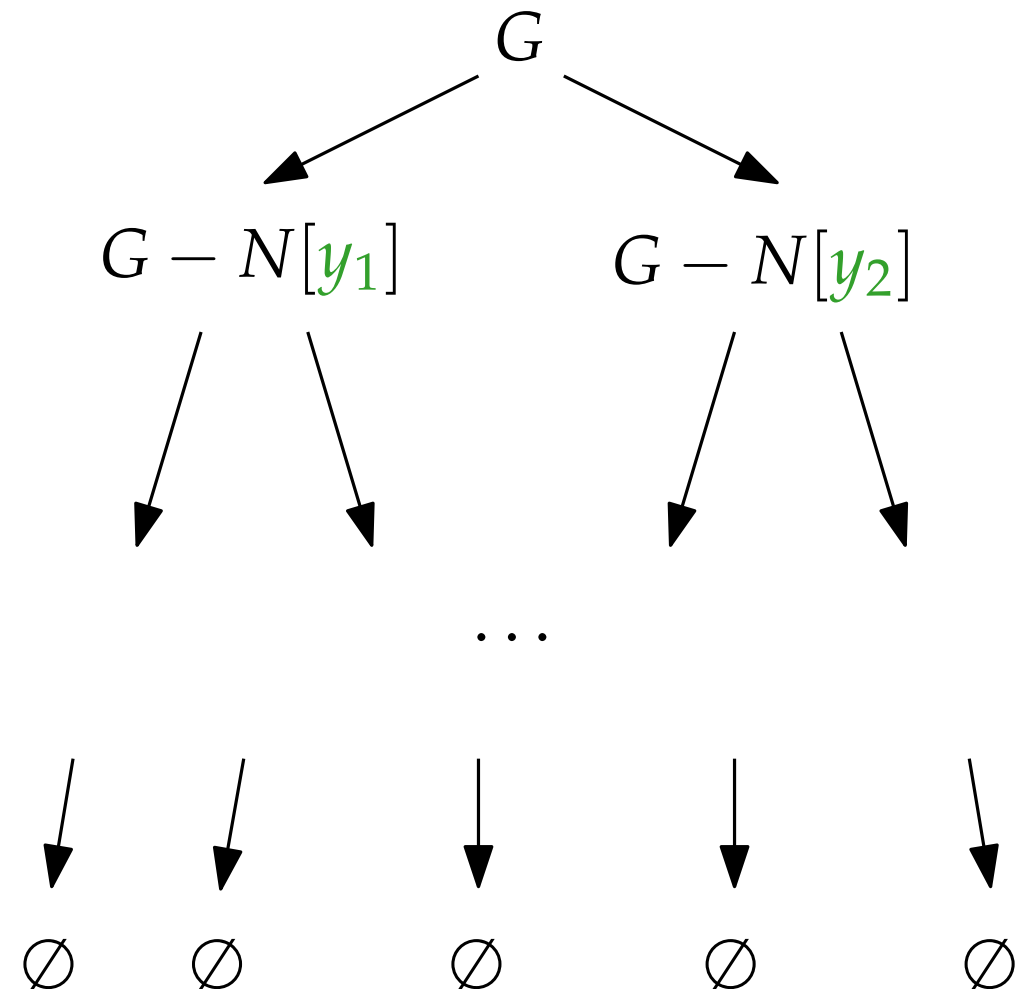


Branching Analyse

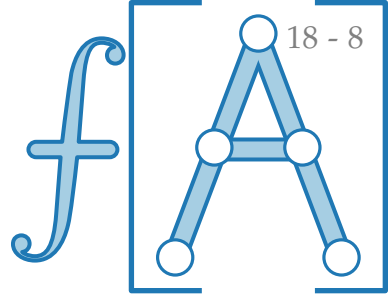


Ausführung entspricht einem **Suchbaum**, dessen Knoten mit dem Input des rekursiven Aufrufs beschriftet sind.

- Sei $B(n)$ die größte Anzahl der Blätter in einem Suchbaum für einen Graph mit n Knoten.
- Suchbaum hat Höhe $\leq n$



Branching Analyse



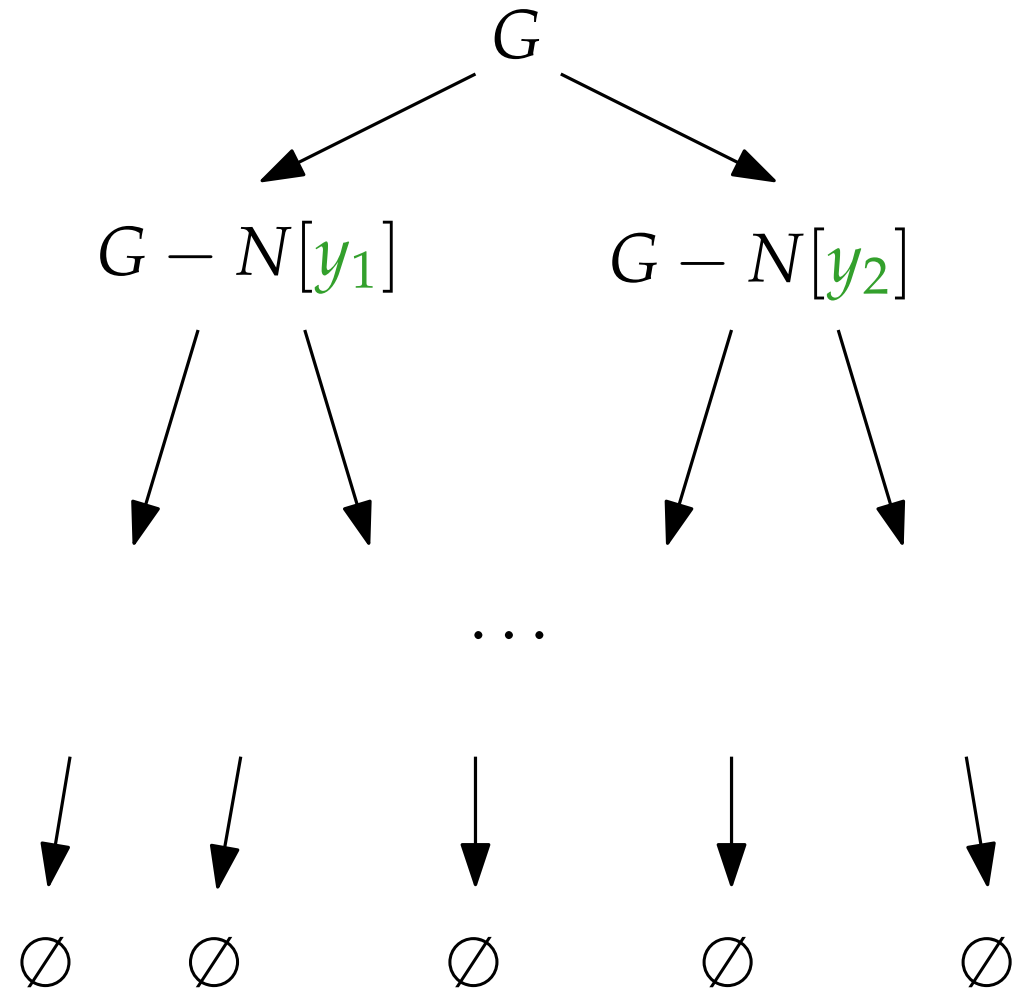
Ausführung entspricht einem **Suchbaum**, dessen Knoten mit dem Input des rekursiven Aufrufs beschriftet sind.

- Sei $B(n)$ die größte Anzahl der Blätter in einem Suchbaum für einen Graph mit n Knoten.

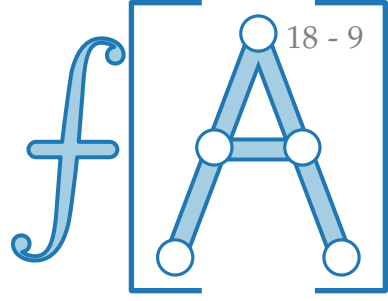
- Suchbaum hat Höhe $\leq n$

- ➔ Die Laufzeit des Algorithmus ist

$$T(n) \in$$



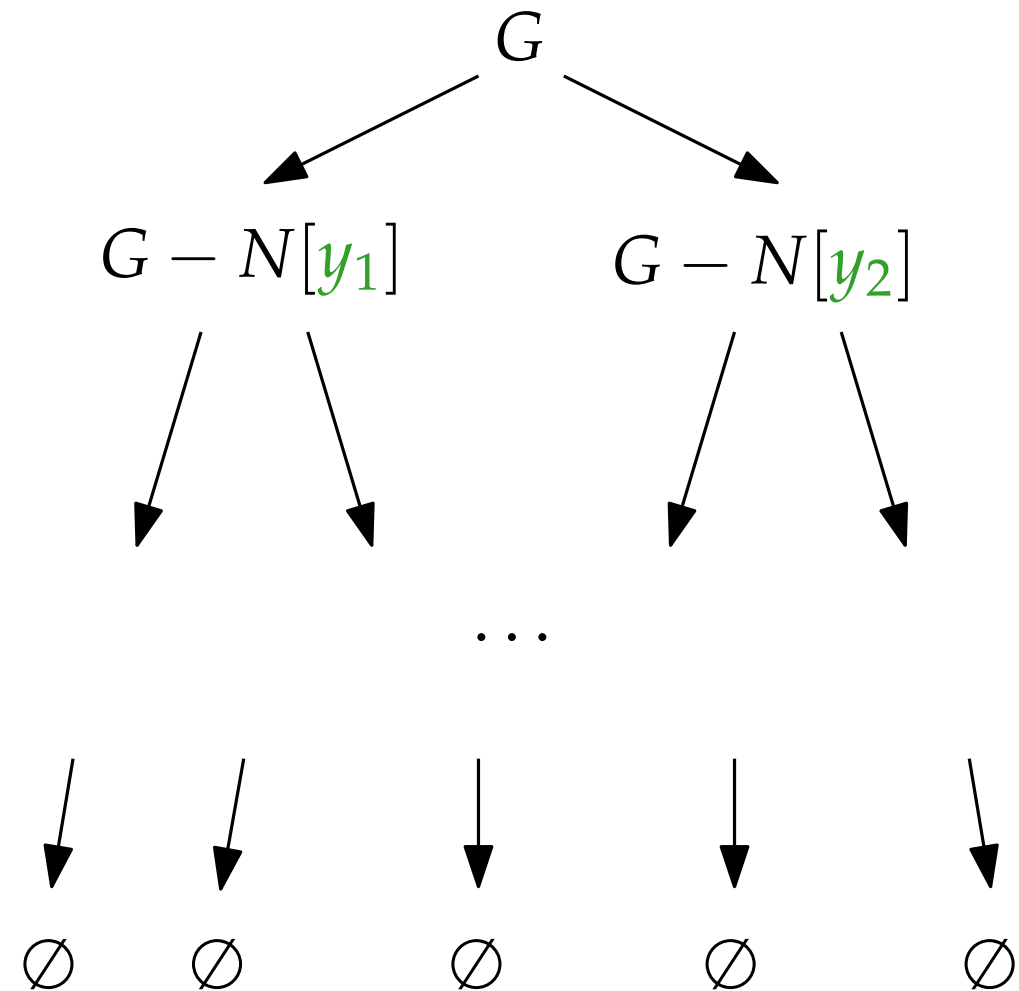
Branching Analyse



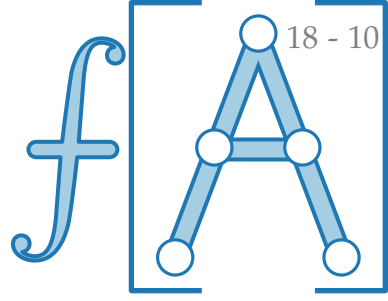
Ausführung entspricht einem **Suchbaum**, dessen Knoten mit dem Input des rekursiven Aufrufs beschriftet sind.

- Sei $B(n)$ die größte Anzahl der Blätter in einem Suchbaum für einen Graph mit n Knoten.
- Suchbaum hat Höhe $\leq n$
 - ➔ Die Laufzeit des Algorithmus ist

$$T(n) \in \mathcal{O}^*(nB(n))$$



Branching Analyse



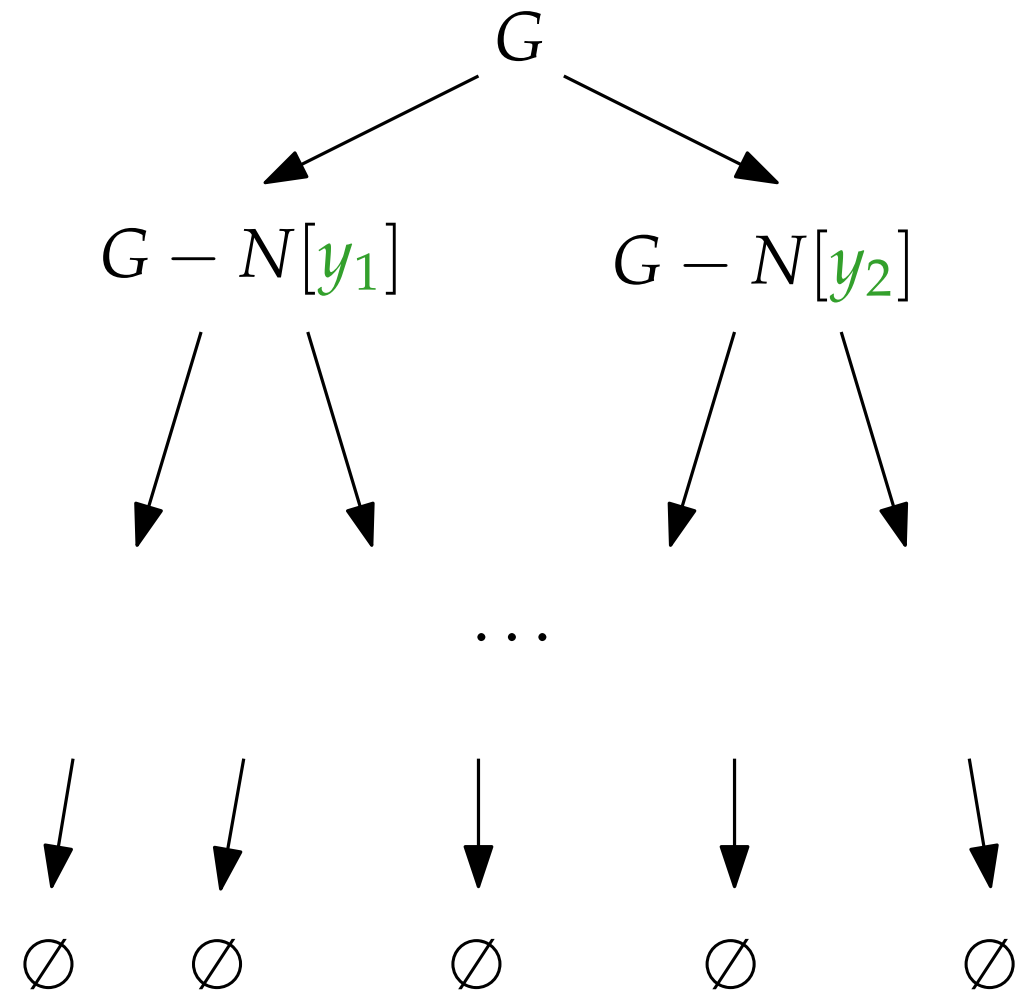
Ausführung entspricht einem **Suchbaum**, dessen Knoten mit dem Input des rekursiven Aufrufs beschriftet sind.

- Sei $B(n)$ die größte Anzahl der Blätter in einem Suchbaum für einen Graph mit n Knoten.

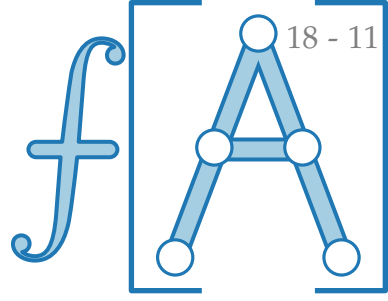
- Suchbaum hat Höhe $\leq n$

- ➔ Die Laufzeit des Algorithmus ist

$$T(n) \in \mathcal{O}^*(nB(n)) = \mathcal{O}^*(B(n)).$$



Branching Analyse



Ausführung entspricht einem **Suchbaum**, dessen Knoten mit dem Input des rekursiven Aufrufs beschriftet sind.

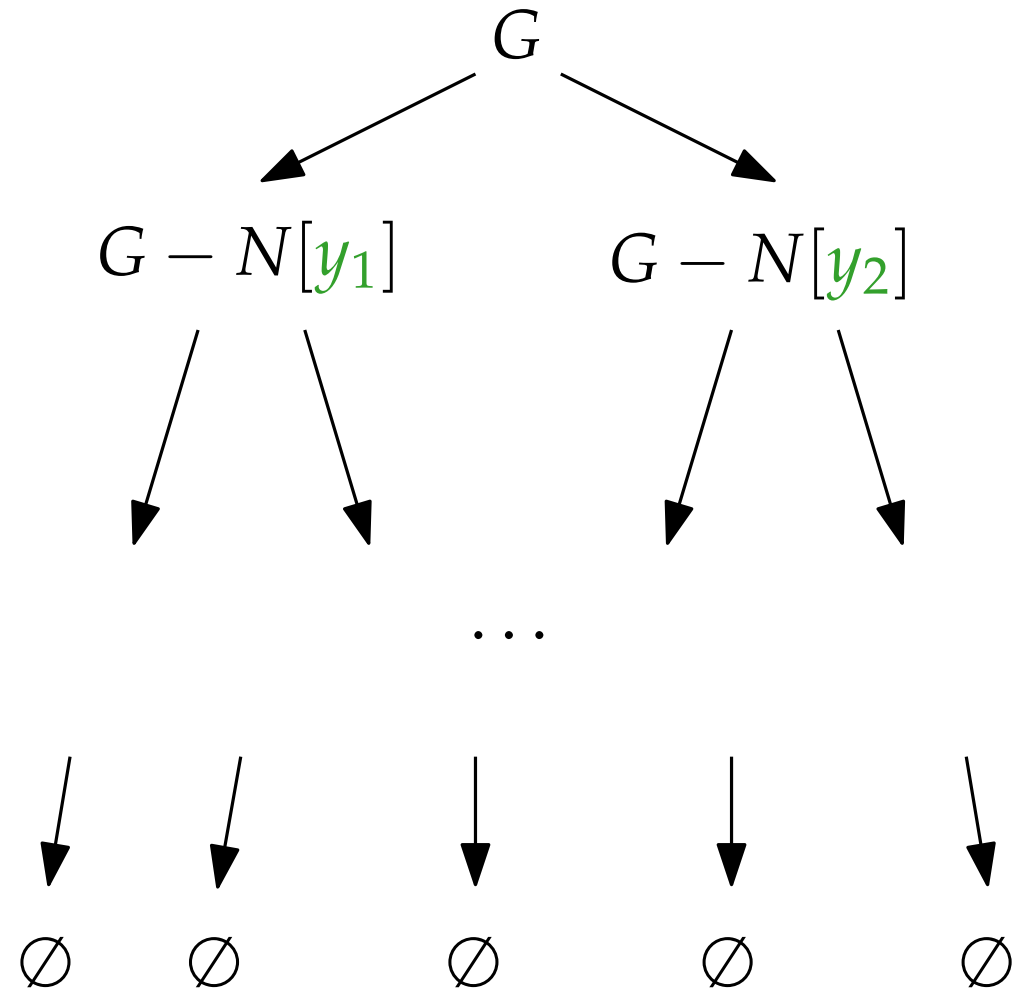
- Sei $B(n)$ die größte Anzahl der Blätter in einem Suchbaum für einen Graph mit n Knoten.

- Suchbaum hat Höhe $\leq n$

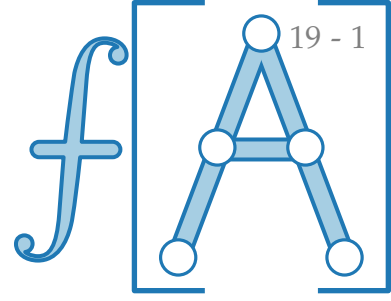
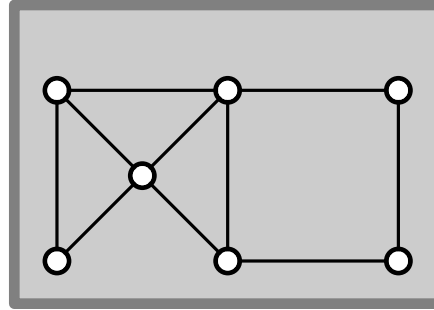
- ➔ Die Laufzeit des Algorithmus ist

$$T(n) \in \mathcal{O}^*(nB(n)) = \mathcal{O}^*(B(n)).$$

- Schauen wir uns ein Beispiel an.



Beispiel



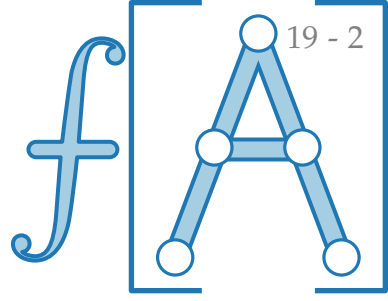
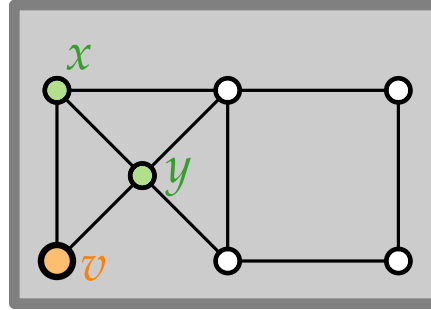
SMARTERMIS(G)

if $V = \emptyset$ **then**
 return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

Beispiel



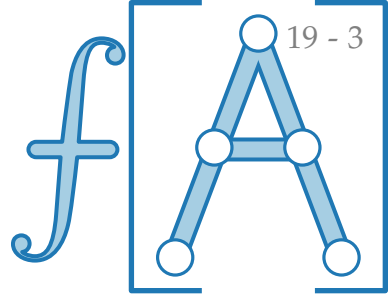
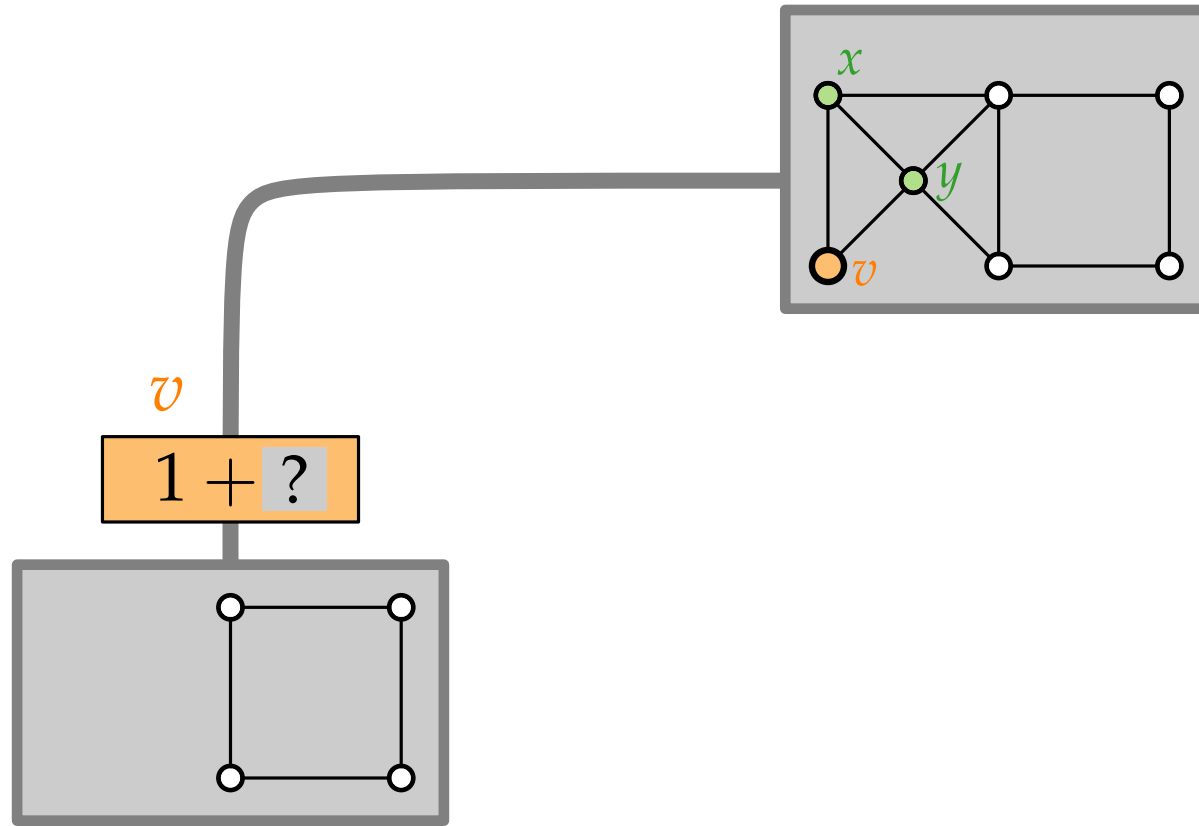
SMARTERMIS(G)

if $V = \emptyset$ **then**
 return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

Beispiel



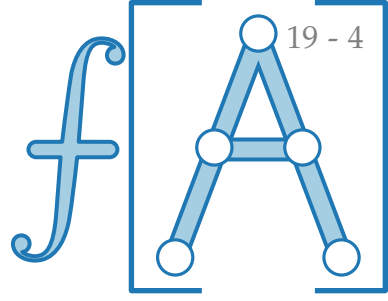
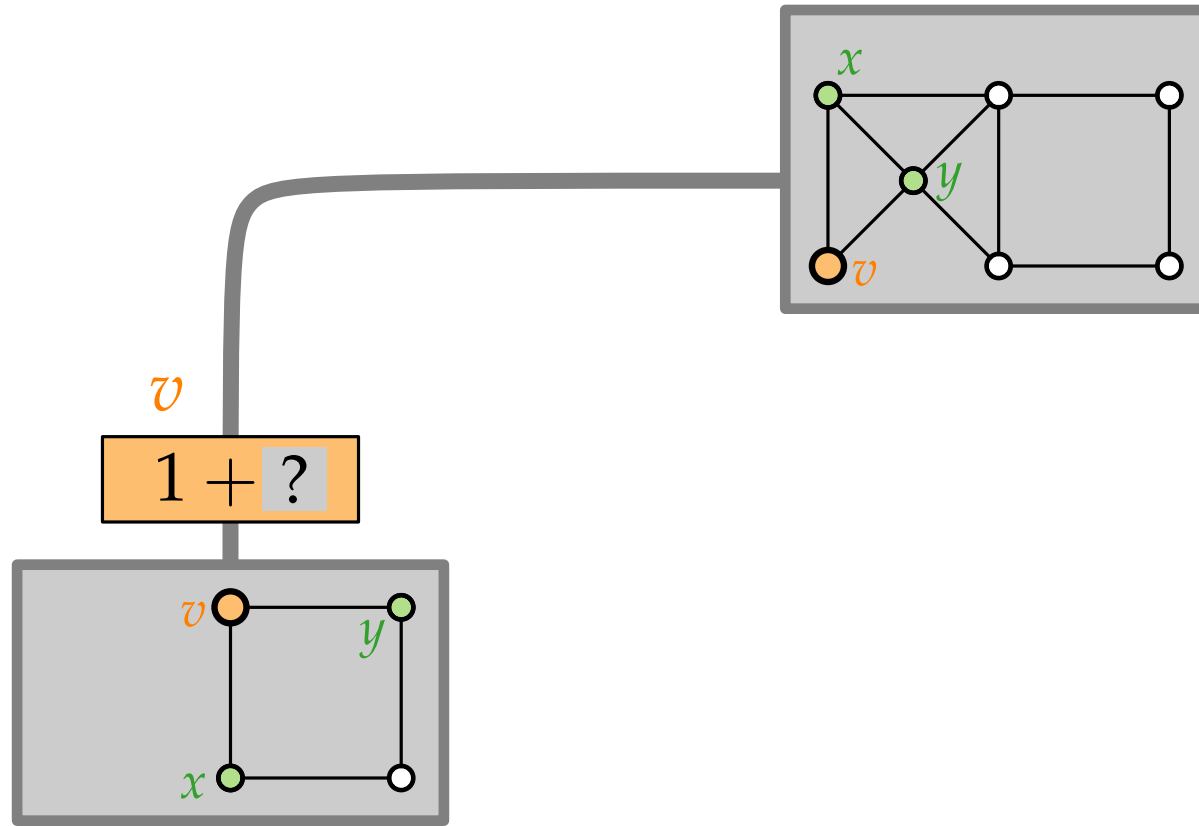
SMARTERMIS(G)

if $V = \emptyset$ **then**
 return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

Beispiel



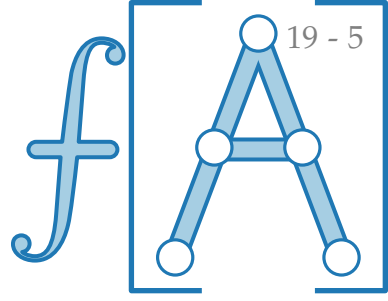
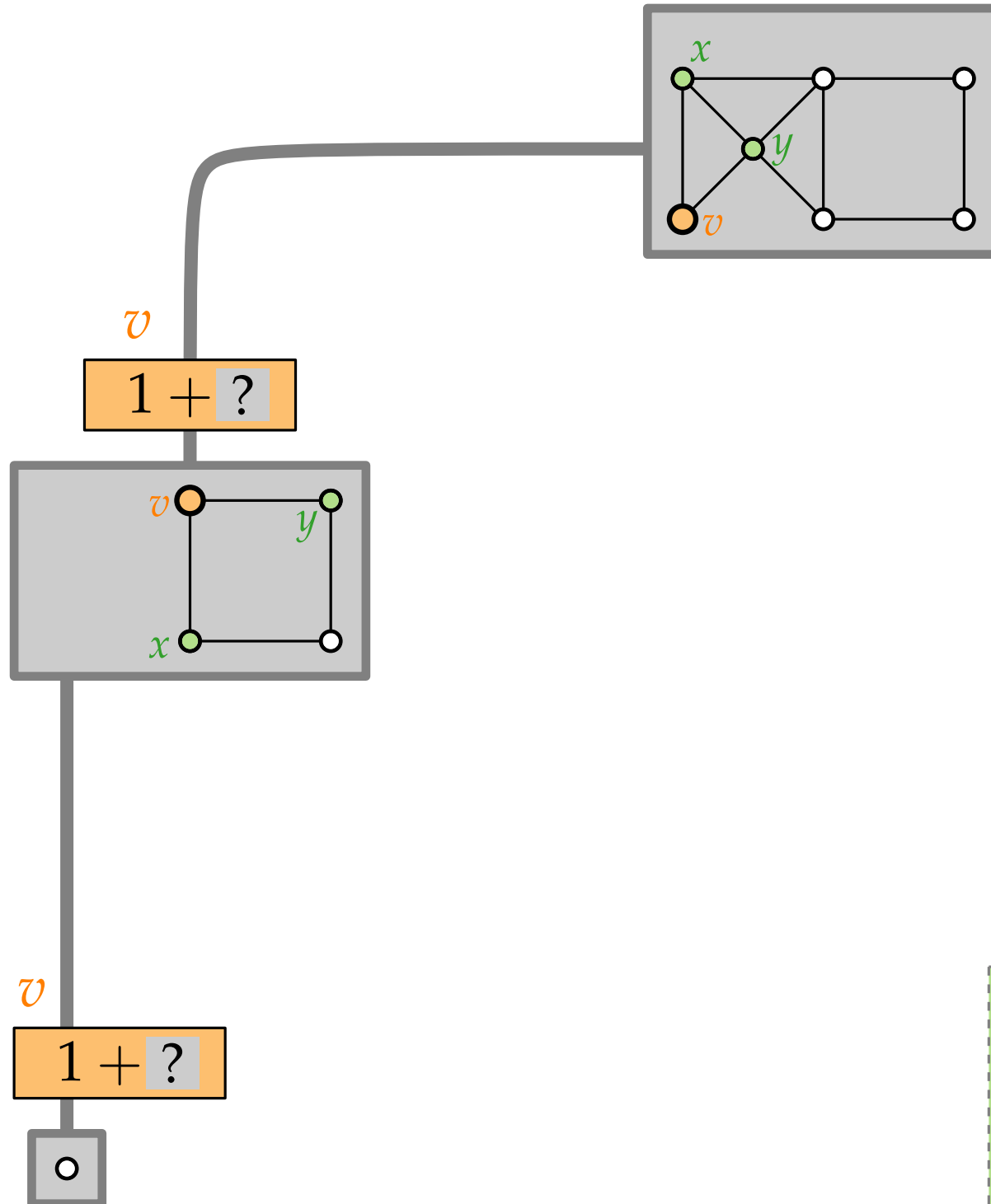
SMARTERMIS(G)

if $V = \emptyset$ **then**
 return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

Beispiel



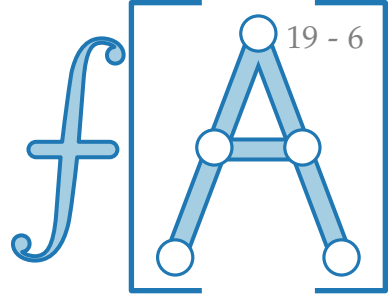
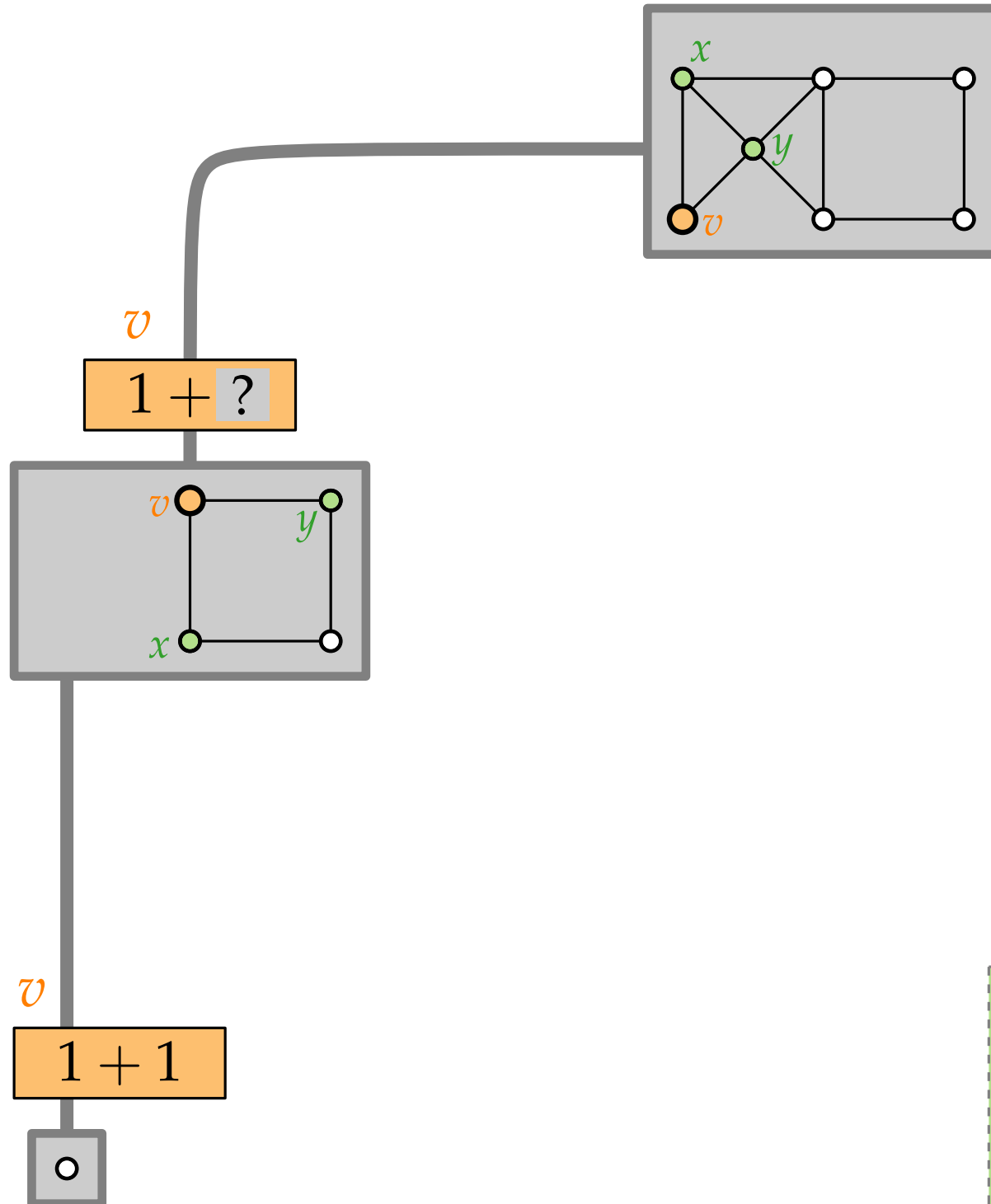
SMARTERMIS(G)

if $V = \emptyset$ **then**
 return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

Beispiel



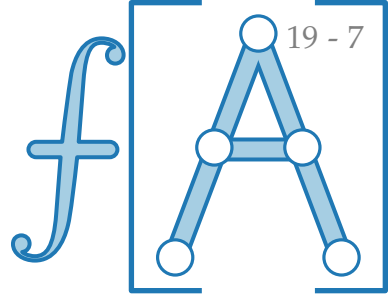
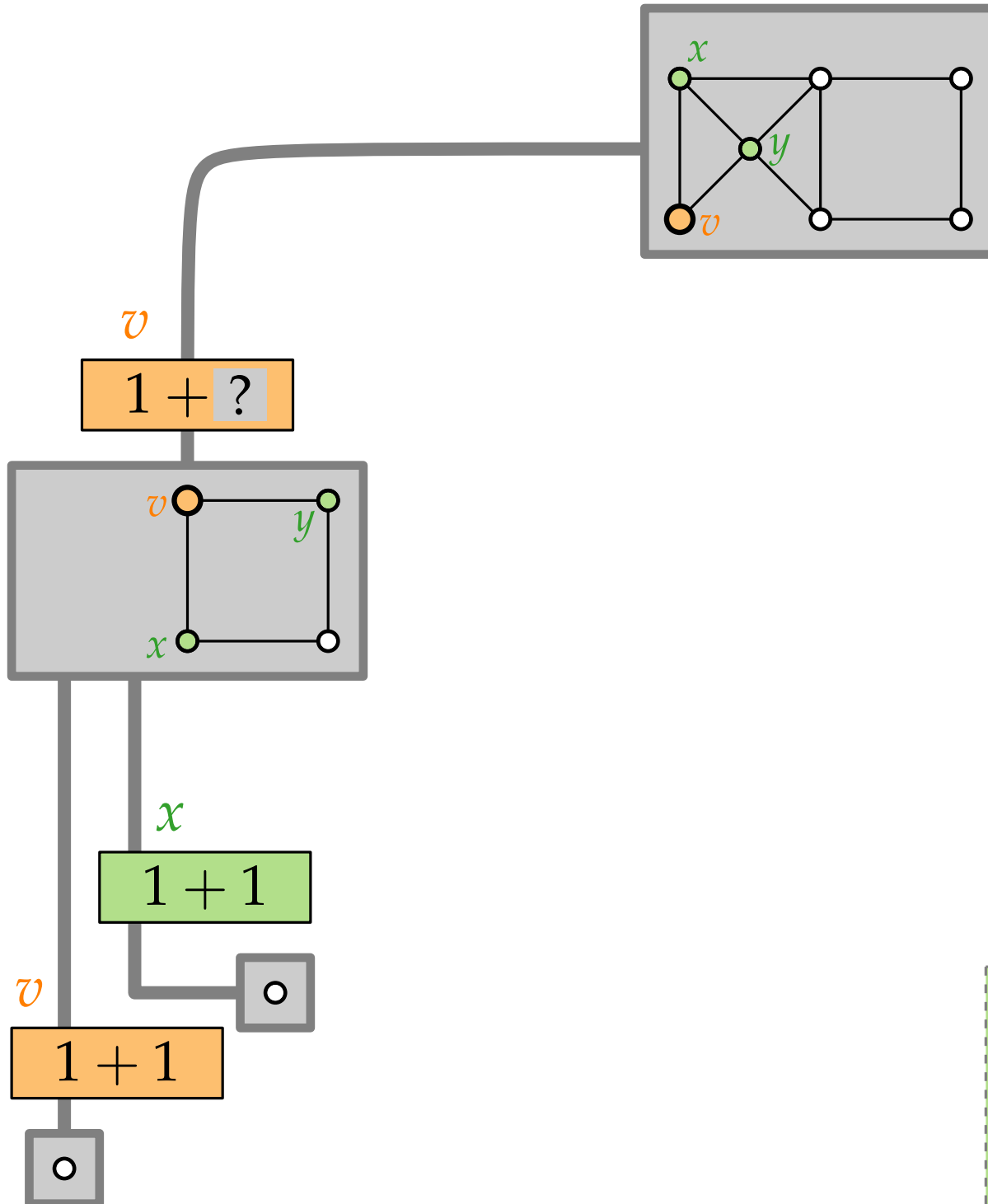
SMARTERMIS(G)

if $V = \emptyset$ **then**
 return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

Beispiel



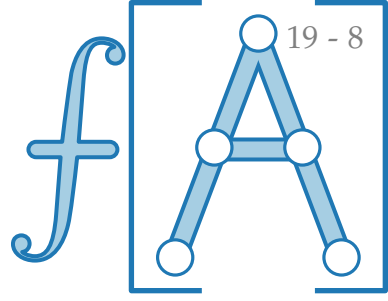
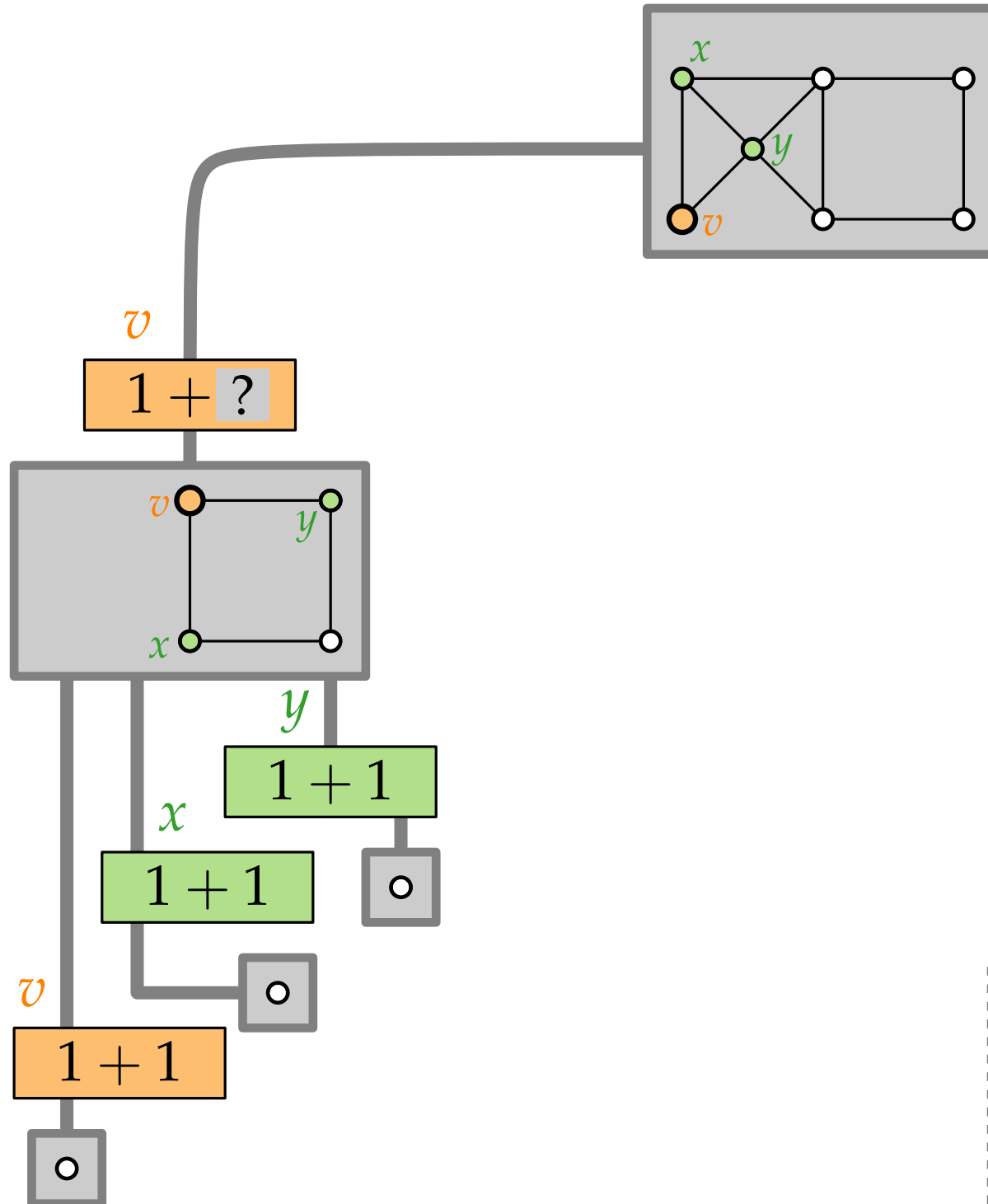
SMARTERMIS(G)

if $V = \emptyset$ **then**
 return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

Beispiel



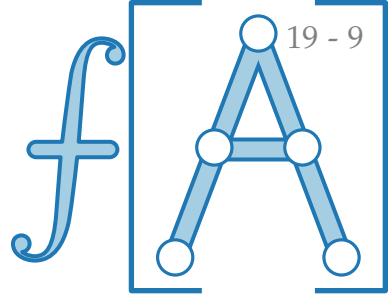
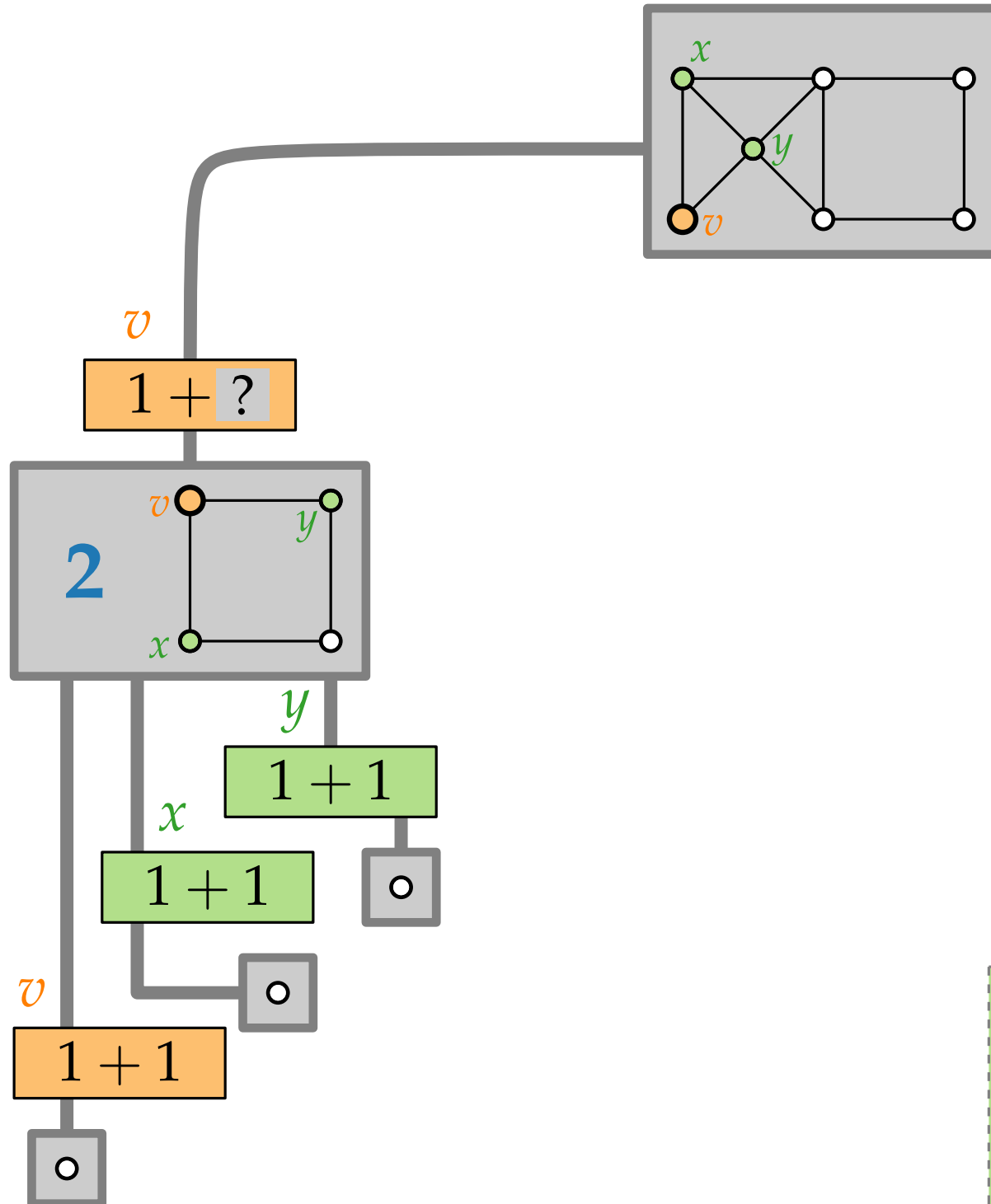
SMARTERMIS(G)

if $V = \emptyset$ **then**
 return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

Beispiel



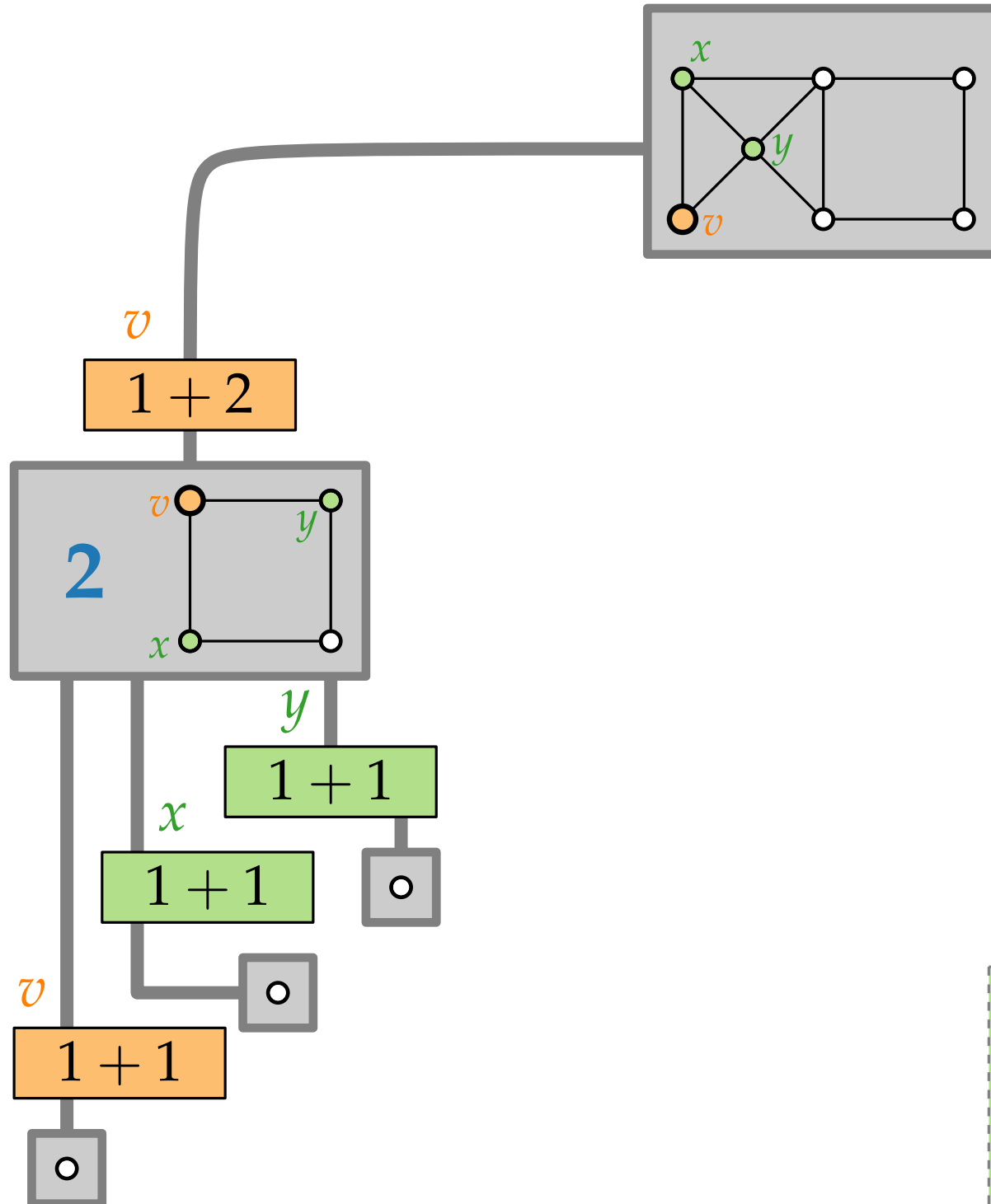
SMARTERMIS(G)

if $V = \emptyset$ **then**
 return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

Beispiel



SMARTERMIS(G)

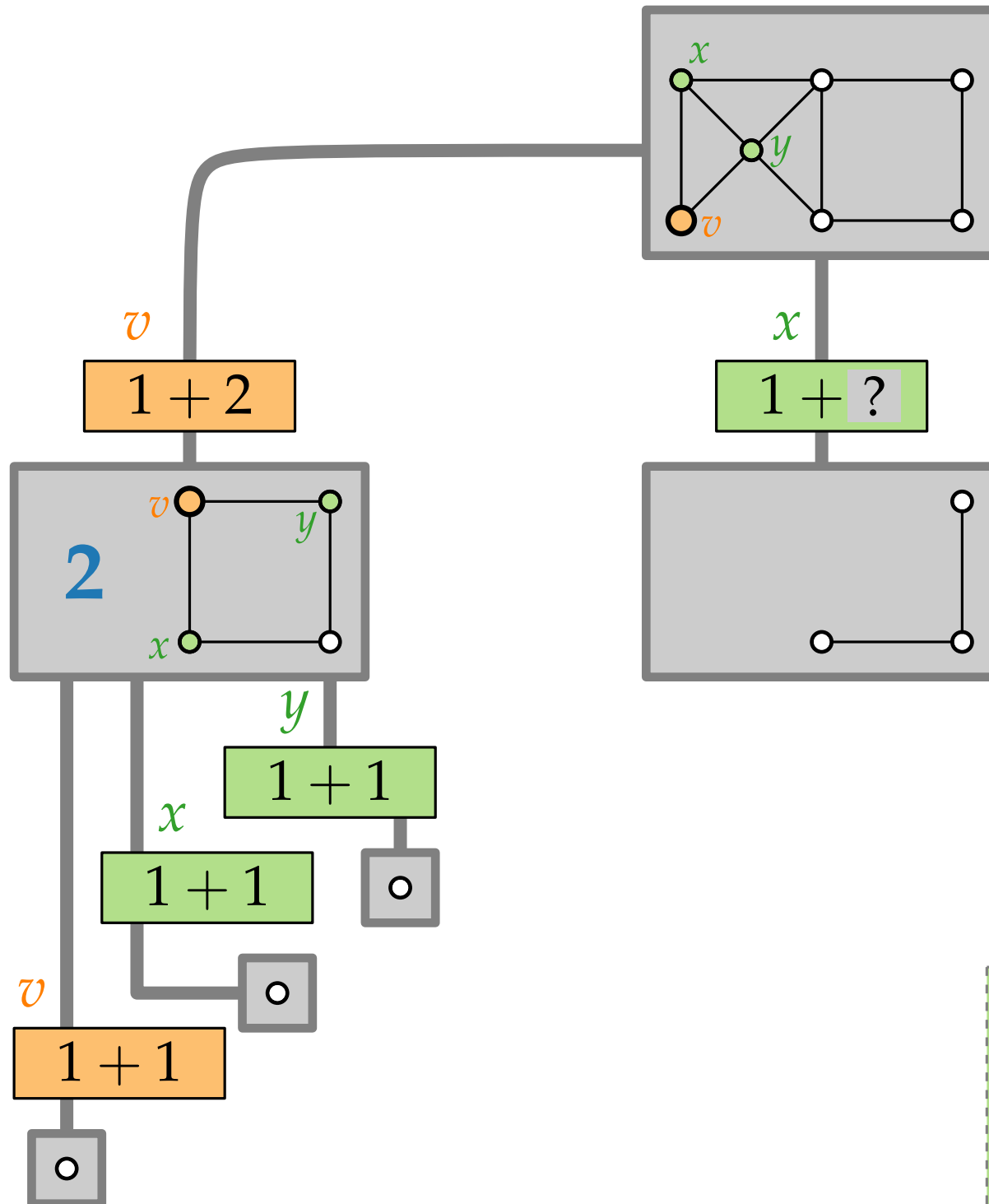
if $V = \emptyset$ **then**

return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

Beispiel


$$\text{SMARTERMIS}(G)$$

```

if  $V = \emptyset$  then
  | return 0

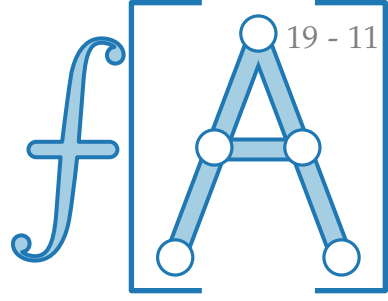
```

v = Knoten mit kleinstem Grad in $V(G)$

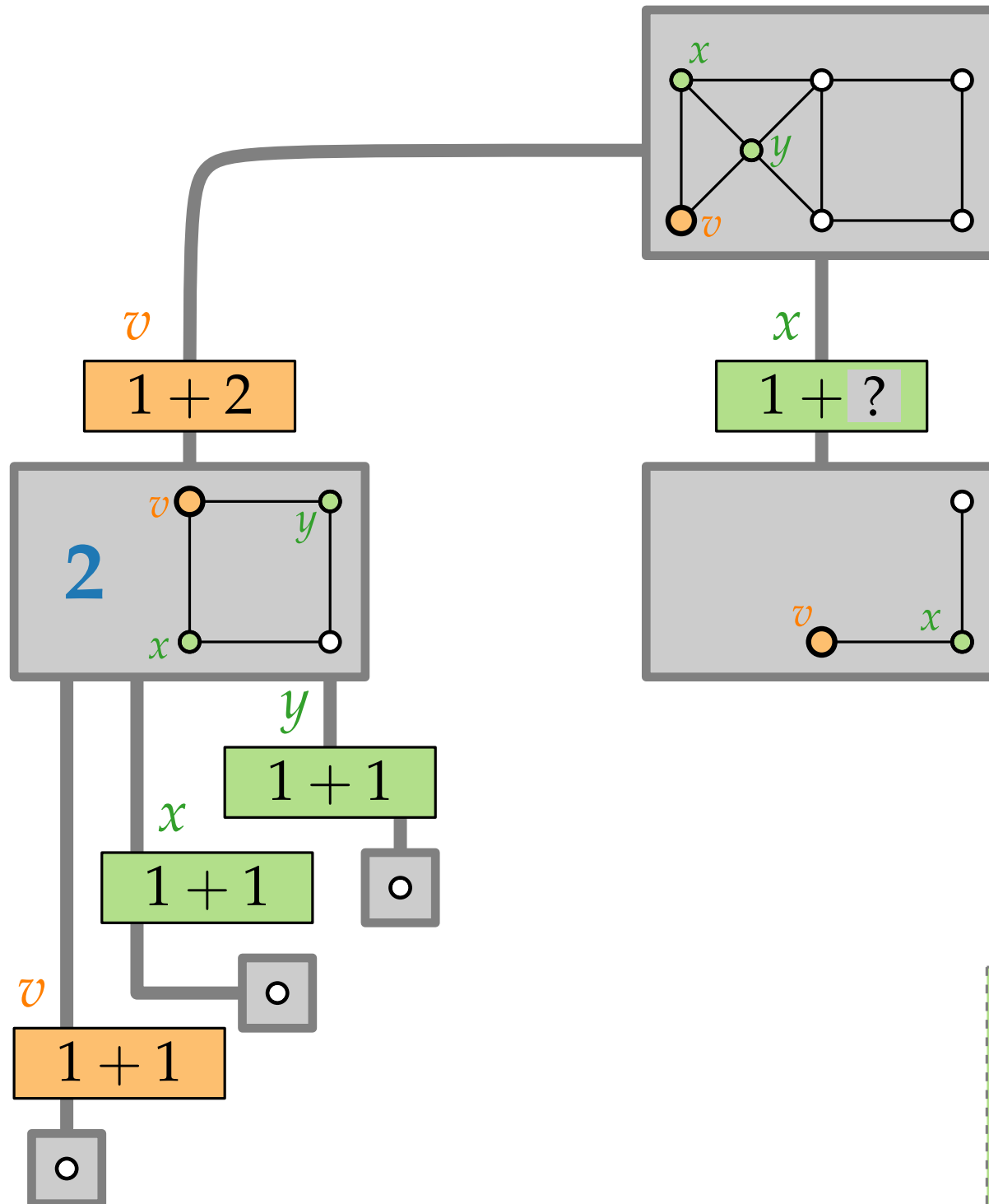
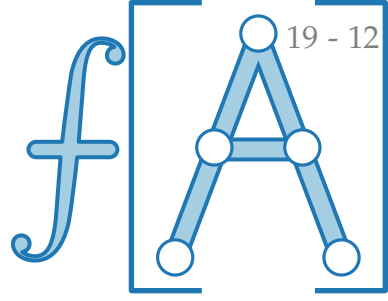
```

return 1 + max{SMARTERMIS( $G - N[y]$ ) |  $y \in N[v]$ }

```



Beispiel



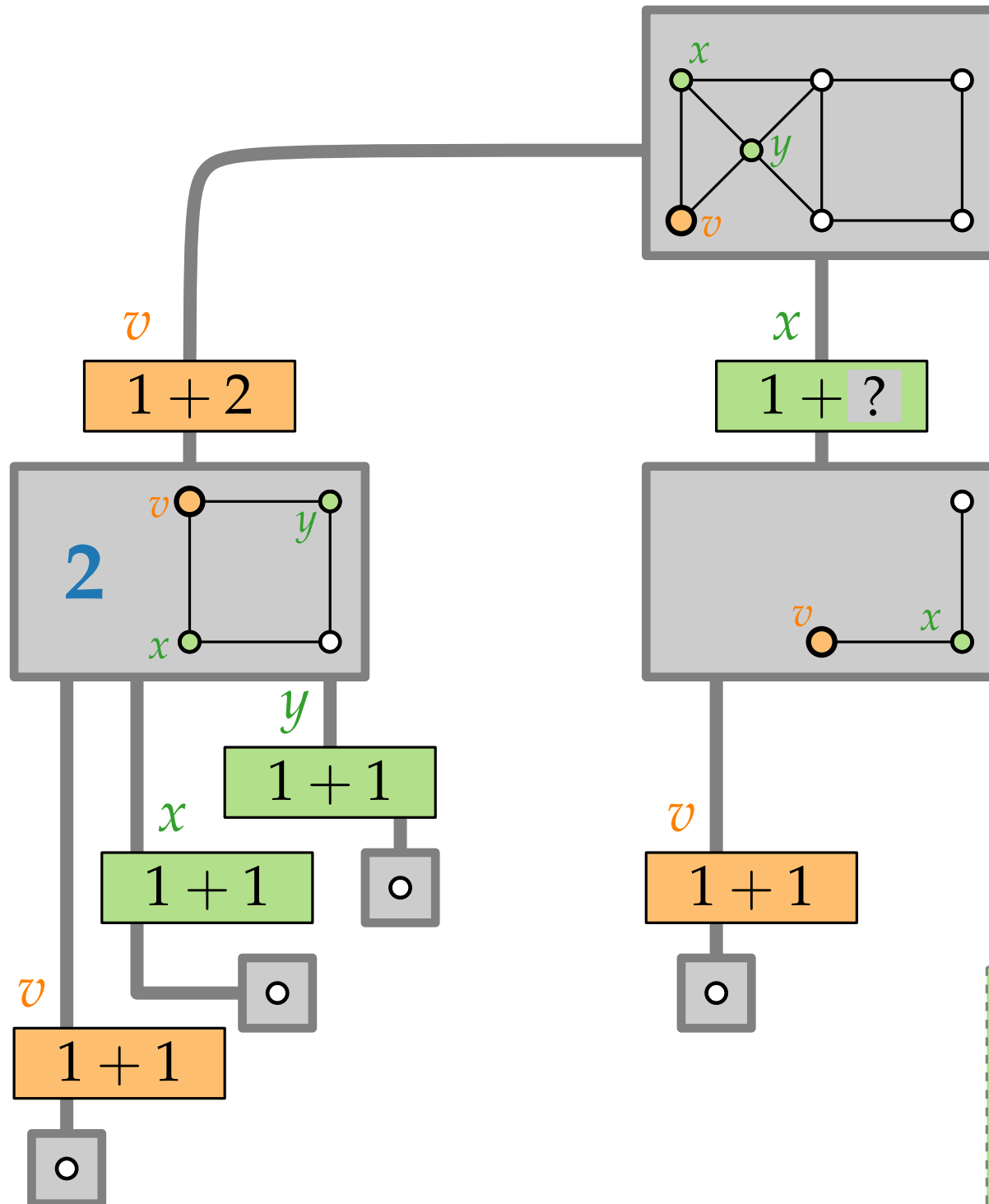
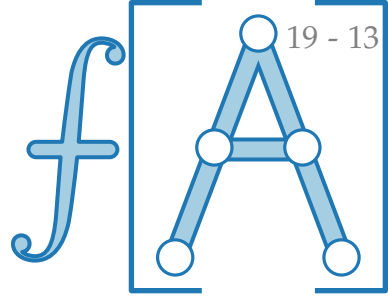
SMARTERMIS(G)

if $V = \emptyset$ **then**
 return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

Beispiel



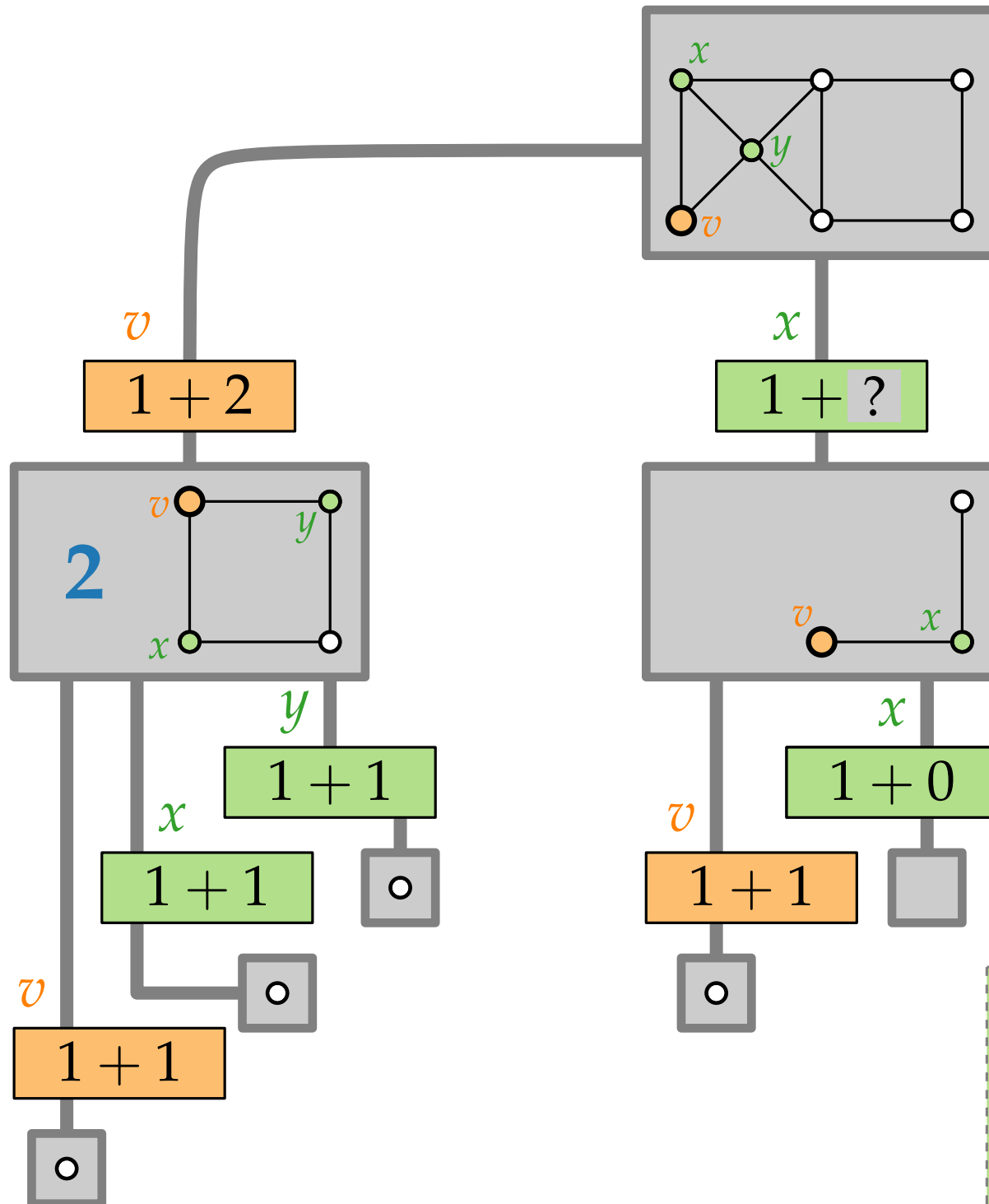
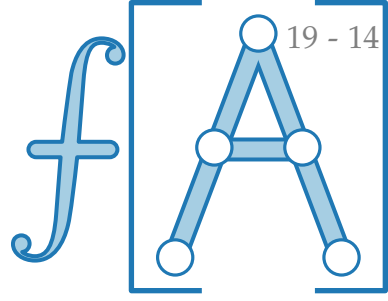
SMARTERMIS(G)

if $V = \emptyset$ **then**
 return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

Beispiel



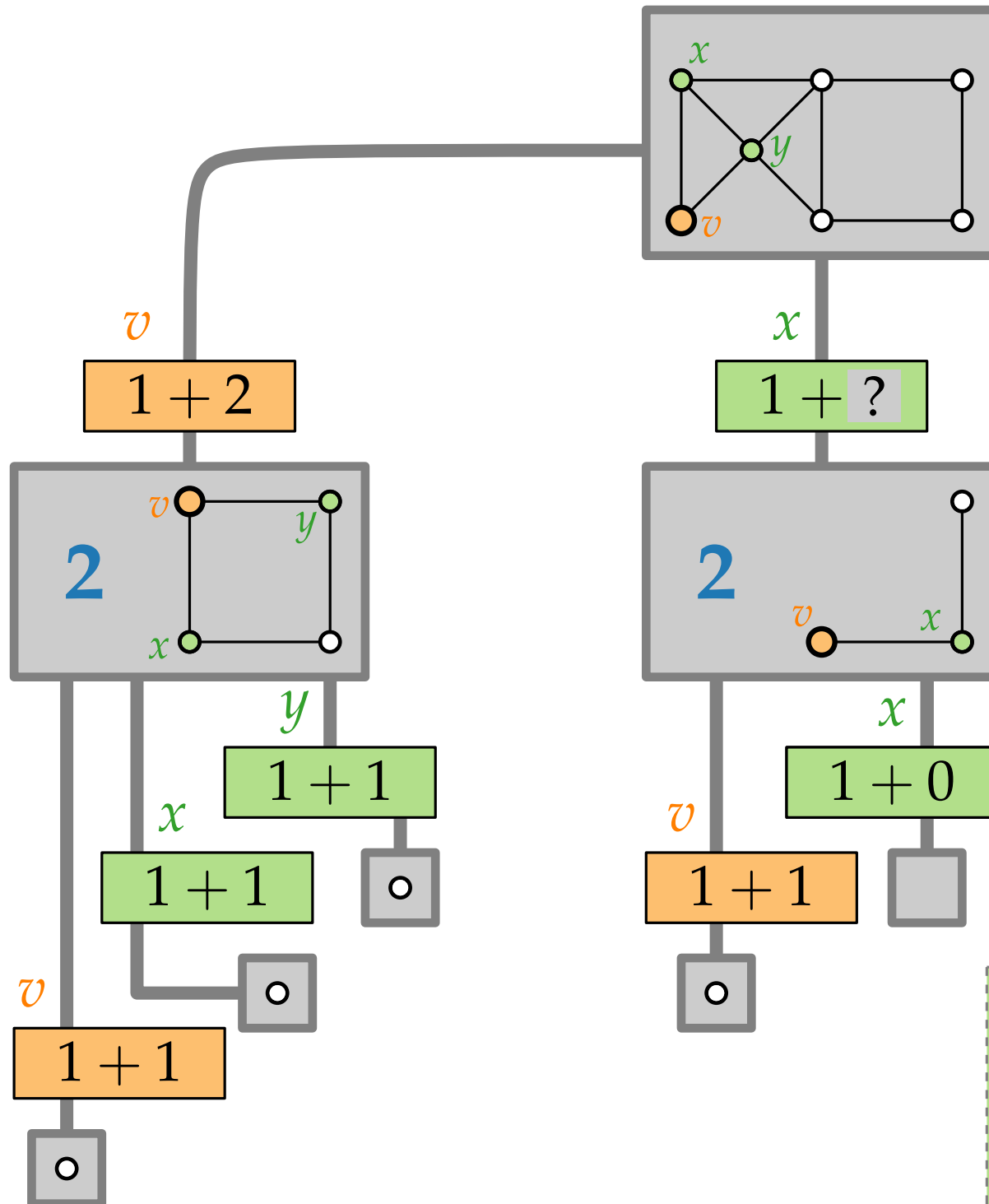
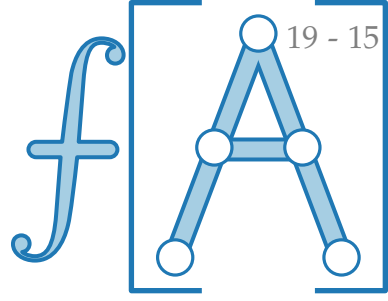
SMARTERMIS(G)

if $V = \emptyset$ **then**
 return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$

Beispiel



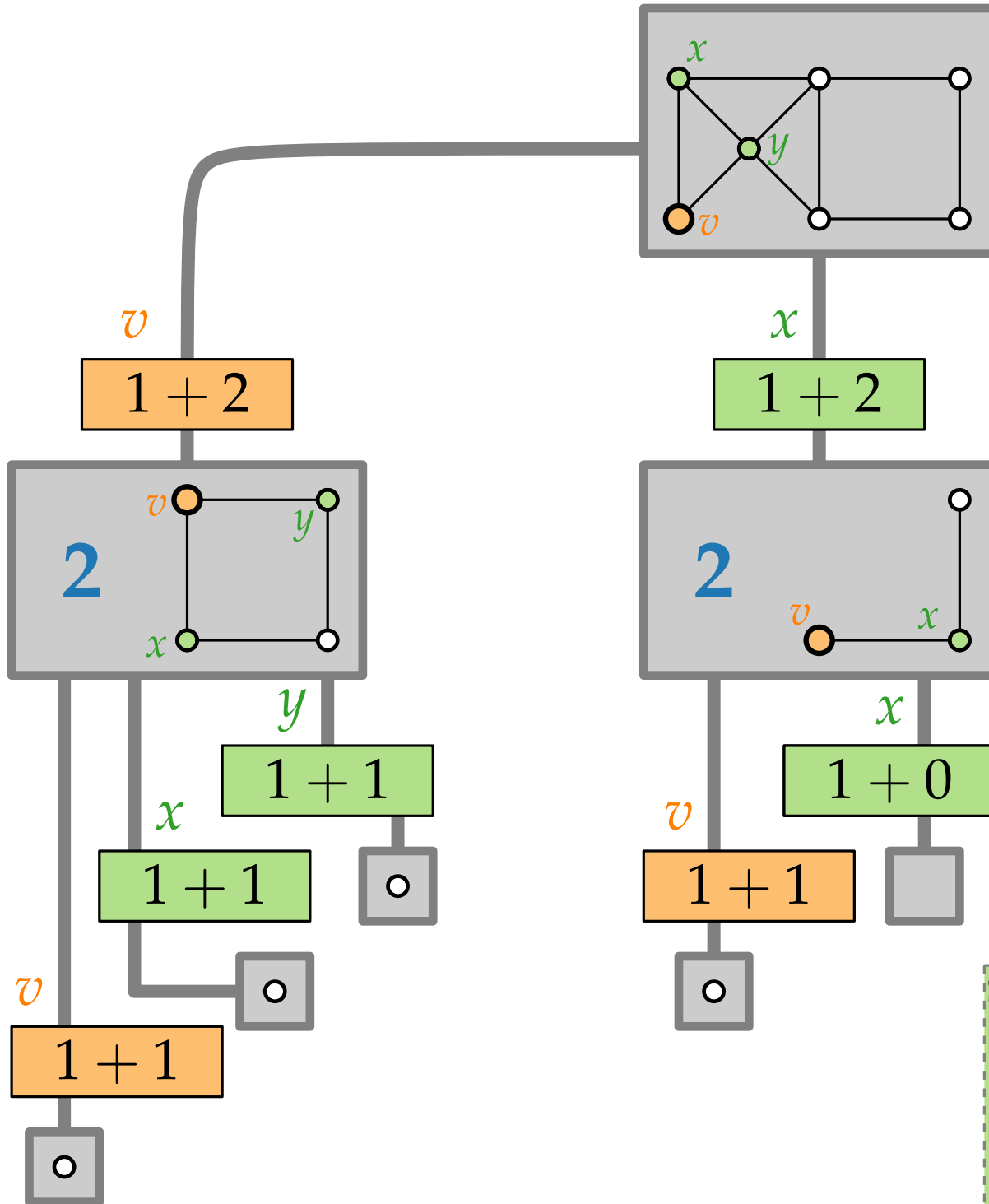
SMARTERMIS(G)

if $V = \emptyset$ **then**
 return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[v]) \mid y \in N[v]\}$

Beispiel



SMARTERMIS(G)

```

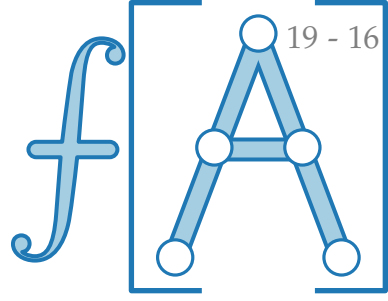
if  $V = \emptyset$  then
  | return 0

```

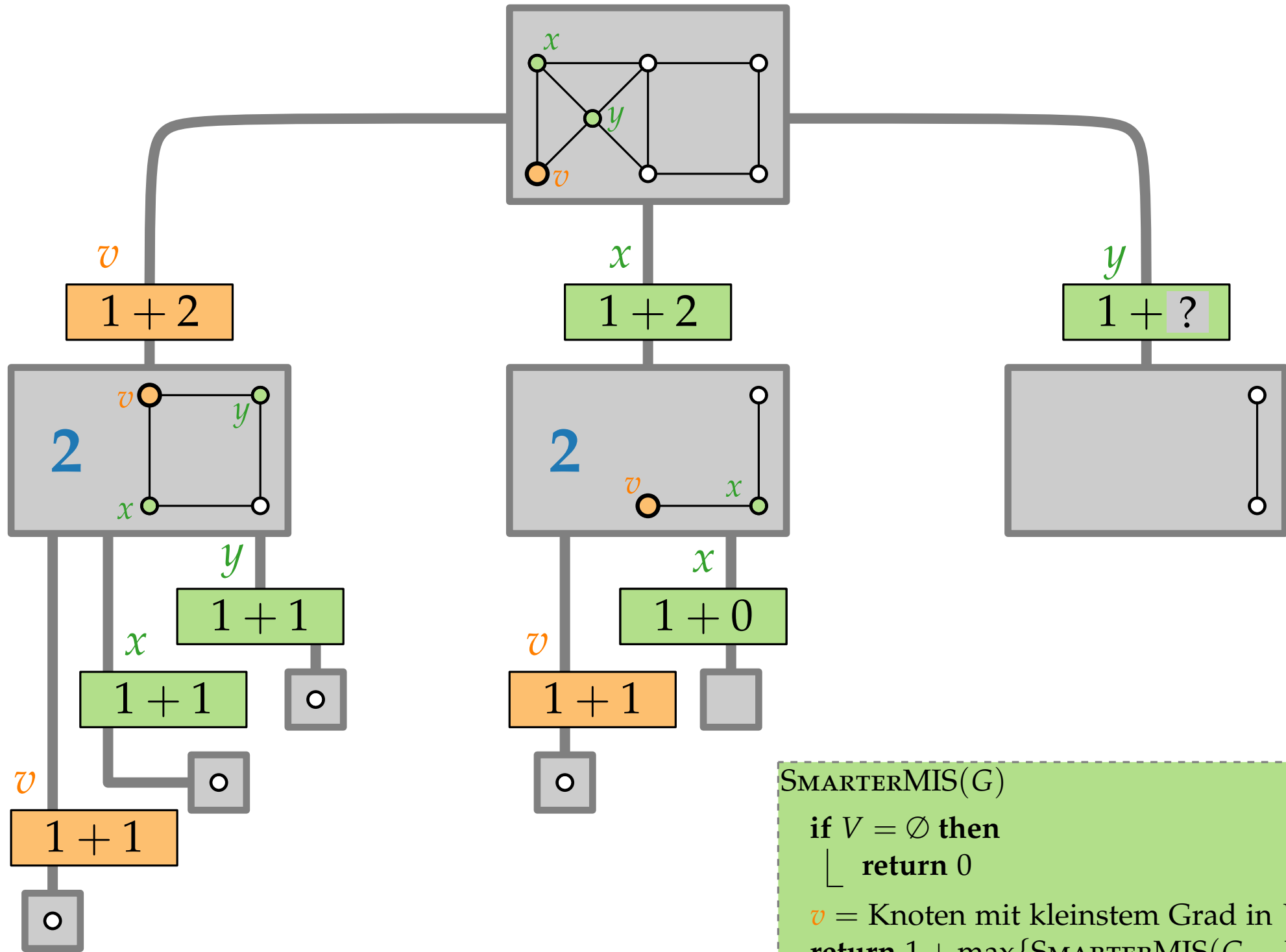
```

return 1 + max{SMARTERMIS( $G - N[y]$ ) |  $y \in N[v]$ }

```



Beispiel



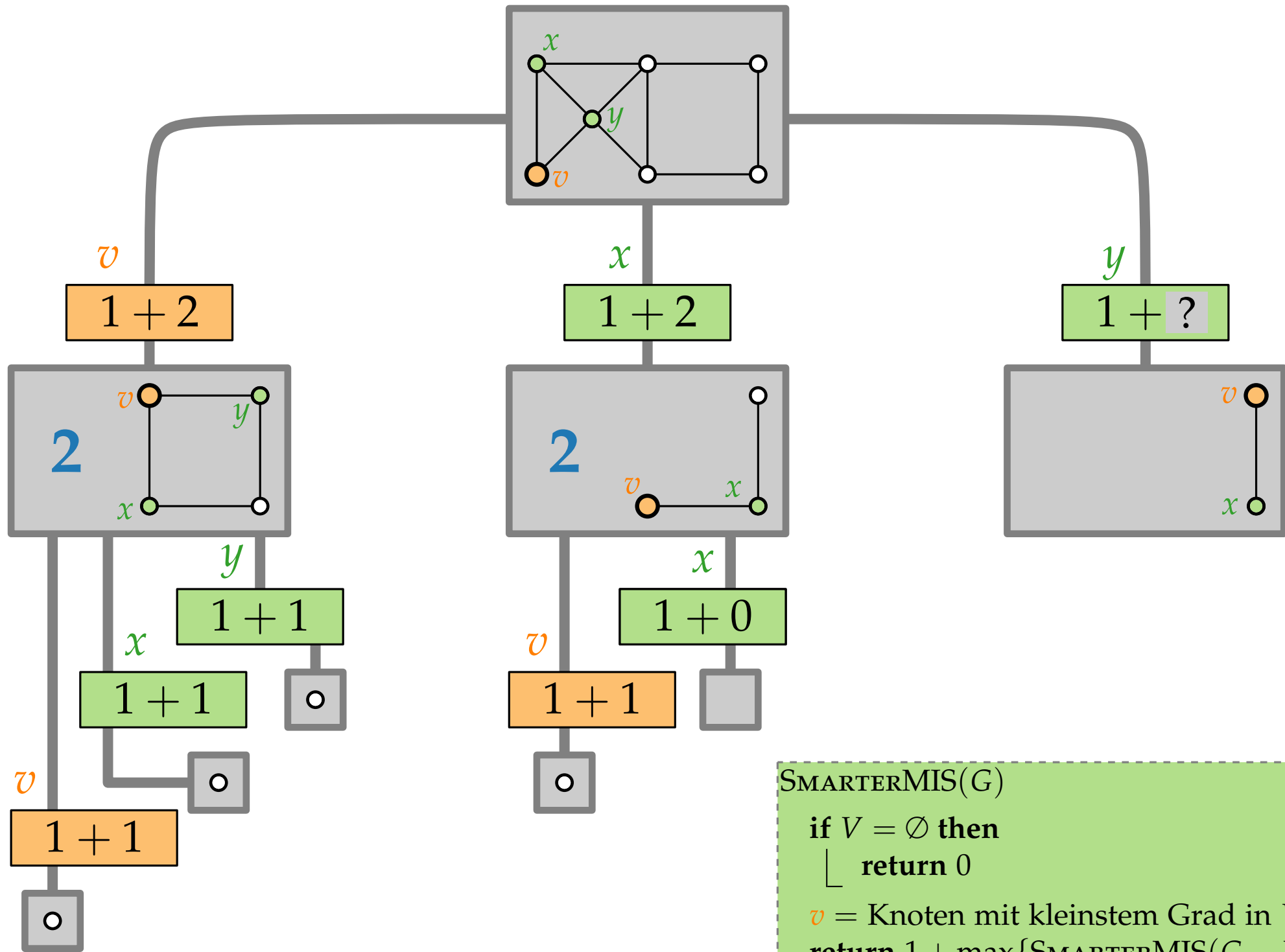
SMARTERMIS(G)

if $V = \emptyset$ then
 | return 0

v = Knoten mit kleinstem Grad in $V(G)$

return $1 + \max\{\text{SMARTERMIS}(G - N[v]) \mid v \in N[v]\}$

Beispiel

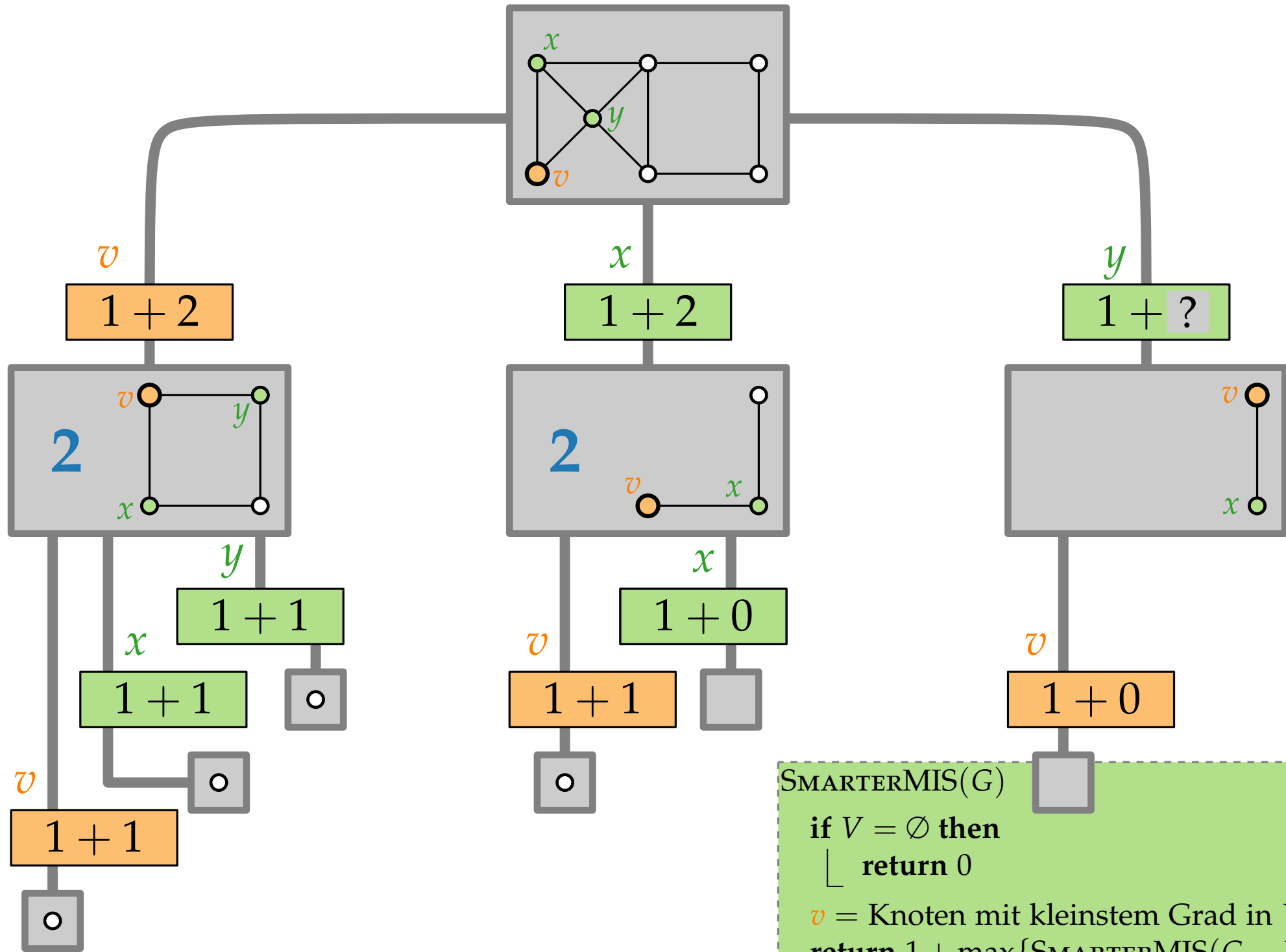
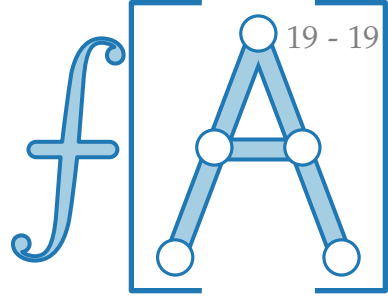


```

SMARTERMIS( $G$ )
    if  $V = \emptyset$  then
        return 0
     $v$  = Knoten mit kleinstem Grad in  $V(G)$ 
    return  $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$ 

```

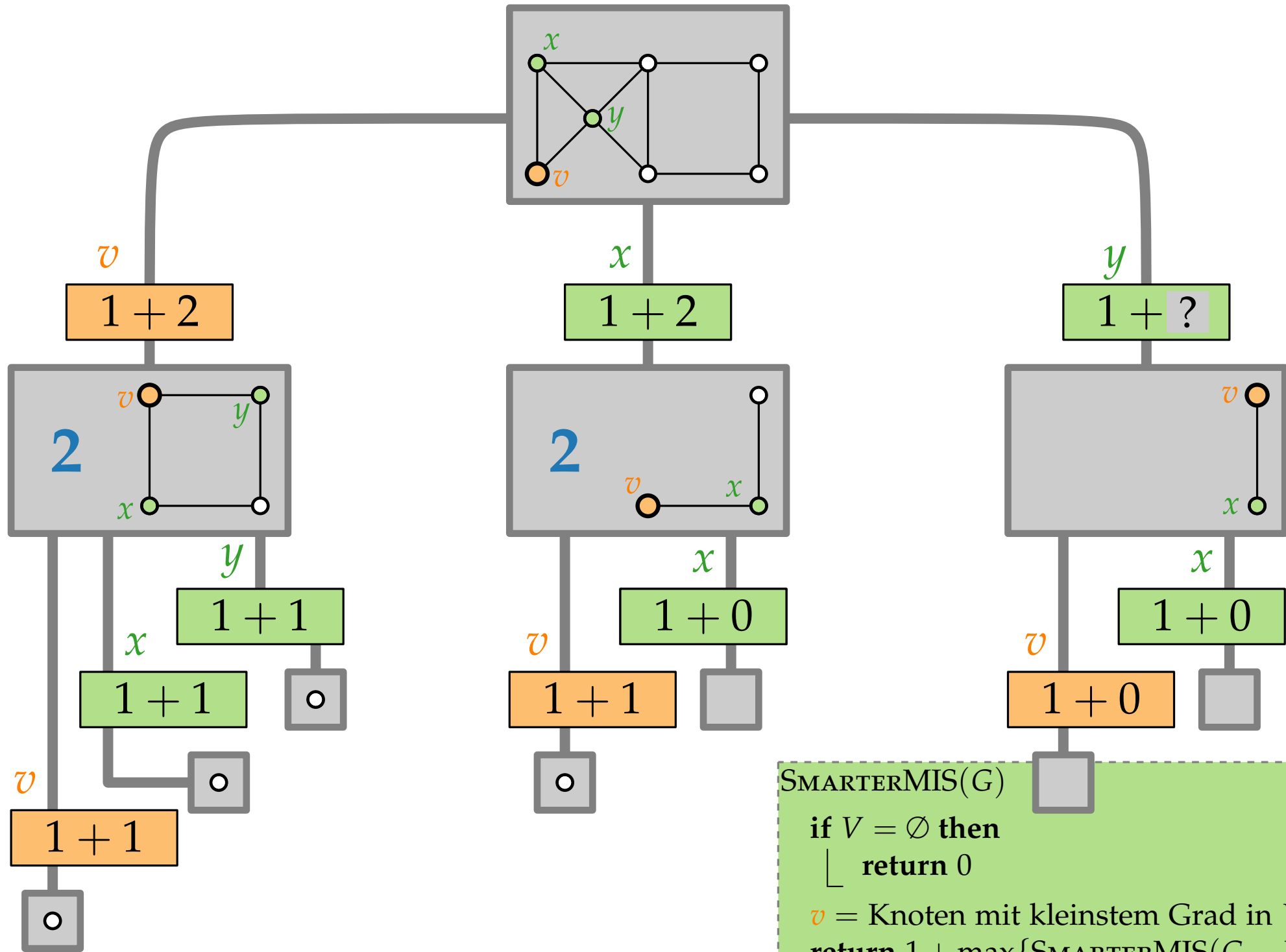
Beispiel



```

SMARTERMIS( $G$ )
  if  $V = \emptyset$  then
    return 0
   $v$  = Knoten mit kleinstem Grad in  $V(G)$ 
  return  $1 + \max\{\text{SMARTERMIS}(G - N[v]) \mid v \in N[v]\}$ 
    
```

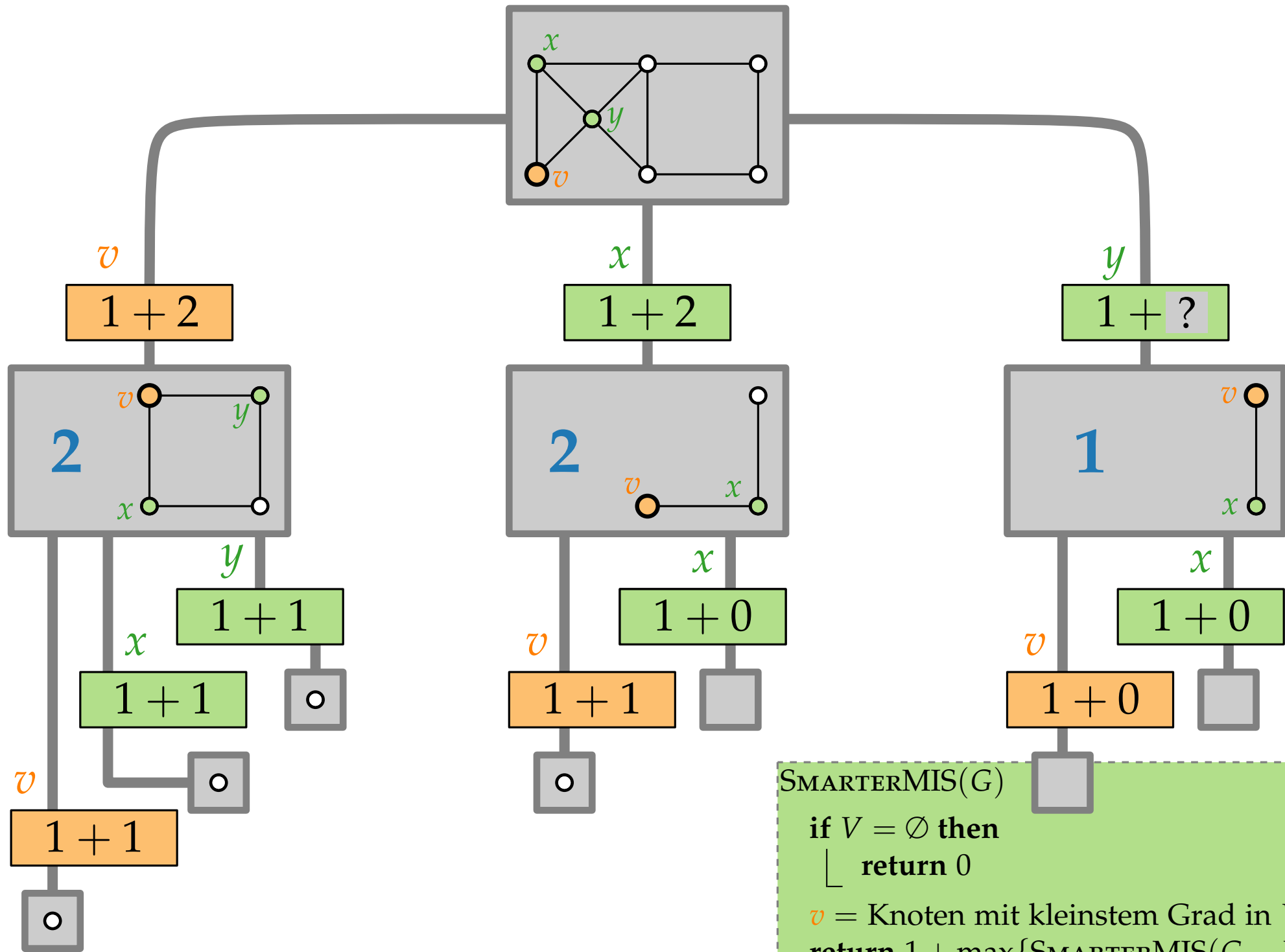
Beispiel



```

SMARTERMIS( $G$ )
  if  $V = \emptyset$  then
    return 0
   $v$  = Knoten mit kleinstem Grad in  $V(G)$ 
  return  $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$ 
    
```

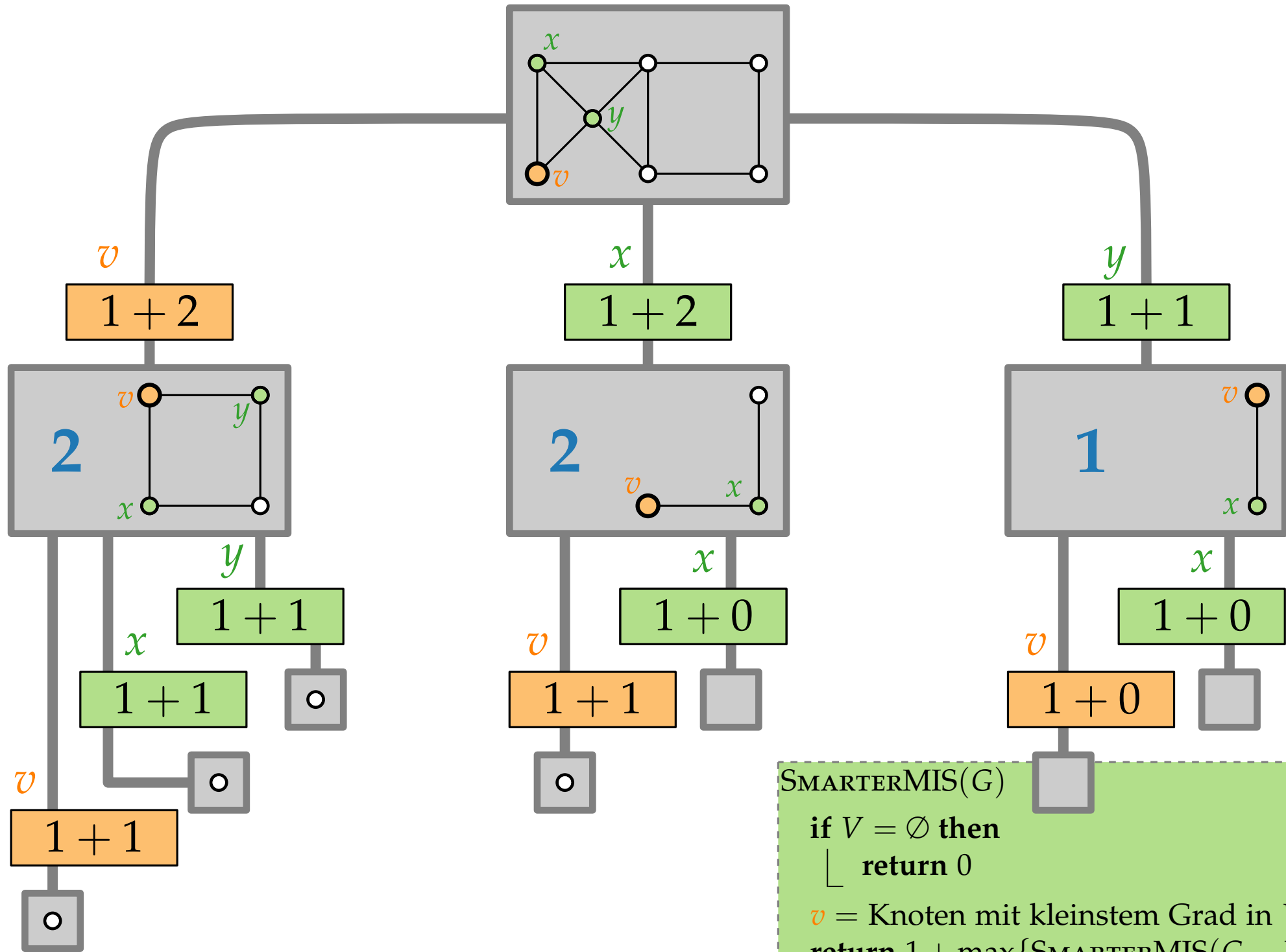
Beispiel



```

SMARTERMIS( $G$ )
  if  $V = \emptyset$  then
    return 0
   $v$  = Knoten mit kleinstem Grad in  $V(G)$ 
  return  $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$ 
    
```

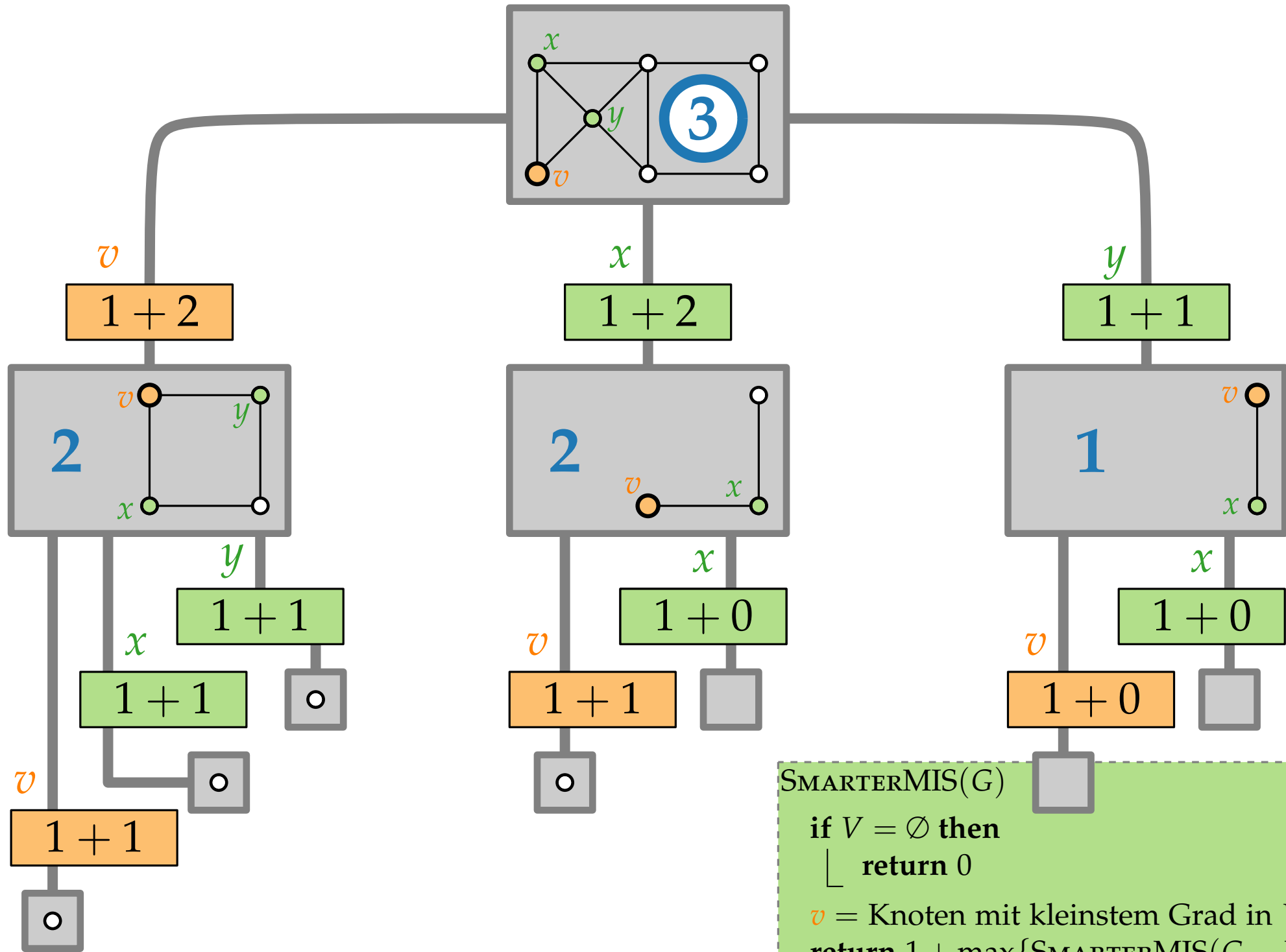
Beispiel



```

SMARTERMIS( $G$ )
  if  $V = \emptyset$  then
    return 0
   $v$  = Knoten mit kleinstem Grad in  $V(G)$ 
  return  $1 + \max\{\text{SMARTERMIS}(G - N[v]) \mid y \in N[v]\}$ 
    
```


Beispiel

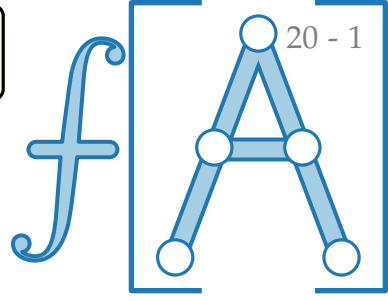


```

SMARTERMIS( $G$ )
  if  $V = \emptyset$  then
    return 0
   $v$  = Knoten mit kleinstem Grad in  $V(G)$ 
  return  $1 + \max\{\text{SMARTERMIS}(G - N[y]) \mid y \in N[v]\}$ 
    
```

Analyse

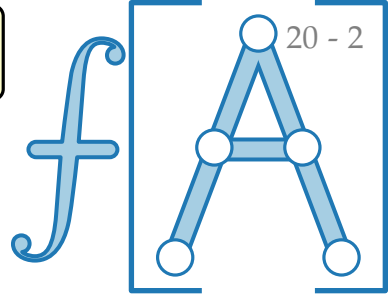
nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

Analyse

nicht prüfungsrelevant

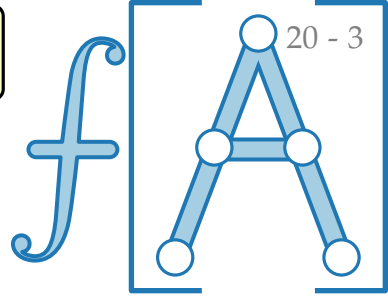


Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq$$

Analyse

nicht prüfungsrelevant

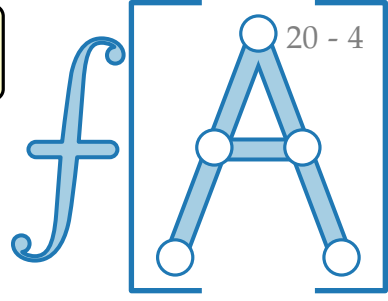


Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1))$$

Analyse

nicht prüfungsrelevant



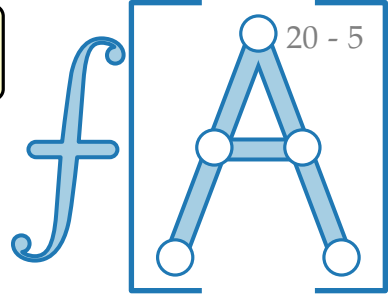
Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1))$$

wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

Analyse

nicht prüfungsrelevant



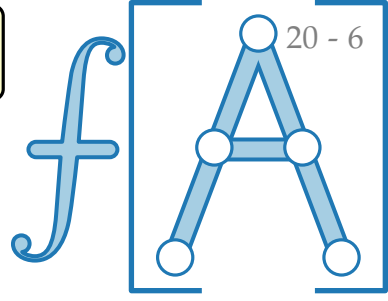
Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1))$$

wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

Analyse

nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

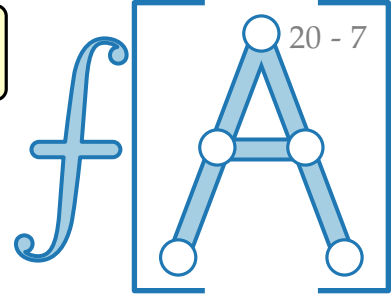
$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1))$$

wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

$$(\deg(y) \geq \deg(v))$$

Analyse

nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

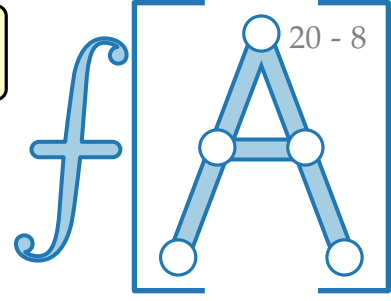
$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1)) \leq (\deg(v) + 1) \cdot B(n - (\deg(v) + 1))$$

wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

$$(\deg(y) \geq \deg(v))$$

Analyse

nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1)) \leq (\deg(v) + 1) \cdot B(n - (\deg(v) + 1))$$

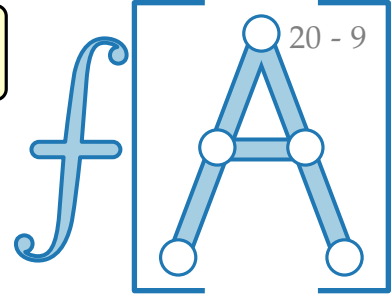
wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

$$(\deg(y) \geq \deg(v))$$

Wir beweisen per Induktion, dass $B(n) \leq 3^{n/3}$.

Analyse

nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1)) \leq (\deg(v) + 1) \cdot B(n - (\deg(v) + 1))$$

wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

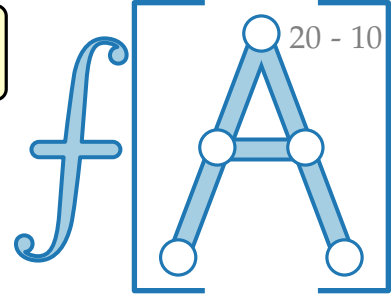
$$(\deg(y) \geq \deg(v))$$

Wir beweisen per Induktion, dass $B(n) \leq 3^{n/3}$.

■ Basisfall: $B(0) = 1 \leq 3^{0/3}$

Analyse

nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1)) \leq (\deg(v) + 1) \cdot B(n - (\deg(v) + 1))$$

wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

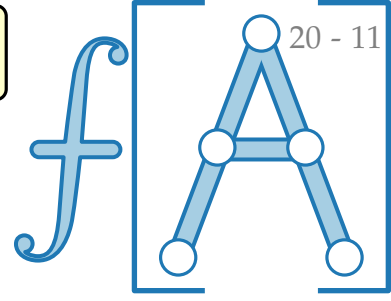
$$(\deg(y) \geq \deg(v))$$

Wir beweisen per Induktion, dass $B(n) \leq 3^{n/3}$.

- Basisfall: $B(0) = 1 \leq 3^{0/3}$
- Sei $s = \deg(v) + 1$ in der oberen Ungleichung

Analyse

nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1)) \leq (\deg(v) + 1) \cdot B(n - (\deg(v) + 1))$$

wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

$$(\deg(y) \geq \deg(v))$$

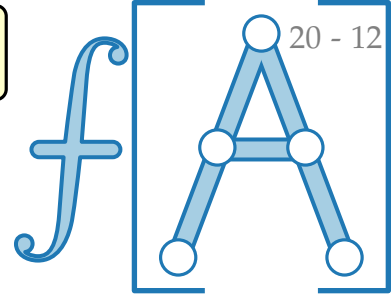
Wir beweisen per Induktion, dass $B(n) \leq 3^{n/3}$.

- Basisfall: $B(0) = 1 \leq 3^{0/3}$
- Sei $s = \deg(v) + 1$ in der oberen Ungleichung

$$B(n) \leq s \cdot B(n - s) \leq$$

Analyse

nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1)) \leq (\deg(v) + 1) \cdot B(n - (\deg(v) + 1))$$

wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

$$(\deg(y) \geq \deg(v))$$

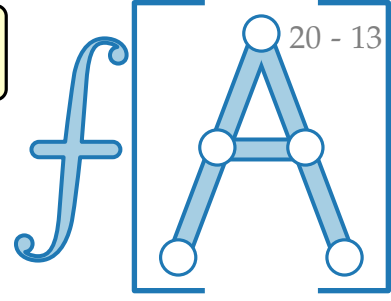
Wir beweisen per Induktion, dass $B(n) \leq 3^{n/3}$.

- Basisfall: $B(0) = 1 \leq 3^{0/3}$
- Sei $s = \deg(v) + 1$ in der oberen Ungleichung

$$B(n) \leq s \cdot B(n - s) \stackrel{\text{(IV)}}{\leq}$$

Analyse

nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1)) \leq (\deg(v) + 1) \cdot B(n - (\deg(v) + 1))$$

wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

$$(\deg(y) \geq \deg(v))$$

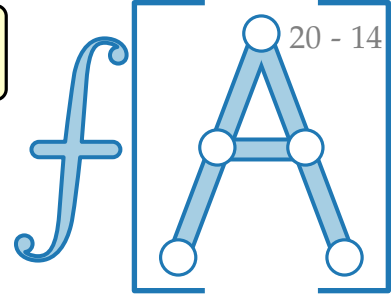
Wir beweisen per Induktion, dass $B(n) \leq 3^{n/3}$.

- Basisfall: $B(0) = 1 \leq 3^{0/3}$
- Sei $s = \deg(v) + 1$ in der oberen Ungleichung

$$B(n) \leq s \cdot B(n - s) \stackrel{\text{(IV)}}{\leq} s \cdot 3^{(n-s)/3} =$$

Analyse

nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1)) \leq (\deg(v) + 1) \cdot B(n - (\deg(v) + 1))$$

wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

$$(\deg(y) \geq \deg(v))$$

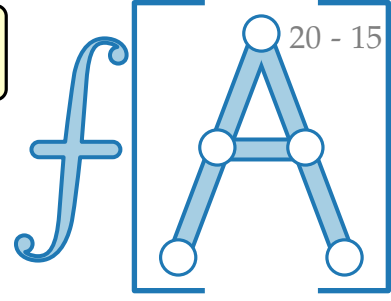
Wir beweisen per Induktion, dass $B(n) \leq 3^{n/3}$.

- Basisfall: $B(0) = 1 \leq 3^{0/3}$
- Sei $s = \deg(v) + 1$ in der oberen Ungleichung

$$B(n) \leq s \cdot B(n - s) \stackrel{\text{(IV)}}{\leq} s \cdot 3^{(n-s)/3} = \frac{s}{3^{s/3}} \cdot 3^{n/3} \leq$$

Analyse

nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1)) \leq (\deg(v) + 1) \cdot B(n - (\deg(v) + 1))$$

wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

$$(\deg(y) \geq \deg(v))$$

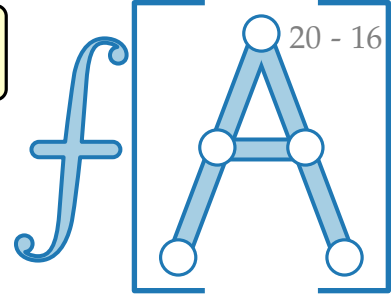
Wir beweisen per Induktion, dass $B(n) \leq 3^{n/3}$.

- Basisfall: $B(0) = 1 \leq 3^{0/3}$
- Sei $s = \deg(v) + 1$ in der oberen Ungleichung

$$B(n) \leq s \cdot B(n - s) \stackrel{\text{(IV)}}{\leq} s \cdot 3^{(n-s)/3} = \frac{s}{3^{s/3}} \cdot 3^{n/3} \stackrel{?}{\leq} 3^{n/3}$$

Analyse

nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1)) \leq (\deg(v) + 1) \cdot B(n - (\deg(v) + 1))$$

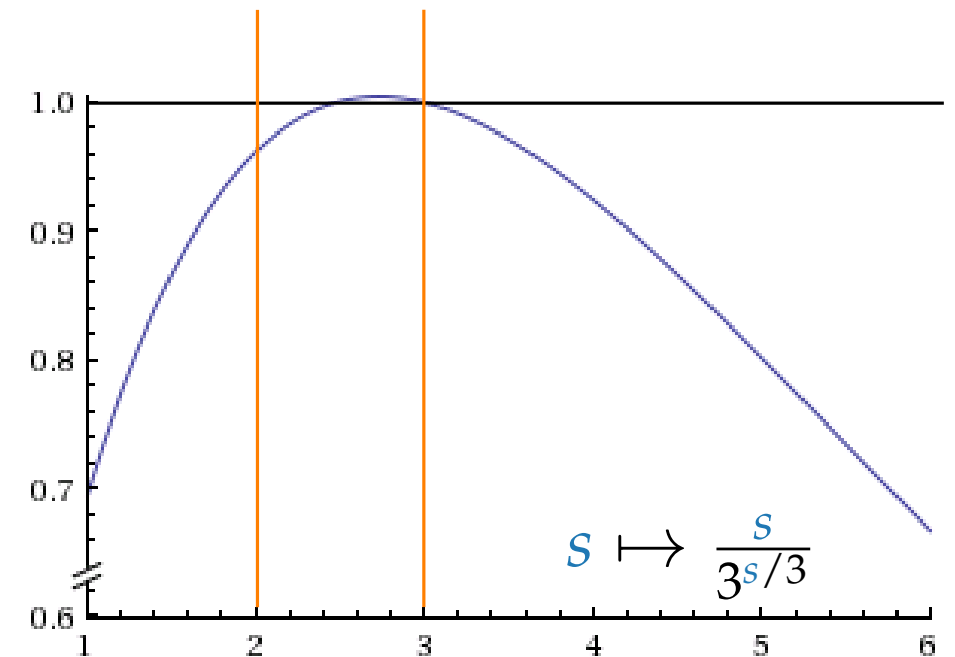
wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

$$(\deg(y) \geq \deg(v))$$

Wir beweisen per Induktion, dass $B(n) \leq 3^{n/3}$.

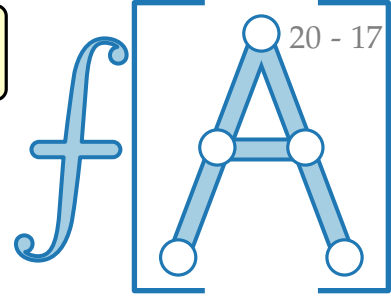
- Basisfall: $B(0) = 1 \leq 3^{0/3}$
- Sei $s = \deg(v) + 1$ in der oberen Ungleichung

$$B(n) \leq s \cdot B(n - s) \stackrel{\text{(IV)}}{\leq} s \cdot 3^{(n-s)/3} = \frac{s}{3^{s/3}} \cdot 3^{n/3} \stackrel{?}{\leq} 3^{n/3}$$



Analyse

nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1)) \leq (\deg(v) + 1) \cdot B(n - (\deg(v) + 1))$$

wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

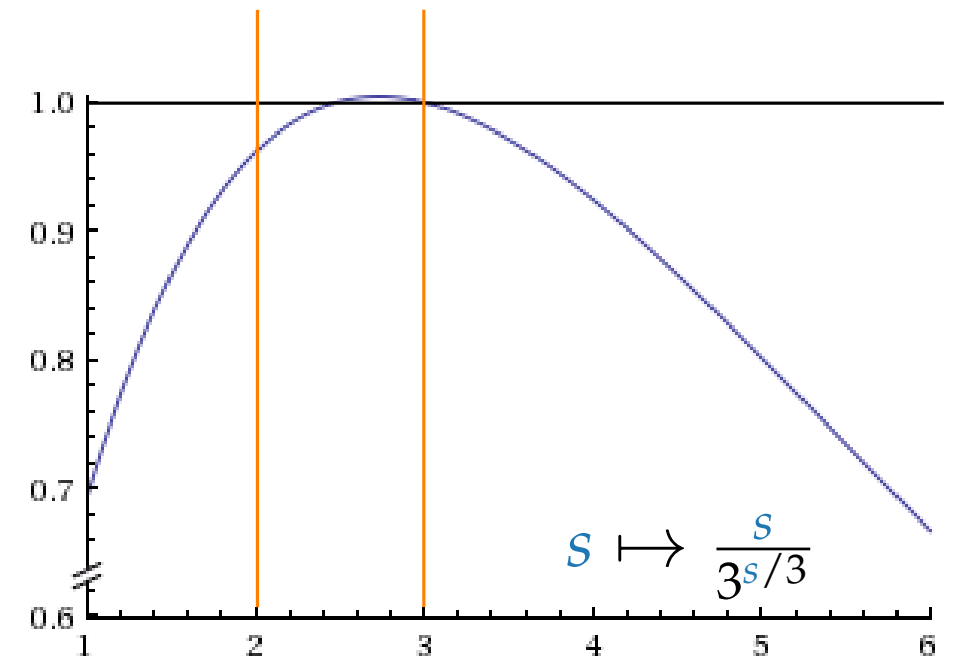
$$(\deg(y) \geq \deg(v))$$

Wir beweisen per Induktion, dass $B(n) \leq 3^{n/3}$.

- Basisfall: $B(0) = 1 \leq 3^{0/3}$
- Sei $s = \deg(v) + 1$ in der oberen Ungleichung

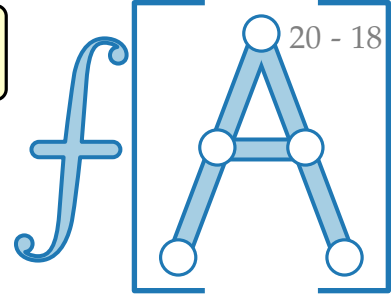
$$B(n) \leq s \cdot B(n - s) \stackrel{\text{(IV)}}{\leq} s \cdot 3^{(n-s)/3} = \frac{s}{3^{s/3}} \cdot 3^{n/3} \stackrel{?}{\leq} 3^{n/3}$$

$$B(n) \in \mathcal{O}^*(\sqrt[3]{3^n})$$



Analyse

nicht prüfungsrelevant



Im schlechtesten Fall gilt für einen Graphen G mit n Knoten ($n \geq 1$):

$$B(n) \leq \sum_{y \in N[v]} B(n - (\deg(y) + 1)) \leq (\deg(v) + 1) \cdot B(n - (\deg(v) + 1))$$

wobei v ein Knoten von G mit Minimalgrad ist, und wir beobachten dass $B(n') \leq B(n)$ für jedes $n' \leq n$.

$$(\deg(y) \geq \deg(v))$$

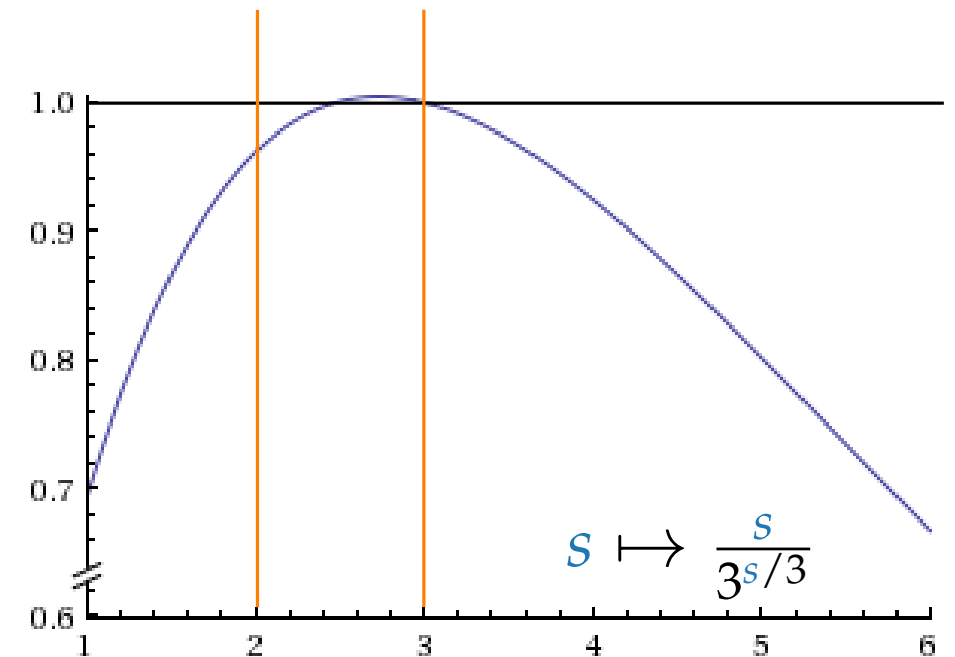
Wir beweisen per Induktion, dass $B(n) \leq 3^{n/3}$.

■ Basisfall: $B(0) = 1 \leq 3^{0/3}$

■ Sei $s = \deg(v) + 1$ in der oberen Ungleichung

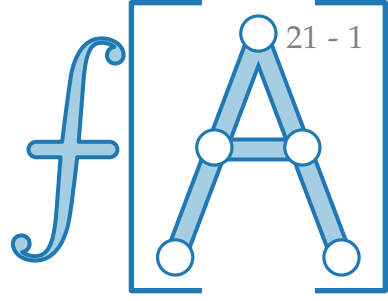
$$B(n) \leq s \cdot B(n - s) \stackrel{\text{(IV)}}{\leq} s \cdot 3^{(n-s)/3} = \frac{s}{3^{s/3}} \cdot 3^{n/3} \stackrel{?}{\leq} 3^{n/3}$$

$$B(n) \in \mathcal{O}^*(\sqrt[3]{3}^n) \subset \mathcal{O}^*(1.4423^n)$$



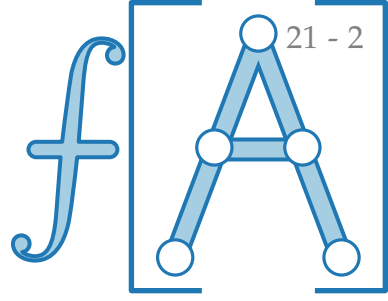
Diskussion

- Schlaueres Branching liefert einen $\mathcal{O}^*(1.44225^n)$ -Zeit Algorithmus



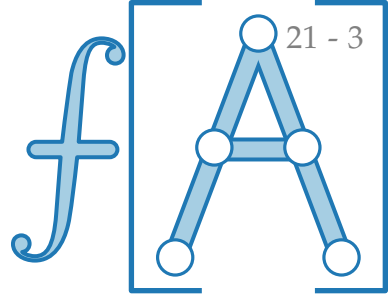
Diskussion

- Schlaueres Branching liefert einen $\mathcal{O}^*(1.44225^n)$ -Zeit Algorithmus
- verglichen mit Brute-force, was $\mathcal{O}(2^n \cdot n)$ Zeit dauert.



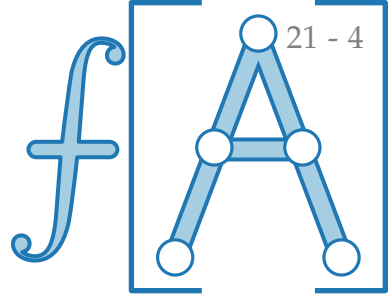
Diskussion

- Schlaueres Branching liefert einen $\mathcal{O}^*(1.44225^n)$ -Zeit Algorithmus
- verglichen mit Brute-force, was $\mathcal{O}(2^n \cdot n)$ Zeit dauert.
- Es gibt Algorithmen für MIS, die in $\mathcal{O}^*(1.2202^n)$ Zeit mit polynomiellem Platz laufen,

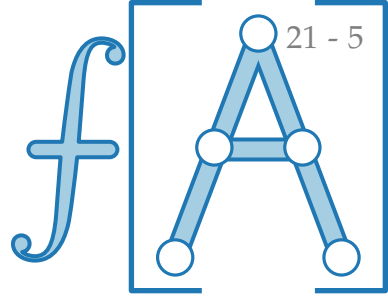


Diskussion

- Schlaueres Branching liefert einen $\mathcal{O}^*(1.44225^n)$ -Zeit Algorithmus
- verglichen mit Brute-force, was $\mathcal{O}(2^n \cdot n)$ Zeit dauert.
- Es gibt Algorithmen für MIS, die in $\mathcal{O}^*(1.2202^n)$ Zeit mit polynomiellem Platz laufen,
- oder in $\mathcal{O}^*(1.2109^n)$ Zeit mit exponentiellem Platz.

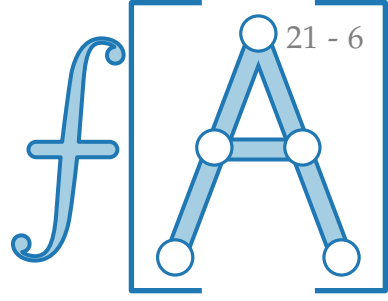


Diskussion



- Schlaueres Branching liefert einen $\mathcal{O}^*(1.44225^n)$ -Zeit Algorithmus
- verglichen mit Brute-force, was $\mathcal{O}(2^n \cdot n)$ Zeit dauert.
- Es gibt Algorithmen für MIS, die in $\mathcal{O}^*(1.2202^n)$ Zeit mit polynomiellem Platz laufen,
- oder in $\mathcal{O}^*(1.2109^n)$ Zeit mit exponentiellem Platz.
- Welche Knoten sind in **jedem** MIS?

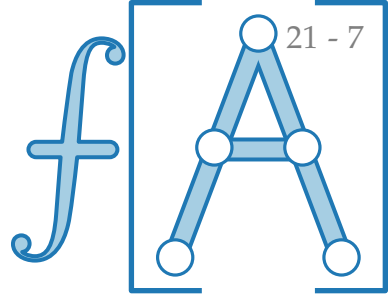
Diskussion



- Schlaueres Branching liefert einen $\mathcal{O}^*(1.44225^n)$ -Zeit Algorithmus
- verglichen mit Brute-force, was $\mathcal{O}(2^n \cdot n)$ Zeit dauert.
- Es gibt Algorithmen für MIS, die in $\mathcal{O}^*(1.2202^n)$ Zeit mit polynomiellem Platz laufen,
- oder in $\mathcal{O}^*(1.2109^n)$ Zeit mit exponentiellem Platz.
- Welche Knoten sind in **jedem** MIS?



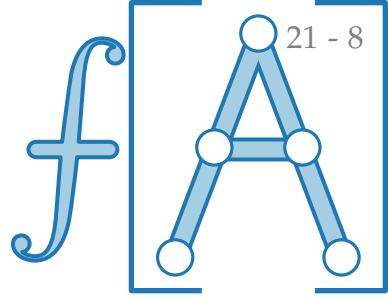
Diskussion



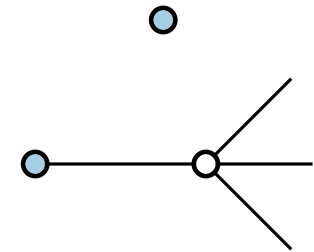
- Schlaueres Branching liefert einen $\mathcal{O}^*(1.44225^n)$ -Zeit Algorithmus
- verglichen mit Brute-force, was $\mathcal{O}(2^n \cdot n)$ Zeit dauert.
- Es gibt Algorithmen für MIS, die in $\mathcal{O}^*(1.2202^n)$ Zeit mit polynomiellem Platz laufen,
- oder in $\mathcal{O}^*(1.2109^n)$ Zeit mit exponentiellem Platz.
- Welche Knoten sind in **jedem** MIS?
- Von welchen Knoten können wir sicher annehmen, dass sie in einem MIS sind?

○

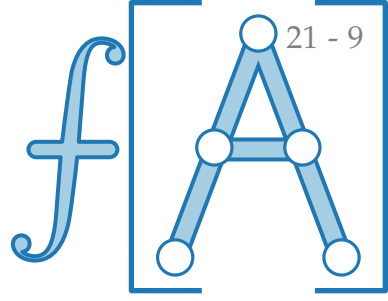
Diskussion



- Schlaueres Branching liefert einen $\mathcal{O}^*(1.44225^n)$ -Zeit Algorithmus
- verglichen mit Brute-force, was $\mathcal{O}(2^n \cdot n)$ Zeit dauert.
- Es gibt Algorithmen für MIS, die in $\mathcal{O}^*(1.2202^n)$ Zeit mit polynomiellem Platz laufen,
- oder in $\mathcal{O}^*(1.2109^n)$ Zeit mit exponentiellem Platz.
- Welche Knoten sind in **jedem** MIS?
- Von welchen Knoten können wir sicher annehmen, dass sie in einem MIS sind?



Diskussion



- Schlaueres Branching liefert einen $\mathcal{O}^*(1.44225^n)$ -Zeit Algorithmus
- verglichen mit Brute-force, was $\mathcal{O}(2^n \cdot n)$ Zeit dauert.
- Es gibt Algorithmen für MIS, die in $\mathcal{O}^*(1.2202^n)$ Zeit mit polynomiellem Platz laufen,
- oder in $\mathcal{O}^*(1.2109^n)$ Zeit mit exponentiellem Platz.
- Welche Knoten sind in **jedem** MIS?
- Von welchen Knoten können wir sicher annehmen, dass sie in einem MIS sind?
- Erweiterte Fallanalyse in [Fomin, Kratsch 2.3] gibt einen $\mathcal{O}^*(1.2786^n)$ -Zeit Algorithmus.

